IFC Alignment Geometry Implementation Guide

A developer's guide to IFC4x3 alignment geometry

Revision: 14 Oct 2026



High Five Interchange at the intersection of I-635 and U.S. Route 75 in Dallas, Texas

Photo by austrini — CC BY 2.0 via Wikimedia Commons

Executive Summary

IFC4x3 — published as ISO 16739-1:2024 — is the first version of the IFC schema to support infrastructure alignment geometry: the horizontal, vertical, and cant (superelevation) curves defining the plan, profile, and cross-slope of roads, railways, and other linear infrastructure. The specification defines the *schema* for these entities but not the *mathematics* needed to evaluate them — the parametric equations for curvature, tangent direction, and position are left to the implementer. This guide fills that gap.

Purpose and audience. This is an implementation guide, not a specification, written for software developers building or extending IFC4x3 alignment support in authoring tools, viewers, and validation software. Where the IFC specification states *what* an entity means, this guide supplies *how* to compute it: closed-form and iterative equations, worked examples, and IFC-conformant sample models.

Scope. The guide covers the full alignment geometry stack: semantic vs. geometric representations and concept templates (Ch. 1); parametric equations for every horizontal, vertical, and cant curve type — lines, circular arcs, parabolas, and the five transition-curve families (Clothoid, Bloss, Cosine, Sine, Helmert, Viennese Bend) (Ch. 2–4); offset curves (Ch. 5); approximate polyline geometry (Ch. 6); alignment reuse across layouts (Ch. 7); linear placement (Ch. 8); referents and stationing, including ISO 19148 linear referencing (Ch. 9); sectioned surfaces and solids (Ch. 10); precision and tolerance (Ch. 11); a validated test set (Ch. 12); references (Ch. 13); and a LandXML-to-IFC appendix.

Each chapter closes with a self-contained implementation checklist and is illustrated with figures, worked examples, and links to IFC example files. Readers new to IFC alignment geometry should start with Chapters 1–2; others may go directly to the chapter covering their feature of interest.

Revision Log

Date	Description	By
30 Jun 2026	Chapter 8 revised and expanded: reorganized offset/unreachable-points content into §8.2.1, added §8.2.2 (Stationing and DistanceAlong) and §8.3.3 (Linear Placement Example), and updated/added several figures.	R. Brice
6 Jul 2026	Chapter 8 cleanup following the 30 Jun restructuring: removed an unused figure, fixed stale cross-references, and regenerated Figure 8.2.1-1.	R. Brice
7 Jul 2026	Chapter 8 §8.2/§8.3 rewritten to separate `IfcAxis2PlacementLinear`'s Location and Orientation roles, anchored on `BasisCurve`'s tangent; fixed Figure 8.1.2-2 units, and added a §12.3.4 vertical-offset test case.	R. Brice
8 Jul 2026	Chapter 9 §9.5.2 example reworked with a non-zero starting station to clarify Station vs. `DistanceAlong`; Figure 9.5.2-1 extended with `IfcAlignment`, a second `IfcRelPositions`, and a starting-station referent; §9.6 checklist updated.	R. Brice
9 Jul 2026	Chapter 9 updated to cover the Product Span Positioning concept.	R. Brice
10 Jul 2026	§9.5 rewritten per ISO 19148 §6.2.12.1.	R. Brice
13 Jul 2026	§9.5.3 simplified: `IfcAnnotation` removed from the intersection-referent correlation pattern.	R. Brice
15 Jul 2026	§10.5.4 expanded to cover cross-section/string-line redundancy.	R. Brice
17 Jul 2026	Chapter 10 expanded to cover an additional guide-curve rooting and tag scoping technique, with examples and figures updated to match.	R. Brice
31 Jul 2026	Chapter 10 §10.5 restructured around bSI-InfraRoom/IFC-Specification#118 (the guide-curve mechanism's origin issue)	R. Brice
1 Aug 2026	Added §9.3.4 (Key-Point Referents) and §9.5.2 (IfcRelInterfersElements)	R. Brice
5 Aug 2026	§9.3.4 corrected: key-point referents nest to `IfcAlignment` (Concept Template 4.1.4.4.3 Object Nesting)	R. Brice

Date	Description	By
14 Aug 2026	Updates based on Scott's review comments.	R. Brice
31 Aug 2026	Added §9.2.7 (Decreasing/Reverse Stationing) with a worked example and Figure 9.2.7-1; generalized the §9.2.5 conversion algorithm and §9.7 checklist for stationing direction; expanded §9.3.1 (`HasIncreasingStation`, new Figure 9.3.1-2) and §9.3.2 (Pattern A list-ordering ambiguity, also IFC spec issue 21).	R. Brice
13 Oct 2026	Helmert test data corrected. The `HELMERTCURVE` test cases (horizontal and cant, all unit variants) and the BPaimio–Kupittaa real-world alignment were generated with an IfcOpenShell defect. The second half of each Helmert curve was placed using the heading at the x coordinate the integration had reached, θ(x), instead of the heading at arc length s, θ(s). Every Helmert curve therefore had a heading kink at its midpoint and ended away from the curve its design parameters describe. The geometry IFC, semantic terminator segments, reference CSVs and plots have been regenerated. Implementations validated against the old Helmert reference data will now differ from it by up to about 2.2 cm (0.071 ft) in position and 4.35×10^{-4} rad in direction at the end of the curve (100 m transition from R 300 m to R 1000 m). The difference is zero over the first half of the curve and grows over the second half. An implementation that matched the old data reproduced the defect; one that follows §2.7 now matches. The real-world 3D reference CSVs are now sampled at every metre instead of at 100 equal intervals. The §2.7.2/§2.7.3 worked example's parent-curve position, normalization matrix and raw-spiral integrals were also corrected; these were arithmetic errors unrelated to the defect.	R. Brice
13 Oct 2026	Test set and example models regenerated with the current IfcOpenShell. Cant transitions that start with the left rail canted (right-hand exit and arc-to-arc transitions, and all right-hand Viennese bends) had their cross-slope axis	R. Brice

Date	Description	By
	leaning the wrong way, and the second half of each Helmert cant transition started half way in sine rather than half way in angle between the start and end cross slopes. Implementations validated against the old cant reference data will now differ from it in the `Axis`/cross-track columns, by up to 0.213 where the lean was reversed; positions and the horizontal and vertical reference data are unchanged. Stationing referents no longer have an `IfcRelPositions` to the alignment (§9.3). In the example models the starting referent is now placed on the alignment curve instead of at the world origin, and offset-curve alignments carry their own stationing referent. The FHWA key-point referents are renamed to the `"E-Line 119+56.79 (P.C.)"` form (§9.3.4). The §4.5 Helmert cant worked example now places the second half at the interpolated cross slope, 1.517361336 rad, instead of 1.517437677 rad, matching the regenerated data; §4.5.3 is recomputed, and its trim-start elevation (0.04 m, not 0.16 m), trim-start cross slope and interpolation formula are corrected.	
13 Oct 2026	§4.1.3/§4.2 cant evaluation algorithm aligned with IfcOpenShell: the end cross slope is that of the next segment's `Placement.Axis`; the placement's `Y` is normalized; the parent-curve frame uses `RefDir` and `Axis` as computed, without re-orthogonalizing; and the trim-start translation is local (0). The §4.5.3 worked example now reproduces IfcOpenShell's evaluation of the test set to 1e-16.	R. Brice
13 Oct 2026	Chapter 4 cant worked examples (§4.3, §4.4, §4.6-§4.10) recomputed with the §4.2 algorithm. Each now reproduces IfcOpenShell's evaluation of the corresponding test set file and the mapping excerpts match the test set. Corrected along the way: hand-derived vectors that disagreed with the matrices, §4.4's parent-curve elevation (`IfcClothoid` has no constant term, so D(0) = 0), §4.6's start elevation (0, not	R. Brice

Date	Description	By
	0.08 m), §4.9's end cross slope, and §4.10's vertical matrix (swapping rows and columns of a flat-grade M_v gives the identity). §4.13 gains two checklist items.	
13 Oct 2026	Cant and spiral reference data regenerated with a corrected IfcOpenShell. The cant slope along the track (the `RefDir` of the cant frame) for `IfcClothoid`, `IfcCosineSpiral` and `IfcSineSpiral` parent curves was L times too small, so the `LINEARTRANSITION`, `COSINECURVE` and `SINECURVE` cant reference CSVs change in their `RefDir` and cross-track columns by up to 1.6e-3. IfcOpenShell now also integrates spiral positions more accurately, so spiral positions in the horizontal, cant 3D and real-world reference CSVs change by up to 1.3e-5 m and now agree with an independent evaluation to the CSVs' rounding. The test set IFC files are unchanged.	R. Brice

Table of Contents

Chapter 1 — IFC Alignment Concepts 1
Chapter 2 — Horizontal Alignments 11
Chapter 3 — Vertical Alignments 52
Chapter 4 — Cant Alignments 70
Chapter 5 — Offset Curves 116
Chapter 6 — Approximate Alignment Geometry 123
Chapter 7 — Alignments Reusing Horizontal Layout 125
Chapter 8 — Linear Placement 131
Chapter 9 — Referents and Stationing 145
Chapter 10 — Sectioned Surfaces and Solids 175
Chapter 11 — Precision and Tolerance 200
Chapter 12 — Alignment Geometry Testset 204
Chapter 13 — References 219
Appendix A — LandXML to IFC Conversion 221

Notation

Scalars and Parameters

Arc-Length and Distance

Symbol	Definition
s	Arc-length parameter along the parent curve
s_0	Arc-length at the start of the trim; equals `SegmentStart`
L	Segment length; equals `SegmentLength`
ℓ	Horizontal distance from the segment start; evaluation variable in vertical and cant sections
d	Distance measured along the horizontal `IfcCompositeCurve`
d_0	Distance along at the trim start; $d_0 = x(s_0)$
h_l	Horizontal length of a vertical segment; equals `HorizontalLength`

Angles

Symbol	Definition
$\theta(s)$	Bearing angle at arc-length s in the horizontal plane
$\theta(\ell)$	Tangent angle at horizontal distance ℓ; grade angle $\tan^{-1}(g(\ell))$ in the vertical plane; slope of deviating elevation $\tan^{-1}(D'(\ell))$ in the cant plane
θ_0	Tangent angle at the trim start s_0
θ_p	Tangent angle of `IfcCurveSegment.Placement.RefDirection`
θ_v	Grade direction angle; $\theta_v = \tan^{-1}(dy_v/dx_v)$
ϕ	Cross-slope angle; angle from the transverse y-axis to the `Axis` vector
$\phi_s,\ \phi_e$	Cross-slope angle at the start and end of a cant segment
ϕ_p	Cross-slope angle of `IfcCurveSegment.Placement.Axis`
Δ	Sweep angle of a circular arc; angle subtended at the center by the arc from the trim start to an evaluation point
Δ_0	Angular position of the trim-start point on the parent circle; angle from the circle's positive x-axis to the

Symbol	Definition
	radius vector at the trim start (Section 3.4)
Δ_{pc}	Radial angle at the evaluation point on the parent circle; $\Delta_{pc} = \Delta_0 - \Delta$ for a clockwise segment, $\Delta_0 + \Delta$ for counter-clockwise (Section 3.4)

Curvature and Radius

Symbol	Definition
$\kappa(s)$	Curvature at arc-length s; $\kappa = 1/R$
κ_s	Curvature at the segment start; $\kappa_s = 1/R_s$
κ_e	Curvature at the segment end; $\kappa_e = 1/R_e$
R	Radius of curvature
R_s	Radius at the segment start; equals `StartRadiusOfCurvature`
R_e	Radius at the segment end; equals `EndRadiusOfCurvature`
f	Cumulative change in curvature over the segment; $f = L(1/R_e - 1/R_s)$

Grade

Symbol	Definition
$g(s)$	Gradient (slope) at arc-length s
g_s	Gradient at the segment start; equals `StartGradient`
g_e	Gradient at the segment end; equals `EndGradient`

Position and Elevation

Symbol	Definition
$x(s),\ y(s)$	Coordinates of the parent curve at arc-length s
$x_0,\ y_0$	Parent curve position at the trim start s_0
$x_p,\ y_p$	Placement location; equals `IfcCurveSegment.Placement.Location`
z	Elevation (vertical y-coordinate mapped to 3D z)
z_0	Elevation at the trim start; equals `StartHeight`
$C_x,\ C_y$	Center coordinates of a circular arc parent curve
c	Chord length from the trim start to an evaluation point

Cant (Deviating Elevation)

Symbol	Definition
$D(s)$	Deviating elevation at arc-length s; vertical offset of the track centerline from the gradient curve
D_0	Deviating elevation at the trim start; $D_0 = D(s_0)$
$D_s,\ D_e$	Deviating elevation at the segment start and end; $D_s = (D_{sl} + D_{sr})/2$
$D_{sl},\ D_{sr}$	Left and right cant elevation at the segment start; equals `StartCantLeft`, `StartCantRight`
$D_{el},\ D_{er}$	Left and right cant elevation at the segment end; equals `EndCantLeft`, `EndCantRight`
D_p	Deviating elevation component of `IfcCurveSegment.Placement.Location`
ΔD	Change in deviating elevation over the segment; $\Delta D = D_e - D_s$
D_{rh}	Rail head distance; center-to-center distance between railheads; equals `RailHeadDistance`
β_s	Cross-slope ratio at the segment start; $\beta_s = (D_{sr} - D_{sl})/D_{rh}$
β_e	Cross-slope ratio at the segment end; $\beta_e = (D_{er} - D_{el})/D_{rh}$
h_{cg}	Gravity centerline height; height of the vehicle center of gravity above the rail plane; equals `GravityCenterLineHeight`
cf	Cant factor; $cf = -420\ (h_{cg}/L)(\beta_e - \beta_s)$; scales the Viennese Bend polynomial coefficients

Spiral Curve Coefficients

Symbol	Definition
A	Clothoid constant
u	Clothoid unit parameterization parameter; $u = s\ /\ \lvert A\sqrt{\pi}\rvert$
A_i	Dimensional polynomial coefficient (with units) for spiral and parabolic curves
a_i	Normalized (dimensionless) polynomial coefficient prior to scaling
$A_{i1},\ A_{i2}$	Coefficient A_i for the first and second halves of a Helmert segment

Vectors

Symbol	Definition
$dx,\ dy$	Horizontal tangent direction components; $dx = \cos\theta$, $dy = \sin\theta$
$dx_p,\ dy_p$	Tangent direction components at the placement; $dx_p = \cos\theta_p$, $dy_p = \sin\theta_p$
$dx_v,\ dy_v$	Grade direction components from M_v; $dx_v = \cos\theta_v$, $dy_v = \sin\theta_v$
$\boldsymbol{RefDir}_p$	Reference direction of `IfcCurveSegment.Placement`; equals `Placement.RefDirection`
$\boldsymbol{Axis}_p$	Axis direction of `IfcCurveSegment.Placement`; equals `Placement.Axis`
$\boldsymbol{X}_p,\ \boldsymbol{Y}_p,\ \boldsymbol{Z}_p$	Orthonormal basis vectors of the placement frame

Matrices

All matrices are 4×4 homogeneous transformation matrices. Column 4 carries position; columns 1–3 carry the frame orientation.

Symbol	Definition
M_{CSP}	Curve segment placement matrix; constructed from `IfcCurveSegment.Placement`; maps the trimmed segment into the alignment coordinate system
M_N	Normalization matrix; translates the trim-start point to the origin and rotates the tangent to align with the positive x-direction
M_{PC}	Parent curve matrix at the evaluation point; encodes $x(s)$, $y(s)$, and $\theta(s)$
M_{PCS}	Cant parent curve matrix evaluated at the trim start s_0
$M_{PC\ell}$	Cant parent curve matrix evaluated at the point under consideration ℓ
M_h	Horizontal placement matrix; $M_h = M_{CSP}\, M_N\, M_{PC}$
M_v	Vertical placement matrix; $M_v = M_{CSP}\, M_N\, M_{PC}$ in the (distance-along, elevation) plane
M_v'	Modified vertical matrix; rows 2 and 3 of M_v swapped, distance-along component zeroed, for multiplication with M_h
M_v''	Orientation-only form of M_v'; column 4 set to $(0,0,0,1)^T$ for use in the cant 3D composition
M_c	Cant placement matrix; $M_c = M_{CSP}\, M_N\, M_{PC}$ in the

Symbol	Definition
	(distance-along, deviating elevation) plane
M_c'	Modified cant matrix; deviating elevation moved from row 2 to row 3, distance-along component zeroed, for multiplication
M_c''	Orientation-only form of M_c'; column 4 set to $(0,0,0,1)^T$ for use in the cant 3D composition
M_{3D}	Full 3D placement matrix combining horizontal and vertical; $M_{3D} = M_h\, M_v'$
M_{3Dcant}	Full 3D placement matrix combining horizontal, vertical, and cant; $M_{3Dcant} = M_h\, M_v''\, M_c''$
I	4×4 identity matrix

Chapter 1 — IFC Alignment Concepts

1.0 Introduction

IFC4x3 ADD2, published as ISO 16739-1:2024 [1], is the current IFC standard and includes full support for infrastructure alignment geometry. Despite the standard's publication, software adoption has lagged — not because the standard is new, but because implementation guidance has been scarce. This guide addresses that gap by documenting the relevant concepts and mathematics a software developer needs to implement alignment-based geometry.

If you are unfamiliar with infrastructure alignment geometry, information is available in a multitude of highway engineering, rail engineering, and surveying texts. A concise primer is available in the US Federal Highway Administration (FHWA), Bridge Geometry Manual [2], Chapters 1 - 5 and Appendix B.

This guide assumes the reader has a basic working knowledge of IFC and is familar with the specifications including the fundamental concepts and assumptions.

The IFC specification provides concept templates for mapping semantic alignment representations to their geometric counterparts. The concept templates — covering aggregation to project, alignment layout nesting, and the four alignment geometry variants (horizontal; horizontal+vertical; horizontal+vertical+cant; segments) — provide the minimum information required to properly structure alignment semantic and geometric entities an IFC model. The *partial* concept templates that cover individual curve segment geometry types, however, are sparse: they show class relationships but do not provide mathematical equations or parameter mappings.

This guide aims to encompass the full scope of IFC alignment implementation: the parametric equations for horizontal, vertical, and cant curve segments (Chapters 2–4); offset curves (Chapter 5); approximate polyline representations (Chapter 6); alignments that share a common horizontal layout (Chapter 7); linear placement of objects along an alignment (Chapter 8); referents and stationing, including station equations (Chapter 9); alignment-based physical geometry — sectioned surfaces and solids (Chapter 10); precision and tolerance guidance (Chapter 11); examples and validation data (Chapter 12); and references (Chapter 13). LandXML-to-IFC conversion is covered in Appendix A.

1.1 Horizontal Alignment

A horizontal alignment typically consists of straight lines called **tangents** (or tangent runs) that are connected by **horizontal curves**. The curves are usually circular arcs. Easement or **transition curves** in the form of spirals can be used to provide gradual transitions between tangents and circular curves. The horizontal alignment is a plan view curvilinear path in a Cartesian coordinate system aligned with East and North. Positions along an alignment, measured along the plan view projection of the 3D alignment curve, are

typically denoted with *stations* (also called *chainage* in British and Australian practice; the two terms are used interchangeably throughout this guide).

1.2 Vertical Alignment

A vertical alignment (often referred to as the vertical profile by civil engineers) consists of straight sections of **grade lines** connected by **vertical curves**. The vertical curves are typically parabolas for roadway and circular arcs or clothoid curves for rail. A vertical alignment is defined along the curvilinear path of a horizontal alignment in a 2D "Distance Along Horizontal Alignment, Elevation" coordinate system. Combined, the horizontal and vertical alignments define a 3D curve. This combination of two 2D curves is sometimes referred to as a 2.5D or 2+1D geometry. The IFC specification notes that contemporary alignment design almost always implements this 2.5D approach, with precision varying based on management priorities, design era, and available software tools.

1.3 Cant Alignment

When a rail vehicle traverses a horizontal curve, centrifugal force acts laterally on passengers and cargo. Tilting the track cross-section — raising the outer (high) rail above the inner (low) rail — converts part of that lateral force into a downward component, improving ride comfort and permitting higher operating speeds through curves. This tilt is called **cant** (or *superelevation*).

Cant is measured as the height difference (deviating elevation) between the two rail heads, perpendicular to the track. The low (inner-curve) rail is the conventional pivot point for rotation; the high (outer-curve) rail rises by the cant amount.

A cant alignment is structured similarly to the horizontal and vertical alignments: **constant-cant zones** — corresponding to the tangent and circular arc sections of the horizontal alignment, where cant is held at a fixed value — are connected by **cant transitions** that ramp over the length of the horizontal transition spiral. Transitions may be linear or may follow the same non-linear curve families used for horizontal spirals.

Like the vertical alignment, the cant alignment is defined in a two-dimensional profile: distance along the horizontal alignment on one axis, cant value on the other. The cant profile encodes not only a deviating elevation but also the cross slope of the rail head, fully defining the rotated cross-section. This profile is then layered onto the combined horizontal and vertical alignment to produce a full 3D alignment curve incorporating cross-sectional banking.

1.4 Semantic Definition (Business Logic)

The semantic definition of alignment allows for the alignment to be described as close as possible to the terminology and concepts used in a business context. Examples include horizontal, vertical and cant layouts, stationing, and anchor points for domain specific

properties such as design speed, cant deficiency, superelevation transitions, and widenings.

Specialized applications can analyze alignments using the semantic information. Alignments can be evaluated against design criteria such as speed requirements, sight distances, and maximum gradient, to name a few.

Importantly, the semantic and geometric representations are independent and can be used and exchanged separately. A proper IFC file may contain only the semantic definition without any geometric representation, which is common in early design stages when geometry has not yet been computed.

An `IfcAlignment` may be defined with a horizontal layout only, a horizontal and vertical layout, or a full horizontal, vertical, and cant layout. Each layout type is composed through an `IfcRelNests` relationship: `IfcAlignmentHorizontal`, `IfcAlignmentVertical`, and `IfcAlignmentCant` are nested within the `IfcAlignment` as applicable. Each layout is in turn composed of one or more `IfcAlignmentSegment` entities, also through `IfcRelNests`. The `IfcAlignmentSegment` models the segment design parameters in a subtype of `IfcAlignmentParameterSegment`: `IfcAlignmentHorizontalSegment`, `IfcAlignmentVerticalSegment`, or `IfcAlignmentCantSegment`, depending on the layout. The semantic definition model is shown in Figure 1.4-1.

> NOTE: Multiple vertical and cant alignments can reuse the same horizontal alignment in more complex modelling scenarios. This is discussed in Chapter 7.

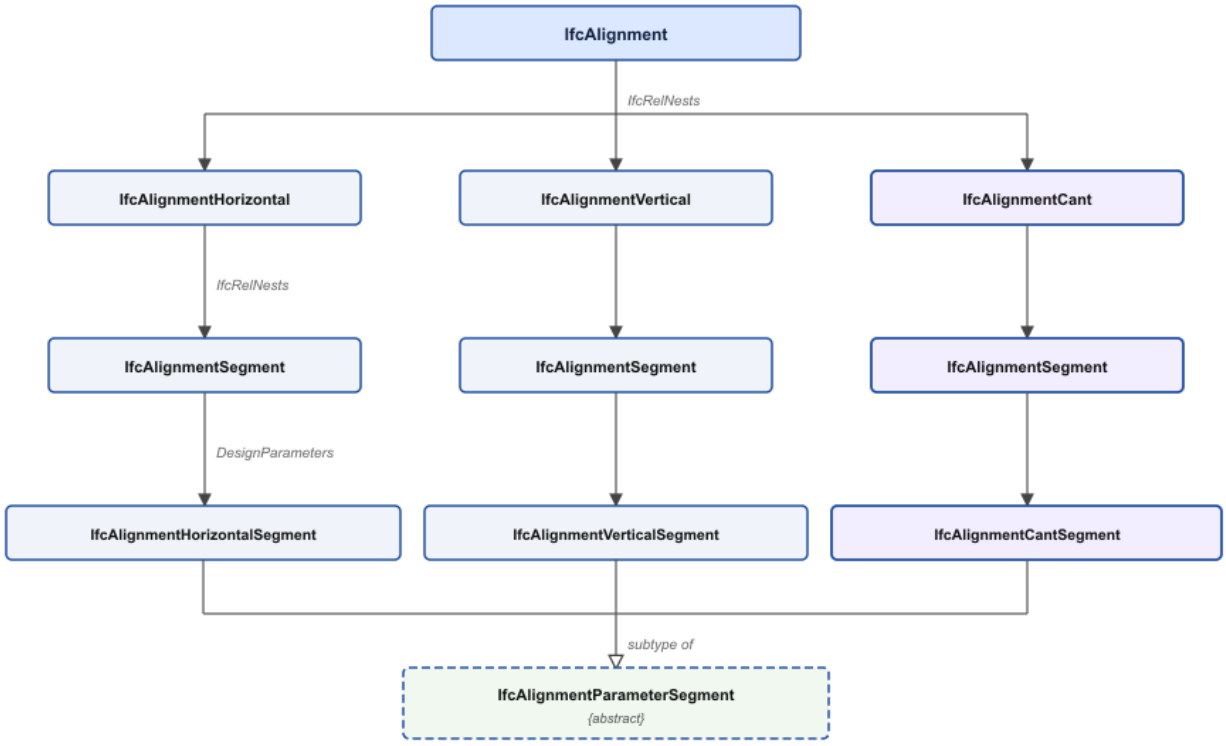


Figure 1.4-1 Semantic alignment model

`IfcRelNests.RelatedObjects` is an ordered collection, so the layout sub-objects have a defined sequence within the nest. The IFC specification does not state this order explicitly, but it can be inferred from the diagrams in the `IfcAlignment` documentation (§5.4.3.1.5): `IfcAlignmentHorizontal` precedes `IfcAlignmentVertical`, which precedes `IfcAlignmentCant`.

The ordering of `IfcAlignmentSegment` instances within each layout's nest is similarly unspecified by the schema. For `IfcAlignmentHorizontalSegment` the only logical order is start to end. For `IfcAlignmentVerticalSegment` and `IfcAlignmentCantSegment`, positional order could theoretically be derived from the `StartDistAlong` attribute regardless of list order, but relying on this is unnecessarily complex. Segments should always be ordered start to end in `IfcRelNests.RelatedObjects`.

An interesting caveat of the alignment layout entities (`IfcAlignmentHorizontal`, `IfcAlignmentVertical`, and `IfcAlignmentCant`) is that the last `IfcAlignmentSegment.DesignParameters` must be zero length. If a geometric representation is provided, then its corresponding last segment must also be zero length. For a continuous composition of segments, the end of one segment is at the same location as the start of the next segment. The zero-length segment is intended to provide the end point of the last segment. See §2.12 for full implementation details.

Aggregation to project. `IfcAlignment` is a mandatory participant in the IFC spatial structure. Every alignment must be aggregated to `IfcProject` — directly, or indirectly through a parent alignment — via an `IfcRelAggregates` relationship. An alignment not connected to the project is invalid.

Referencing to site. Alignments can traverse multiple administrative or physical sites. For this reason, `IfcAlignment` associates with `IfcSite` using `IfcRelReferencedInSpatialStructure` rather than `IfcRelContainedInSpatialStructure`. The referenced-in relationship is non-exclusive: a single alignment may be referenced by every `IfcSite` it crosses. This is the key distinction between *containment* (one element, one spatial zone) and *referencing* (one element, many zones). Figure 1.4-2 shows multiple sites along an alignment. The central East-West alignment is contained in the spatial structure of the project and referenced into the spatial structure of all the sites it traverses.



Figure 1.4-2 Alignments spanning multiple sites

1.5 Geometric Representation

1.5.1 Overview

Section 4.1.7.1.1 of the IFC4x3 specification indicates that the valid representations of `IfcAlignment` are:

- `IfcCompositeCurve` as a 2D horizontal alignment (represented by its horizontal alignment segments), without a vertical layout.
- `IfcGradientCurve` as a 3D horizontal and vertical alignment (represented by their alignment segments), without a cant layout.
- `IfcSegmentedReferenceCurve` as a 3D curve defined relative to an IfcGradientCurve to incorporate the application of cant.
- `IfcOffsetCurveByDistances` as a 2D or 3D curve defined relative to an `IfcGradientCurve` or another `IfcOffsetCurveByDistances`.
- `IfcPolyline` or `IfcIndexedPolyCurve` as a 3D alignment by a 3D polyline representation (such as coming from a survey).
- `IfcPolyline` or `IfcIndexedPolyCurve` as a 2D horizontal alignment by a 2D polyline representation (such as in very early planning phases or as a map representation).

The `RepresentationIdentifier` and `RepresentationType` values required by the IFC concept templates for each alignment geometry variant are listed in Table 1.5.1-1.

Alignment variant	Curve entity	`RepresentationIdentifier`	`RepresentationType`
Horizontal only	`IfcCompositeCurve`	`'Axis'`	`'Curve2D'`

Alignment variant	Curve entity	RepresentationIdentifier	RepresentationType
Horizontal + Vertical	IfcCompositeCurve (plan view)	'FootPrint'	'Curve2D'
Horizontal + Vertical	IfcGradientCurve (3D)	'Axis'	'Curve3D'
Horizontal + Vertical + Cant	IfcCompositeCurve (plan view)	'FootPrint'	'Curve2D'
Horizontal + Vertical + Cant	IfcSegmentedReferenceCurve (3D)	'Axis'	'Curve3D'

Table 1.5.1-1 — Required RepresentationIdentifier and RepresentationType for each alignment geometry variant

IfcGradientCurve and IfcSegmentedReferenceCurve inherit from IfcCompositeCurve. These curves consist of a sequence of segments defined by IfcCurveSegment. The mathematical computations for IfcCurveSegment geometry are the primary focus of Chapters 2 - 4.

IfcPolyline and IfcIndexedPolyCurve produce approximate, discrete geometry rather than exact parametric curves. They arise in survey-derived alignments and early planning contexts. Their use, limitations, and incompatibility with the full semantic alignment definition are discussed in Chapter 6. The IFC specification does not indicate the proper RepresentationIdentifier or RepresentationType; by analogy with the parametric curve variants, 'Axis' with either 'Curve2D' or 'Curve3D' is appropriate.

IfcOffsetCurveByDistances is treated in Chapter 5. Again, the representation identifier and representation type are not indicated in the specification. The most logical choice is to match the BasisCurve.

1.5.2 Understanding Geometric Representation of Alignment

The semantic definition of an alignment and its geometric representation carry overlapping information — both describe segment types, lengths, and radii — but they serve different consumers. The semantic representation encodes design intent in domain vocabulary: a horizontal segment typed as CLOTHOID with a StartRadiusOfCurvature of infinity, an EndRadiusOfCurvature of 300 m, and a SegmentLength of 100 m tells a design application *what* the engineer specified and enables evaluation against standards for minimum radius, design speed, and sight distance. The geometric representation encodes the computed mathematical form: the exact start coordinates, the tangent bearing at each point, and parametric equations that yield position and direction at any arc-length along the curve — the form that a rendering engine, clash-detection tool, or quantity-takeoff calculation requires.

The geometric representation of `IfcAlignment` consists of one or more `IfcShapeRepresentation` instances: a plan-view 2D curve and, where vertical or cant geometry is present, a 3D curve. These are illustrated in Figure 1.5.2-1.

Geometrically, the horizontal alignment is defined by an `IfcCompositeCurve` and the vertical alignment is defined by an `IfcGradientCurve`. The horizontal alignment is the basis curve for the vertical alignment.

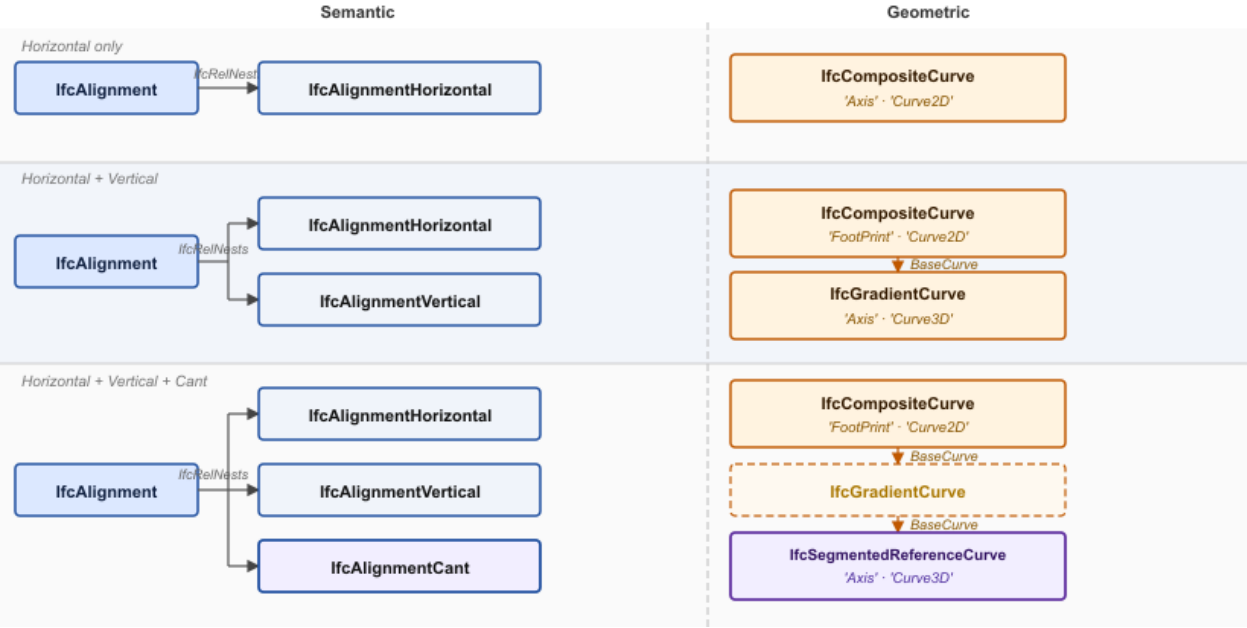


Figure 1.5.2-1 Geometric representation variants for the three alignment constructs (H, H+V, H+V+C). The dashed `IfcGradientCurve` box in the H+V+C row appears as a `BaseCurve` intermediate only — it is not itself a shape representation.

The geometric elements are modeled with `IfcCurveSegment`. `IfcCurveSegment` cuts a segment from a parent curve and places it in the alignment coordinate system. A horizontal circular arc is modeled with an `IfcCurveSegment` that cuts an arc from an `IfcCircle`. Figure 1.5.2-2 shows an alignment consisting of a tangent run (red) and a horizontal curve (green). The line and curve segments are cut from their `IfcLine` and `IfcCircle` parent curves, respectively. The process of defining a parent curve, cutting a segment from that curve, and placing the cut segment into the alignment is repeated to define the entire horizontal alignment. All the horizontal alignment `IfcCurveSegment` objects are then combined to create an `IfcCompositeCurve` to complete the plan view geometric representation of the horizontal alignment.

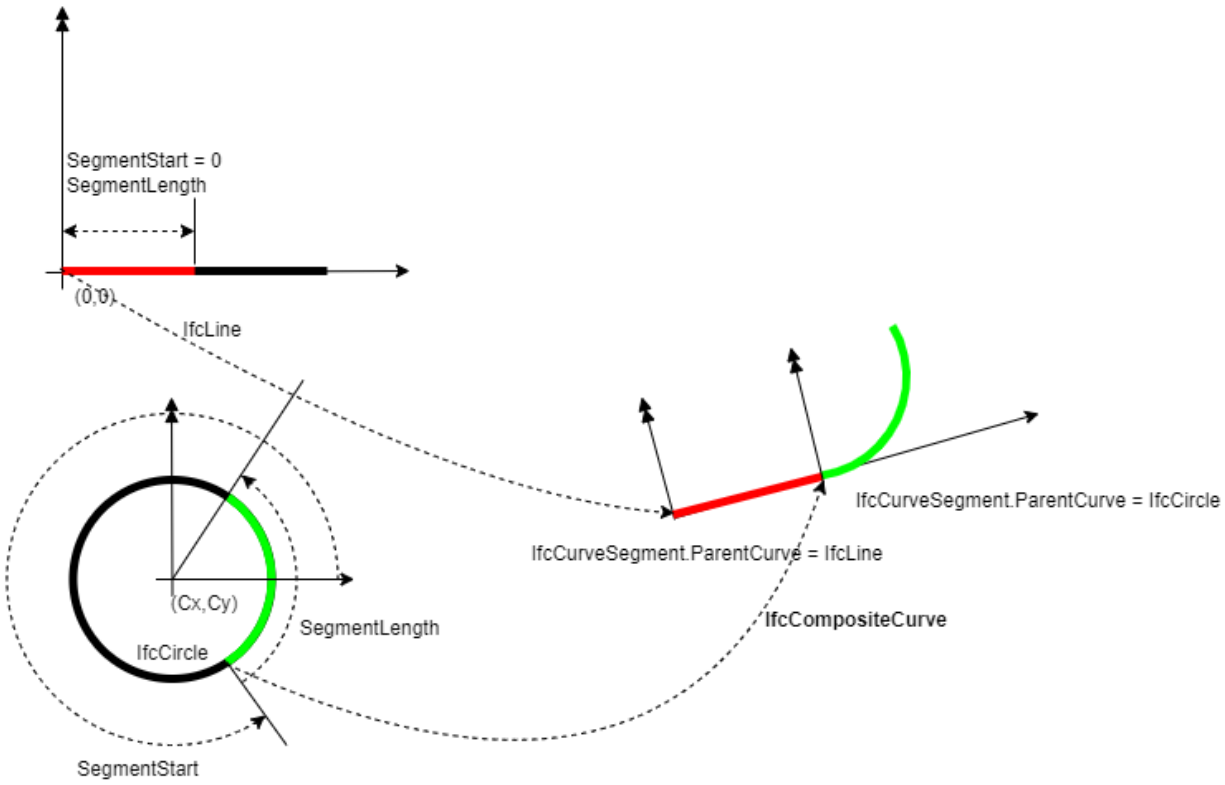


Figure 1.5.2-2 Illustration of parent curves and their placement with `IfcCurveSegment.Placement`

NOTE: Angles start on the positive coordinate axis (X-axis for this example) and increase in counter-clockwise direction.

A similar process is used to create the vertical alignment. `IfcCurveSegment` instances trim geometry from parent curves and combine end to start to create an `IfcGradientCurve`. `IfcGradientCurve` is a sub-type of `IfcCompositeCurve` with the additional attribute `BaseCurve`. The segments of the gradient curve are placed in a 2-dimensional plane in a "distance along the horizontal alignment, elevation" coordinate system. The `IfcGradientCurve` uses the horizontal alignment `IfcCompositeCurve` as its base curve.

`IfcSegmentedReferenceCurve` is also a sub-type of `IfcCompositeCurve`. It defines a deviating elevation and rail head cross slope along the base curve. The base curve is an `IfcGradientCurve`. The "distance along" value is along the gradient curve's base curve, which is the horizontal alignment curve.

1.5.2.1 Curve segment geometry

When defining curve segment geometry with an `IfcCurveSegment`, the location and orientation of the parent curve are not particularly important. The `IfcCurveSegment.Placement` defines the location of the start point of the trimmed curve as well as the orientation of the tangent at the start of the trimmed curve. In the example

above, the `IfcLine` parent curve starts at (0,0) and progresses in horizontal direction, but the tangent segment of the alignment starts at a specific (X,Y) position and is oriented in a North-East direction. The `IfcCurveSegment.Placement` attribute establishes this position and orientation.

1.5.2.2 Vertical and Cant Curve Extents

For `IfcGradientCurve` and `IfcSegmentedReferenceCurve` the segments do not need to start or end at the same location of their base curve. These curves can be shorter or longer than the horizontal alignment. This is illustrated in Figure 1.5.2.2-1 where the blue curve is the full horizontal curve and the orange curve is the portion of the horizontal curve coinciding with the vertical curve. The `IfcGradientCurve` (#770) has the `IfcCompositeCurve` (#657) as its base curve. The placement of the first `IfcCurveSegment` (#771) in the `IfcGradientCurve` is defined by `IfcAxis2Placement2D` (#777) with Location as `IfcCartesianPoint` (#778). The first segment is located at a distance of 250.0036 along the horizontal `IfcCompositeCurve` at an elevation of 145.3129 m. The IFC specification does not provide guidance on how to define complete 3D geometry in the regions of the horizontal alignment that do not overlap with the vertical or cant curves.

```
#657=IFCCOMPOSITECURVE((#658,#671,#684,#697,#710,#723,#736,#749,#762),.F.)
#770=IFCGRADIENTCURVE((#771,#784,#796,#809,#821,#834,#846,#859,#871),.F.,#657
,#887)
#771 = IFCCURVESEGMENT(.CONTSAMEGRADIENTSAMECURVATURE., #777,
IFCLENGTHMEASURE(0.),
IFCLENGTHMEASURE(40.0002408172751), #780);
#777 = IFCAXIS2PLACEMENT2D(#778, #779);
#778 = IFCCARTESIANPOINT((250.003634293681, 145.312908277025));
```

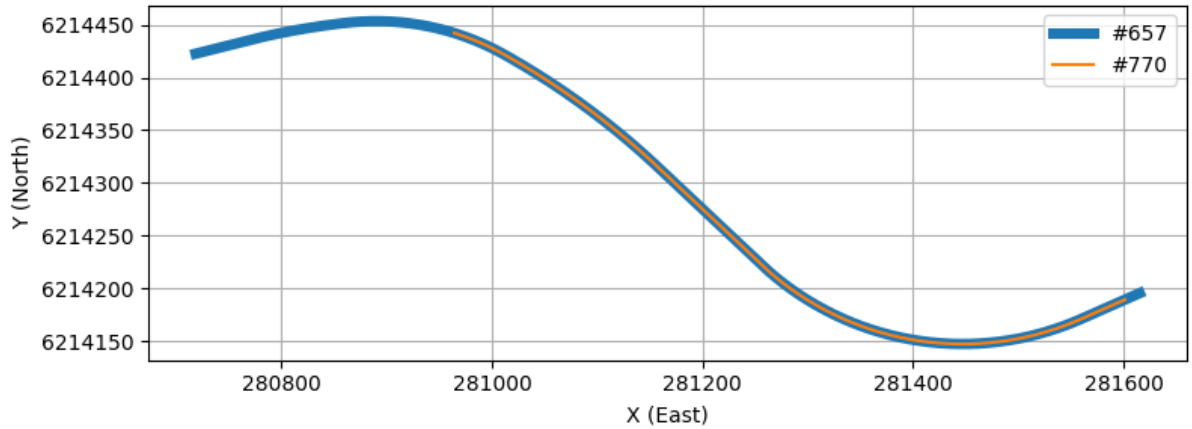


Figure 1.5.2.2-1 — Example of `IfcGradientCurve` with a start location and length different than the `IfcCompositeCurve` BaseCurve.

1.6 Reference Implementation and Segment Mapping

Chapters 2, 3, and 4 discuss the mapping of the semantic definition of alignment segments (`IfcAlignmentSegment` subtypes) to their corresponding geometric representation

(`IfcCurveSegment.ParentCurve`). In general, the mapping formulas are given without derivation and have been developed from the reference implementation, EnrichIFC4x3, published at IFC-Rail-Unit-Test-Reference-Code/EnrichIFC4x3/EnrichIFC4x3/business2geometry at master · bSI-RailwayRoom/IFC-Rail-Unit-Test-Reference-Code (github.com) [3].

The cant segment mappings in the EnrichIfc4x3 reference implementation are incorrect. These errors are documented in Chapter 4, which presents corrected formulas and explains why the reference implementation is incorrect.

1.7 Units of Measure

Unless otherwise specified, the following unit conventions apply throughout this guide:

Quantity	Unit
Length, distance, elevation, offset, cant	meter (m)
Spiral and polynomial coefficients (A, A_0, A_1, ...)	meter (m)
Curvature ($\kappa = 1/R$)	reciprocal meter (m^{-1})
Angles (bearings, grade angles, cross-slope angles)	radian (rad)
Gradient (vertical slope)	dimensionless ratio (m/m)

Table 1.7-1 — Unit conventions used throughout this guide

Gradient is expressed as a decimal rise-over-run value — 0.02 represents a 2% grade — not as a percentage or angle.

Chapter 2 — Horizontal Alignments

2.0 Introduction

The geometric representation of a horizontal alignment is accomplished with an `IfcCompositeCurve`. The composite curve consists of a sequence of `IfcCurveSegment` entities whose geometry is defined by a parent curve. This section defines the mathematical relationships and equations for each parent curve type and the algorithm for evaluating points on those curves.

Table 2.0-1 maps each `IfcAlignmentHorizontalSegment.PredefinedType` to its corresponding parent curve type.

Business Logic (`IfcAlignmentHorizontalSegment.PredefinedType`)	Geometric Representation (`IfcCurveSegment.ParentCurve`)
LINE	`IfcLine`
CIRCULARARC	`IfcCircle`
CLOTHOID	`IfcClothoid`
CUBIC	`IfcPolynomialCurve`
HELMERTCURVE	`IfcSecondOrderPolynomialSpiral`
BLOSSCURVE	`IfcThirdOrderPolynomialSpiral`
COSINECURVE	`IfcCosineSpiral`
SINECURVE	`IfcSineSpiral`
VIENNESEBEND	`IfcSeventhOrderPolynomialSpiral`

Table 2.0-1 — Mapping of business logic to geometric representation for horizontal alignment

This chapter covers:

- The three-step evaluation algorithm for computing position and tangent direction at any point on a horizontal curve segment.
- Parametric equations and geometry mapping examples for all nine curve types in Table 2.0-1.
- The zero-length closing segment required at the end of every `IfcCompositeCurve`.

2.1 Conventions

Each curve is parameterized by arc-length s, where $s = 0$ at the start of the parent curve. The start and end radii R_s and R_e are taken from `IfcAlignmentHorizontalSegment.StartRadiusOfCurvature` and `EndRadiusOfCurvature`; a value of zero indicates infinite radius (zero curvature, i.e. a straight line). Radii follow a

positive-left sign convention: a positive value indicates curvature to the left of the direction of travel; a negative value indicates curvature to the right. The segment length L is `IfcAlignmentHorizontalSegment.SegmentLength`. The tangent angle $\theta(s)$ is measured from the positive x-axis; its cosine and sine form the `RefDirection` of the curve at s.

The x and y coordinates of all horizontal parent curves are defined by the integrals:

$$x(s) = \int \cos\theta(s)\, ds \qquad y(s) = \int \sin\theta(s)\, ds$$

2.2 Curve Segment Evaluation Algorithm

This algorithm evaluates the 2D position and tangent direction of a point on an `IfcCurveSegment` in the alignment coordinate system. Let s_0 = `SegmentStart` and s = the arc-length parameter of the point to evaluate, where $s_0 \le s < s_0 + \mathtt{SegmentLength}$.

Step 1 — Form the curve segment placement matrix M_{CSP}

M_{CSP} places the trimmed segment into the alignment coordinate system. It is constructed directly from `IfcCurveSegment.Placement`, where (x_p, y_p) is the `Location` and θ_p is the bearing of the `RefDirection`.

$$M_{CSP} = \begin{bmatrix} \cos\theta_p & -\sin\theta_p & 0 & x_p \\ \sin\theta_p & \cos\theta_p & 0 & y_p \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Step 2 — Evaluate the parent curve at the trim start and form the normalization matrix M_N

Compute the position (x_0, y_0) and tangent angle θ_0 of the parent curve at s_0. This establishes the frame of the parent curve at the point where trimming begins.

M_N simultaneously translates the trim-start point to the origin and rotates so that the tangent at s_0 aligns with the positive x-direction.

$$M_N = \begin{bmatrix} \cos\theta_0 & \sin\theta_0 & 0 & -x_0\cos\theta_0 - y_0\sin\theta_0 \\ -\sin\theta_0 & \cos\theta_0 & 0 & x_0\sin\theta_0 - y_0\cos\theta_0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Step 3 — Evaluate and map each point

For the point at arc-length s, compute the parent curve position $(x(s), y(s))$ and tangent angle $\theta(s)$ and form:

$$M_{PC} = \begin{bmatrix} \cos(\theta(s)) & -\sin(\theta(s)) & 0 & x(s) \\ \sin(\theta(s)) & \cos(\theta(s)) & 0 & y(s) \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Apply the normalization and placement in sequence:

$$M_h = M_{CSP} M_N M_{PC}$$

The position of the point in the alignment coordinate system is the fourth column of M_h, and its tangent direction is the first column. Numerical examples of this algorithm are given in Sections 2.3 through 2.11.

2.3 Line

A tangent run is geometrically represented with a segment trimmed from an `IfcLine` parent curve.

2.3.1 Parent Curve Parametric Equations

The curve tangent angle and curvature are given by the following equations

$$\theta(t) = \theta$$

$$\kappa(t) = 0$$

The point on line equation can be parameterized with a **unit parameterization** where the parameter u ranges from 0 to 1 over the full extent of the curve, independent of its physical length.

$$\lambda(u) = C + L\left(\int_0^{u=1} \cos(\theta(t))\, dt\ x,\ \int_0^{u=1} \sin(\theta(t))\, dt\ y\right)$$

It can also be parameterized with an **arc length parameterization** where the parameter u equals the distance along the line in units of length, so $u = s$.

$$\lambda(u) = C + \left(\int_0^{u=L} \cos(\theta(t))\, dt\ x,\ \int_0^{u=L} \sin(\theta(t))\, dt\ y\right)$$

Other parent curve types can also be parameterized by unit value or arc length. However, the polynomial spirals do not have a well defined unit value parameterization. For this reason, the IFC specification mandates that all parameterization for alignment geometry be by arc length. For `IfcCurveSegment`, the `SegmentStart` and `SegmentLength` attributes **must** be of type `IfcLengthMeasure`. See Section 8.9.3.28 IfcCurveSegment, Informal Proposition 1.

This matters because `IfcCurveSegment.SegmentStart` and `IfcCurveSegment.SegmentLength` are of type `IfcCurveMeasureSelect`, which can be either `IfcParameterValue` (a dimensionless scalar based on unit value) or `IfcLengthMeasure` (a physical length).

2.3.2 Semantic Definition to Geometry Mapping

Mapping of the semantic definition of the linear segment to the geometric definition is described with the following example.

Given a horizontal alignment segment is a line segment starting at point (500,2500), bearing in the direction S 57° E (5.70829654085293 radian) and has a length of 1956.785654.

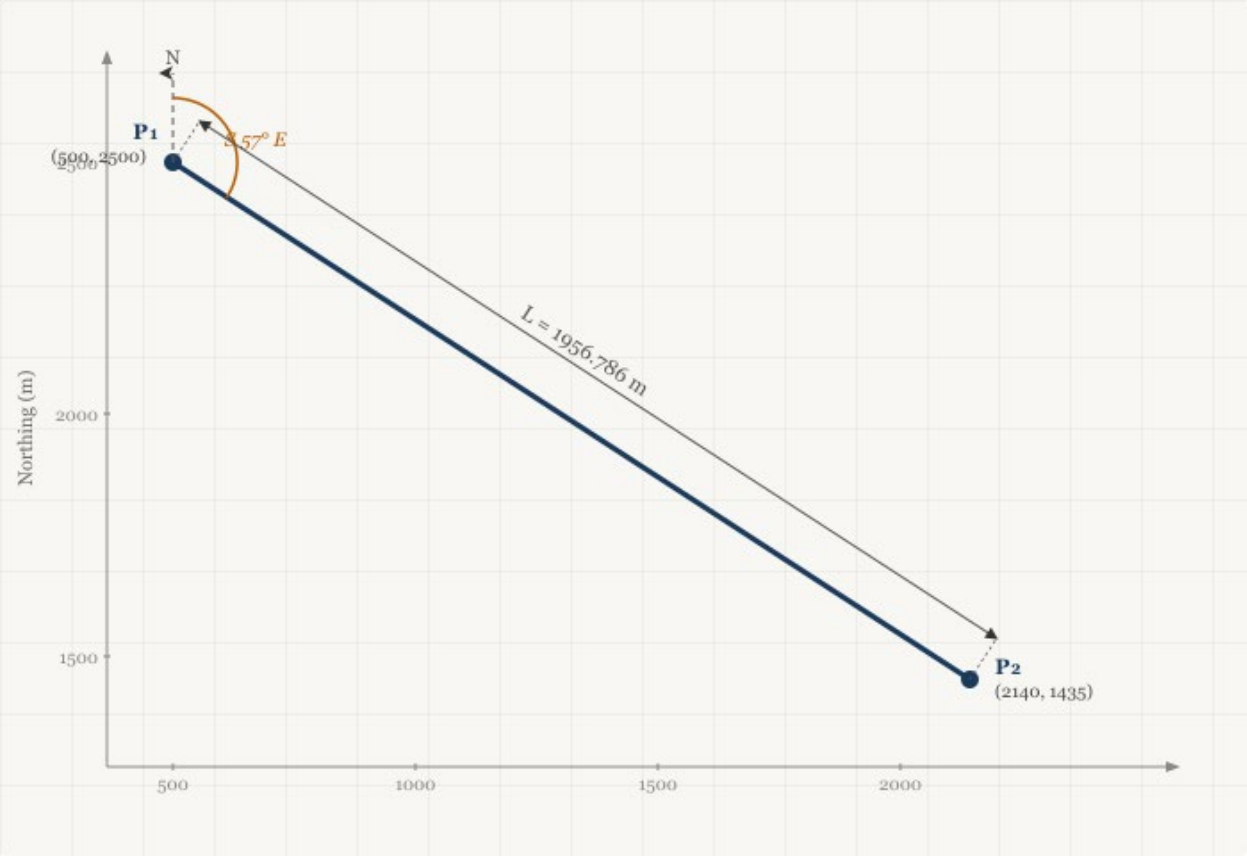


Figure 2.3.2-1 Tangent segment

The semantic definition of this segment is:

```
#31=IFCCARTESIANPOINT((500.,2500.));
#32=IFCALIGNMENTHORIZONTALSEGMENT($,$,#31,5.70829654085293,0.,0.,1956.785654,
$,.LINE.);
```

The geometric representation is an `IfcLine`. The line can be defined in different ways and is generally not important, as will be shown in the example below. `IfcLine` is an infinitely long line that passes through a specified point at some direction in the X-Y plane. A valid, but unnecessarily complex parent curve is shown in Figure 2.3.2-2. The `IfcLine` passes through point (1000,-800) and is directed at 135° from the x-axis. The `IfcCurveSegment` trim starts 100 m from (1000,-800) and the trimmed segment has a length of 1956.796 m.

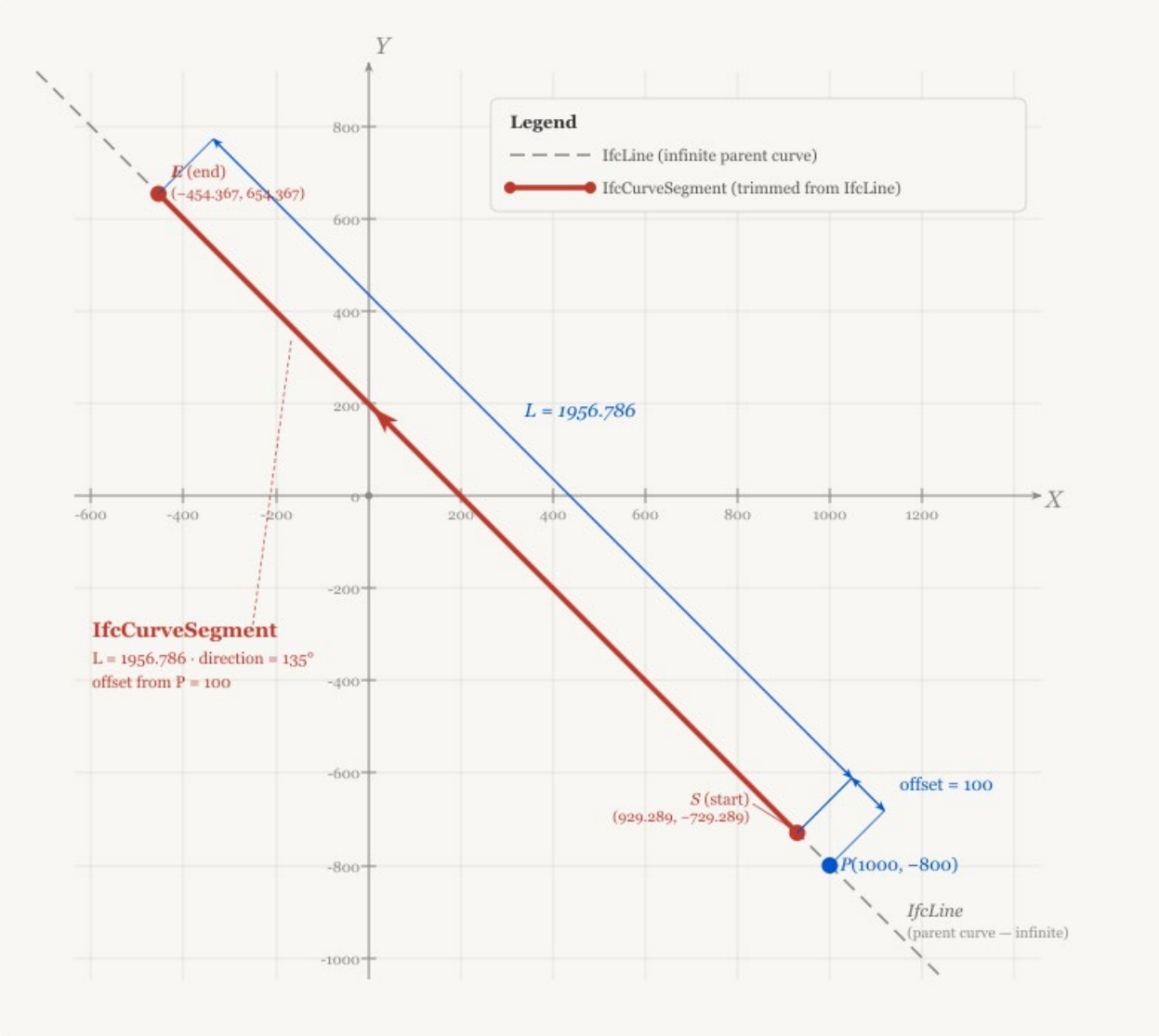


Figure 2.3.2-2 IfcLine at arbitrary placement

Figure 2.3.2-2 represents a valid parent curve definition and implementations must be prepared to properly interpret this parent curve geometry. This guide defines general algorithms to accommodate this geometry.

In practice, it is far easier to define the parent curve passing through point (0,0) and in the direction of the X-axis. The trimming can begin at the origin as shown in Figure 2.3.2-3.

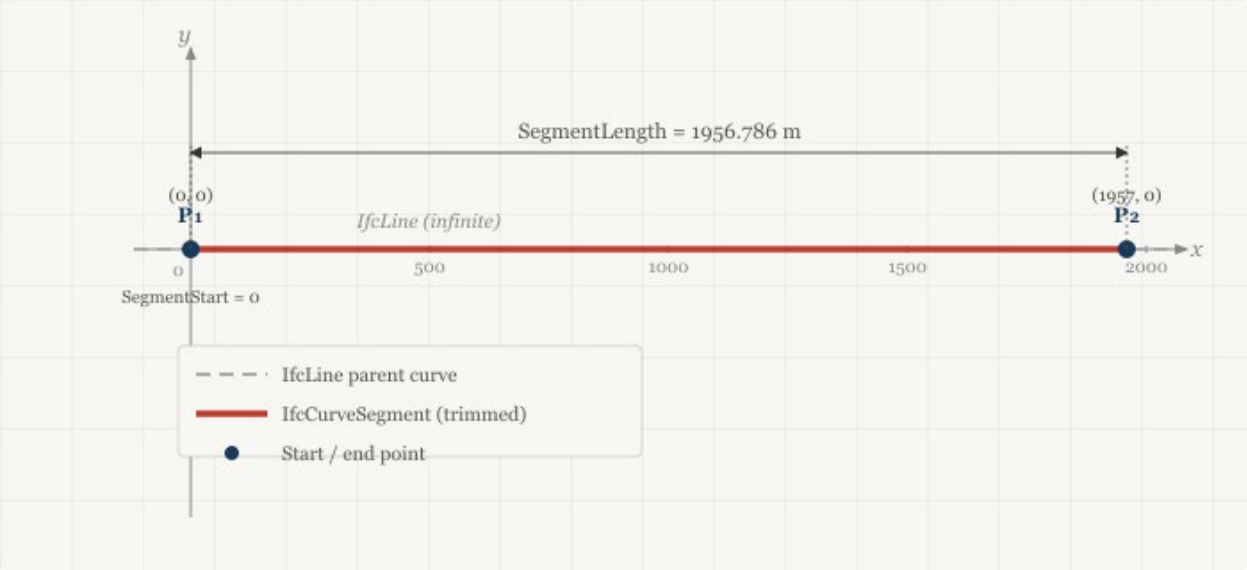


Figure 2.3.2-3 IfcLine parent curve at origin

The parent curve definition is:

```
#51=IFCCARTESIANPOINT((0.,0.));
#52=IFCDIRECTION((1.,0.));
#53=IFCVECTOR(#52,1.);
#54=IFCLINE(#51,#53);
```

`IfcCurveSegment` defines the portion of the line used and how it is placed and oriented in the X-Y plane.

`IfcCurveSegment.SegmentStart` defines where the parent curve trimming begins. It can be anywhere along the line. It is easiest to begin the trimming at the origin but could be anywhere. The `SegmentStart` attribute is 0.0 in this case.

`IfcCurveSegment.SegmentLength` defines where the parent curve trimming ends relative to `SegmentStart`. This is the length of the curve we want to trim. The `SegmentLength` attribute is 1956.785654 for this example.

The `SegmentStart` and `SegmentLength` attributes trim a portion of the `IfcLine` parent curve, which is oriented in the direction (1,0) with origin at (0,0).

The trimmed portion of the curve needs to be placed and oriented into the horizontal alignment. The tangent segment begins at (500,2500) and runs in the direction 5.70829654085293 radian.

From the segment direction,

$$dy = \sin(5.70829654085293) = -0.54374144087698$$

$$dx = \cos(5.70829654085293) = 0.839252789970355$$

The segment placement is

```
#31=IFCCARTESIANPOINT((500.,2500.));
#49=IFCDIRECTION((0.839252789970355,-0.54374144087698));
#50=IFCAXIS2PLACEMENT2D(#31,#49);
```

The `IfcCurveSegment` is defined as:

```
#55=IFCCURVESEGMENT(.CONTSAMEGRADIENT.,#50,IFCLENGTHMEASURE(0.),IFCLENGTHMEAS
URE(1956.785654),#54);
```

2.3.3 Compute Point on Curve

Compute the position matrix for a point 1500 m from the start of the curve segment.

Step 1 — Form the curve segment placement matrix M_{CSP}

From the `IfcCurveSegment.Placement`:

$$x = 500, y = 2500$$

$$dx = 0.839252789970355,\ dy = -0.54374144087698$$

$$M_{CSP} = \begin{bmatrix} 0.839252789970355 & 0.54374144087698 & 0 & 500 \\ -0.54374144087698 & 0.839252789970355 & 0 & 2500 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Step 2 — Evaluate the parent curve at the trim start and form the normalization matrix M_N

Because the parent curve is located at (0,0) in the direction (1,0), $x_0 = 0, y_0 = 0, \theta_0 = 0$.

Since $x_0 = 0$, $y_0 = 0$, and $\theta_0 = 0$, $M_N = \boldsymbol{I}$.

Step 3 — Evaluate and map each point

Evaluate the parent curve at $u = 1500$

$$\lambda(u) = C + \left(\int_0^u \cos(\theta(t))\, dt\ x,\ \int_0^u \sin(\theta(t))\, dt\ y \right)$$

$$\theta(1500) = \tan^{-1}\left(\frac{0}{1}\right) = 0$$

$$x = 0 + \cos(0) \int_0^{1500} dt = 0 + 1(1500\ m - 0\ m) = 1500\ m$$

$$y = 0 + \sin(0) \int_0^{1500} dt = 0 + 0(1500\ m - 0\ m) = 0\ m$$

Though it is much easier to use the following calculation

$$x(u) = p_x + u(dx)$$

$$y(u) = p_y + u(dy)$$

$$x = 0 + 1(1500) = 1500$$

$$y = 0 - 0(1500) = 0$$

$$M_{PC} = \begin{bmatrix} 1 & 0 & 0 & 1500 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

The resulting position matrix is

$$M_h = M_{CSP} M_N M_{PC}$$

$$M_h = \begin{bmatrix} 0.839252789970355 & 0.54374144087698 & 0 & 500 \\ -0.54374144087698 & 0.839252789970355 & 0 & 2500 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \boldsymbol{I} \begin{bmatrix} 1 & 0 & 0 & 1500 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$M_h = \begin{bmatrix} 0.83925279 & 0.54374114 & 0 & 1758.879185 \\ -0.54374114 & 0.83925279 & 0 & 1684.3878387 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

2.4 Circular Arc

Circular alignment curves are geometrically represented by an arc trimmed from an `IfcCircle` parent curve.

2.4.1 Parent Curve Parametric Equations

The curve tangent angle, curvature, and point on the curve are given by the following equations

$$\theta(s) = \frac{s}{R}$$

$$\kappa(s) = \frac{1}{R}$$

$$x(s) = \int \cos(\theta(s))ds = R\sin(\theta(s))$$

$$y(s) = \int \sin(\theta(s))ds = R\big(1 - \cos(\theta(s))\big)$$

2.4.2 Semantic Definition to Geometry Mapping

`IfcCircle` is defined by its center (`IfcCircle.Position`) and radius (`IfcCircle.Radius`).

Consider a horizontal curve segment that starts at (0,0), has a radius of 300 m, and an arc length of 100 m. The semantic definition is

```
#28 = IFCCARTESIANPOINT((0., 0.));
#29 = IFCALIGNMENTHORIZONTALSEGMENT($, $, #28, 0., 300., 300., 100., $,
.CIRCULARARC.);
```

The parent curve can be defined as follows:

```
#45 = IFCCIRCLE(#46, 300.);
#46 = IFCAXIS2PLACEMENT2D(#47, #48);
#47 = IFCCARTESIANPOINT((0., 300.));
#48 = IFCDIRECTION((0., -1.));
```

This is a circle centered at point (0,300) with the local X-axis aligned with the negative global Y-axis as shown in Figure 2.4.2-1.

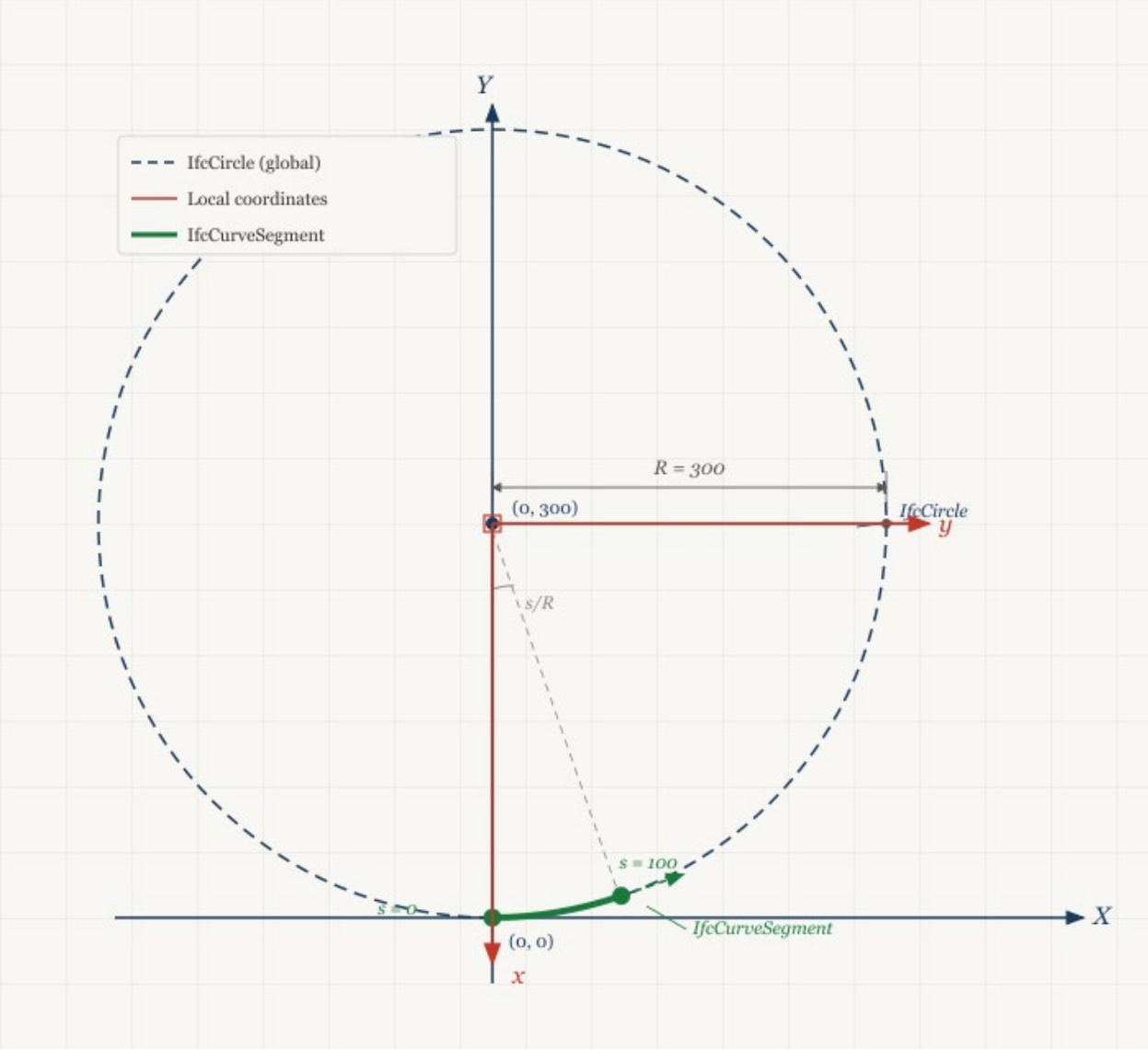


Figure 2.4.2-1 — Curve segment trimmed from an IfcCircle parent curve

The parent curve is placed such that a trim starting at 0.0 is at (0,0) and the tangent is in the direction (1,0).

The curve segment is defined by its placement at a segment trimmed from the parent curve starting at 0.0 for a length of 100.0 along the curve.

```
#36 = IFCCURVESEGMENT(.CONTINUOUS., #42,
IFCLENGTHMEASURE(0.),IFCLENGTHMEASURE(100.), #45);
#42 = IFCAXIS2PLACEMENT2D(#43, #44);
#43 = IFCCARTESIANPOINT((0., 0.));
#44 = IFCDIRECTION((1., 0.));
```

2.4.3 Compute Point on Curve

Compute the placement matrix for a point 50 m from the start of the curve segment.

Step 1 — Form the curve segment placement matrix M_{CSP}

$$M_{CSP} = \boldsymbol{I}$$

Step 2 — Evaluate the parent curve at the trim start and form the normalization matrix M_N

The trim begins where the local x-axis of the circle intersects the circumference. The circle is centered at (0,300) with radius = 300 and the local x-axis is in the direction of the global y-axis. This puts the trim start point at $x_0 = 0, y_0 = 0$ and the tangent direction (1,0) with $\theta_0 = 0$

Since $x_0 = 0, y_0 = 0$, and $\theta_0 = 0$, $M_N = \boldsymbol{I}$.

Step 3 — Evaluate and map each point

Compute point on parent curve at $u = 50$

$$\theta(s) = \frac{s}{R} = \frac{50}{300}$$

$$d_x = \cos(\theta(s)) = \cos\left(\frac{50}{300}\right) = 0.98614323$$

$$d_y = \sin(\theta(s)) = \sin\left(\frac{50}{300}\right) = 0.16589613$$

$$x(s) = R\sin(\theta(s)) = 300\sin\left(\frac{50}{300}\right) = 49.76883981$$

$$y(s) = R\big(1 - \cos(\theta(s))\big) = 300\left(1 - \cos\left(\frac{50}{300}\right)\right) = 4.1570305$$

$$M_{PC} = \begin{bmatrix} 0.98614323 & -0.16589613 & 0 & 49.76883981 \\ 0.16589613 & 0.98614323 & 0 & 4.1570305 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

The resulting position matrix is

$$M_h = M_{CSP} M_N M_{PC}$$

$$M_h = \boldsymbol{II} \begin{bmatrix} 0.98614323 & -0.16589613 & 0 & 49.76883981 \\ 0.16589613 & 0.98614323 & 0 & 4.1570305 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$M_h = \begin{bmatrix} 0.98614323 & -0.16589613 & 0 & 49.76883981 \\ 0.16589613 & 0.98614323 & 0 & 4.1570305 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

2.5 Clothoid

Spiral transition curves are geometrically represented by a segment trimmed from one of the many spiral types in IFC. `IfcClothoid` geometry is used for clothoid curves.

2.5.1 Parent Curve Parametric Equations

The curve tangent angle, curvature, and point on the curve are given by the following equations

$$\theta(s) = \frac{\pi}{2} \frac{A}{|A|} s^2$$

$$\kappa(s) = \frac{A}{|A^3|} s$$

$$x(u) = A\sqrt{\pi} \int_0^u \cos\left(\frac{\pi}{2} \frac{A}{|A|} t^2\right) \, dt$$

$$y(u) = A\sqrt{\pi} \int_0^u \sin\left(\frac{\pi}{2} \frac{A}{|A|} t^2\right) \, dt$$

IFC provides a unit parametrization of clothoid curve.

$$u = \frac{s}{\left|A\sqrt{\pi}\right|}$$

with $-\infty < u < \infty$. When $\theta = \pi/2$, $u = 1.0$.

Care must be taken when evaluating clothoids because `IfcCurveSegment.SegmentStart` and `IfcCurveSegment.SegmentLength` are defined with arc length. The details are illustrated in the example calculations.

2.5.2 Semantic Definition to Geometry Mapping

Consider a horizontal clothoid segment that starts at (0,0) with a start direction of (1.0,0.0). The radii at the start and end of the segment are 300 and 1000, respectively. The arc length of the segment is 100. The semantic definition is

```
#28 = IFCCARTESIANPOINT((0., 0.));
#29 = IFCALIGNMENTHORIZONTALSEGMENT($, $, #28, 0., 300., 1000., 100., $,
.CLOTHOID.);
```

From the semantic definition, compute the clothoid constant, A

$$R_s = 300,\ R_e = 1000,\ L = 100$$

$$f = \frac{L}{R_e} - \frac{L}{R_s} = \frac{100}{1000} - \frac{100}{300} = -0.23333$$

$$A = \frac{L}{|f|^{\frac{1}{2}}}\frac{f}{|f|} = \frac{100}{|-0.23333|^{\frac{1}{2}}}\frac{-0.23333}{|-0.23333|} = -207.0196678$$

Place the parent curve at (0,0) with a tangent direction of (1,0)

```
#45 = IFCCLOTHOID(#46, -207.019667802706);
#46 = IFCAXIS2PLACEMENT2D(#47, $);
#47 = IFCCARTESIANPOINT((0., 0.));
```

The curve segment starts where the radius is 300. A positive radius indicates a counter-clockwise curve which is a curve towards the left. The distance from the origin where this radius occurs can be computed from the curvature equation. The curvature at the start is 1/300 and the clothoid constant is -207.0196678. Solve for distance from origin.

$$\kappa(s) = \frac{A}{|A^3|}s$$

$$\frac{1}{300} = \frac{-207.0196678}{|-207.0196678^3|}s$$

$$s = -142.8571429$$

Figure 2.5.2-1 shows the parent curve clothoid with the segment having $\kappa_s = \frac{1}{300}$ and $\kappa_e = \frac{1}{1000}$ highlighted. This is the segment that `IfcCurveSegment` must trim from the parent curve. The negative clothoid constant results in the parent curve in quadrants 2 and 4. The curvature is positive in quadrant 2. For this reason, the `SegmentStart` attribute is negative. From this starting point, the trimming progresses to a larger radius, which is towards the origin. For this reason `SegmentLength` is a positive value. If the trimming were to progress to a smaller radius, `SegmentLength` would be a negative value. The `SegmentStart` and `SegmentLength` attributes can be positive and negative values in any combination. This is true for all parent curve types.

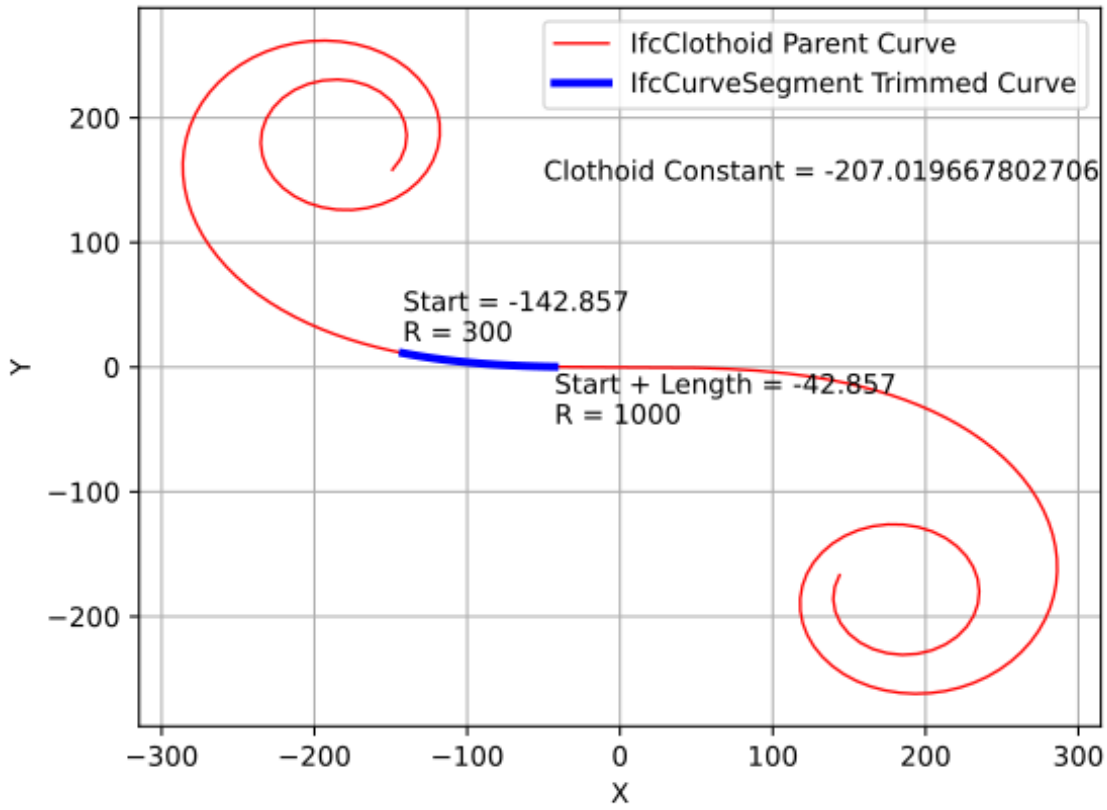


Figure 2.5.2-1 — `IfcCurveSegment` trimmed from an `IfcClothoid` parent curve

Place the curve segment at (0,0) with the tangent in the direction (1,0).

```
#42 = IFCAXIS2PLACEMENT2D(#43, #44);
#43 = IFCCARTESIANPOINT((0., 0.));
#44 = IFCDIRECTION((1., 0.));
```

Define the curve segment

```
#36 = IFCCURVESEGMENT(.CONTINUOUS., #42, IFCLENGTHMEASURE(-142.857142857143),
IFCLENGTHMEASURE(100.), #45);
```

2.5.3 Compute Point on Curve

Compute the placement matrix for a point 50 m from the start of the curve segment.

Step 1 — Form the curve segment placement matrix M_{CSP}

Represent `IfcCurveSegment.Placement` in matrix form. In this example, the placement is at (0,0) with RefDirection (1,0) which results in an identity matrix. This is not true in all cases.

$$M_{CSP} = \boldsymbol{I}$$

Step 2 — Evaluate the parent curve at the trim start and form the normalization matrix M_N

Start by computing the point and curve tangent at the start of the parent curve trim.

Recall that the clothoid equations are in terms of a unit parameterization. Compute the parametric position on the curve.

$$u = \frac{-142.8571428}{\left|-207.0196678\sqrt{\pi}\right|} = -0.3893278$$

Compute the point on curve tangent direction

$$\theta_0(-0.3893278) = \frac{\pi}{2}\frac{-207.0196678}{|-207.0196678|}(-0.3893278)^2 = -0.238095237$$

and compute the point on the curve

$$x_0(-0.3893278) = -207.0196678\sqrt{\pi}\int_0^{-0.3893278}\cos\left(\frac{\pi}{2}\frac{-207.0196678}{|-207.0196678|}t^2\right)\,dt$$
$$= -142.04941746210602\ m$$

$$y_0(-0.3893278) = -207.0196678\sqrt{\pi}\int_0^{-0.3893278}\sin\left(\frac{\pi}{2}\frac{-207.0196678}{|-207.0196678|}t^2\right)\,dt$$
$$= 11.292042785713347\ m$$

The terms of the normalization matrix are:

$$\cos(-0.238095237) = 0.971788979$$

$$\sin(-0.238095237) = -0.235852028$$

$$-(-142.04941746210602\ m)\cos(-0.238095237)$$
$$-\ (11.292042785713347\ m)\sin(-0.238095237) = 140.705309648\ m$$

$$(-142.04941746210602\ m)\sin(-0.238095237)$$
$$-\ (11.292042785713347\ m)\cos(-0.238095237) = 22.529160405\ m$$

$$M_N = \begin{bmatrix} \cos\theta_0 & \sin\theta_0 & 0 & -x_0\cos\theta_0 - y_0\sin\theta_0 \\ -\sin\theta_0 & \cos\theta_0 & 0 & x_0\sin\theta_0 - y_0\cos\theta_0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$
$$= \begin{bmatrix} 0.971788979 & -0.235852028 & 0 & 140.705309648 \\ 0.235852028 & 0.971788979 & 0 & 22.529160405 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Step 3 — Evaluate and map each point

Compute point and curve tangent at 50 m from the start,

$$s = -142.8571428 + 50 = -92.8571428$$

$$u = \frac{-92.8571428}{\left|-207.0196678\sqrt{\pi}\right|} = -0.25306307$$

$$x(-0.25306307) = -207.0196678\sqrt{\pi}\int_0^{-0.25306307} \cos\left(\frac{\pi}{2}\frac{-207.0196678}{|-207.0196678|}s^2\right)\,ds$$
$$= -92.763220844871512$$

$$y(-0.25306307) = -207.0196678\sqrt{\pi}\int_0^{-0.25306307} \sin\left(\frac{\pi}{2}\frac{-207.0196678}{|-207.0196678|}s^2\right)\,ds$$
$$= 3.1114126254443812$$

$$\theta(-0.25306307) = \frac{\pi}{2}\frac{-207.0196678}{|-207.0196678|}(-0.25306307)^2 = -0.100595238$$

$$dx = \cos(-0.100595238) = 0.994944564$$

$$dy = \sin(-0.100595238) = -0.100425663$$

In matrix form

$$M_{PC} = \begin{bmatrix} 0.994944564 & 0.100425663 & 0 & -92.763220844871512 \\ -0.100425663 & 0.994944564 & 0 & 3.1114126254443812 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Apply the normalization and curve segment placement to the parent curve point

$$M_h = M_{CSP}M_N M_{PC}$$

$$= \boldsymbol{I}\begin{bmatrix} 0.971788979 & -0.235852028 & 0 & 140.705309648 \\ 0.235852028 & 0.971788979 & 0 & 22.529160405 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}\begin{bmatrix} 0.994944564 & 0.100425663 & 0 & -92.763 \\ -0.100425663 & 0.994944564 & 0 & 3.1114 \\ 0 & 0 & 1 & \\ 0 & 0 & 0 & \end{bmatrix}$$

$$M_h = \begin{bmatrix} 0.99056175921247780 & -0.13706714116038599 & 0 & 49.825200930152405 \\ 0.13706714116038599 & 0.99056175921247780 & 0 & 3.6744032279728032 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

2.6 Cubic Transition Curve

`IfcPolynomialCurve` is the geometric type for cubic transition spirals.

2.6.1 Parent Curve Parametric Equations

The approach for calculating the coordinates of a horizontal cubic curve is unique compared to the other curve types. For a horizontal alignment, the coordinate point and curve tangent angle is generally defined by parametric equations where the parameter is the distance along the curve. A cubic curve is represented with the `IfcPolynomialCurve` parent curve type. When `IfcPolynomialCurve` is used as a cubic transition curve, the x

coefficients are $[0,1]$ and the y coefficients are $[0,0,0,A_3]$ resulting in a function of the form $y(x) = A_3x^3$. See 4.2.2.1.5 Cubic Transition Segment.

The distance along a curve is defined by

$$d = \int \sqrt{(y'(x))^2 + 1}\, dx$$

$$y'(x) = 3A_3x^2$$

$$d = \int \sqrt{9A_3^2x^4 + 1}\, dx$$

This equation is solved for x and then y can be computed. This can be accomplished with numerical methods.

1. For a distance u along the curve, find x for $d - u = 0$, within a tolerance consistent with the modeled elements. See Chapter 11 for a discussion on tolerances.
2. Compute $y(x) = A_3x^3$
3. Compute curve tangent slope as $\frac{dy}{dx} = 3A_3x^2$

2.6.2 Semantic Definition to Geometry Mapping

Consider a horizontal cubic transition curve segment that starts at (0,0) with a start direction of 0.0. The radius at the start is infinite and the radius at the end is 300. The arc length of the segment is 100. The semantic definition is

```
#28 = IFCCARTESIANPOINT((0., 0.));
#29 = IFCALIGNMENTHORIZONTALSEGMENT($, $, #28, 0., 0., 300., 100.,
$,.CUBIC.);
```

Compute the polynomial curve coefficients

$$A_0 = A_1 = A_2 = 0$$

$$A_3 = \frac{1}{6R_eL} - \frac{1}{6R_sL} = \frac{1}{6(300\ m)(100\ m)} - \frac{1}{6(\infty)(100\ m)} = 5.55555 \cdot 10^{-6}\ m^{-2}$$

The geometric representation is

```
#36 = IFCCURVESEGMENT(.CONTINUOUS., #42, IFCLENGTHMEASURE(0.),
IFCLENGTHMEASURE(100.), #45);
#42 = IFCAXIS2PLACEMENT2D(#43, #44);
#43 = IFCCARTESIANPOINT((0., 0.));
#44 = IFCDIRECTION((1., 0.));
#45 = IFCPOLYNOMIALCURVE(#46, (0., 1.), (0., 0., 0., 5.55555555555556E-6),
$);
#46 = IFCAXIS2PLACEMENT2D(#47, $);
#47 = IFCCARTESIANPOINT((0., 0.));
```

Warning: There is a flaw in the IFC Specification. IfcAlignmentHorizontalSegment indicates the cubic formula is $y = \frac{x^3}{6RL}$ which means the `IfcPolynomialCurve.CoefficentY[3]` attribute must have unit of $Length^{-2}$. This is a direct contradiction to IfcPolynomialCurve which clearly states the coefficients are real values (i.e. scalar values) with `IfcReal`.

Why is this a problem? The calculation of a point on the polynomial curve is different depending on how the coefficients are defined.

The parametric form of the polynomial curve is $\lambda(u) = (x(u), y(u), z(u))$. The polynomial curve represents real geometry so the resulting values, x, y, and z must have units of $Length$.

When u is parametrized as a Length measure, as required by IfcCurveSegment, the parametric terms of the polynomial equation are:

$$x(u) = \sum_{i=0}^{l} a_i \, u^i$$

$$y(u) = \sum_{j=0}^{m} b_j \, u^j$$

$$z(u) = \sum_{k=0}^{n} c_k \, u^k$$

For x, y, and z to have values in $Length$ measure, the implicit unit of measure of the coefficients must be:

$Length^{(1-i)}$ for a_i $Length^{(1-j)}$ for b_j $Length^{(1-k)}$ for c_k

When u is parametrized as a scalar, as required by IfcPolynomialCurve, the parametric terms of the polynomial equation are:

$$x(u) = L \sum_{i=0}^{l} a_i \, u^i$$

$$y(u) = L \sum_{j=0}^{m} b_j \, u^j$$

$$z(u) = L \sum_{k=0}^{n} c_k \, u^k$$

The parameter u and the coefficients are scalar so they must be multiplied with the curve length L to result in values of Length.

The equations for x, y, and z differ depending on the parameterization.

Concept Template 4.2.2.1.5 Cubic Transition Segment and IfcAlignmentHorizontalSegment provide clear direction (not withstanding typographical errors) that `IfcPolynomialCurve` is to be evaluated as $y = CoefficientsY[3]x^3$ which dictates that $CoefficientsY[3]$ must be in units of $Length^{-2}$ and must be encoded in the `IfcPolynomialCurve.CoefficientY` attribute as an `IfcReal`.

An official Implementation Agreement does not appear to exist; however, this interpretation is consistent with discussions on GitHub [4].

2.6.3 Compute Point on Curve

Compute the curve coordinates at a distance along the curve, $u = 100$

Step 1 — Form the curve segment placement matrix M_{CSP}

Represent `IfcCurveSegment.Placement` in matrix form. In this example, the placement is at (0,0) with RefDirection (1,0) which results in an identity matrix.

$$M_{CSP} = \boldsymbol{I}$$

Step 2 — Evaluate the parent curve at the trim start and form the normalization matrix M_N

Because the parent curve is located at (0,0) in the direction (1,0), $x_0 = 0, y_0 = 0, \theta_0 = 0$.

Since $x_0 = 0$, $y_0 = 0$, and $\theta_0 = 0$, $M_N = \boldsymbol{I}$.

Step 3 — Evaluate and map each point

Compute point and curve tangent at 100 m from the start.

From the `IfcPolynomialCurve` definition,

X Coefficients = $(0m^1, 1m^0)$

Y Coefficients = $(0m^1, 0m^0, 0m^{-1}, 5.55555 \cdot 10^{-6}m^{-2})$

$$y(x) = (5.55555 \cdot 10^{-6}m^{-2})x^3$$

Find x such that

$$|d - u| = \left| \int_0^x \left(\sqrt{(y'(x))^2 + 1} \right) dx - u \right| < 10^{-4} \approx 0$$

$$y'(x) = 3(5.55555 \cdot 10^{-6}m^{-2})x^2$$

Solve numerically

$$x = 99.72593255m$$

Check solution

$$d = \int_0^{99.72593255\ m} \sqrt{(3 \cdot (5.55555 \cdot 10^{-6} m^{-2})(x\ m)^2)^2 + 1}\, dx = 100\ m$$

Compute y

$$y(x) = (5.55555 \cdot 10^{-6} m^{-2})(99.72593255\ m)^3 = 5.5100\ m$$

The tangent vector at $u = 100\ m$ along the curve is

$$y'(99.72593255\ m) = 3(5.55555 \cdot 10^{-6} m^{-2})(99.72593255\ m)^2 = 0.165753$$

Normalizing the tangent vector

$$dx = \frac{1}{\sqrt{1^2 + 0.165753^2}} = 0.986539$$

$$dy = \frac{0.165753}{\sqrt{1^2 + 0.165753^2}} = 0.1635219$$

The resulting matrix is

$$M_{PC} = \begin{bmatrix} 0.986539 & -0.1635219 & 0 & 99.72593255 \\ 0.1635219 & 0.986539 & 0 & 5.5100 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Apply the normalization and curve segment placement to the parent curve point

$$M_h = M_{CSP}\ M_N\ M_{PC} = \boldsymbol{I}\ \boldsymbol{I} \begin{bmatrix} 0.986539 & -0.1635219 & 0 & 99.72593255 \\ 0.1635219 & 0.986539 & 0 & 5.5100 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$M_h = \begin{bmatrix} 0.986539 & -0.1635219 & 0 & 99.72593255 \\ 0.1635219 & 0.986539 & 0 & 5.5100 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

2.7 Helmert Transition Curve

The Helmert curve is unique in that it is semantically defined as a single layout segment (`IfcAlignmentHorizontalSegment`) but geometrically defined with two curve segments (`IfcCurveSegment`), each representing half of the transition curve. The parent curve for a helmert transition curve is `IfcSecondOrderPolynomialSpiral`.

2.7.1 Parent Curve Parametric Equations

The curve tangent angle and curvature are given by the following equations

$$\theta(t) = \frac{t^3}{3A_2^3} + \frac{A_1}{2|A_1^3|}t^2 + \frac{t}{A_0}$$

$$\kappa(t) = \frac{1}{A_2^3}t^2 + \frac{A_1}{|A_1^3|}t + \frac{1}{A_0}$$

The polynomial coefficients carry a second subscript to indicate first half, 1, and second half, 2. For example, A_{21} is coefficient A_2 for the first half and A_{02} is coefficient A_0 for the second half.

First Half

$$a_{01} = \frac{L}{R_s},\ A_{01} = \frac{L}{|a_0|}\frac{a_0}{|a_0|}$$

$$a_{11} = 0,\ A_{11} = 0$$

$$a_{21} = 2f,\ A_{21} = \frac{L}{|a_2|^{\frac{1}{3}}}\frac{a_2}{|a_2|}$$

Second Half

$$a_{02} = -f + \frac{L}{R_s},\ A_{02} = \frac{L}{|a_0|}\frac{a_0}{|a_0|}$$

$$a_{12} = 4f,\ A_{12} = \frac{L}{|a_1|^{\frac{1}{2}}}\frac{a_1}{|a_1|}$$

$$a_{22} = -2f,\ A_{22} = \frac{L}{|a_2|^{\frac{1}{3}}}\frac{a_2}{|a_2|}$$

2.7.2 Semantic Definition to Geometry Mapping

Consider a horizontal Helmert transition curve segment that starts at (0,0) with a start direction of (1,0). The radius at the start is infinite and the radius at the end is 300. The arc length of the segment is 100. The semantic definition is

```
#28 = IFCCARTESIANPOINT((0., 0.));
#29 = IFCALIGNMENTHORIZONTALSEGMENT($, $, #28, 0., 0., 300., 100., $,
.HELMERTCURVE.);
```

Compute the curve parameters

$$L = 100\ m$$

$$R_s = \infty$$

$$R_e = 300\ m$$

$$f = \frac{L}{R_e} - \frac{L}{R_s} = \frac{100}{300} - \frac{100}{\infty} = 0.33333$$

First Half

$$a_{01} = \frac{100}{\infty} = 0,\ A_0 = 0$$

$$a_{11} = 0,\ A_{11} = 0$$

$$a_{21} = 2(0.33333) = 0.66667,\ A_2 = \frac{100\ m}{|0.66667|^{\frac{1}{3}}}\frac{0.66667}{|0.66667|} = 114.4714255\ m$$

The geometric representation of the first half is

```
#36 = IFCCURVESEGMENT(.CONTSAMEGRADIENTSAMECURVATURE., #42,
IFCLENGTHMEASURE(0.), IFCLENGTHMEASURE(50.), #45);
#37 = IFCLOCALPLACEMENT($, #38);
#38 = IFCAXIS2PLACEMENT3D(#39, $, $);
#39 = IFCCARTESIANPOINT((0., 0., 0.));
#42 = IFCAXIS2PLACEMENT2D(#43, #44);
#43 = IFCCARTESIANPOINT((0., 0.));
#44 = IFCDIRECTION((1., 0.));
#45 = IFCSECONDORDERPOLYNOMIALSPIRAL(#46, 114.471424255333, $, $);
#46 = IFCAXIS2PLACEMENT2D(#47, $);
#47 = IFCCARTESIANPOINT((0., 0.));
```

> **Note:** when a polynomial coefficient is zero, the correspond term is unused and coded as `$` in the IFC entity

Second Half

$$a_{02} = -0.33333 + \frac{100}{\infty} = -0.33333,\ A_0 = \frac{100\ m}{|-0.33333|}\frac{-0.33333}{|-0.33333|} = -300\ m$$

$$a_{12} = 4(0.33333) = 1.33333,\ A_1 = \frac{100\ m}{|1.33333|^{\frac{1}{2}}}\frac{1.33333}{|1.33333|} = 86.602540378\ m$$

$$a_{22} = -2(0.33333) = -0.66667,\ A_2 = \frac{100\ m}{|-0.66667|^{\frac{1}{3}}}\frac{-0.66667}{|-0.66667|} = -114.4714255\ m$$

Figure 2.7.2-1 shows the first and second half parent curves, without any adjustments.

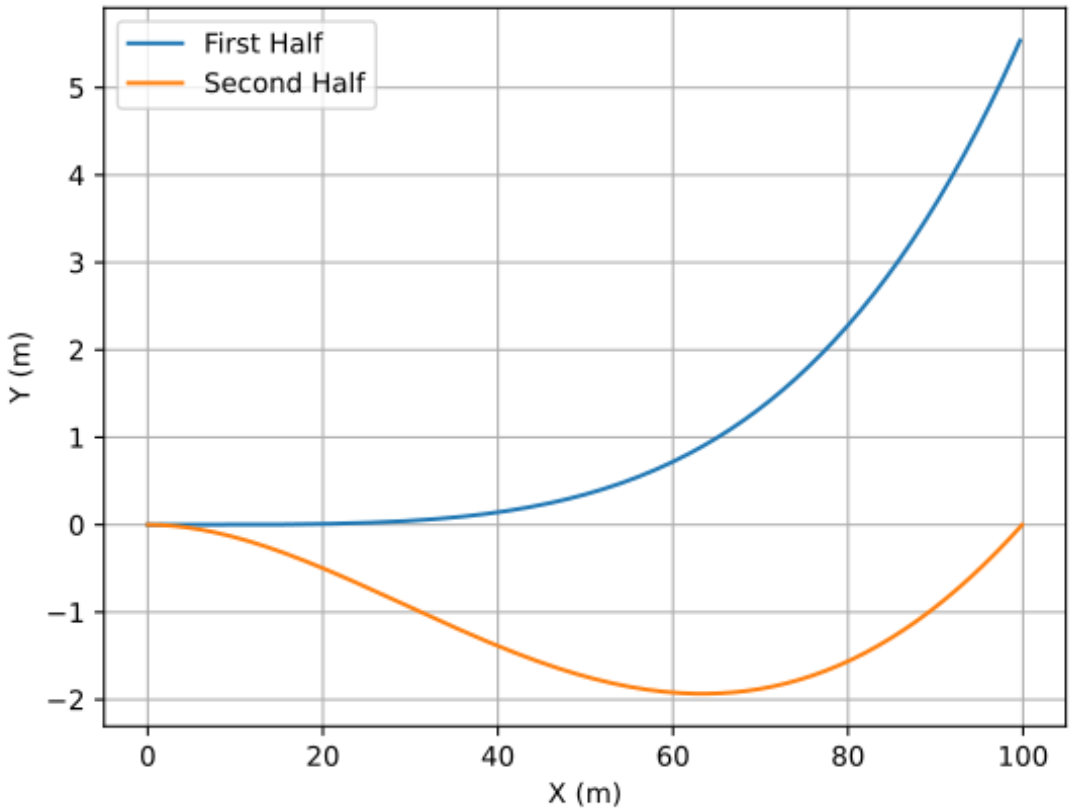


Figure 2.7.2-1 First and second half of parent curves, without placement adjustments

Two-level placement for the second half

`IfcCurveSegment` evaluation normalizes the parent curve at `SegmentStart`, then applies the curve segment `Placement`. For the second half, `SegmentStart = L/2`, so the second half parent curve's value at $t = L/2$ is the normalization origin. The second half parent curve's own `Position` attribute is therefore set so that the point at $L/2$ exactly matches the endpoint of the first half.

With this alignment the normalization cancels the offset, and the curve segment `Placement` supplies only the world-space join position.

Level	Attribute	Purpose
`IfcSecondOrderPolynomialSpiral.Position`	$(x_p,\ y_p,\ \theta_p)$	Shift/rotate the raw spiral so its value at $t = L/2$ equals (x_1, y_1, θ_1)
`IfcCurveSegment.Placement`	World-space join point	Place the normalized second-half curve at the correct position in the composite curve

Table 2.7.2-1 — Two-level placement for the second Helmert half

Finding the parent curve Position

Evaluate the raw second half spiral (with `Position` at origin) at $t = L/2$ to obtain (x_2, y_2, θ_2). The rigid-body transform that maps this to (x_1, y_1, θ_1) is

$$\theta_p = \theta_1 - \theta_2 = 0.0277777777 - (-0.0277777777) = 0.05555555$$

$$x_p = x_1 - x_2\cos\theta_p + y_2\sin\theta_p$$
$$= 49.997244 - 49.966385\cos(0.0555556) + (-1.735658)\sin(0.0555556)$$
$$= 0.011573\ m$$

$$y_p = y_1 - x_2\sin\theta_p - y_2\cos\theta_p$$
$$= 0.347204 - 49.966385\sin(0.0555556) - (-1.735658)\cos(0.0555556)$$
$$= -0.694298\ m$$

$$dx_p = \cos(\theta_p) = \cos(0.055555555) = 0.998457$$

$$dy_p = \sin(\theta_p) = \sin(0.055555555) = 0.055527$$

Figure 2.7.2-2 shows the same first and second half parent curves, with the origin of the second half curve translated (x_p, y_p) and rotated so that the curve tangent is (dx_p, dy_p)

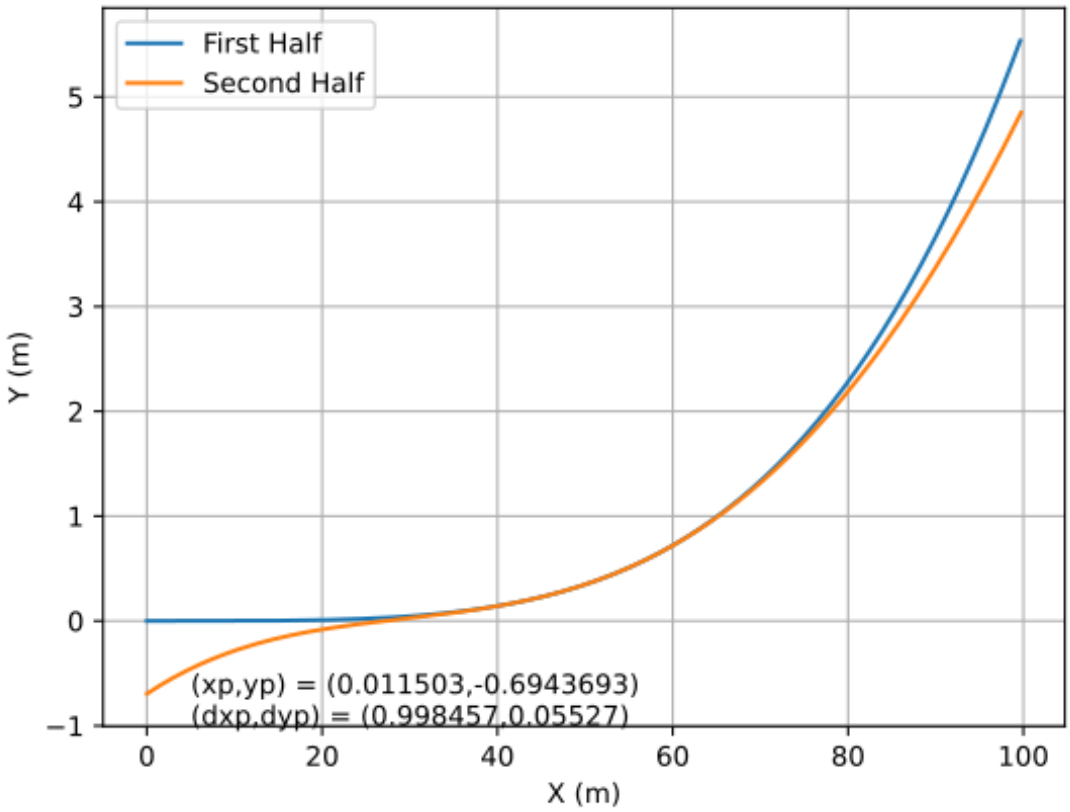


Figure 2.7.2-2 — First and second half parent curves with second half curve positioned

Finding the curve segment Placement

The `Placement` is the world-space position and direction at the join point, obtained by rotating (x_1, y_1) by the overall segment start direction θ_s and translating by the start point (x_s, y_s):

$$x_{join} = x_s + x_1 \cos\theta_s - y_1 \sin\theta_s$$

$$y_{join} = y_s + x_1 \sin\theta_s + y_1 \cos\theta_s$$

$$\theta_{join} = \theta_s + \theta_1$$

For the example with $\theta_s = 0$:

$$x_{join} = 0.0 + 49.99724\cos(0.0) - 0.347204\sin(0.0) = 49.99724 \text{ m}$$

$$y_{join} = 0.0 + 49.99724\sin(0.0) + 0.347204\cos(0.0) = 0.347204 \text{ m}$$

$$\theta_{join} = 0.0 + 0.02777777 = 0.02777777 \text{ rad}$$

The `RefDirection` of the curve segment `Placement` is $(\cos\theta_{join}, \sin\theta_{join})$:

$$\cos\theta_{join} = \cos(0.02777777) = 0.999614$$

$$\sin\theta_{join} = \sin(0.02777777) = 0.027774$$

The geometric representation of the second half

```
#48 = IFCCURVESEGMENT(.CONTINUOUS., #49, IFCLENGTHMEASURE(50.),
IFCLENGTHMEASURE(50.), #52);
#49 = IFCAXIS2PLACEMENT2D(#50, #51);
#50 = IFCCARTESIANPOINT((49.9972443634885, 0.347204361427475));
#51 = IFCDIRECTION((0.999614222337484, 0.0277742056705088));
#52 = IFCSECONDORDERPOLYNOMIALSPIRAL(#53, -114.471424255333,
86.6025403784439, -300.);
#53 = IFCAXIS2PLACEMENT2D(#54, #55);
#54 = IFCCARTESIANPOINT((0.0115725277555399, -0.694297741112199));
#55 = IFCDIRECTION((0.998457186998745, 0.0555269820047339));
```

Figure 2.7.2-3 shows the trimmed and positioned first and second half parent curves resulting in the full Helmert transition curve.

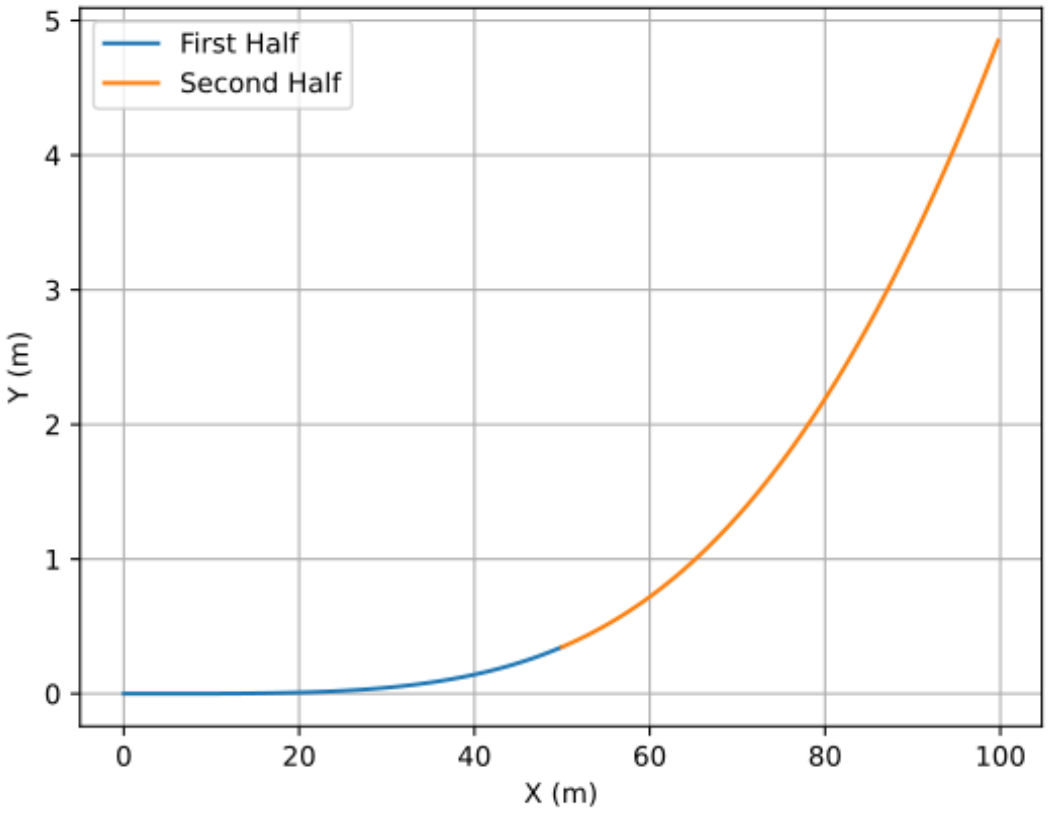


Figure 2.7.2-3 — Final Helmert transition curve

2.7.3 Compute Point on Curve

Compute the curve coordinates at a distance along the curve, $u = 100$ (the end of the full Helmert segment).

The point falls in the second half ($50 < u \leq 100$). The parent curve parameter is $t =$ SegmentStart $+ (u - 50) = 50 + 50 = 100$.

Step 1 — Form the curve segment placement matrix M_{CSP}

The curve segment `Placement` for the second half is at $(x_{join}, y_{join}) = (49.9972,\ 0.347204)$ with direction $\theta_{join} = 0.027778$ rad:

$$M_{CSP} = \begin{bmatrix} 0.999614 & -0.027774 & 0 & 49.9972 \\ 0.027774 & 0.999614 & 0 & 0.347204 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Step 2 — Evaluate the second half parent curve at SegmentStart and form the normalization matrix M_N

The second half parent curve has `Position` at $(x_p, y_p) = (0.011573,\ -0.694298)$ with direction $\theta_p = 0.055556$ rad. This was chosen so that the parent curve at $t = L/2 = 50$ yields $(x_1, y_1, \theta_1) = (49.9972,\ 0.347204,\ 0.027778)$ — the endpoint of the first half.

M_N maps (x_1, y_1, θ_1) to the origin with the x-axis tangent direction:

$$M_N = \begin{bmatrix} \cos\theta_1 & \sin\theta_1 & 0 & -x_1\cos\theta_1 - y_1\sin\theta_1 \\ -\sin\theta_1 & \cos\theta_1 & 0 & x_1\sin\theta_1 - y_1\cos\theta_1 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$M_N = \begin{bmatrix} 0.999614 & 0.027774 & 0 & -49.987600 \\ -0.027774 & 0.999614 & 0 & 1.041563 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Step 3 — Evaluate and map each point

Evaluate the second half parent curve at $t = 100$. The raw spiral (before applying `Position`) uses $(A_{02},\ A_{12},\ A_{22}) = (-300,\ 86.6025,\ -114.4714)$:

$$\theta_{raw}(t) = \frac{t^3}{3(-114.4714)^3} + \frac{86.6025}{2|86.6025^3|}t^2 + \frac{t}{-300}$$

$$\theta_{raw}(100) = -0.22222 + 0.66666 - 0.33333 = 0.11111 \text{ rad}$$

$$x_{raw} = \int_0^{100} \cos\theta_{raw}(t)\, dt = 99.8942275 \text{ m}$$

$$y_{raw} = \int_0^{100} \sin\theta_{raw}(t)\,dt = -0.00146897 \text{ m}$$

Note: These integrals have no closed form and must be evaluated numerically. The values above are converged to better than 10^{-12} m. A production implementation's integrator may be less precise. For example, IfcOpenShell evaluates $(x_{raw}, y_{raw}) = (99.8942272,\ -0.0014677)$, about 1.3×10^{-6} m from the converged values. Expect agreement between implementations, and with the test set in Chapter 12, at the 10^{-6} m level for a 100 m Helmert curve, not to the last printed digit.

Apply the parent curve `Position` (x_p, y_p, θ_p):

$$\begin{aligned} x_{pos} &= x_p + x_{raw}\cos\theta_p - y_{raw}\sin\theta_p \\ &= 0.011573 + 99.8942275(0.998457) - (-0.00146897)(0.055527) \\ &= 99.751763 \text{ m} \end{aligned}$$

$$\begin{aligned} y_{pos} &= y_p + x_{raw}\sin\theta_p + y_{raw}\cos\theta_p \\ &= -0.694298 + 99.8942275(0.055527) + (-0.00146897)(0.998457) \\ &= 4.851061 \text{ m} \end{aligned}$$

$$\theta_{pos}(100) = \theta_p + \theta_{raw}(100) = 0.055556 + 0.11111 = 0.16\overline{6} \text{ rad}$$

Note: $\theta_{pos}(100) = \frac{1}{6} = \frac{L}{2R_e} = \frac{100}{2\times300}$, confirming the total tangent angle change matches a Clothoid with the same endpoints.

Form M_{PC} from the parent curve point at $t = 100$:

$$M_{PC} = \begin{bmatrix} \cos\theta_{pos} & -\sin\theta_{pos} & 0 & x_{pos} \\ \sin\theta_{pos} & \cos\theta_{pos} & 0 & y_{pos} \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} 0.986143 & -0.165896 & 0 & 99.751763 \\ 0.165896 & 0.986143 & 0 & 4.851061 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Apply normalization and curve segment placement:

$$M_h = M_{CSP}\, M_N\, M_{PC}$$

$$M_h$$

$$= \begin{bmatrix} 0.999614 & -0.027774 & 0 & 49.9972 \\ 0.027774 & 0.999614 & 0 & 0.347204 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 0.999614 & 0.027774 & 0 & -49.987600 \\ -0.027774 & 0.999614 & 0 & 1.041563 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 0.986143 & -0.1 \\ 0.165896 & 0.98 \\ 0 & \\ 0 & \end{bmatrix}$$

$$M_h = \begin{bmatrix} 0.98614323 & -0.16589613 & 0. & 99.75176345 \\ 0.16589613 & 0.98614323 & 0. & 4.85106052 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

2.8 Bloss Transition Curve

`IfcThirdOrderPolynomialSpiral` is the geometric type for Bloss transition spirals.

2.8.1 Parent Curve Parametric Equations

The curve tangent angle and curvature are given by the following equations

$$\theta(t) = \frac{A_3}{4|A_3^5|}t^4 + \frac{1}{3A_2^3}t^3 + \frac{A_1}{2|A_1^3|}t^2 + \frac{t}{A_0}$$

$$\kappa(t) = \frac{A_3}{|A_3^5|}t^3 + \frac{t^2}{A_2^3} + \frac{A_1}{2|A_1^3|}t + \frac{1}{A_0}$$

2.8.2 Semantic Definition to Geometry Mapping

Consider a horizontal Bloss transition curve segment that starts at (0,0) with a start direction of 0.0. The radius at the start is infinite and the radius at the end is 300. The arc length of the segment is 100. The semantic definition is

```
#28 = IFCCARTESIANPOINT((0., 0.));
#29 = IFCALIGNMENTHORIZONTALSEGMENT($, $, #28, 0., 0., 300., 100., $,
.BLOSSCURVE.);
```

Compute the polynomial spiral parameters

$$L = 100\ m$$

$$R_s = \infty$$

$$R_e = 300\ m$$

$$f = \frac{L}{R_e} - \frac{L}{R_s} = \frac{100}{300} - \frac{100}{\infty} = 0.33333$$

$$a_0 = \frac{L}{R_s} = \frac{100}{\infty} = 0$$

$$A_0 = \frac{L}{|a_0|}\frac{a_0}{|a_0|} = 0$$

$$A_1 = 0$$

$$a_2 = 3f = 3(0.33333) = 1$$

$$A_2 = \frac{L}{|a_2|^{\frac{1}{3}}}\frac{a_2}{|a_2|} = \frac{100}{|1|^{\frac{1}{3}}}\frac{1}{|1|} = 100\ m$$

$$a_3 = -2f = -2(0.33333) = -0.66667$$

$$A_3 = \frac{L}{|a_3|^{\frac{1}{4}}}\frac{a_3}{|a_3|} = \frac{100\ m}{|-0.66667|^{\frac{1}{4}}}\frac{-0.66667}{|-0.66667|} = -110.668192\ m$$

```
#36 = IFCCURVESEGMENT(.CONTINUOUS., #42, IFCLENGTHMEASURE(0.),
IFCLENGTHMEASURE(100.), #45);
#42 = IFCAXIS2PLACEMENT2D(#43, #44);
#43 = IFCCARTESIANPOINT((0., 0.));
#44 = IFCDIRECTION((1., 0.));
#45 = IFCTHIRDORDERPOLYNOMIALSPIRAL(#46, -110.668191970032, 100., $, $);
#46 = IFCAXIS2PLACEMENT2D(#47, $);
#47 = IFCCARTESIANPOINT((0., 0.));
```

2.8.3 Compute Point on Curve

Compute the curve coordinates at a distance along the curve, $u = 50$

Step 1 — Form the curve segment placement matrix M_{CSP}

Represent `IfcCurveSegment.Placement` in matrix form. In this example, the placement is at (0,0) with RefDirection (1,0) which results in an identity matrix.

$$M_{CSP} = \boldsymbol{I}$$

Step 2 — Evaluate the parent curve at the trim start and form the normalization matrix M_N

Because the parent curve is located at (0,0) in the direction (1,0), $x_0 = 0, y_0 = 0, \theta_0 = 0$.

Since $x_0 = 0$, $y_0 = 0$, and $\theta_0 = 0$, $M_N = \boldsymbol{I}$.

Step 3 — Evaluate and map each point

Compute point and curve tangent at 50 m from the start.

$$x = \int_0^{50} \cos\theta(s)\ ds = 49.9962109$$

$$y = \int_0^{50} \sin\theta(s)\ ds = 0.416638875$$

$$\theta(50) = \frac{-110.668192}{4|-110.668192^5|}(50)^4 + \frac{1}{3(100)^3}(50)^3 = 0.03125$$

$$dx = \cos(0.03125) = 0.999512$$

$$dy = \sin(0.03125) = 0.031245$$

In matrix form

$$M_{PC} = \begin{bmatrix} 0.999512 & -0.031245 & 0 & 49.9962109 \\ 0.031245 & 0.999512 & 0 & 0.416638875 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Apply the normalization and curve segment placement to the parent curve point

$$M_h = M_{CSP} M_N M_{PC} = \boldsymbol{I}\,\boldsymbol{I} \begin{bmatrix} 0.999512 & -0.031245 & 0 & 49.9962109 \\ 0.031245 & 0.999512 & 0 & 0.416638875 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$M_h = \begin{bmatrix} 0.999512 & -0.031245 & 0 & 49.9962109 \\ 0.031245 & 0.999512 & 0 & 0.416638875 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

2.9 Cosine Transition Curve

`IfcCosineSpiral` is the geometric type for cosine transition spirals.

2.9.1 Parent Curve Parametric Equations

The curve tangent angle and curvature are given by the following equations

$$\kappa(t) = \frac{1}{A_0} + \frac{1}{A_1}\cos\left(\frac{\pi}{L}t\right)$$

$$\theta(t) = \frac{1}{A_0}t + \frac{L}{\pi A_1}\sin\left(\frac{\pi}{L}t\right)$$

2.9.2 Semantic Definition to Geometry Mapping

Consider a horizontal cosine transition curve segment that starts at (0,0) with a start direction of 0.0. The radius at the start is infinite and the radius at the end is 300. The arc length of the segment is 100. The semantic definition is

```
#28 = IFCCARTESIANPOINT((0., 0.));
#29 = IFCALIGNMENTHORIZONTALSEGMENT($, $, #28, 0., 0., 300., 100., $,
.COSINECURVE.);
```

Compute the cosine spiral parameters

$$R_s = \infty,\ R_e = 300\ m,\ L = 100\ m$$

$$f = \frac{L}{R_e} - \frac{L}{R_s} = \frac{100}{300} - \frac{100}{\infty} = 0.33333$$

Constant Term:

$$a_0 = 0.5f + \frac{L}{R_s} = 0.5(0.33333) + \frac{100}{\infty} = 0.16667$$

$$A_0 = \frac{L}{|a_0|}\frac{a_0}{|a_0|} = \frac{100\ m}{|0.16667|}\frac{0.16667}{|0.16667|} = 600\ m$$

Cosine Term:

$$a_1 = -0.5f = -0.5(0.33333) = -0.16667$$

$$A_1 = \frac{L}{|a_1|}\frac{a_1}{|a_1|} = \frac{100}{|-0.16667|}\frac{-0.16667}{|-0.16667|} = -600\ m$$

```
#36 = IFCCURVESEGMENT(.CONTINUOUS., #42, IFCLENGTHMEASURE(0.),
IFCLENGTHMEASURE(100.), #45);
#42 = IFCAXIS2PLACEMENT2D(#43, #44);
#43 = IFCCARTESIANPOINT((0., 0.));
#44 = IFCDIRECTION((1., 0.));
#45 = IFCCOSINESPIRAL(#46, -600., 600.);
#46 = IFCAXIS2PLACEMENT2D(#47, $);
#47 = IFCCARTESIANPOINT((0., 0.));
```

2.9.3 Compute Point on Curve

Compute the curve coordinates at a distance along the curve, $u = 100$

Step 1 — Form the curve segment placement matrix M_{CSP}

Represent `IfcCurveSegment.Placement` in matrix form. In this example, the placement is at (0,0) with RefDirection (1,0) which results in an identity matrix.

$$M_{CSP} = \boldsymbol{I}$$

Step 2 — Evaluate the parent curve at the trim start and form the normalization matrix M_N

Because the parent curve is located at (0,0) in the direction (1,0), $x_0 = 0, y_0 = 0, \theta_0 = 0$.

Since $x_0 = 0$, $y_0 = 0$, and $\theta_0 = 0$, $M_N = \boldsymbol{I}$.

Step 3 — Evaluate and map each point

Compute point and curve tangent at 100 m from the start.

$$\theta(t) = \frac{1}{600}t - \frac{100}{600\pi}\sin\left(\frac{\pi}{100}t\right)$$

$$x = \int_0^{100} \cos\theta(t)\ dt = 99.7485\ m$$

$$y = \int_0^{100} \sin\theta(t)\ dt = 4.9458\ m$$

$$\theta(100) = \frac{1}{600}100 - \frac{100}{600\pi}\sin\left(\frac{\pi}{100}100\right) = \frac{1}{6} = 0.1666667$$

$$dx = \cos\left(\frac{1}{6}\right) = 0.98614$$

$$dy = \sin\left(\frac{1}{6}\right) = 0.16589$$

In matrix form

$$M_{PC} = \begin{bmatrix} 0.98614 & -0.16589 & 0 & 99.7485 \\ 0.16589 & 0.98614 & 0 & 4.9458 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Apply the normalization and curve segment placement to the parent curve point

$$M_h = M_{CSP} M_N M_{PC} = I\ I \begin{bmatrix} 0.98614 & -0.16589 & 0 & 99.7485 \\ 0.16589 & 0.98614 & 0 & 4.9458 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$M_h = \begin{bmatrix} 0.98614 & -0.16589 & 0 & 99.7485 \\ 0.16589 & 0.98614 & 0 & 4.9458 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

2.10 Sine Transition Curve

`IfcSineSpiral` is the geometric type for sine transition spirals.

2.10.1 Parent Curve Parametric Equations

The curve tangent angle and curvature are given by the following equations

$$\kappa(t) = \frac{1}{A_0} + \frac{A_1}{|A_1|}\left(\frac{1}{A_1}\right)^2 t + \frac{1}{A_2}\sin\left(\frac{2\pi}{L}t\right)$$

$$\theta(t) = \frac{1}{A_0}t + \frac{1}{2}\left(\frac{A_1}{|A_1|}\right)\left(\frac{t}{A_1}\right)^2 - \frac{L}{2\pi A_2}\left(\cos\left(\frac{2\pi}{L}t\right) - 1\right)$$

$$x = \int_0^L \cos\theta(t)\ dt$$

$$y = \int_0^L \sin\theta(t)\ dt$$

2.10.2 Semantic Definition to Geometry Mapping

Consider a horizontal sine transition curve segment that starts at (0,0) with a start direction of 0.0. The radius at the start is infinite and the radius at the end is 300. The arc length of the segment is 100. The semantic definition is

```
#28 = IFCCARTESIANPOINT((0., 0.));
#29 = IFCALIGNMENTHORIZONTALSEGMENT($, $, #28, 0., 0., 300., 100., $,
.SINECURVE.);
```

Compute the sine spiral parameters

$$R_s = \infty,\ R_e = 300\ m,\ L = 100\ m$$

$$f = \frac{L}{R_e} - \frac{L}{R_s} = \frac{100}{300} - \frac{100}{\infty} = 0.33333$$

Constant Term:

$$a_0 = \frac{L}{R_s} = \frac{100}{\infty} = 0$$

$$A_0 = \frac{L}{|a_0|}\frac{a_0}{|a_0|} = 0$$

Linear Term:

$$a_1 = f = 0.33333$$

$$A_1 = \frac{L}{|a_1|^{\frac{1}{2}}}\frac{a_1}{|a_1|} = \frac{100}{|0.33333|^{\frac{1}{2}}}\frac{0.33333}{|0.33333|} = 173.2050808\ m$$

Sine Term:

$$a_2 = -\frac{1}{2\pi}f = -\frac{1}{2\pi}(0.33333) = -0.053051647$$

$$A_2 = \frac{L}{|a_2|}\frac{a_2}{|a_2|} = \frac{100}{|-0.053051647|}\frac{-0.053051647}{|-0.053051647|} = -1884.955592\ m$$

```
#36 = IFCCURVESEGMENT(.CONTINUOUS., #42, IFCLENGTHMEASURE(0.),
IFCLENGTHMEASURE(100.), #45);
#42 = IFCAXIS2PLACEMENT2D(#43, #44);
#43 = IFCCARTESIANPOINT((0., 0.));
#44 = IFCDIRECTION((1., 0.));
#45 = IFCSINESPIRAL(#46, -1884.95559215388, 173.205080756888, $);
#46 = IFCAXIS2PLACEMENT2D(#47, $);
#47 = IFCCARTESIANPOINT((0., 0.));
```

2.10.3 Compute Point on Curve

Compute the curve coordinates at a distance along the curve, $u = 100$

Step 1 — Form the curve segment placement matrix M_{CSP}

Represent `IfcCurveSegment.Placement` in matrix form. In this example, the placement is at (0,0) with RefDirection (1,0) which results in an identity matrix.

$$M_{CSP} = \boldsymbol{I}$$

Step 2 — Evaluate the parent curve at the trim start and form the normalization matrix M_N

Because the parent curve is located at (0,0) in the direction (1,0), $x_0 = 0, y_0 = 0, \theta_0 = 0$.

Since $x_0 = 0$, $y_0 = 0$, and $\theta_0 = 0$, $M_N = \boldsymbol{I}$.

Step 3 — Evaluate and map each point

Compute point and curve tangent at 100 m from the start.

$$\theta(t) = \frac{1}{2}\left(\frac{t}{173.2050808}\right)^2 + \frac{100}{2\pi(1884.955592)}\left(\cos\left(\frac{2\pi}{100}t\right) - 1\right)$$

$$x = \int_0^{100} \cos\theta(t)\ dt = 99.75698\ m$$

$$y = \int_0^{100} \sin\theta(t)\ dt = 4.70132\ m$$

$$\theta(100) = \frac{1}{2}\left(\frac{100}{173.2050808}\right)^2 + \frac{100}{2\pi(1884.955592)}\left(\cos\left(\frac{2\pi}{100}100\right) - 1\right) = \frac{1}{6} = 0.1666667$$

$$dx = \cos\left(\frac{1}{6}\right) = 0.98614$$

$$dy = \sin\left(\frac{1}{6}\right) = 0.16589$$

In matrix form

$$M_{PC} = \begin{bmatrix} 0.98614 & -0.16589 & 0 & 99.75698 \\ 0.16589 & 0.98614 & 0 & 4.70132 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Apply the normalization and curve segment placement to the parent curve point

$$M_h = M_{CSP} M_N M_{PC} = \boldsymbol{I}\,\boldsymbol{I} \begin{bmatrix} 0.98614 & -0.16589 & 0 & 99.75698 \\ 0.16589 & 0.98614 & 0 & 4.70132 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$M_h = \begin{bmatrix} 0.98614 & -0.16589 & 0 & 99.75698 \\ 0.16589 & 0.98614 & 0 & 4.70132 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

2.11 Viennese Transition Curve

The Viennese Bend is a transition spiral developed for high-speed railway applications. Unlike every other transition curve type in this guide — all of which define horizontal

geometry purely from geometric parameters (start radius, end radius, and arc length) — the Viennese Bend incorporates vehicle dynamics: its curvature variation is shaped by the superelevation (cant) transition that accompanies it. The result is a curve that simultaneously satisfies continuity requirements for curvature, cant, and their first derivatives, producing lower lateral jerk experienced by passengers.

`IfcAlignmentHorizontalSegment.GravityCenterLineHeight` is required for `VIENNESEBEND` but is optional for all other horizontal segment types. The associated cant segment (`IfcAlignmentCantSegment` with `PredefinedType = VIENNESEBEND`) must span the same horizontal length.

The IFC geometric type is `IfcSeventhOrderPolynomialSpiral`.

2.11.1 Parent Curve Parametric Equations

The curve tangent angle and curvature are given by the following equations

$$\theta(s) = \frac{A_7}{8|A_7^9|}s^8 + \frac{1}{7A_6^7}s^7 + \frac{A_5}{6|A_5^7|}s^6 + \frac{1}{5A_4^5}s^5 + \frac{A_3}{4|A_3^5|}s^4 + \frac{1}{3A_2^3}s^3 + \frac{A_1}{2|A_1^3|}s^2 + \frac{1}{A_0}s$$

$$\kappa(s) = \frac{A_7}{|A_7^9|}s^7 + \frac{1}{A_6^7}s^6 + \frac{A_5}{|A_5^7|}s^5 + \frac{1}{A_4^5}s^4 + \frac{A_3}{|A_3^5|}s^3 + \frac{1}{A_2^3}s^2 + \frac{A_1}{|A_1^3|}s + \frac{1}{A_0}$$

2.11.2 Semantic Definition to Geometry Mapping

Consider a horizontal Viennese Bend transition curve segment that starts at (0,0) with a start direction of 0.0. The radius at the start is infinite and the radius at the end is 300. The arc length is 100. The Gravity Center Line Height is 1.8 (this optional parameter is required for the Viennese Bend). The semantic definition is

```
#28 = IFCCARTESIANPOINT((0., 0.));
#29 = IFCALIGNMENTHORIZONTALSEGMENT($, $, #28, 0., 0., 300., 100., 1.8,
.VIENNESEBEND.);
```

Viennese Bend transition curves are unique in that the horizontal geometry of the curve depends on the cant. For the example, the cant segment horizontal length is 100, the start cant is 0.0 and the end cant is 0 at the left rail and 0.1 at the right rail. The semantic definition is

```
#64 = IFCALIGNMENTCANTSEGMENT($, $, 0., 100., 0., 0., 0., 0.1,
.VIENNESEBEND.);
```

h_{cg} = Gravity Center Line Height = `IfcAlignmentHorizontalSegment.GravityCenterLineHeight`

D_{rh} = Rail Head Distance = `IfcAlignmentCant.RailHeadDistance`

D_{sl} = `IfcAlignmentCantSegment.StartCantLeft`

D_{el} = `IfcAlignmentCantSegment.EndCantLeft`

D_{sr} = `IfcAlignmentCantSegment.StartCantRight`

D_{er} = `IfcAlignmentCantSegment.EndCantRight`

$\beta_s = \frac{(D_{sr} - D_{sl})}{D_{rh}}$, Start Cross Slope

$\beta_e = \frac{(D_{er} - D_{el})}{D_{rh}}$, End Cross Slope

$cf = -420.\left(\frac{h_{cg}}{L}\right)(\beta_e - \beta_s)$, Cant Factor

Compute the polynomial spiral parameters

$$R_s = \infty,\ R_e = 300\ m,\ L = 100\ m$$

$$f = \frac{L}{R_e} - \frac{L}{R_s} = \frac{100}{300} - \frac{100}{\infty} = 0.33333$$

$$\beta_s = \frac{(0. - 0.)}{1.5} = 0.0$$

$$\beta_e = \frac{0.1 - 0.0}{1.5} = 0.066667$$

$$cf = -420\left(\frac{1.8}{100}\right)(0.066667 - 0.) = -0.504$$

Constant term

$$a_0 = \frac{L}{R_s} = \frac{100}{\infty} = 0$$

$$A_0 = \frac{L}{|a_0|}\frac{a_0}{|a_0|} = 0$$

Linear term

$$a_1 = 0$$

$$A_1 = 0$$

Quadratic term

$$a_2 = cf = -0.504$$

$$A_2 = \frac{L}{|a_2|^{\frac{1}{3}}}\frac{a_2}{|a_2|} = \frac{100}{|-0.504|^{\frac{1}{3}}}\frac{-0.504}{|-0.504|} = -125.6579069\ m$$

Cubic term

$$a_3 = -4cf = -4(-0.504) = 2.016$$

$$A_3 = \frac{L}{|a_3|^{\frac{1}{4}}}\frac{a_3}{|a_3|} = \frac{100}{|2.016|^{\frac{1}{4}}}\frac{2.016}{|2.016|} = 83.92229813\ m$$

Quartic term

$$a_4 = 5cf + 35f = 5(-0.504) + 35(0.33333) = 9.1466655$$

$$A_4 = \frac{L}{|a_4|^{\frac{1}{5}}}\frac{a_4}{|a_4|} = \frac{100}{|9.1466655|^{\frac{1}{5}}}\frac{9.1466655}{|9.1466655|} = 64.231406\ m$$

Quintic term

$$a_5 = -2cf - 84f = -2(-0.504) - 84(0.33333) = -26.9919999$$

$$A_5 = \frac{L}{|a_5|^{\frac{1}{6}}}\frac{a_5}{|a_5|} = \frac{100}{|-26.9919999|^{\frac{1}{6}}}\frac{-26.9919999}{|-26.9919999|} = -57.7378785\ m$$

Sextic term

$$a_6 = 70f = 70(0.33333) = 23.33333333$$

$$A_6 = \frac{L}{|a_6|^{\frac{1}{7}}}\frac{a_6}{|a_6|} = \frac{100}{|23.33333333|^{\frac{1}{7}}}\frac{23.33333333}{|23.33333333|} = 63.76388134\ m$$

Septic term

$$a_7 = -20f = -20(0.33333) = -6.66666666$$

$$A_7 = \frac{L}{|a_7|^{\frac{1}{8}}}\frac{a_7}{|a_7|} = \frac{100}{|-6.66666666|^{\frac{1}{8}}}\frac{-6.66666666}{|-6.66666666|} = -78.8880838\ m$$

```
#66 = IFCCURVESEGMENT(.CONTINUOUS., #72, IFCLENGTHMEASURE(0.),
IFCLENGTHMEASURE(100.), #75);
#72 = IFCAXIS2PLACEMENT2D(#73, #74);
#73 = IFCCARTESIANPOINT((0., 0.));
#74 = IFCDIRECTION((1., 0.));
#75 = IFCSEVENTHORDERPOLYNOMIALSPIRAL(#76, -78.8880838459446,
63.7638813456506, -57.7378785242934, 64.2314061308743, 83.922298125931, -
125.657906854859, $, $);
#76 = IFCAXIS2PLACEMENT2D(#77, $);
#77 = IFCCARTESIANPOINT((0., 0.));
```

2.11.3 Compute Point on Curve

Compute the curve coordinates at a distance along the curve, $u = 100$

Step 1 — Form the curve segment placement matrix M_{CSP}

Represent `IfcCurveSegment.Placement` in matrix form. In this example, the placement is at (0,0) with RefDirection (1,0) which results in an identity matrix.

$$M_{CSP} = \boldsymbol{I}$$

Step 2 — Evaluate the parent curve at the trim start and form the normalization matrix M_N

Because the parent curve is located at (0,0) in the direction (1,0), $x_0 = 0, y_0 = 0, \theta_0 = 0$.

Since $x_0 = 0$, $y_0 = 0$, and $\theta_0 = 0$, $M_N = \boldsymbol{I}$.

Step 3 — Evaluate and map each point

Compute point and curve tangent at 100 m from the start.

$$\theta(s) = \frac{A_7}{8|A_7^9|}s^8 + \frac{1}{7A_6^7}s^7 + \frac{A_5}{6|A_5^7|}s^6 + \frac{1}{5A_4^5}s^5 + \frac{A_3}{4|A_3^5|}s^4 + \frac{1}{3A_2^3}s^3 + \frac{A_1}{2|A_1^3|}s^2 + \frac{1}{A_0}s$$

$$x = \int_0^{100} \cos\theta(s)\ ds = 99.76319823871657$$

$$y = \int_0^{100} \sin\theta(s)\ ds = 4.499916129045404$$

$$\theta(100) = \frac{-78.8880838}{8|(-78.8880838)^9|}100^8 + \frac{1}{7(63.76388134)^7}100^7 + \frac{-57.7378785}{6|(-57.7378785)^7|}100^6 + \frac{1}{5(64.231406)^5}100^5 + \frac{83.92229813}{4|(83.92229813)^5|}100^4 + \frac{1}{3(-125.6579069)^3}100^3 = \frac{1}{6} = 0.1666666667$$

$$dx = \cos(0.1666666667) = 0.9861432315629198$$

$$dy = \sin(0.1666666667) = 0.16589613269344608$$

In matrix form

$$M_{PC} = \begin{bmatrix} 0.9861432315629198 & -0.16589613269344608 & 0 & 99.76319823871657 \\ 0.16589613269344608 & 0.9861432315629198 & 0 & 4.499916129045404 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Apply the normalization and curve segment placement to the parent curve point

$$M_h = M_{CSP}M_N M_{PC}$$

$$= \boldsymbol{I}\,\boldsymbol{I}\begin{bmatrix} 0.9861432315629198 & -0.16589613269344608 & 0 & 99.76319823871657 \\ 0.16589613269344608 & 0.9861432315629198 & 0 & 4.499916129045404 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$M_h = \begin{bmatrix} 0.9861432315629198 & -0.16589613269344608 & 0 & 99.76319823871657 \\ 0.16589613269344608 & 0.9861432315629198 & 0 & 4.499916129045404 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

2.12 Zero Length Segment

IFC Concept Templates 4.1.4.4.1.1, 4.1.4.4.1.2, 4.1.7.1.1.1, 4.1.7.1.1.2, 4.1.7.1.1.3, and 4.1.7.1.1.4 require that the final `IfcAlignmentSegment` nested within `IfcAlignmentHorizontal`, `IfcAlignmentVertical`, and `IfcAlignmentCant` has a `SegmentLength` of zero. A corresponding zero-length `IfcCurveSegment` is required in each associated geometric representation (`IfcCompositeCurve`, `IfcGradientCurve`, or `IfcSegmentedReferenceCurve`).

The specification imposes no constraint on which curve type backs the zero-length geometric segment. As best practice, use `.LINE.` (`IfcAlignmentHorizontalSegment`), `.CONSTANTGRADIENT.` (`IfcAlignmentVerticalSegment`), or `.CONSTANTCANT.` (`IfcAlignmentCantSegment`) as the `PredefinedType` of the semantic closing segment, and an `IfcLine` as the parent curve of the corresponding `IfcCurveSegment`. The placement of both the semantic and geometric closing segments must match the endpoint and end direction of the preceding segment.

2.12.1 Semantic Closing Segment

The `StartPoint` and `StartDirection` of the semantic segment are taken from the end state of the preceding segment, and `SegmentLength` is set to zero. One implementation strategy is to map the preceding semantic segment definition to its geometric representation as described in this chapter, as well as in Chapters 3 and 4, and evaluate it at its end point.

```
// Last non-zero length segment
#88=IFCCARTESIANPOINT((7790.93212831259,4006.73076454875));
#89=IFCALIGNMENTHORIZONTALSEGMENT($,$,#88,-
1.23849516741382,0.,0.,2112.2850844257,$,.LINE.);

// Final segment, zero length.
#91=IFCCARTESIANPOINT((8480.,2010.));
#92=IFCALIGNMENTHORIZONTALSEGMENT($,$,#91,-
1.23849516741382,0.,0.,0.,$,.LINE.);
```

2.12.2 Geometric Closing Segment

The zero-length `IfcCurveSegment` is placed at the endpoint of the alignment with its axis aligned to the end tangent direction. The parent curve is an `IfcLine` whose point and direction vector match that placement. `SegmentStart` and `SegmentLength` are both zero. Because the segment has zero length, the choice of `IfcLine` here has no geometric effect — it represents a point at the end of the alignment.

```
// Last non-zero length segment
#169=IFCCARTESIANPOINT((0.,0.));
#170=IFCDIRECTION((1.,0.));
#171=IFCVECTOR(#170,1.);
#172=IFCLINE(#169,#171);
#173=IFCDIRECTION((0.326219162729524,-0.945294164727599));
#174=IFCAXIS2PLACEMENT2D(#88,#173);
#175=IFCCURVESEGMENT(.CONTSAMEGRADIENTSAMECURVATURE.,#174,IFCLENGTHMEASURE(0.
```

```
),IFCLENGTHMEASURE(2112.2850844257),#172);

// Final segment, zero length
#176=IFCCARTESIANPOINT((0.,0.));
#177=IFCDIRECTION((1.,0.));
#178=IFCVECTOR(#177,1.);
#179=IFCLINE(#176,#178);
#180=IFCDIRECTION((0.326219162729524,-0.945294164727599));
#181=IFCAXIS2PLACEMENT2D(#91,#180);
#182=IFCCURVESEGMENT(.DISCONTINUOUS.,#181,IFCLENGTHMEASURE(0.),IFCLENGTHMEASU
RE(0.),#179);
```

2.13 Summary and Implementation Checklist

#	Item	Notes
1	Use `IfcLengthMeasure` for `SegmentStart` and `SegmentLength` on every `IfcCurveSegment`	Arc-length parameterization is mandatory per IFC schema Informal Proposition 1; `IfcParameterValue` (unit parameterization) is not valid for alignment geometry
2	Handle negative `SegmentStart` and `SegmentLength`	A negative `SegmentStart` locates the trim origin on the opposite side of the parent curve's reference point; a negative `SegmentLength` indicates trimming in the direction of decreasing parameter. Both can occur in any combination.
3	For clothoid curves, convert `SegmentStart` from arc-length to unit parameter before evaluating	Compute $u = s/\left\|A\sqrt{\pi}\right\|$ where A is the clothoid constant; the Fresnel integrals are defined in terms of u, not s
4	Expect one `IfcAlignmentHorizontalSegment` with `PredefinedType = HELMERTCURVE` to map to two `IfcCurveSegment` entities	The Helmert curve is the only horizontal type where one semantic segment produces two geometric segments; evaluate the correct half based on the query distance relative to $L/2$
5	Treat `IfcPolynomialCurve` coefficients as having implicit units of Length$^{(1-i)}$	For a cubic curve, `CoefficientsY[3]` has implicit units of m^{-2}; evaluate as $y = b_3x^3$ where x is a length. Do not multiply by curve length L as you would for a unit-parameterized polynomial.
6	Viennese Bend horizontal geometry requires `GravityCenterLineHeight` and a paired cant segment	`IfcAlignmentHorizontalSegment.GravityCenterLineHeight` is mandatory for `VIENNESEBEND`; the associated `IfcAlignmentCantSegment` with `PredefinedType = VIENNESEBEND` must span the same horizontal length, and its

#	Item	Notes
		cant values enter the curvature formula via the cant factor cf
7	Do not assume the parent curve is centered at the origin or aligned with the x-axis	The three-step algorithm (M_{CSP}, M_N, M_{PC}) handles arbitrary parent curve placement; `SegmentStart` locates where trimming begins regardless of the parent curve's own position and orientation
8	Apply the positive-left sign convention for radii	Positive `StartRadiusOfCurvature` / `EndRadiusOfCurvature` indicates a curve to the left of the direction of travel; negative indicates a curve to the right; zero indicates infinite radius (straight tangent)

Chapter 3 — Vertical Alignments

3.0 Introduction

The geometric representation of a vertical alignment is accomplished with `IfcGradientCurve`. It represents the elevation profile as a two-dimensional curve in a (distance along, elevation) coordinate system, where distance along is measured along the horizontal `IfcCompositeCurve` (`IfcGradientCurve.BaseCurve`). Combined with the horizontal alignment, the gradient curve establishes the three-dimensional geometry of the route.

Table 3.0-1 maps each `IfcAlignmentVerticalSegment.PredefinedType` to its corresponding parent curve type.

Business Logic (`IfcAlignmentVerticalSegment.PredefinedType`)	Geometric Representation (`IfcCurveSegment.ParentCurve`)
CONSTANTGRADIENT	`IfcLine`
CIRCULARARC	`IfcCircle`
CLOTHOID	`IfcClothoid`
PARABOLICARC	`IfcPolynomialCurve`

Table 3.0-1 — Mapping of business logic to geometric representation for vertical alignment

This chapter covers:

- The four-step evaluation algorithm: Steps 1–3 follow the same matrix composition as horizontal segments; Step 4 combines the vertical result with the horizontal placement matrix to produce a full 3D placement.
- Parametric equations and geometry mapping examples for the curve types in Table 3.0-1.
- The combined 3D evaluation that produces position, tangent, cross-track, and up-axis vectors from the `IfcGradientCurve`.
- Known limitation: vertical clothoid curves are not addressed in this guide.

3.1 Conventions

Each vertical segment is parameterized by arc-length s along its parent curve, where $s = 0$ at the start of the parent curve. The key semantic attributes are `IfcAlignmentVerticalSegment.StartDistAlong` (horizontal distance from the alignment start to the segment start), `HorizontalLength` (the horizontal projection of the segment), `StartHeight` (elevation at the segment start), and `StartGradient` and `EndGradient` (decimal slopes; e.g., $0.005 = 0.5\%$).

`IfcCurveSegment.SegmentLength` stores arc-length along the parent curve, **not** horizontal distance. The evaluator parameter is horizontal distance ℓ, which must be converted to arc-length s for use in the parent curve equations. For the grades typical in road and railway design ($\leq 5\%$), the difference is small but non-zero.

The grade angle at horizontal distance ℓ is $\theta(\ell) = \tan^{-1}(g(\ell))$, where $g(\ell)$ is the gradient. The cosine and sine of θ form the `RefDirection` of the vertical curve at that point.

3.2 Curve Segment Evaluation Algorithm

Steps 1–3 follow the identical procedure described in Section 2.2 for horizontal segments, substituting distance along for the horizontal x-coordinate and elevation for the horizontal y-coordinate. Let s_0 = `IfcCurveSegment.SegmentStart`.

Step 1 — Form the curve segment placement matrix M_{CSP}

M_{CSP} is constructed from `IfcCurveSegment.Placement`, where (d_p, y_p) is the `Location` (distance along, elevation) and θ_p is the grade angle of the `RefDirection`.

$$M_{CSP} = \begin{bmatrix} \cos\theta_p & -\sin\theta_p & 0 & d_p \\ \sin\theta_p & \cos\theta_p & 0 & y_p \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Step 2 — Evaluate the parent curve at the trim start and form the normalization matrix M_N

Compute the distance along $d_0 = x(s_0)$, elevation $y_0 = y(s_0) =$ IfcAlignmentVerticalSegment.StartHeight, and grade angle $\theta_0 = \theta(s_0)$.

M_N simultaneously translates the trim-start point to the origin and rotates so that the tangent at s_0 aligns with the positive x-direction.

$$M_N = \begin{bmatrix} \cos\theta_0 & \sin\theta_0 & 0 & -d_0\cos\theta_0 - y_0\sin\theta_0 \\ -\sin\theta_0 & \cos\theta_0 & 0 & d_0\sin\theta_0 - y_0\cos\theta_0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Step 3 — Evaluate and map each point in the vertical plane

For the point at distance along s, compute $x(s)$, $y(s)$, and $\theta(\ell)$:

$$M_{PC} = \begin{bmatrix} \cos\theta(\ell) & -\sin\theta(\ell) & 0 & x(s) \\ \sin\theta(\ell) & \cos\theta(\ell) & 0 & y(s) \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$M_v = M_{CSP} M_N M_{PC}$$

Column 4 of M_v contains the distance along d and elevation z of the evaluated point. Column 1 contains $(dx_v, dy_v) = (\cos\theta_v, \sin\theta_v)$, the grade direction at that point. Step 4 is performed immediately for this point before moving to the next point.

Step 4 — Combine with the horizontal alignment point to produce the 3D placement matrix

The vertical result lives in the (distance along, elevation) plane and must be merged with the horizontal alignment to form a 3D position and orientation. The "distance along" axis corresponds to the curve tangent of the horizontal alignment and the vertical y is elevation (Z in 3D).

Evaluate the horizontal placement matrix M_h at distance d along the `IfcCompositeCurve`. This yields the horizontal 2D position (x_h, y_h) and the horizontal alignment tangent direction $(dx_h, dy_h) = (\cos\theta_h, \sin\theta_h)$.

M_v is a two-dimensional matrix whose rows index the vertical frame: row 1 = distance-along, row 2 = elevation, row 3 = out-of-plane. Three modifications produce M'_v, the form required for multiplication with M_h.

Column 4 — The distance-along value d is replaced with zero. The horizontal position already comes from M_h; retaining d would displace the point a second time along the horizontal tangent.

Rows 2 and 3 — M_h's third row $(0,\ 0,\ 1,\ 0)$ passes row 3 of the right operand through unchanged into row 3 of the product. Because row 3 of M_{3D} must carry the elevation (Z) component of every vector, elevation must occupy row 3 of M'_v — not row 2 as in M_v. The row semantics are therefore swapped: in M'_v, row 2 is the cross-track (lateral) component and row 3 is the elevation (Z) component.

Columns 2 and 3 — The row reordering reindexes the column values and exchanges the columns. M_v column 2, the in-plane normal $(-dy_v,\ dx_v,\ 0)$ in [d-along, elevation, out-of-plane] ordering, becomes $(-dy_v,\ 0,\ dx_v)$ in [d-along, cross-track, elevation] ordering — the 3D up direction — and moves to column 3. M_v column 3, the out-of-plane unit direction $(0,\ 0,\ 1)$, becomes $(0,\ 1,\ 0)$ — the cross-track direction — and moves to column 2.

$$M'_v = \begin{bmatrix} dx_v & 0 & -dy_v & 0 \\ 0 & 1 & 0 & 0 \\ dy_v & 0 & dx_v & z \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

The columns of M'_v define directions in the 3D frame local to the alignment:

- **Column 1** $(dx_v,\ 0,\ dy_v)$: the grade direction — dx_v scales the contribution along the horizontal tangent; dy_v is the elevation rate of change (Z component of the 3D tangent).
- **Column 2** $(0,\ 1,\ 0)$: the cross-track direction, always lateral to the alignment and always horizontal.

- **Column 3** $(-dy_v,\ 0,\ dx_v)$: the "up" direction in the vertical plane of the alignment, orthogonal to the grade direction.
- **Column 4** $(0,\ 0,\ z)$: the elevation offset with no horizontal displacement.

The 3D placement matrix is:

$$M_{3D} = M_h \cdot M'_v$$

Multiplying out, the columns of M_{3D} are:

- **Column 1** (RefDirection / 3D tangent): $(dx_h \cdot dx_v,\ dy_h \cdot dx_v,\ dy_v)$ — the horizontal tangent scaled by $\cos\theta_v$, with $\sin\theta_v$ as the Z component.
- **Column 2** (Y / cross-track): $(-dy_h,\ dx_h,\ 0)$ — the horizontal normal direction, unchanged from M_h.
- **Column 3** (Axis / up): $(-dx_h \cdot dy_v,\ -dy_h \cdot dy_v,\ dx_v)$ — orthogonal to both the 3D tangent and the cross-track direction.
- **Column 4** (Location): $(x_h,\ y_h,\ z)$ — the full 3D position.

Numerical examples of Steps 1–3 are given in Sections 3.3 through 3.6. A complete worked example including Step 4 is in Section 3.7.

3.3 Constant Gradient

A constant gradient is geometrically represented with a segment trimmed from an `IfcLine` parent curve.

3.3.1 Parent Curve Parametric Equations

$$x(s) = p_x + s \cdot dx$$

$$y(s) = p_y + s \cdot dy$$

where $(p_x,\ p_y)$ is the `IfcLine` origin and dx and dy are the direction parameters.

3.3.2 Semantic Definition to Geometry Mapping

Mapping of the semantic definition of the linear segment to the geometric definition is described with the following example.

Consider a vertical gradient at an uphill slope of 0.5 starting at point (0,10). The horizontal projection of the segment length is 100.

```
#44 = IFCALIGNMENTVERTICALSEGMENT($, $, 0., 100., 10., 0.5, 0.5, $,
.CONSTANTGRADIENT.);
```

Define the direction of the `IfcLine` so it is horizontal

$$dx = 1$$

$$dy = 0$$

Define the `IfcLine` as passing through point (0,0)

```
#80 = IFCLINE(#81, #82);
#81 = IFCCARTESIANPOINT((0., 0.));
#82 = IFCVECTOR(#83, 1.);
#83 = IFCDIRECTION((1., 0.));
```

Place the curve segment at (0,10) with a tangent direction

$$dx = \cos(\tan^{-1}(0.5)) = 0.894427191$$

$$dy = \sin(\tan^{-1}(0.5)) = 0.44713595$$

```
#77 = IFCAXIS2PLACEMENT2D(#78, #79);
#78 = IFCCARTESIANPOINT((0., 10.));
#79 = IFCDIRECTION((0.894427190999916, 0.447213595499958));
```

`IfcCurveSegment.SegmentLength` is the length measured along the trimmed curve. For a horizontal length of $100\ m$, the length along the curve is

$$100/0.894427190999916 = 111.803398874989\ m$$

The curve segment is defined as

```
#71 = IFCCURVESEGMENT(.CONTINUOUS., #77, IFCLENGTHMEASURE(0.),
IFCLENGTHMEASURE(111.803398874989), #80);
```

3.3.3 Compute Point on Curve

Compute the vertical placement matrix for the point at horizontal distance $\ell = 50$ m from the start of the curve segment.

Step 1 — Form the curve segment placement matrix M_{CSP}

From `IfcCurveSegment.Placement`: $(d_p,\ y_p) = (0,\ 10)$, $\theta_p = 0.463647609$ rad:

$$\cos\theta_p = \cos(0.463647609) = 0.894427191 \quad \sin\theta_p = \sin(0.463647609) = 0.447213595$$

$$M_{CSP} = \begin{bmatrix} 0.894427191 & -0.447213595 & 0 & 0 \\ 0.447213595 & 0.894427191 & 0 & 10 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Step 2 — Evaluate the parent curve at the trim start and form the normalization matrix M_N

The trim begins at $s_0 = \text{SegmentStart} = 0$. For the `IfcLine` through the origin with direction $(dx,\ dy) = (1.,\ 0.)$:

$$d_0 = x(0) = 0 \quad y_0 = y(0) = 0 \quad \theta_0 = \tan^{-1}\left(\frac{dy}{dx}\right) = \tan^{-1}\left(\frac{0.}{1.}\right) = 0.\ \text{rad}$$

$$M_N = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Step 3 — Evaluate and map each point in the vertical plane

The horizontal distance $\ell = 50$ m is not the arc-length parameter. Convert to arc-length s along the `IfcLine`:

$$s = s_0 + \frac{\ell}{dx} = 0 + \frac{50}{0.894427191} = 55.9016994 \text{ m}$$

Evaluate the parent curve at $s = 55.9016994$:

$$d = x(55.9016994) = 55.9016994 \times 1 = 55.9016994 \text{ m}$$

$$z = y(55.9016994) = 55.9016994 \times 0 = 0 \text{ m}$$

$$\theta = 0.$$

$$M_{PC} = \begin{bmatrix} 1 & 0 & 0 & 55.9016994 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

The vertical placement matrix is:

$$M_v = M_{CSP}\ M_N\ M_{PC}$$

$$= \begin{bmatrix} 0.894427191 & -0.447213595 & 0 & 0 \\ 0.447213595 & 0.894427191 & 0 & 10 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & 55.9016994 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$M_v = \begin{bmatrix} 0.894427191 & -0.447213595 & 0 & 50.000 \\ 0.447213595 & 0.894427191 & 0 & 35.000 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

3.4 Circular Arc

A circular vertical curve is geometrically represented with a segment trimmed from an `IfcCircle` parent curve.

3.4.1 Parent Curve Parametric Equations

The `IfcCircle` parametric equations for a vertical circular arc are identical to the horizontal case (Section 2.4.1), with $x(s)$ representing distance along the horizontal alignment and $y(s)$ representing elevation.

$$\theta(s) = \frac{s}{R}$$

$$\kappa(s) = \frac{1}{R}$$

$$x(s) = \int \cos(\theta(s))ds = R\sin(\theta(s))$$

$$y(s) = \int \sin(\theta(s))ds = R\big(1 - \cos(\theta(s))\big)$$

where R is `IfcCircle.Radius` and s is arc length along the circle. The grade at arc length s is $g(s) = \tan(\theta(s))$.

3.4.2 Semantic Definition to Geometry Mapping

Vertical circular arcs are tricky. The "distance along" dimension is horizontal while the `IfcCurveSegment` trimming parameters `SegmentStart` and `SegmentLength` are measured along the `IfcCircle`.

The following procedure maps the semantic parameters of a vertical circular arc to its geometric definition.

Given the following semantic definition of a vertical circular arc, create the geometric definition.

```
#44 = IFCALIGNMENTVERTICALSEGMENT($, $, 0., 100., 10., -5.E-1, -1.,
384.773458895502, .CIRCULARARC.);
```

Determine the tangent slope angle at the start and end of the segment.

$$g_{start} = -0.5$$

$$\theta_{start} = \tan^{-1}(g_{start})$$

$$\theta_{start} = \tan^{-1}(-0.5) = -0.463648$$

$$g_{end} = -1$$

$$\theta_{end} = \tan^{-1}(g_{end})$$

$$\theta_{end} = \tan^{-1}(-1) = -0.785398$$

Compute the radius of the circle.

`IfcAlignmentVerticalSegment.RadiusOfCurvature` is optional. If provided, it should be consistent with the `HorizontalLength`, `StartGradient` and `EndGradient`, but is not guaranteed. For this reason, the radius is computed from the required `HorizontalLength`, `StartGradient` and `EndGradient` attributes. Note that in computing the radius, it is taken to be an absolute value because `IfcCircle.Radius` is a `IfcPositiveLengthMeasure`.

$$R = \left|\frac{h_l}{\sin(\theta_{start}) - \sin(\theta_{end})}\right|$$

$$h_l = \text{IfcAlignmentVerticalSegment.HorizontalLength} = 100.$$

$$R = \left|\frac{100}{\sin(-0.785398) - \sin(-0.463648)}\right| = 384.773458895502$$

Compute the arc-length trimming parameters `SegmentStart` and `SegmentLength`.

For an `IfcCircle` with CCW parametrization and `RefDirection` = (1, 0), the arc-length parameter at a point where the grade angle is θ depends on which half of the circle the vertical curve occupies.

If $\theta_{start} < \theta_{end}$ — sagging curve (lower half of circle):

$$\text{SegmentStart} = R\left(\theta_{start} + \frac{3}{2}\pi\right)$$

If $\theta_{start} > \theta_{end}$ — cresting curve (upper half of circle):

$$\text{SegmentStart} = R\left(\theta_{start} + \frac{1}{2}\pi\right)$$

In both cases:

$$\text{SegmentLength} = R(\theta_{end} - \theta_{start})$$

For a cresting curve $\theta_{end} < \theta_{start}$, so `SegmentLength` is negative — the segment is traversed clockwise.

For this example (cresting):

$$\text{SegmentStart} = R\left(\theta_{start} + \frac{\pi}{2}\right) = (384.773458895502)(-0.463647609 + \frac{\pi}{2})$$
$$= 426.001441657352$$

$$\text{SegmentLength} = R(\theta_{end} - \theta_{start})$$
$$= (384.773458895502)(-0.785398 - (-0.463647609))$$
$$= -123.801073716741$$

Determine the placement of the trimmed curve

X = `IfcAlignmentVerticalSegment.StartDistAlong` = 0.0

Y = `IfcAlignmentVerticalSegment.StartHeight` = 10.0

$$C_x = Rd_y = 384.773458895502(-0.447213595) = -172.075922$$

$$C_y = -Rd_x = -384.773458895502(0.894427191) = -344.151844$$

$$dx = \cos(\theta_{start}) = \cos(-0.463647609) = 0.894427191$$

$$dy = \sin(\theta_{start}) = \sin(-0.463647609) = -0.447213595$$

The geometric representation is

```
#71 = IFCCURVESEGMENT(.CONTINUOUS., #77, IFCLENGTHMEASURE(426.001441657352),
IFCLENGTHMEASURE(-123.801073716741), #80);
#77 = IFCAXIS2PLACEMENT2D(#78, #79);
#78 = IFCCARTESIANPOINT((0., 10.));
#79 = IFCDIRECTION((8.94427190999916E-1, -4.47213595499958E-1));
#80 = IFCCIRCLE(#81, 384.773458895502);
#81 = IFCAXIS2PLACEMENT2D(#82, #83);
#82 = IFCCARTESIANPOINT((-172.075922005613, -344.151844011225));
#83 = IFCDIRECTION((1., 0.));
```

3.4.3 Compute Point on Curve

Compute the vertical placement matrix for the point at horizontal distance $\ell = 50$ m from the start of the curve segment.

Step 1 — Form the curve segment placement matrix M_{CSP}

From `IfcCurveSegment.Placement`: $(d_p,\ y_p) = (0.,10.)$, $dx_p = 0.894427190999916$, $dy_p = -0.447213595499958$

$$M_{CSP} = \begin{bmatrix} 0.894427190999916 & 0.447213595499958 & 0 & 0 \\ -0.447213595499958 & 0.894427190999916 & 0 & 10. \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Step 2 — Evaluate the parent curve at the trim start and form the normalization matrix M_N

$$\Delta_0 = \tan^{-1}\left(\frac{-344.151844011225}{-172.075922005613}\right) = 1.1071487177940900$$

$$x_0 = C_x + R\cos(\Delta_0) = -172.075922 + 384.773458895502\cos(1.1071487177940900) = 0$$

$$y_0 = C_y + R\sin(\Delta_0) = -344.151844 + 384.773458895502\sin(1.1071487177940900) = 0$$

$$\theta_0 = \Delta_0 - \frac{\pi}{2} = -0.463647609$$

$$dx_0 = \cos(\theta_0) = 0.89442719099991$$

$$dy_0 = \sin(\theta_0) = -0.447213595499958$$

$$M_N = \begin{bmatrix} 0.89442719099991 & -0.447213595499958 & 0 & 0 \\ 0.447213595499958 & 0.89442719099991 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Step 3 — Evaluate and map each point in the vertical plane

Compute the vertical placement matrix for the point at horizontal distance $\ell = 50\ m$ from the start of the curve segment.

The center point of the trimmed circular arc is

$$C_x = -172.075922005613,\ C_y = -344.151844011225$$

From Step 2

$$x_0 = 0,\ y_0 = 0,\ \theta_0 = -0.463647609$$

Compute the chord length, c between the start and end of the trimmed segment. This is accomplished by locating the point on the parent curve that corresponds to a horizontal distance of $\ell = 50\ m$. From Figure 3.4.3-1 the point on the parent curve is located using the right triangle shown. Once the point on the parent curve is known, a chord distance between the start of the trim and this point can be computed (see inset chord line detail). From here a sweep angle Δ is computed and then the tangent direction at the point.

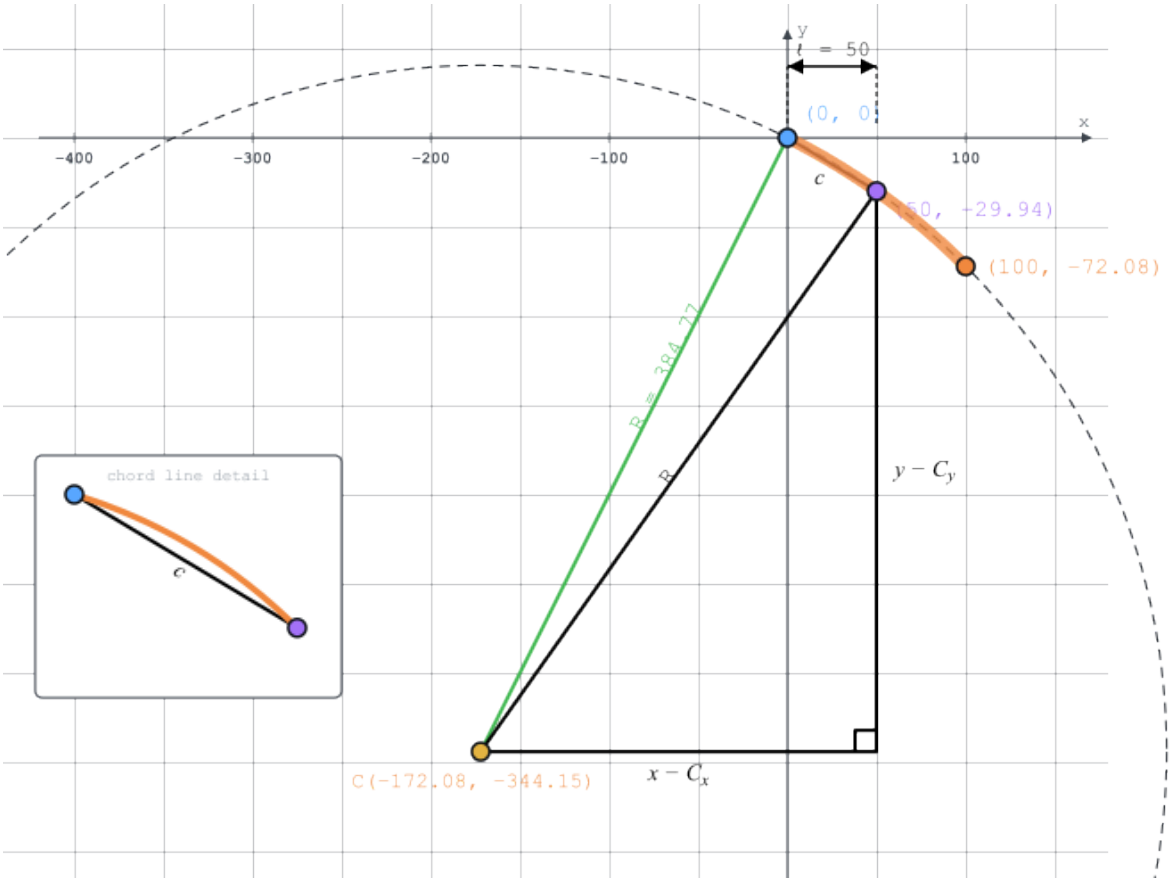


Figure 3.4.3-1 — Geometry of a vertical circular arc. The chord c connects two points on the arc. The right triangle simplifies locating points on the arc. The inset highlights the distinction between the straight chord and the curved arc.

Location point on parent curve corresponding to $\ell = 50$

$$x = x_0 + \ell = 0 + 50 = 50\ m$$

From the right triangle

$$R^2 = (x - C_x)^2 + (y - C_y)^2$$

Solving for y,

$$y = C_y - \frac{L}{|L|}\sqrt{R^2 - (x - C_x)^2}$$

$$= -344.151844011225$$
$$- \frac{-123.801073716741}{|-123.801073716741|}\sqrt{(384.773458895502)^2 - (50 - (-172.075922005613))^2}$$
$$= -29.933926737614911$$

Compute the chord length from the trim start to the point

$$c = \sqrt{(x - x_0)^2 + (y - y_0)^2} = \sqrt{(50 - (0))^2 + (-29.933926737614911 - 10)^2}$$
$$= 58.275552077461235\ m$$

Angle subtended by the trimmed segment

$$\Delta = 2\sin^{-1}\left(\frac{c}{2R}\right) = 2\sin^{-1}\left(\frac{58.275552077461235}{2 \cdot 384.773458895502}\right) = 0.15159931828379261$$

Arc-length of the trimmed segment = $R\Delta =$
$(384.773458895502)(0.15159931828379261) = 58.331394062255001$

Radial angle at $\ell = 50\ m$

$$\Delta_{pc} = \Delta_0 - \Delta = 1.1071487177940900 - 0.15159931828379261$$
$$= 0.95554939951029738$$

Compute point on trimmed segment at ℓ.

$$x_{pc} = C_x + R\cos(\Delta_{pc})$$
$$= (-172.075922005613)$$
$$+ 384.77345889550202\cos(0.95554939951029738) = 50$$

$$y_{pc} = C_y + R\sin(\Delta_{pc})$$
$$= (-344.15184401122502)$$
$$+ 384.77345889550202\sin(0.95554939951029738)$$
$$= -29.933926737614570$$

$$dx_{pc} = -\frac{L}{|L|}\sin(\Delta_{pc}) = -\frac{-123.801073716741}{|-123.801073716741|}\sin(0.95554939951029738)$$
$$= 0.81663095520043849$$

$$dy_{pc} = \frac{L}{|L|}\cos(\Delta_{pc}) = \frac{-123.801073716741}{|-123.801073716741|}\cos(0.95554939951029738)$$
$$= -0.57716018834325311$$

$$M_{PC} = \begin{bmatrix} 0.81663095520043849 & 0.57716018834325311 & 0 & 50 \\ -0.57716018834325311 & 0.81663095520043849 & 0 & -29.933926737614570 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

The vertical placement matrix is:

$$M_v = M_{CSP}\, M_N\, M_{PC}$$

$$= \begin{bmatrix} 0.894427190999916 & 0.447213595499958 & 0 & 0 \\ -0.447213595499958 & 0.894427190999916 & 0 & 10. \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 0.89442719099991 & -0.4472135954999 \\ 0.447213595499958 & 0.8944271909999 \\ 0 & 0 \\ 0 & 0 \end{bmatrix}$$

$$M_v = \begin{bmatrix} 0.81663096 & 0.57716019 & 0. & 50. \\ -0.57716019 & 0.81663096 & 0. & -19.93392674 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

3.5 Clothoid

Vertical clothoid transition curves are not addressed in this guide. The available reference implementation (bSI-RailwayRoom/IFC-Rail-Unit-Test-Reference-Code) produces degenerate results for this case — the generated IFC files have zero-length vertical alignments and a clothoid constant of zero, making them unsuitable as reference material. Until a correct reference or authoritative worked example is available, the geometric mapping for `IfcClothoid` in a vertical context remains undocumented here. An example of a file generated from the reference implementation is https://github.com/bSI-RailwayRoom/IFC-Rail-Unit-Test-Reference-Code/blob/master/alignment_testset/IFC-WithGeneratedGeometry/GENERATED__VerticalAlignment_Clothoid_100.0_10.0_0.5_0.0_1_Meter.ifc.

The mapping from the semantic curve definition to the geometric definition is shown in this IFC coding.

```
// Semantic definition
#41 = IFCALIGNMENTVERTICAL('1FNFyDAJeHwv87wDZHIYI1', $, $, $, $, #89, $);
#42 = IFCALIGNMENTSEGMENT('1FNFyHAJeHwuDtwDZHIYI2', #3, $, $, $, #72, #75,
#44);
#43 = IFCRELNESTS('4CGecNrjCHwxOSbERtTLTf', $, $, $, #41, (#42));
#44 = IFCALIGNMENTVERTICALSEGMENT($, $, 0., 100., 10., 5.E-1, 0., $,
.CLOTHOID.);

// Geometric definition
#70 = IFCGRADIENTCURVE((#71), .F., #45, #86);
#71 = IFCCURVESEGMENT(.CONTINUOUS., #77, IFCLENGTHMEASURE(0.),
IFCLENGTHMEASURE(0.), #80);
#77 = IFCAXIS2PLACEMENT2D(#78, #79);
#78 = IFCCARTESIANPOINT((0., 10.));
#79 = IFCDIRECTION((8.94427190999916E-1, 4.47213595499958E-1));
#80 = IFCCLOTHOID(#81, 0.); // <--- degenerate clothoid
#81 = IFCAXIS2PLACEMENT2D(#82, $);
#82 = IFCCARTESIANPOINT((0., 0.));
```

3.6 Parabolic Arc

The most common transition curve in a vertical profile is a parabola. The geometric representation is `IfcPolynomialCurve`. Mapping of the semantic definition to the geometric definition can be a bit tricky.

3.6.1 Parent Curve Parametric Equations

The general form of a parabola is

$$y(x) = A_2x^2 + A_1x + A_0$$

where:

A_2 = (end gradient - start gradient)/(2 * horizontal length)

A_1 = start gradient

A_0 = start height

The gradient of the curve is

$$y'(x) = 2A_2x + A_1$$

3.6.2 Semantic Definition to Geometry Mapping

Consider a 1600 m parabolic vertical curve that starts 1200 m along the horizontal alignment. The entry grade is 1.75% and the exit grade is -1%. The elevation at the start of the curve is 121 m.

The semantic definition is

```
#289=IFCALIGNMENTVERTICALSEGMENT($,$,1200.,1600.,121.,0.017500000000000002,-
0.01,$,.PARABOLICARC.);
```

Compute the polynomial curve coefficients

$$A_0 = 121.\ m$$

$$A_1 = 0.0175$$

$$A_2 = \frac{-0.01 - 0.0175}{2 \cdot 1600.} = -8.59375 \cdot 10^{-6}\ m^{-1}$$

It is easiest to place the parent curve at the origin and orient it with the global coordinate system. The parent curve is defined as

```
#291=IFCCARTESIANPOINT((0.,0.));
#292=IFCDIRECTION((1.,0.));
#293=IFCAXIS2PLACEMENT2D(#291,#292);
#294=IFCPOLYNOMIALCURVE(#293,(0.,1.),(121.,0.017500000000000002,-
8.5937500000000005E-06),$);
```

Note: The coefficients A_0, A_1, and A_2 must have the following unit of measure, consistent with the project units:

$$A_0 = Length^1$$
$$A_1 = Length^0$$
$$A_2 = Length^{-1}$$

The coefficients of `IfcPolynomialCurve` expect real numbers without explicit unit of measure. This is a problem with the IFC Specification. See the discussion of `IfcAlignmentHorizontalSegment` and `IfcPolynomialCurve` for Cubic Transition Curve in Chapter 2 - Horizontal Alignments. Implicit units of measure are required for the polynomial coefficients.

The polynomial curve is trimmed using `IfcCurveSegment.SegmentStart` and `IfcCurveSegment.SegmentLength`. These parameters are measured along the length of the curve. The horizontal projection of the segment length, from `IfcAlignmentVerticalSegment.HorizontalLength` is $h_l = 1600$. `SegmentLength` will be slightly longer because it is the distance along the curve.

The distance along a curve is

$$s(x) = \int (\sqrt{(y')^2 + 1})dx$$

The length along the parabolic curve is then:

$$s(x) = \int \sqrt{4A_2^2x^2 + 4A_2A_1x + A_1^2 + 1}\, dx$$

The solution to this integral is:

$$s(x) = \int \left(\sqrt{ax^2 + bx + c}\right) dx$$
$$= \frac{b + 2ax}{4a}\sqrt{ax^2 + bx + c} + \frac{4ac - b^2}{8a^{\frac{3}{2}}}\ln\left|2ax + b + 2\sqrt{a(ax^2 + bx + c)}\right|$$

The length along the curve is $L_c = s(h_l) - s(0.0)$.

Let $\quad a = 4A_2^2 \quad b = 4A_2A_1 \quad c = A_1^2 + 1$

Compute the coefficients a, b, c, substitute curve length equation and solve. The curve length is

$$L_c = 1600.0616641340894$$

The placement of the trimmed curve segment is noteworthy. From the `IfcAlignmentVerticalSegment` attributes, the parabolic segment of the vertical alignment starts at 1200 from the beginning of the horizontal alignment and is at an elevation of 121. The RefDirection vector at the start of the curve is needed as well.

The gradient is $y'(x = 0) = 0.0175$

$$\theta_p = \tan^{-1}(0.0175) = 0.017498213869856595$$

$$dx = \cos(0.017498213869856595) = 0.017497320927833689$$

$$dy = \sin(0.017498213869856595) = 0.99984691016192495$$

The geometric representation is

```
#291=IFCCARTESIANPOINT((0.,0.));
#292=IFCDIRECTION((1.,0.));
#293=IFCAXIS2PLACEMENT2D(#291,#292);
#294=IFCPOLYNOMIALCURVE(#293,(0.,1.),(121.,0.017500000000000002,-
8.5937500000000005E-06),$);
#295=IFCCARTESIANPOINT((1200.,121.));
#296=IFCDIRECTION((0.99984691016192495,0.017497320927833689));
#297=IFCAXIS2PLACEMENT2D(#295,#296);
#298=IFCCURVESEGMENT(.CONTSAMEGRADIENT.,#297,IFCLENGTHMEASURE(0.),IFCLENGTHME
ASURE(1600.0616641340894),#294);
```

3.6.3 Compute Point on Curve

Compute the vertical placement matrix for the point at horizontal distance 1500 m from the start of the alignment. This vertical alignment segments starts at 1200 from the start of the alignment. The evaluation point is $\ell = 1500 - 1200 = 300$ from the start of the segment.

Step 1 — Form the curve segment placement matrix M_{CSP}

From `IfcCurveSegment.Placement`: $(d_p,\ y_p) = (1200,\ 121), dx_p = 0.99984691016192495, dy_p = 0.017497320927833689$

$$M_{CSP} = \begin{bmatrix} 0.99984691016192495 & -0.017497320927833689 & 0 & 1200 \\ 0.017497320927833689 & 0.99984691016192495 & 0 & 121 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Step 2 — Evaluate the parent curve at the trim start and form the normalization matrix M_N

Because the parent curve is located at (0,0) in the direction (1,0), $x_0 = 0,\ y_0 = 0,\ \theta_0 = 0.$

Since $x_0 = 0,\ y_0 = 0,\ \theta_0 = 0,\ M_N = \boldsymbol{I}$

Step 3 — Evaluate and map each point in the vertical plane

Evaluate the parent curve at $x = 300$

$$y(300) = (-8.59375 \cdot 10^{-6})300^2 + 0.0175(300) + 121 = 125.4765625$$

$$y'(300) = 2(-8.59375 \cdot 10^{-6})300 + 0.0175 = 0.01234375$$

$$dx = \frac{1}{\sqrt{(0.01234375)^2 + 1}} = 0.999923825$$

$$dy = \frac{0.01234375}{\sqrt{(0.01234375)^2 + 1}} = 0.01234281$$

$$M_{PC} = \begin{bmatrix} 0.999923825 & -0.01234281 & 0 & 300 \\ 0.01234281 & 0.999923825 & 0 & 125.4765625 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

The vertical placement matrix is:

$$M_v = M_{CSP}\, M_N\, M_{PC}$$

$$= \begin{bmatrix} 0.99984691016192495 & -0.017497320927833689 & 0 & 1200 \\ 0.017497320927833689 & 0.99984691016192495 & 0 & 121 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \boldsymbol{I} \begin{bmatrix} 0.999923825 & -0.01234281 & 0 \\ 0.01234281 & 0.999923825 & 0 \\ 0 & 0 & 1 \\ 0 & 0 & 0 \end{bmatrix}$$

$$M_v = \begin{bmatrix} 0.9999998299568322 & -0.0005831691921746294 & 0.0 & 1500.0 \\ 0.0005831691921746294 & 0.9999998299568322 & 0.0 & 125.4765625 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

3.7 Combined 3D

To complete evaluation of a point on the alignment, the 2D vertical is combined with the 2D horizontal resulting in a point in 3D space.

From line example in Section 2.3, the horizontal alignment point at $1500\ m$ is

$$M_h = \begin{bmatrix} 0.83925279 & 0.54374114 & 0 & 1758.879185 \\ -0.54374114 & 0.83925279 & 0 & 1684.3878387 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

From the parabolic arc example in Section 3.6, the corresponding vertical alignment point is

$$M_v = \begin{bmatrix} 0.9999998299568322 & -0.0005831691921746294 & 0.0 & 1500.0 \\ 0.0005831691921746294 & 0.9999998299568322 & 0.0 & 125.4765625 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Step 4 — Combine with the horizontal alignment point to produce the 3D placement matrix

Modify the M_v matrix into M_v' to put the vertical point into the same reference frame as the horizontal. Swap rows and columns 2 and 3. Set the distance along to 0.0.

$$M'_v = \begin{bmatrix} dx_v & 0 & -dy_v & 0 \\ 0 & 1 & 0 & 0 \\ dy_v & 0 & dx_v & z \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$= \begin{bmatrix} 0.9999998299568322 & 0.0 & -0.0005831691921746294 & 0.0 \\ 0 & 1 & 0 & 0 \\ 0.0005831691921746294 & 0 & 0.9999998299568322 & 125.4765625 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

The 3D placement matrix is:

$$M_{3D} = M_h \cdot M'_v$$

$$M_{3D}$$

$$= \begin{bmatrix} 0.83925279 & 0.54374114 & 0 & 1758.879185 \\ -0.54374114 & 0.83925279 & 0 & 1684.3878387 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 0.9999998299568322 & 0.0 & -0.0005831691 \\ 0 & 1 & 0 \\ 0.0005831691921746294 & 0 & 0.999999829 \\ 0 & 0 & 0 \end{bmatrix}$$

$$M_{3D}$$

$$= \begin{bmatrix} 0.8391888595725846 & 0.5437414408769801 & -0.010358737485349092 & 1758.879184955533 \\ -0.5437000211676682 & 0.8392527899703555 & 0.006711297136288405 & 1684.3878386845 \\ 0.012342809710188737 & 0.0 & 0.999923824622885 & 125.4765625 \\ 0.0 & 0.0 & 0.0 & 1.0 \end{bmatrix}$$

3.8 Zero-Length Closing Segment and `IfcGradientCurve.EndPoint`

The zero-length closing segment requirement described in Section 2.12 applies equally to `IfcGradientCurve`. The final `IfcAlignmentVerticalSegment` nested within `IfcAlignmentVertical` must have `SegmentLength = 0`, and the corresponding geometric counterpart in `IfcGradientCurve` must be a zero-length `IfcCurveSegment` placed at the endpoint of the profile with its tangent direction matching the end grade direction.

`IfcGradientCurve.EndPoint` is an *optional* `IfcCartesianPoint` attribute whose purpose overlaps with the zero-length segment: it encodes the 3D endpoint of the gradient curve, providing the same geometric information as the placement of the zero-length `IfcCurveSegment`. Because `EndPoint` is optional and the zero-length segment is required by the IFC Concept Templates, the zero-length segment is the primary, normative source of the endpoint geometry. When `EndPoint` is present, it must be consistent with the placement of the zero-length closing segment — a discrepancy between the two is a data error. Implementations should not rely on `EndPoint` as a substitute for the zero-length segment, nor assume it will be populated.

3.9 Summary and Implementation Checklist

#	Item	Notes
1	`SegmentStart` and `SegmentLength` are arc-length parameters, but vertical	Convert horizontal distance ℓ to arc-length s using the parent curve geometry

#	Item	Notes
	segments are evaluated at horizontal distance	before passing to the evaluator; for low grades the difference is small but non-zero
2	`SegmentLength` is negative for crest (cresting) circular arc curves	A negative `SegmentLength` indicates the arc is traversed clockwise; use its absolute value for length accumulation and preserve the sign for trim direction
3	Do not rely on `IfcAlignmentVerticalSegment.RadiusOfCurvature`	This attribute is optional and may be absent or inconsistent; always compute the radius from `HorizontalLength`, `StartGradient`, and `EndGradient`
4	Treat `IfcPolynomialCurve` coefficients as having implicit units of Length$^{(1-i)}$	For a parabolic arc, A_2 has implicit units of Length^{-1}; evaluate as $y(x) = A_2x^2 + A_1x + A_0$ where x is a length
5	When forming M_v' for Step 4, zero the distance-along component and swap rows/columns 2 and 3	This maps elevation from the vertical plane's row 2 to row 3 of the 3D matrix, placing it on the Z axis where M_h expects it
6	Include a zero-length `IfcCurveSegment` at the end of `IfcGradientCurve`; treat `IfcGradientCurve.EndPoint` as secondary	The zero-length segment is required by the IFC Concept Templates and is the normative endpoint; `EndPoint` is optional — if present, it must agree with the zero-length segment placement

Chapter 4 — Cant Alignments

4.0 Introduction

Cant (also called superelevation) is the transverse inclination of a railway track cross-section: the elevation difference between the two railheads when one rail is raised above the other. By tilting the track into a curve, cant directs a component of gravitational force inward, partially offsetting the centrifugal acceleration experienced by vehicles negotiating the curve. The result is improved ride quality, reduced wheel–rail lateral forces, and the ability to operate at higher speeds through curves without exceeding comfort or safety limits.

Cant is measured as a length — the height difference between railheads — rather than as an angle. The conversion between the two depends on track gauge. `IfcAlignmentCant.RailHeadDistance` supplies the center-to-center distance between railheads, which is the lever arm for this conversion.

The geometric representation of a cant alignment is accomplished with `IfcSegmentedReferenceCurve`. As a subtype of `IfcCompositeCurve`, it is an end-to-start collection of `IfcCurveSegment` segments. Its `BaseCurve` is typically an `IfcGradientCurve`, which supplies the combined horizontal and vertical geometry to which the cant offsets are applied to produce the full 3D track centerline.

Table 4.0-1 maps each `IfcAlignmentCantSegment.PredefinedType` to its corresponding parent curve type used in the geometric representation.

Business Logic (`IfcAlignmentCantSegment.PredefinedType`)	Geometric Representation (`IfcCurveSegment.ParentCurve`)
`CONSTANTCANT`	`IfcLine`
`LINEARTRANSITION`	`IfcClothoid`
`HELMERTCURVE`	`IfcSecondOrderPolynomialSpiral`
`BLOSSCURVE`	`IfcThirdOrderPolynomialSpiral`
`COSINECURVE`	`IfcCosineSpiral`
`SINECURVE`	`IfcSineSpiral`
`VIENNESEBEND`	`IfcSeventhOrderPolynomialSpiral`

Table 4.0-1 — Mapping of business logic to geometric representation for cant alignment

This chapter covers:

- The cant segment types in Table 4.0-1 and the geometric curve types that represent them.
- Parametric equations and geometry mapping examples for the curve types in Table 4.0-1.

- An algorithm for composing cant, horizontal, and vertical matrices to produce a full 3D placement frame.
- An implementation checklist in Section 4.13.

4.1 Fundamentals

Each `IfcAlignmentCantSegment` is parameterized by distance along the horizontal alignment. The key semantic attributes are `StartDistAlong`, `SegmentLength` (the horizontal length of the cant segment), `StartCantLeft`, `EndCantLeft`, `StartCantRight`, and `EndCantRight`. From these, the deviating elevation D_s at the start and D_e at the end are derived as the averages of their respective left and right rail values (Section 4.1.3). Each cant curve segment uses `IfcAxis2Placement3D` for its placement, in contrast to the 2D placements used for horizontal and vertical segments, because cant carries an out-of-plane cross-slope rotation.

4.1.1 Cant Transition

Cant transitions occur when the segment start elevation differs from the segment start elevation of the next segment. The left section in Figure 4.1.1-1 shows the cross section of a rail line. At the start of the segment, right rail is elevated above the left rail. The y-ordinate of `IfcCurveSegment.Placement.Location` indicates the deviating elevation at the centerline between rails, which is one-half the cant value at the right rail. The y-ordinate of `IfcCurveSegment.Placement.Location` for the next segment indicates a value of 0.0 meaning that the cant transitioned from the starting value to zero over the length of the segment. This transition is the rotation of the right rail about the left rail, and the corresponding change in deviating elevation at the centerline over the length of the segment. The cant transition is shown in the right section of Figure 4.1.1-1 as a linear transition. `IfcCurveSegment.ParentCurve` defines how the transition occurs. Figure 4.1.1-2 shows a non-linear transition of the centerline and right rail deviating elevations. Notice that the deviating elevation of the left rail is constant at zero because it is the point of rotation.

When the segment start elevation is the same as the start of the next segment, the deviating elevation along the length of the segment is constant. This occurs when centrifugal forces are constant or zero, in constant radius curvature and straight sections of the railway, respectively.

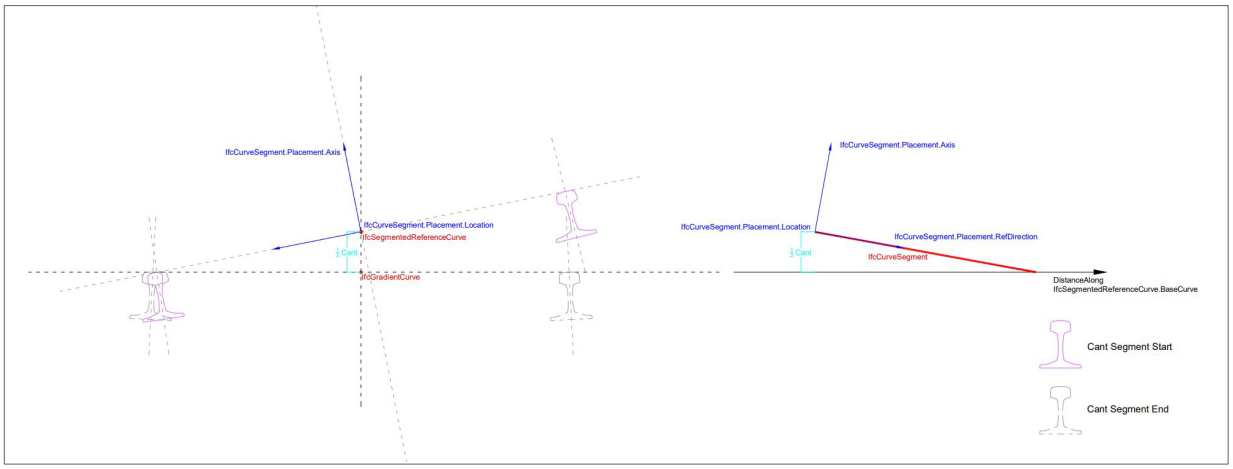


Figure 4.1.1-1 — cross section and elevation of a cant segment (source: bSI)

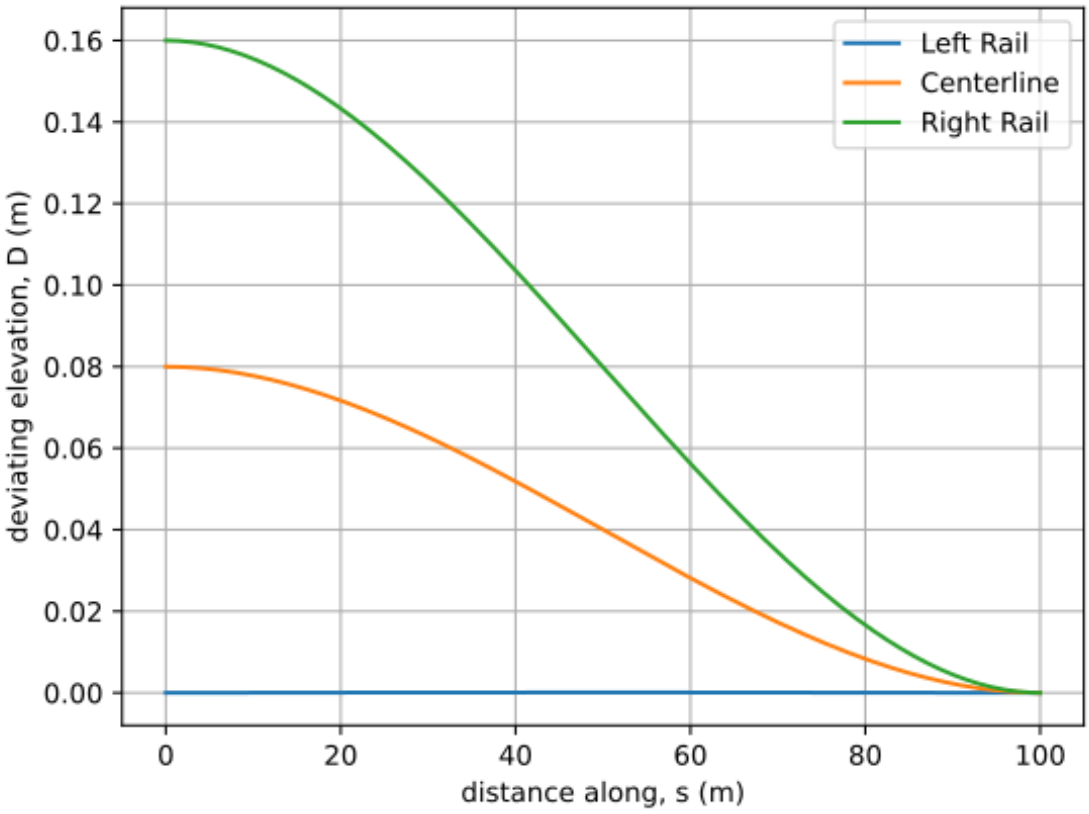


Figure 4.1.1-2 — Deviating elevation of rails

The railhead cross slope angle is defined by the `IfcCurveSegment.Placement.Axis` attribute. The `Axis` direction is generally upwards. The cross slope angle is measured from the y-axis and orients a vector that is perpendicular to the plane of the railheads. Figure

4.1.1-3 shows the cross slope angle for the transitions in Figure 4.1.1-2. The direction perpendicular to the plane of the railhead is about 1.46 rad at the start of the segment and increases to about 1.57 rad as the rotation decreases. When the left and right rails are at the same elevation the cross slope angle is $\frac{\pi}{2}$.

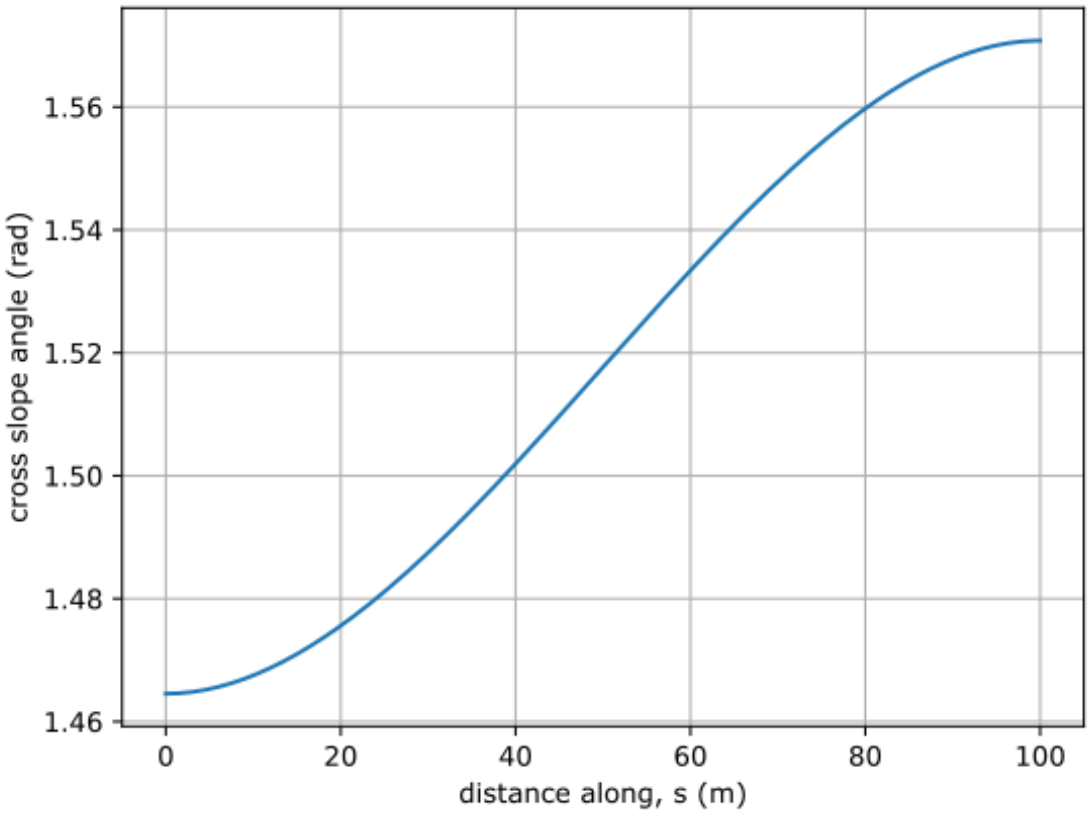


Figure 4.1.1-3 — Rail head cross slope

4.1.2 Coordinate System

Figure 4.1.1-1 illustrates the cant coordinate system.

`IfcCurveSegment.Placement.RefDirection` is tangent to the "distance along" the base curve in the horizontal plane, (e.g. `IfcSegmentedReferenceCurve.BaseCurve` => `IfcGradientCurve.BaseCurve` => `IfcCompositeCurve`).

The railway cross section is in the plane defined by `Y` and `Z`. Looking in the positive sense down the alignment, `Y` is towards the left and `Z` is upwards. The cross slope angle is measured clockwise from `Y`.

4.1.3 Transition Functions

The transition functions used to shape cant variation have the same functional form as functions used for horizontal transition curves: line, clothoid (linear cant), Helmert, Bloss, cosine, sine, and Viennese bend. In practice, the cant transition type always matches the horizontal transition type and segment length, though IFC does not enforce this constraint.

As an example, the horizontal tangent direction, $\theta(t)$, and the radius of curvature, $\kappa(t)$, for a Helmert curve is a second order polynomial of the form

$$\theta(t) = \frac{t^3}{3A_2^3} + \frac{A_1}{2|A_1^3|}t^2 + \frac{t}{A_0}$$

$$\kappa(t) = \frac{1}{A_2^3}t^2 + \frac{A_1}{|A_1^3|}t + \frac{1}{A_0}$$

The cant deviating elevation for a Helmert transition is

$$\frac{D(s)}{L^2} = \frac{1}{A_2^3}s^2 + \frac{A_1}{|A_1^3|}s + \frac{1}{A_0}$$

The deviating elevation has the same functional form as the radius of curvature of the horizontal alignment segment.

In IFC, the semantic cant profile is encoded in `IfcAlignmentCant` and its child `IfcAlignmentCantSegment` entities. The geometric representation is an `IfcSegmentedReferenceCurve`, which evaluates the cant at every point along the alignment and, in conjunction with the horizontal alignment `IfcCompositeCurve` and the vertical alignment `IfcGradientCurve`, produces the full 3D track centerline geometry.

Each `IfcCurveSegment` in an `IfcSegmentedReferenceCurve` is evaluated in a two-dimensional coordinate system whose axes are *distance along the horizontal alignment* ℓ and *deviating elevation* D. The deviating elevation is the vertical offset applied to the track centerline, away from the gradient curve, to produce the correct cross-slope angle at every point along the segment.

The `StartCantLeft` D_{sl}, `EndCantLeft` D_{el}, `StartCantRight` D_{sr}, and `EndCantRight` D_{er} attributes of `IfcAlignmentCantSegment` determine D_s and D_e as the averages of their respective left and right values:

$$D_s = \frac{D_{sl} + D_{sr}}{2}, \quad D_e = \frac{D_{el} + D_{er}}{2} \quad \Delta D = D_e - D_s$$

The cross-slope angle ϕ at a given point is not obtained directly from the parent curve equation; instead it is interpolated between the cross-slope angle of the segment's own `IfcCurveSegment.Placement.Axis` and that of the next segment's `Placement.Axis`, using the deviating elevation as the interpolation parameter:

$$\phi(\ell) = \phi_s + \left(\frac{\phi_e - \phi_s}{\Delta D}\right)(D(\ell) - D_s)$$

$$\phi_s = \cos^{-1}\left(\frac{D_{sr} - D_{sl}}{D_{rh}}\right)$$

where ϕ_s is the cross-slope angle of this segment's `Placement.Axis`, ϕ_e is that of the next segment's `Placement.Axis` (for the last segment, the zero-length closing segment of §4.12), D_s and ΔD are the deviating elevation of the parent curve at the start of the trimmed segment and its change over the segment, and $D(\ell)$ is the deviating elevation at ℓ. The cross-slope angle ϕ is the angle from the transverse y axis to the projection of the `Axis` vector onto the Y-Z plane. In sections without cant, $\phi = \frac{\pi}{2}$.

The cross-slope angle can be determined from the `Axis` vector as $\phi = \tan^{-1}\left(\frac{Axis.dz}{Axis.dy}\right)$

Together, the distance along, the deviating elevation $D(\ell)$, and the cross-slope angle $\phi(\ell)$ fully specify a 3D placement frame for the track centerline at each ℓ. Section 4.2 describes an algorithm for constructing that frame and composing it with the horizontal and vertical matrices to produce a 3D position.

All examples use a railhead distance $D_{rh} = 1.5\ m$.

4.2 Curve Segment Evaluation Algorithm

Cant segments are evaluated in a two-dimensional "distance along, deviating elevation" $(\ell, D(\ell))$ coordinate system in which ℓ is the distance measured along the horizontal `IfcCompositeCurve` and $D(\ell)$ is the deviating elevation: the vertical offset applied to the track centerline to accommodate the cross slope. Unlike horizontal and vertical segments, each cant point also carries a cross slope angle $\phi(\ell)$, making the local frame inherently three-dimensional.

Let s_0 = `IfcCurveSegment.SegmentStart`.

Step 1 — Form the curve segment placement matrix M_{CSP}

M_{CSP} is constructed from `IfcCurveSegment.Placement`: (s_p, D_p) is the `Location` (distance along, deviating elevation).

$$\boldsymbol{Y}_p = \frac{\boldsymbol{Axis}_p \times \boldsymbol{RefDir}_p}{\left|\boldsymbol{Axis}_p \times \boldsymbol{RefDir}_p\right|}$$

$$\boldsymbol{X}_p = \boldsymbol{Y}_p \times \boldsymbol{Axis}_p$$

`RefDir` need not be perpendicular to `Axis` (a cant segment's `RefDir` follows the slope of the deviating elevation while its `Axis` is tilted in the cross section), so $\boldsymbol{Y}_p$ is normalized as for any `IfcAxis2Placement3D`.

$$\boldsymbol{Z}_p = \boldsymbol{Axis}_p$$

The placement in matrix form is

$$M_{CSP} = \begin{bmatrix} X_p.x & Y_p.x & Z_p.x & s_p \\ X_p.y & Y_p.y & Z_p.y & D_p \\ X_p.z & Y_p.z & Z_p.z & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Step 2 — Evaluate the parent curve at the trim start - M_{PCS}

Compute the deviating elevation $D_s = D(s_0)$ and the slope angle $\theta_s = \tan^{-1}(D'(s_0))$.

Compute the cross slope

$$\phi(s_0) = \phi_s + \left(\frac{\phi_e - \phi_s}{\Delta D}\right)(D(s_0) - D_s)$$

Because $D(s_0) = D_s,\ \phi(s_0) = \phi_s$.

Construct the frame vectors

$$\boldsymbol{RefDir}_{pcs} = (\cos\theta_s,\ \sin\theta_s,\ 0)$$

$$\boldsymbol{Axis}_{pcs} = (0,\ \cos\phi_s,\ \sin\phi_s)$$

$$\boldsymbol{Y}_{pcs} = \boldsymbol{Axis}_{pcs} \times \boldsymbol{RefDir}_{pcs}$$

`RefDir` lies in the distance-along/deviating-elevation plane and `Axis` in the cross section. They are used as computed and are not re-orthogonalized, so the frame is not exactly orthonormal when both the slope and the cross slope are non-zero. This is how IfcOpenShell evaluates cant segments.

In matrix form

$$M_{PCS} = \begin{bmatrix} RefDir_{pcs}.x & Y_{pcs}.x & Axis_{pcs}.x & 0 \\ RefDir_{pcs}.y & Y_{pcs}.y & Axis_{pcs}.y & D_s \\ RefDir_{pcs}.z & Y_{pcs}.z & Axis_{pcs}.z & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Step 3 — Evaluate the parent curve at the point under consideration, ℓ - $M_{PC\ell}$

This step follows the same calculations as Step 2, except D, D' and ϕ are evaluated at ℓ, the distance from the start of the trimmed segment (parent curve parameter $s_0 + \ell$).

The resulting matrix is

$$M_{PC\ell} = \begin{bmatrix} RefDir_{\ell}.x & Y_{\ell}.x & Axis_{\ell}.x & \ell \\ RefDir_{\ell}.y & Y_{\ell}.y & Axis_{\ell}.y & D_{\ell} \\ RefDir_{\ell}.z & Y_{\ell}.z & Axis_{\ell}.z & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Step 4 — Compute the cant placement matrix M_c

The cant frame at ℓ is obtained by applying the incremental change in the parent curve — from the trim start to ℓ — to the curve segment placement. Because the 4×4 matrices encode both orientation and parameter-space coordinates (distance along, deviating elevation) in column 4, the rotation and translation must be computed separately to prevent the rotation from acting on the position coordinates.

Rotation

Let R_{CSP}, R_{PCS}, and $R_{PC\ell}$ denote the upper-left 3×3 rotation submatrix of M_{CSP}, M_{PCS}, and $M_{PC\ell}$ respectively. The incremental rotation from trim start to ℓ is:

$$\Delta R = R_{PC\ell} \cdot R_{PCS}^{T}$$

Apply the increment to the curve segment placement:

$$R_c = \Delta R \cdot R_{CSP} = R_{PC\ell} \cdot R_{PCS}^{T} \cdot R_{CSP}$$

Translation

Let $\boldsymbol{T}_{CSP}$, $\boldsymbol{T}_{PCS}$, and $\boldsymbol{T}_{PC\ell}$ denote column 4, rows 1–3, of M_{CSP}, M_{PCS}, and $M_{PC\ell}$ respectively. The incremental translation from trim start to ℓ is:

$$\Delta \boldsymbol{T} = \boldsymbol{T}_{PC\ell} - \boldsymbol{T}_{PCS}$$

Apply the increment to the curve segment placement:

$$\boldsymbol{T}_c = \Delta \boldsymbol{T} + \boldsymbol{T}_{CSP} = \boldsymbol{T}_{PC\ell} - \boldsymbol{T}_{PCS} + \boldsymbol{T}_{CSP}$$

Assemble

$$M_c = \begin{bmatrix} R_c & \boldsymbol{T}_c \\ \boldsymbol{0}^T & 1 \end{bmatrix}$$

Step 5 is performed immediately for this point before moving to the next ℓ.

Step 5 — Combine with horizontal and vertical to produce the 3D placement matrix

The cant result must be combined with the horizontal matrix M_h (evaluated at distance ℓ along the `IfcCompositeCurve`) and the vertical matrix M_v (evaluated at the same ℓ). All three coordinate systems share the distance-along axis, so the positional components of M_v and M_c must be extracted before multiplication and added back afterward to avoid incorrectly rotating the position offsets.

Construct M_v' as described in Section 3.2.

Construct M_c' by zeroing the distance-along ℓ component in row 1, column 4 and moving the deviating elevation D_ℓ from row 2 to row 3 in column 4 of M_c:

$$M'_c = \begin{bmatrix} X.x & Y.x & Z.x & 0 \\ X.y & Y.y & Z.y & 0 \\ X.z & Y.z & Z.z & D_\ell \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Extract position vectors from each modified matrix, $M'_v,\ M'_c$, (setting row 4 to zero), then zero column 4 before multiplying:

$$P_v = M'_v \text{ column 4, row 4 set to 0} = \begin{bmatrix} 0 \\ 0 \\ z \\ 0 \end{bmatrix}, \qquad P_c = M'_c \text{ column 4, row 4 set to 0} = \begin{bmatrix} 0 \\ 0 \\ D \\ 0 \end{bmatrix}$$

$$M''_v = M'_v \text{ with column 4 set to } (0,0,0,1)^T, \qquad M''_c = M'_c \text{ with column 4 set to } (0,0,0,1)^T$$

Multiply the three orientation matrices, then add back both position offsets:

$$M' = M_h \cdot M''_v \cdot M''_c$$

$$M_{3Dcant} = M' + \begin{bmatrix} 0 & 0 & 0 & P_{v.x} \\ 0 & 0 & 0 & P_{v.y} \\ 0 & 0 & 0 & P_{v.z} \\ 0 & 0 & 0 & 0 \end{bmatrix} + \begin{bmatrix} 0 & 0 & 0 & P_{c.x} \\ 0 & 0 & 0 & P_{c.y} \\ 0 & 0 & 0 & P_{c.z} \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

Column 4 of the final matrix M_{3Dcant} is the full 3D position of the track centerline. Column 3 is the cross-slope direction, encoding the railhead superelevation at that point.

Numerical examples of Steps 1–4 are given in Sections 4.3 through 4.9. A complete worked example including Step 5 is in Section 4.10.

4.3 Constant Cant

Cant has a constant deviating elevation and cross-slope angle in rail segments between curves or in the circular portion of curves where the tracks are banked at a constant rate.

The parent curve is `IfcLine`. The slope-intercept form of a line is $y = mx + b$, where m is the slope and b is the y-intercept. The derivative is $y' = m$, and the second derivative is $y'' = 0$.

In the context of cant, $D(s) = y'(x)$, so the deviating elevation is a constant value, m, and the rate of change of the deviating elevation $D'(s) = y''(x) = 0$.

4.3.1 Parent Curve Parametric Equations

The deviating elevation and its rate of change are given by the following equations.

$$D(s) = D_s = D_e$$

$$\frac{d}{ds} D(s) = D'(s) = 0.0$$

4.3.2 Semantic Definition to Geometry Mapping

Consider 100 m long segment of a railway that is turning towards the left. The right rail is raised 0.16 m through the circular portion of the curve. The semantic definition is

```
#64 = IFCALIGNMENTCANTSEGMENT($, $, 0., 100., 0., 0., 0.16, 0.16,
.CONSTANTCANT.);
```

$$D_s = \frac{0.0\ m + 0.16\ m}{2} = 0.08\ m$$

$$D_e = \frac{0.0\ m + 0.16\ m}{2} = 0.08\ m$$

$$\Delta D = D_e - D_s = 0.08\ m - 0.08\ m = 0.0\ m$$

The parent curve has a constant deviating elevation at $0.08\ m$ and a slope of 0.0.

Define the parent curve as an `IfcLine` passing through (0,0) in the direction (1,0).

```
#96=IFCCARTESIANPOINT((0.,0.));
#97=IFCDIRECTION((1.,0.));
#98=IFCVECTOR(#97,1.);
#99=IFCLINE(#96,#98);
```

The `IfcCurveSegment` trims a segment from the parent curve. The curve segment placement must capture the cross-slope rotation between the rail heads. For this reason the placement is an `IfcAxis2Placement3D` (instead of `IfcAxis2Placement2D` used for horizontal and vertical representations).

The cross-slope at the start of the segment is

$$\phi_s = \cos^{-1}\left(\frac{D_{sr} - D_{sl}}{D_{rh}}\right) = \cos^{-1}\left(\frac{0.16 - 0.0}{1.5}\right) = 1.463926346$$

The cross slope orientation is

$$dy_z = \cos(\phi_s) = \cos(1.463926346) = 0.106666667$$

$$dz_z = \sin(\phi_s) = \sin(1.463926346) = 0.994294837$$

```
#100=IFCCARTESIANPOINT((0.,0.08,0.));
#101=IFCDIRECTION((0.,0.10666666666666667,0.99429483666678176));
#102=IFCDIRECTION((1.,0.,0.));
#103=IFCAXIS2PLACEMENT3D(#100,#101,#102);
#104=IFCCURVESEGMENT(.CONTSAMEGRADIENTSAMECURVATURE.,#103,IFCLENGTHMEASURE(0.
),IFCLENGTHMEASURE(100.),#99);
```

4.3.3 Compute Point on Curve

Compute the placement matrix for a point $\ell = 50\ m$ from the start of the curve segment using the algorithm in Section 4.2.

Step 1 — Form the curve segment placement matrix M_{CSP}

$$\boldsymbol{RefDir}_p = (1,\ 0,\ 0)$$

$$\boldsymbol{Axis}_p = (0,\ 0.106667,\ 0.994295)$$

$$\boldsymbol{Y}_p = \boldsymbol{Axis}_p \times \boldsymbol{RefDir}_p = (0,\ 0.994295,\ -0.106667)$$

$$M_{CSP} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0.994295 & 0.106667 & 0.08 \\ 0 & -0.106667 & 0.994295 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Step 2 — Evaluate the parent curve at the trim start — M_{PCS}

$$s_0 = 0,\ D(0) = 0\ m,\ D'(0) = 0,\ \theta_s = 0$$

For constant cant $\Delta D = 0$, so $\phi(s_0) = \phi_s = \phi_e = 1.463926346$.

$$\boldsymbol{RefDir}_s = (1,\ 0,\ 0)$$

$$\boldsymbol{Axis}_s = (0,\ 0.106667,\ 0.994295)$$

$$\boldsymbol{Y}_s = \boldsymbol{Axis}_s \times \boldsymbol{RefDir}_s = (0,\ 0.994295,\ -0.106667)$$

$$M_{PCS} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0.994295 & 0.106667 & 0 \\ 0 & -0.106667 & 0.994295 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Step 3 — Evaluate the parent curve at $\ell = 50\ m$ — $M_{PC\ell}$

$$D(50\ m) = 0\ m,\ D'(50\ m) = 0,\ \theta_\ell = 0$$

Since $\Delta D = 0$, $\phi(50\ m) = \phi_s = 1.463926346$.

$$\boldsymbol{RefDir}_\ell = (1,\ 0,\ 0)$$

$$\boldsymbol{Axis}_\ell = (0,\ 0.106667,\ 0.994295)$$

$$\boldsymbol{Y}_\ell = \boldsymbol{Axis}_\ell \times \boldsymbol{RefDir}_\ell = (0,\ 0.994295,\ -0.106667)$$

$$M_{PC\ell} = \begin{bmatrix} 1 & 0 & 0 & 50 \\ 0 & 0.994295 & 0.106667 & 0 \\ 0 & -0.106667 & 0.994295 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Step 4 — Compute the cant placement matrix M_c

Rotation

$$R_{CSP} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 0.994295 & 0.106667 \\ 0 & -0.106667 & 0.994295 \end{bmatrix}$$

$$R_{PCS} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 0.994295 & 0.106667 \\ 0 & -0.106667 & 0.994295 \end{bmatrix}$$

$$R_{PC\ell} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 0.994295 & 0.106667 \\ 0 & -0.106667 & 0.994295 \end{bmatrix}$$

$$R_c = R_{PC\ell} \cdot R_{PCS}^T \cdot R_{CSP}$$

$$= \begin{bmatrix} 1 & 0 & 0 \\ 0 & 0.994295 & 0.106667 \\ 0 & -0.106667 & 0.994295 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 \\ 0 & 0.994295 & -0.106667 \\ 0 & 0.106667 & 0.994295 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 \\ 0 & 0.994295 & 0.106667 \\ 0 & -0.106667 & 0.994295 \end{bmatrix}$$

$$R_c = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 0.994295 & 0.106667 \\ 0 & -0.106667 & 0.994295 \end{bmatrix}$$

Translation

$$\boldsymbol{T_c} = \boldsymbol{T_{CSP}} + \boldsymbol{T_{PC\ell}} - \boldsymbol{T_{PCS}}$$

$$\boldsymbol{T_{CSP}} = (0,\ 0.08,\ 0)^T$$

$$\boldsymbol{T_{PC\ell}} = (50,\ 0,\ 0)^T$$

$$\boldsymbol{T_{PCS}} = (0,\ 0,\ 0)^T$$

$$\boldsymbol{T_c} = (0,\ 0.08,\ 0)^T + (50,\ 0,\ 0)^T - (0,\ 0,\ 0)^T$$

$$\boldsymbol{T_c} = (50,\ 0.08,\ 0\ ,\ 0)^T$$

Assemble

$$M_c = \begin{bmatrix} R_c & \boldsymbol{T}_c \\ \boldsymbol{0}^T & 1 \end{bmatrix}$$

$$M_c = \begin{bmatrix} 1 & 0 & 0 & 50 \\ 0 & 0.994295 & 0.106667 & 0.08 \\ 0 & -0.106667 & 0.994295 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

4.4 Linear Transition

A linear transition in cant is represented with an `IfcClothoid`.

4.4.1 Parent Curve Parametric Equations

The deviating elevation and its rate of change are given by the following equations.

$$\frac{D(s)}{L^2} = \frac{1}{A_0} + \frac{A_1}{|A_1^3|}s$$

$$\frac{d}{ds}D(s) = L^2\frac{A_1}{|A_1^3|}$$

$$a_0 = D_s,\ A_0 = \frac{L^{2/1}}{a_0^1}\frac{a_0}{|a_0|},\ A_0 = \frac{L^2}{a_0}$$

$$a_1 = \Delta D,\ A_1 = \frac{L^{\frac{3}{2}}}{|a_1|^{\frac{1}{2}}}\frac{a_1}{|a_1|}$$

A_1 is the clothoid constant.

4.4.2 Semantic Definition to Geometry Mapping

Consider an alignment segment that has a clothoid transition curve towards the left. The track banks at a constantly increasing rate resulting in a linear deviation in the right rail elevation relative to the left rail. The right rail raises 160 mm over the 100 m segment length.

The semantic representation of the cant alignment is

```
#64 = IFCALIGNMENTCANTSEGMENT($, $, 0., 100., 0., 0., 0.16, 0.,
.LINEARTRANSITION.);
```

Compute the parent curve parameters.

$$D_{sl} = 0\ m \quad D_{el} = 0\ m$$

$$D_{sr} = 0.16\ m \quad D_{er} = 0\ m$$

$$D_s = \frac{(0.16 + 0)}{2} = 0.08\ m$$

$$D_e = \frac{(0 + 0)}{2} = 0\ m$$

$$\Delta D = D_e - D_s = 0.0 - 0.08 = -0.08\ m$$

$$a_0 = D_s = 0.08\ m,\ A_0 = \frac{(100\ m)^2}{0.08\ m} = 125000\ m$$

$$a_1 = \Delta D = -0.08\ m \quad A_1 = \frac{(100\ m)^{\frac{3}{2}}}{|-0.08\ m|^{\frac{1}{2}}}\frac{-0.08\ m}{|-0.08\ m|} = -3535.533906\ m$$

The clothoid parent curve is

```
#96=IFCCARTESIANPOINT((0.,0.));
#97=IFCDIRECTION((1.,0.));
#98=IFCAXIS2PLACEMENT2D(#96,#97);
#99=IFCCLOTHOID(#98,-3535.533905932738);
```

The cant segment begins with a deviating elevation of

$$D(0\ m) = (100\ m)^2\left(\frac{1}{125000} + \frac{-3535.533906}{|-3535.533906|^3}(0\ m)\right) = 0.08\ m$$

The slope at the start of the segment is

$$D'(0\ m) = (100\ m)^2\frac{-3535.533906}{|-3535.533906^3|} = -0.0008$$

$$\theta_0 = \tan^{-1}(-0.0008) = -0.00079999$$

$$dx_x = \cos(\theta_0) = 0.999999680$$

$$dy_x = \sin(\theta_0) = -0.000799999744$$

The cross-slope at the start of the segment is

$$\phi_s = \cos^{-1}\left(\frac{D_{sr} - D_{sl}}{D_{rh}}\right) = \cos^{-1}\left(\frac{0.16 - 0.0}{1.5}\right) = 1.463926346$$

The cross slope orientation is

$$dy_z = \cos(\phi_s) = \cos(1.463926346) = 0.106666667$$

$$dz_z = \sin(\phi_s) = \sin(1.463926346) = 0.994294837$$

```
#100=IFCCARTESIANPOINT((0.,0.08,0.));
#101=IFCDIRECTION((0.,0.1066666666666667,0.99429483666678176));
#102=IFCDIRECTION((0.999999680,-0.000799999744,0.));
#103=IFCAXIS2PLACEMENT3D(#100,#101,#102);
#104=IFCCURVESEGMENT(.CONTINUOUS.,#103,IFCLENGTHMEASURE(0.),IFCLENGTHMEASURE(
100.),#99);
```

4.4.3 Compute Point on Curve

Compute the cant placement matrix for a point $\ell = 50\ m$ from the start of the curve segment using the algorithm in Section 4.2.

Step 1 — Form the curve segment placement matrix M_{CSP}

$$\boldsymbol{RefDir}_p = (0.9999996800001536,\ -0.0007999997440001227,\ 0)$$

$$\boldsymbol{Axis}_p = (0,\ 0.10666666666666667,\ 0.9942948366667818)$$

$$\boldsymbol{Y}_p = \frac{\boldsymbol{Axis}_p \times \boldsymbol{RefDir}_p}{|\boldsymbol{Axis}_p \times \boldsymbol{RefDir}_p|} = (0.0007954356177,\ 0.9942945221,\ -0.1066666329)$$

$$M_{CSP} = \begin{bmatrix} 0.999999683641039 & 0.00079543561769016 & 0 & 0 \\ -0.000790897527570178 & 0.9942945221127 & 0.106666666666667 & 0.08 \\ 8.48464658869504e-05 & -0.106666632921711 & 0.994294836666782 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Step 2 — Evaluate the parent curve at the trim start — M_{PCS}

`IfcClothoid` has no constant term, so the parent curve's deviating elevation is $D(s) = L^2 \frac{A_1}{|A_1^3|} s$, which is 0 at the trim start. The $0.08\ m$ deviating elevation at the start of the segment is carried by the placement `Location`, and only the change in D from the trim start enters M_c.

$$s_0 = 0,\ D(0) = 0\ m,\ D'(0) = -0.0008,\ \theta_s = \tan^{-1}(D'(0)) = -0.0007999998293$$

The cross slope at the trim start is that of the segment's `Placement.Axis`, $\phi_s = \tan^{-1}\left(\frac{Axis_p.dz}{Axis_p.dy}\right) = 1.463926346$.

$$\boldsymbol{RefDir}_s = (\cos\theta_s,\ \sin\theta_s,\ 0) = (0.99999968,\ -0.000799999744,\ 0)$$

$$\boldsymbol{Axis}_s = (0,\ \cos\phi_s,\ \sin\phi_s) = (0,\ 0.1066666667,\ 0.9942948367)$$

$$\boldsymbol{Y}_s = \boldsymbol{Axis}_s \times \boldsymbol{RefDir}_s = (0.0007954356148,\ 0.9942945185,\ -0.1066666325)$$

$$M_{PCS} = \begin{bmatrix} 0.999999680000154 & 0.000795435614794069 & 0 & 0 \\ -0.000799999744000123 & 0.994294518492587 & 0.106666666666667 & 0 \\ 0 & -0.10666663253335 & 0.994294836666782 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Step 3 — Evaluate the parent curve at $\ell = 50\ m$ — $M_{PC\ell}$

$$D(50\ m) = -0.04\ m,\ D'(50\ m) = -0.0008,\ \theta_\ell = \tan^{-1}(D'(50\ m)) = -0.0007999998293$$

The cross slope at the end of the segment is that of the next segment's `Placement.Axis`, $\phi_e = 1.570796327$, and the parent curve's deviating elevation changes by $\Delta D = D(100\ m) - D(0) = -0.08\ m$ over the segment.

$$\phi(\ell) = 1.463926346 + \left(\frac{1.570796327 - 1.463926346}{-0.08}\right)(-0.04 - 0) = 1.517361336$$

$$\boldsymbol{RefDir}_\ell = (\cos\theta_\ell,\ \sin\theta_\ell,\ 0) = (0.99999968,\ -0.000799999744,\ 0)$$

$$\boldsymbol{Axis}_\ell = (0,\ \cos\phi(\ell),\ \sin\phi(\ell)) = (0,\ 0.05340956531,\ 0.9985726906)$$

$$\boldsymbol{Y}_\ell = \boldsymbol{Axis}_\ell \times \boldsymbol{RefDir}_\ell = (0.0007988578968,\ 0.998572371,\ -0.05340954822)$$

$$M_{PC\ell}$$
$$= \begin{bmatrix} 0.999999680000154 & 0.00079885789681383 & 0 & 50 \\ -0.00079999744000123 & 0.998572371017288 & 0.0534095653134642 & -0.04 \\ 0 & -0.0534095482224115 & 0.998572690560395 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Step 4 — Compute the cant placement matrix M_c

Rotation

$$R_c = R_{PC\ell} \cdot R_{PCS}^T \cdot R_{CSP}$$

Translation

$$\boldsymbol{T}_c = \boldsymbol{T}_{PC\ell} - \boldsymbol{T}_{PCS} + \boldsymbol{T}_{CSP}$$

Assemble

$$M_c = \begin{bmatrix} R_c & \boldsymbol{T}_c \\ \boldsymbol{0}^T & 1 \end{bmatrix}$$

$$M_c$$
$$= \begin{bmatrix} 0.999999676359268 & 0.000798857893905279 & -8.53332787200349e-05 & 50 \\ -0.00079999741087415 & 0.998572367381599 & 0.0534096335800873 & 0.04 \\ 0 & -0.0534095480279535 & 0.998572690560395 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

4.5 Helmert Curve

Like the Helmert transition spiral, the Helmert cant semantic definition maps to two `IfcCurveSegment` instances for the geometric representation. The parent curve is `IfcSecondOrderPolynomialSpiral`.

4.5.1 Parent Curve Parametric Equations

The deviating elevation and its rate of change are given by the following equations.

$$\frac{D(s)}{L^2} = \frac{1}{A_2^3}s^2 + \frac{A_1}{|A_1^3|}s + \frac{1}{A_0}$$

$$\frac{d}{ds}D(s) = L^2\left(\frac{2s}{A_2^3} + \frac{A_1}{|A_1^3|}\right)$$

The polynomial coefficients carry a second subscript to indicate first half 1, and second half 2. For example, A_{21} is coefficient A_2 for the first half and A_{02} is coefficient A_0 for the second half.

In the first half of the cant transition

Constant Term:

$a_{01} = 4D_s$,

$$A_{01} = \frac{L^2}{|a_{01}|}\frac{a_{01}}{|a_{01}|}$$

Linear Term:

$a_{11} = 0$,

$$A_{11} = \frac{L^{\frac{3}{2}}}{|a_{11}|^{\frac{1}{2}}}\frac{a_{11}}{|a_{11}|} = 0$$

Quadratic Term:

$a_{21} = 8\Delta D$,

$$A_{21} = \frac{L^{4/3}}{|a_{21}|^{\frac{1}{3}}}\frac{a_{21}}{|a_{21}|}$$

In the second half

Constant Term:

$a_{02} = -4\Delta D + 4D_s$,

$$A_{02} = \frac{L^2}{|a_{02}|}\frac{a_{02}}{|a_{02}|}$$

Linear Term:

$a_{12} = 16\Delta D$,

$$A_{12} = \frac{L^{\frac{3}{2}}}{|a_{12}|^{\frac{1}{2}}}\frac{a_{12}}{|a_{12}|}$$

Quadratic Term:

$a_{22} = -8\Delta D$,

$$A_{22} = \frac{L^{\frac{4}{3}}}{|a_{22}|^{\frac{1}{3}}}\frac{a_{22}}{|a_{22}|}$$

4.5.2 Semantic Definition to Geometry Mapping

Consider an alignment segment that has a Helmert transition curve towards the left. The start cant is $160\ mm$ and transitions to zero over $100\ m$.

```
#64=IFCALIGNMENTCANTSEGMENT($,$,0.,100.,0.,0.,0.16,0.,.HELMERTCURVE.);
```

Compute the parent curve parameters

$$L = 100\ m$$

$$D_{sl} = 0\ m,\ D_{el} = 0\ m$$

$$D_{sr} = 0.16\ m,\ D_{er} = 0\ m$$

$$D_s = \frac{0 + 0.16\ m}{2} = 0.08\ m$$

$$D_e = \frac{0 + 0}{2} = 0.\ m$$

$$\Delta D = 0.0 - 0.08 = -0.08\ m$$

First half:

$$a_{01} = 4D_s = 4(0.08\ m) = 0.32\ m$$

$$A_{01} = \frac{L^2}{|a_{01}|}\frac{a_{01}}{|a_{01}|} = \frac{(100\ m)^2}{|0.32\ m|}\frac{0.32\ m}{|0.32\ m|} = 31250\ m$$

$$a_{11} = 0,\ A_{11} = 0$$

$$a_{21} = 8\Delta D = 8(-0.08\ m) = -0.64\ m$$

$$A_{21} = \frac{L^{4/3}}{|a_{21}|^{\frac{1}{3}}}\frac{a_{21}}{|a_{21}|} = \frac{(100\ m)^{\frac{4}{3}}}{|-0.64\ m|^{\frac{1}{3}}}\frac{-0.64\ m}{|-0.64\ m|} = -538.6086725\ m$$

The first half parent curve `IfcSecondOrderPolynomialSpiral` is

```
#104=IFCCARTESIANPOINT((0.,0.));
#105=IFCDIRECTION((1.,0.));
#106=IFCAXIS2PLACEMENT2D(#104,#105);
#107=IFCSECONDORDERPOLYNOMIALSPIRAL(#106,-538.60867250797071,$,31250.);
```

Determine the first half placement and trimming.

> **Note:** The length of the first half curve is $L_1 = 50\ m$, half the length of the full Helmert transition.

The spiral coefficients A_{01} and A_{21} are computed from the **full** transition length $L = 100\ m$. The evaluation formula, by contrast, multiplies by the curve segment length squared — $(L_1)^2 = (50\ m)^2$ — because $L_1 = 50\ m$ is the trim length of the first-half curve segment. These two lengths work together by design. The A coefficients fix the boundary conditions of the deviating elevation profile at $s = 0$, $s = L/2$, and $s = L$ relative to the complete $100\ m$ transition. Using $(L_1)^2$ in the evaluation then scales those boundary conditions correctly to physical distances within the trimmed half. Substituting $A_{01} = L^2/a_{01}$ confirms this:

$$\left(\frac{L}{2}\right)^2 \cdot \frac{a_{01}}{L^2} = \frac{a_{01}}{4} = \frac{4D_s}{4} = D_s$$

Figure 4.5.2-1 illustrates this geometrically. Each parent curve is drawn solid over the portion used by its curve segment and dashed beyond the trim boundary. The dashed extension of the first-half curve diverges from the second-half curve after $s = 50\ m$, confirming that the two polynomials are distinct — each is correct only within its own half.

The deviating elevation at the start of the first half transition is

$$D(0\ m) = (50\ m)^2\left(\frac{1}{31250\ m} + \frac{1}{(-538.6086725\ m)^3}(0\ m)^2\right) = 0.08\ m$$

$$D'(0\ m) = (50\ m)^2\left(\frac{2(0\ m)}{(-538.6086725\ m)^3}\right) = 0$$

$$\theta = 0,\ dx_x = 1, dy_x = 0$$

The cross-slope at the start of the segment is

$$\phi_s = \cos^{-1}\left(\frac{D_{sr} - D_{sl}}{D_{rh}}\right) = \cos^{-1}\left(\frac{0.16 - 0.0}{1.5}\right) = 1.463926346$$

The cross slope orientation is

$$dy_z = \cos(\phi_s) = \cos(1.463926346) = 0.106666667$$

$$dz_z = \sin(\phi_s) = \sin(1.463926346) = 0.994294837$$

```
#108=IFCCARTESIANPOINT((0.,0.08,0.));
#109=IFCDIRECTION((0.,0.1066666666666667,0.99429483666678176));
#110=IFCDIRECTION((1.,0.,0.));
#111=IFCAXIS2PLACEMENT3D(#108,#109,#110);
#112=IFCCURVESEGMENT(.CONTSAMEGRADIENTSAMECURVATURE.,#111,IFCLENGTHMEASURE(0.
),IFCLENGTHMEASURE(50.),#107);
```

Second half:

$$a_{02} = -4\Delta D + 4D_s = -4(-0.08\ m) + 4(0.08)\ m = 0.64\ m$$

$$A_{02} = \frac{L^2}{|a_{02}|}\frac{a_{02}}{|a_{02}|} = \frac{(100\ m)^2}{|0.64\ m|}\frac{0.64\ m}{|0.64\ m|} = 15625\ m$$

$$a_{12} = 16\Delta D = 16(-0.08\ m) = -1.28\ m$$

$$A_{12} = \frac{L^{\frac{3}{2}}}{|a_{12}|^{\frac{1}{2}}}\frac{a_{12}}{|a_{12}|} = \frac{(100\ m)^{\frac{3}{2}}}{|-1.28\ m|^{\frac{1}{2}}}\frac{-1.28\ m}{|-1.28\ m|} = -883.8834765\ m$$

$$a_{22} = -8\Delta D = -8(-0.08\ m) = 0.64\ m$$

$$A_{22} = \frac{L^{\frac{4}{3}}}{|a_{22}|^{\frac{1}{3}}} \frac{a_{22}}{|a_{22}|} = \frac{(100\ m)^{\frac{4}{3}}}{|0.64\ m|^{\frac{1}{3}}} \frac{0.64\ m}{|0.64\ m|} = 538.6086725\ m$$

Figure 4.5.2-1 shows the first and second half parent curves.

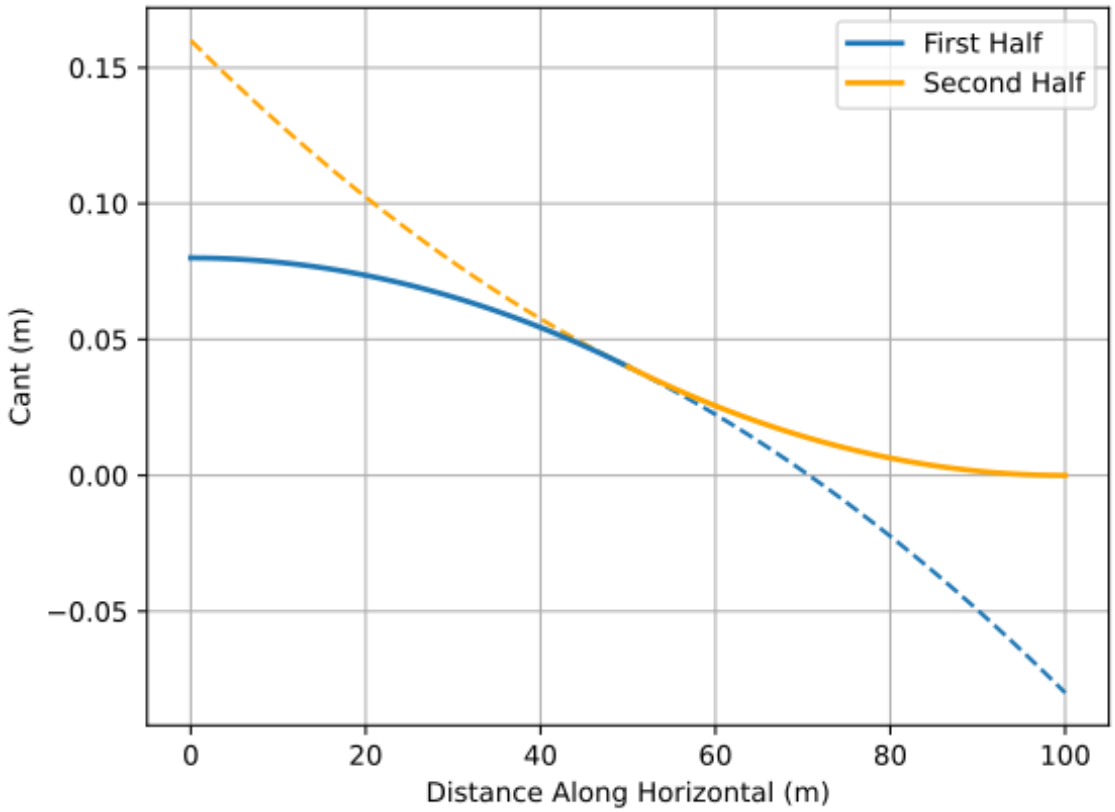


Figure 4.5.2-1 — `IfcSecondOrderPolynomialSpiral` parent curves. The dashed lines represent the projection of the parent curve over the full length of the semantic segment.

The second half parent curve `IfcSecondOrderPolynomialSpiral` is

```
#113=IFCCARTESIANPOINT((0.,0.));
#114=IFCDIRECTION((1.,0.));
#115=IFCAXIS2PLACEMENT2D(#113,#114);
#116=IFCSECONDORDERPOLYNOMIALSPIRAL(#115,538.60867250797071,-
883.8834764831845,15625.);
```

Determine the second half placement and trimming.

The starting elevation of the second half is the end elevation of the first half. Use the first half parent curve to determine the start of the second half curve.

$$D(50\ m) = (50\ m)^2\left(\frac{1}{31250\ m} + \frac{1}{(-538.6086725\ m)^3}(50\ m)^2\right) = 0.04\ m$$

$$D'(50\ m) = (50\ m)^2\left(\frac{2 \cdot 50\ m}{(-538.6086725\ m)^3}\right) = -0.0016$$

$$dx_x = \frac{1}{\sqrt{(-0.0016)^2 + 1}} = 0.999998724,\ dy_x = \frac{-0.0016}{\sqrt{(-0.0016)^2 + 1}} = -0.00159999795$$

The second half starts where the first half ends. The cross slope is interpolated linearly in the deviating elevation between the start and end cross slopes of the full transition (§4.2). The deviating elevation at the end of the first half is half way between D_s and D_e, so the cross slope there is half way, in angle, between them:

$$\phi(50\ m) = \frac{\phi_s + \phi_e}{2} = \frac{1.463926346 + \frac{\pi}{2}}{2} = 1.517361336$$

> **Note:** This is not the angle whose cosine is the cant ratio at the midpoint, $\cos^{-1}\left(\frac{0.08}{1.5}\right) = 1.517437677$. The two differ by $7.6 \times 10^{-5}\ rad$. The second half's `Axis` must use the interpolated angle so that the cross slope is continuous where the halves meet.

$$dy_z = \cos(\phi(50\ m)) = \cos(1.517361336) = 0.053409565$$

$$dz_z = \sin(\phi(50\ m)) = \sin(1.517361336) = 0.998572691$$

The trimming begins at $50\ m$ and progresses for a length of $50\ m$ for the second half segment.

```
#117=IFCCARTESIANPOINT((50.,0.04,0.));
#118=IFCDIRECTION((0.,0.05340956531005583,0.9985726905605775));
#119=IFCDIRECTION((0.9999987200024576,-0.0015999979520039342,0.));
#120=IFCAXIS2PLACEMENT3D(#117,#118,#119);
#121=IFCCURVESEGMENT(.CONTSAMEGRADIENTSAMECURVATURE.,#120,IFCLENGTHMEASURE(50
.),IFCLENGTHMEASURE(50.),#116);
```

4.5.3 Compute Point on Curve

Compute the cant placement matrix for a point $75\ m$ from the start of the curve segment using the algorithm in Section 4.2. $75\ m$ falls in the second half of the Helmert transition, $\ell = 75\ m - 50\ m = 25\ m$.

Step 1 — Form the curve segment placement matrix M_{CSP}

$$\boldsymbol{RefDir}_p = (0.9999987200024576,\ -0.0015999979520039342,\ 0.)$$

$$\boldsymbol{Axis}_p = (0.,\ 0.05340956531005583,\ 0.9985726905605775)$$

$$\boldsymbol{Y}_p = \frac{\boldsymbol{Axis}_p \times \boldsymbol{RefDir}_p}{\left|\boldsymbol{Axis}_p \times \boldsymbol{RefDir}_p\right|} = (0.001597714,\ 0.9985714,\ -0.0534095)$$

$$M_{CSP}$$

$$= \begin{bmatrix} 0.999998723653748 & 0.00159771426565771 & 0 & 50 \\ -0.00159543383300484 & 0.998571416036067 & 0.0534095653100558 & 0.04 \\ 8.53332244184533e-05 & -0.0534094971409574 & 0.998572690560578 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Step 2 — Evaluate the parent curve at the trim start — M_{PCS}

$$D_s = D(50\ m)$$
$$= (50\ m)^2 \left(\frac{1}{(538.6086725\ m)^3}(50\ m)^2 + \frac{-883.8834765\ m}{|(-883.8834765\ m)^3|}(50\ m) + \frac{1}{15625\ m}\right) = 0.04\ m$$

$$D'(s) = D'(50\ m) = (50\ m)^2 \left(\frac{(2)(50\ m)}{(538.6086725\ m)^3} + \frac{(-883.8834765\ m)}{|(-883.8834765\ m)^3|}\right) = -0.0016$$

$$dx_x = \frac{1}{\sqrt{(-0.0016)^2 + 1}} = 0.99999872$$

$$dy_x = \frac{-0.0016}{\sqrt{(-0.0016)^2 + 1}} = -0.001599998$$

Because $D(s_0) = D_s$, the cross slope is this segment's start cross slope, $\phi(s_0) = \phi_s = 1.517361336$.

$$\boldsymbol{RefDir}_s = (0.9999987200024576,\ -0.0015999979520039342,\ 0)$$

$$\boldsymbol{Axis}_s = (0,\ 0.05340956531005585,\ 0.9985726905605775)$$

$$\boldsymbol{Y}_s = \boldsymbol{Axis}_s \times \boldsymbol{RefDir}_s$$
$$= (0.0015977142598239782,\ 0.9985714123899877,\ -0.05340949694594351)$$

$$M_{PCS}$$

$$= \begin{bmatrix} 0.999998720002458 & 0.00159771425982398 & 0 & 0 \\ -0.00159999795200393 & 0.998571412389988 & 0.0534095653100558 & 0.0400000000000001 \\ 0 & -0.0534094969459435 & 0.998572690560578 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Step 3 — Evaluate the parent curve at $\ell = 25\ m$ — $M_{PC\ell}$

Add curve start $25 + 50 = 75\ m$.

$$D_\ell = D(75\ m)$$
$$= (50\ m)^2\left(\frac{1}{(538.6086725\ m)^3}(75\ m)^2 + \frac{-883.8834765\ m}{|(-883.8834765\ m)^3|}(75\ m) + \frac{1}{15625\ m}\right) = 0.01\ m$$

$$D'_\ell = D'(75\ m) = (50\ m)^2\left(\frac{(2)(75\ m)}{(538.6086725\ m)^3} + \frac{(-883.8834765\ m)}{|(-883.8834765\ m)^3|}\right) = -0.0008$$

$$dx_x = \cos(\theta) = 0.99999968000015360$$

$$dx_y = \sin(\theta) = -0.00079999974400011946$$

The second half runs from $D_s = 0.04\ m$ to $D_e = 0$, so $\Delta D = -0.04\ m$, and its cross slope runs from $\phi_s = 1.517361336$ to $\phi_e = \frac{\pi}{2}$.

$$\phi(\ell) = 1.517361336 + \left(\frac{\frac{\pi}{2} - 1.517361336}{-0.04}\right)(0.01 - 0.04) = 1.5574375791735484$$

$$dz_y = \cos(1.5574375791735484) = 0.013358350299808584$$

$$dz_z = \sin(1.5574375791735484) = 0.9999107732579281$$

$$\boldsymbol{RefDir}_\ell = (0.9999996800001536,\ -0.00079999997440001195,\ 0)$$

$$\boldsymbol{Axis}_\ell = (0,\ 0.013358350299808584,\ 0.9999107732579281)$$

$$\boldsymbol{Y}_\ell = \boldsymbol{Axis}_\ell \times \boldsymbol{RefDir}_\ell$$
$$= (0.00079999928362629304,\ 0.9999104532866342,\ -0.01335834602513854)$$

$$M_{PC\ell} = \begin{bmatrix} 0.999999680000154 & 0.00079999928362629304 & 0 & 25 \\ -0.00079999999744000119 & 0.999910453286634 & 0.0133583502998086 & 0.010000000000000 \\ 0 & -0.0133583460251385 & 0.999910773257928 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Step 4 — Compute the cant placement matrix M_c

Rotation

$$R_c = R_{PC\ell} \cdot R_{PCS}^T \cdot R_{CSP}$$

Translation

$$\boldsymbol{T}_c = \boldsymbol{T}_{PC\ell} - \boldsymbol{T}_{PCS} + \boldsymbol{T}_{CSP}$$

Assemble

$$M_c = \begin{bmatrix} R_c & \boldsymbol{T}_c \\ \boldsymbol{0}^T & 1 \end{bmatrix}$$

$$= \overset{M_c}{\begin{bmatrix} 0.99999967634886 & 0.000799928359708533 & -8.54551677678602e-05 & 75 \\ -0.000799999741079088 & 0.999910449635666 & 0.0133584186639428 & 0.0100000000 \\ 0 & -0.0133583459763633 & 0.999910773257928 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}}$$

4.6 Bloss Curve

A Bloss transition in cant is represented with an `IfcThirdOrderPolynomialSpiral`.

4.6.1 Parent Curve Parametric Equations

The deviating elevation and its rate of change are given by the following equations.

$$\frac{D(s)}{L^2} = \frac{A_3}{|A_3^5|}s^3 + \frac{1}{A_2^3}s^2 + \frac{A_1}{2|A_1^3|}s + \frac{1}{A_0}$$

$$\frac{d}{ds}D(s) = L^2\left(\frac{3A_3}{|A_3^5|}s^2 + \frac{2}{A_2^3}s + \frac{A_1}{2|A_1^3|}\right)$$

Constant Term

$$a_0 = D_s,\ A_0 = \frac{L^{\frac{2}{1}}}{|a_0|^1}\frac{a_0}{|a_0|} = \frac{L^2}{|a_0|}\frac{a_0}{|a_0|}$$

Linear Term

$$a_1 = 0,\ A_1 = 0$$

Quadratic Term

$$a_2 = 3\Delta D,\ A_2 = \frac{L^{\frac{4}{3}}}{|a_2|^{\frac{1}{3}}}\frac{a_2}{|a_2|}$$

Cubic Term

$$a_3 = -2\Delta D,\ A_3 = \frac{L^{\frac{5}{4}}}{|a_3|^{\frac{1}{4}}}\frac{a_3}{|a_3|}$$

4.6.2 Semantic Definition to Geometry Mapping

Consider an alignment segment that has a Bloss transition curve towards the left. The cant transitions from zero to 160 mm over 100 m.

```
#64=IFCALIGNMENTCANTSEGMENT($,$,0.,100.,0.,0.,0.,0.16,.BLOSSCURVE.);
```

Compute the parent curve parameters

$$D_s = \frac{0+0}{2} = 0.0\ m,\ D_e = \frac{0+0.16\ m}{2} = 0.08\ m,\ \Delta D = 0.08 - 0.0 = 0.08\ m$$

Constant Term

$$a_0 = 0\ m,\ A_0 = 0\ m$$

Linear Term

$$A_1 = 0\ m$$

Quadratic Term

$$a_2 = 3(0.08\ m) = 0.24\ m,\ A_2 = \frac{(100\ m)^{\frac{4}{3}}}{|0.24\ m|^{\frac{1}{3}}}\left(\frac{0.24\ m}{|0.24\ m|}\right) = 746.9007911\ m$$

Cubic Term

$$a_3 = -2\Delta D = -2(0.08\ m) = -0.16\ m,\ A_3 = \frac{(100\ m)^{\frac{5}{4}}}{|-0.16\ m|^{\frac{1}{4}}}\left(\frac{-0.16\ m}{|-0.16\ m|}\right) = -500\ m$$

The parent curve is

```
#96=IFCCARTESIANPOINT((0.,0.));
#97=IFCDIRECTION((1.,0.));
#98=IFCAXIS2PLACEMENT2D(#96,#97);
#99=IFCTHIRDORDERPOLYNOMIALSPIRAL(#98,-500.0,746.90079109286057,$,$);
```

$$D_0 = D(0\ m) = (100\ m)^2\left(\frac{-500\ m}{|(-500\ m)^5|}(0\ m)^3 + \frac{1}{(746.9007911\ m)^3}(0\ m)^2 + 0\ m\right) = 0.0\ m$$

$$D'(0) = 0,\ \theta_0 = 0$$

$$dx_x = \cos(\theta_0) = 1$$

$$dy_x = \sin(\theta_0) = 0$$

The cross-slope at the start of the segment is

$$\phi_s = \cos^{-1}\left(\frac{D_{sr} - D_{sl}}{D_{rh}}\right) = \cos^{-1}\left(\frac{0.0 - 0.0}{1.5}\right) = 1.57079633$$

The cross slope orientation is

$$dy_z = \cos(\phi_s) = \cos(1.57079633) = 0.0$$

$$dz_z = \sin(\phi_s) = \sin(1.57079633) = 1.0$$

```
#100=IFCCARTESIANPOINT((0.,0.,0.));
#101=IFCDIRECTION((0.,0.0,1.0));
```

```
#102=IFCDIRECTION((1.,0.,0.));
#103=IFCAXIS2PLACEMENT3D(#100,#101,#102);
#104=IFCCURVESEGMENT(.CONTSAMEGRADIENT.,#103,IFCLENGTHMEASURE(0.),IFCLENGTHME
ASURE(100.),#99);
```

4.6.3 Compute Point on Curve

Compute the cant placement matrix for a point $\ell = 50\ m$ from the start of the curve segment using the algorithm in Section 4.2.

Step 1 — Form the curve segment placement matrix M_{CSP}

$$\boldsymbol{RefDir}_p = (1,\ 0,\ 0)$$

$$\boldsymbol{Axis}_p = (0,\ 0,\ 1)$$

$$\boldsymbol{Y}_p = \frac{\boldsymbol{Axis}_p \times \boldsymbol{RefDir}_p}{\left|\boldsymbol{Axis}_p \times \boldsymbol{RefDir}_p\right|} = (0,\ 1,\ 0)$$

$$M_{CSP} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Step 2 — Evaluate the parent curve at the trim start — M_{PCS}

$$s_0 = 0,\ D(0) = 0\ m,\ D'(0) = 0,\ \theta_s = \tan^{-1}(D'(0)) = 0$$

The cross slope at the trim start is that of the segment's `Placement.Axis`, $\phi_s = \tan^{-1}\left(\frac{Axis_p.dz}{Axis_p.dy}\right) = 1.570796327$.

$$\boldsymbol{RefDir}_s = (\cos\theta_s,\ \sin\theta_s,\ 0) = (1,\ 0,\ 0)$$

$$\boldsymbol{Axis}_s = (0,\ \cos\phi_s,\ \sin\phi_s) = (0,\ 6.123233996e-17,\ 1)$$

$$\boldsymbol{Y}_s = \boldsymbol{Axis}_s \times \boldsymbol{RefDir}_s = (0,\ 1,\ -6.123233996e-17)$$

$$M_{PCS} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 6.12323399573677e-17 & 0 \\ 0 & -6.12323399573677e-17 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

The rotation part of M_{PCS} equals that of M_{CSP} because $\theta_s = 0$ and the parent curve starts with the placement's cross slope.

Step 3 — Evaluate the parent curve at $\ell = 50\ m$ — $M_{PC\ell}$

$$D(50\ m) = 0.04\ m,\ D'(50\ m) = 0.0012,\ \theta_\ell = \tan^{-1}(D'(50\ m)) = 0.001199999424$$

The cross slope at the end of the segment is that of the next segment's `Placement.Axis`, $\phi_e = 1.463926346$, and the parent curve's deviating elevation changes by $\Delta D = D(100\ m) - D(0) = 0.08\ m$ over the segment.

$$\phi(\ell) = 1.570796327 + \left(\frac{1.463926346 - 1.570796327}{0.08}\right)(0.04 - 0) = 1.517361336$$

$$\boldsymbol{RefDir}_\ell = (\cos\theta_\ell,\ \sin\theta_\ell,\ 0) = (0.99999928,\ 0.001199999136,\ 0)$$

$$\boldsymbol{Axis}_\ell = (0,\ \cos\phi(\ell),\ \sin\phi(\ell)) = (0,\ 0.05340956531,\ 0.9985726906)$$

$$\boldsymbol{Y}_\ell = \boldsymbol{Axis}_\ell \times \boldsymbol{RefDir}_\ell = (-0.001198286366,\ 0.9985719716,\ -0.05340952686)$$

$$M_{PC\ell} = \begin{bmatrix} 0.999999280000778 & -0.00119828636590682 & 0 & 50 \\ 0.00119999913600094 & 0.998571971589017 & 0.0534095653100558 & 0.0400000000000001 \\ 0 & -0.0534095268552104 & 0.998572690560578 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Step 4 — Compute the cant placement matrix M_c

Rotation

$$R_c = R_{PC\ell} \cdot R_{PCS}^T \cdot R_{CSP}$$

Translation

$$\boldsymbol{T}_c = \boldsymbol{T}_{PC\ell} - \boldsymbol{T}_{PCS} + \boldsymbol{T}_{CSP}$$

Assemble

$$M_c = \begin{bmatrix} R_c & \boldsymbol{T}_c \\ \boldsymbol{0}^T & 1 \end{bmatrix}$$

$$M_c = \begin{bmatrix} 0.999999280000778 & -0.00119828636590682 & 0 & 50 \\ 0.00119999913600094 & 0.998571971589017 & 0.0534095653100558 & 0.0400000000000001 \\ 0 & -0.0534095268552103 & 0.998572690560578 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

4.7 Cosine Curve

A Cosine transition in cant is represented with an `IfcCosineSpiral`.

4.7.1 Parent Curve Parametric Equations

The deviating elevation and its rate of change are given by the following equations.

$$\frac{D(s)}{L^2} = \frac{1}{A_0} + \frac{1}{A_1}\cos\left(\pi\frac{s}{L}\right)$$

$$\frac{d}{ds}D(s) = L^2\left(-\frac{\pi}{A_1 L}\sin\left(\pi\frac{s}{L}\right)\right)$$

Constant term,

$$a_0 = D_s + \frac{\Delta D}{2}$$

$$A_0 = L^2\frac{1}{a_0}\frac{a_0}{|a_0|}$$

Cosine term,

$$a_1 = -\frac{1}{2}\Delta D$$

$$A_1 = L^2\frac{1}{a_1}\frac{a_1}{|a_1|}$$

4.7.2 Semantic Definition to Geometry Mapping

Consider an alignment segment that has a Cosine transition curve towards the left. The start cant is $160\ mm$ and transitions to zero over $100\ m$.

```
#64=IFCALIGNMENTCANTSEGMENT($,$,0.,100.,0.,0.,0.16,0.,.COSINECURVE.);
```

Compute the parent curve parameters

$$D_s = \frac{0 + 0.16}{2} = 0.08\ m,\ D_e = \frac{0 + 0\ m}{2} = 0.\ m,\ \Delta D = 0.0\ m - 0.08\ m = -0.08\ m$$

$$a_0 = 0.08\ m + \frac{-0.08\ m}{2} = 0.04$$

$$A_0 = (100\ m)^2\frac{1}{0.04\ m}\frac{0.04\ m}{|0.04\ m|} = 250000\ m$$

$$a_1 = -\frac{1}{2}(-0.08\ m) = -0.04\ m$$

$$A_1 = (100\ m)^2\frac{1}{-0.04\ m}\frac{-0.04\ m}{|-0.04\ m|} = 250000\ m$$

The parent curve is

```
#96=IFCCARTESIANPOINT((0.,0.));
#97=IFCDIRECTION((1.,0.));
#98=IFCAXIS2PLACEMENT2D(#96,#97);
#99=IFCCOSINESPIRAL(#98,250000.,250000.);
```

$$D(0\ m) = (100\ m)^2\left(\frac{1}{250000\ m} + \frac{1}{250000\ m}\cos\left(\pi\frac{0\ m}{100\ m}\right)\right) = 0.08\ m$$

$$D'(0\ m) = (100\ m)^2\left(-\frac{\pi}{(250000\ m)(100\ m)}\sin\left(\pi\frac{0\ m}{100\ m}\right)\right) = 0$$

$$\theta_0 = 0,\ dx_x = 1,\ dy_x = 0$$

The cross-slope at the start of the segment is

$$\phi_s = \cos^{-1}\left(\frac{D_{sr} - D_{sl}}{D_{rh}}\right) = \cos^{-1}\left(\frac{0.16 - 0.0}{1.5}\right) = 1.463926346$$

The cross slope orientation is

$$dy_z = \cos(\phi_s) = \cos(1.463926346) = 0.106666667$$

$$dz_z = \sin(\phi_s) = \sin(1.463926346) = 0.994294837$$

```
#100=IFCCARTESIANPOINT((0.,0.08,0.));
#101=IFCDIRECTION((0.,0.1066666666666667,0.99429483666678176));
#102=IFCDIRECTION((1.,0.,0.));
#103=IFCAXIS2PLACEMENT3D(#100,#101,#102);
#104=IFCCURVESEGMENT(.CONTSAMEGRADIENTSAMECURVATURE.,#103,IFCLENGTHMEASURE(0.
),IFCLENGTHMEASURE(100.),#99);
```

4.7.3 Compute Point on Curve

Compute the cant placement matrix for a point $\ell = 50\ m$ from the start of the curve segment using the algorithm in Section 4.2.

Step 1 — Form the curve segment placement matrix M_{CSP}

$$\boldsymbol{RefDir}_p = (1,\ 0,\ 0)$$

$$\boldsymbol{Axis}_p = (0,\ 0.10666666666666667,\ 0.9942948366667818)$$

$$\boldsymbol{Y}_p = \frac{\boldsymbol{Axis}_p \times \boldsymbol{RefDir}_p}{\left|\boldsymbol{Axis}_p \times \boldsymbol{RefDir}_p\right|} = (0,\ 0.9942948367,\ -0.1066666667)$$

$$M_{CSP} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0.994294836666782 & 0.106666666666667 & 0.08 \\ 0 & -0.106666666666667 & 0.994294836666782 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Step 2 — Evaluate the parent curve at the trim start — M_{PCS}

$$s_0 = 0,\ D(0) = 0.08\ m,\ D'(0) = 0,\ \theta_s = \tan^{-1}(D'(0)) = 0$$

The cross slope at the trim start is that of the segment's `Placement.Axis`, $\phi_s = \tan^{-1}\left(\frac{Axis_p.dz}{Axis_p.dy}\right) = 1.463926346.$

$$\boldsymbol{RefDir}_s = (\cos\theta_s,\ \sin\theta_s,\ 0) = (1,\ 0,\ 0)$$

$$\boldsymbol{Axis}_s = (0,\ \cos\phi_s,\ \sin\phi_s) = (0,\ 0.1066666667,\ 0.9942948367)$$

$$\boldsymbol{Y}_s = \boldsymbol{Axis}_s \times \boldsymbol{RefDir}_s = (0,\ 0.9942948367,\ -0.1066666667)$$

$$M_{PCS} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0.994294836666782 & 0.106666666666667 & 0.08 \\ 0 & -0.106666666666667 & 0.994294836666782 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

The rotation part of M_{PCS} equals that of M_{CSP} because $\theta_s = 0$ and the parent curve starts with the placement's cross slope.

Step 3 — Evaluate the parent curve at $\ell = 50\ m$ — $M_{PC\ell}$

$$D(50\ m) = 0.04\ m,\ D'(50\ m) = -0.001256637061,\ \theta_\ell = \tan^{-1}(D'(50\ m)) = -0.0012566364$$

The cross slope at the end of the segment is that of the next segment's `Placement.Axis`, $\phi_e = 1.570796327$, and the parent curve's deviating elevation changes by $\Delta D = D(100\ m) - D(0) = -0.08\ m$ over the segment.

$$\phi(\ell) = 1.463926346 + \left(\frac{1.570796327 - 1.463926346}{-0.08}\right)(0.04 - 0.08) = 1.517361336$$

$$\boldsymbol{RefDir}_\ell = (\cos\theta_\ell,\ \sin\theta_\ell,\ 0) = (0.9999992104,\ -0.001256636069,\ 0)$$

$$\boldsymbol{Axis}_\ell = (0,\ \cos\phi(\ell),\ \sin\phi(\ell)) = (0,\ 0.05340956531,\ 0.9985726906)$$

$$\boldsymbol{Y}_\ell = \boldsymbol{Axis}_\ell \times \boldsymbol{RefDir}_\ell = (0.001254842461,\ 0.9985719021,\ -0.05340952314)$$

$$M_{PC\ell} = \begin{bmatrix} 0.999999210432583 & 0.0012548424607127 & 0 & 50 \\ -0.00125663606923624 & 0.998571902120117 & 0.0534095653100558 & 0.04 \\ 0 & -0.0534095231396033 & 0.998572690560578 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Step 4 — Compute the cant placement matrix M_c

Rotation

$$R_c = R_{PC\ell} \cdot R_{PCS}^T \cdot R_{CSP}$$

Translation

$$\boldsymbol{T}_c = \boldsymbol{T}_{PC\ell} - \boldsymbol{T}_{PCS} + \boldsymbol{T}_{CSP}$$

Assemble

$$M_c = \begin{bmatrix} R_c & \boldsymbol{T}_c \\ \boldsymbol{0}^T & 1 \end{bmatrix}$$

$$= \overset{M_c}{\begin{bmatrix} 0.999999210432583 & 0.0012548424607127 & 0 & 50 \\ -0.00125663606923624 & 0.998571902120117 & 0.0534095653100559 & 0.04 \\ 0 & -0.0534095231396034 & 0.998572690560578 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}}$$

4.8 Sine Curve

A Sine transition in cant is represented with an `IfcSineSpiral`

4.8.1 Parent Curve Parametric Equations

The deviating elevation and its rate of change are given by the following equations.

$$\frac{D(s)}{L^2} = \frac{1}{A_0} + \frac{A_1}{|A_1|}\left(\frac{1}{A_1}\right)^2 s + \frac{1}{A_2}\sin\left(2\pi\frac{s}{L}\right)$$

$$\frac{d}{ds}D(s) = L^2\left(\frac{A_1}{|A_1|}\left(\frac{1}{A_1}\right)^2 + \frac{2\pi}{LA_2}\cos\left(2\pi\frac{s}{L}\right)\right)$$

Constant term:

$$a_0 = D_s$$

$$A_0 = L^2\frac{1}{a_0}\frac{a_0}{|a_0|}$$

Linear term:

$$a_1 = \Delta D$$

$$A_1 = L^{\frac{3}{2}}\frac{1}{|a_1|^{\frac{1}{2}}}\frac{a_1}{|a_1|}$$

Sine term:

$$a_2 = -\frac{1}{2\pi}\Delta D$$

$$A_2 = L^2\frac{1}{a_2}\frac{a_2}{|a_2|}$$

4.8.2 Semantic Definition to Geometry Mapping

Consider an alignment segment that has a Sine transition curve towards the left. The start cant is $160\ mm$ and transitions to zero over $100\ m$.

```
#64=IFCALIGNMENTCANTSEGMENT($,$,0.,100.,0.,0.,0.16,0.,.SINECURVE.);
```

Compute the parent curve parameters

$$D_s = \frac{0 + 0.16}{2} = 0.08\ m,\ D_e = \frac{0 + 0\ m}{2} = 0.\ m,\ \Delta D = 0.0\ m - 0.08\ m = -0.08\ m$$

Constant term:

$$a_0 = 0.08\ m$$

$$A_0 = (100\ m)^2 \frac{1}{0.08\ m} \frac{0.08\ m}{|0.08\ m|} = 125000\ m$$

Linear term:

$$a_1 = -0.08\ m$$

$$A_1 = (100\ m)^{\frac{3}{2}} \frac{1}{|-0.08\ m|^{\frac{1}{2}}} \frac{-0.08\ m}{|-0.08\ m|} = -3535.533906\ m$$

Sine term:

$$a_2 = -\frac{1}{2\pi}(0.08\ m) = -0.0127324\ m$$

$$A_2 = (100\ m)^2 \frac{1}{-0.0127324\ m} \frac{-0.0127324\ m}{|-0.0127324\ m|} = 785398.1634\ m$$

The parent curve is

```
#96=IFCCARTESIANPOINT((0.,0.));
#97=IFCDIRECTION((1.,0.));
#98=IFCAXIS2PLACEMENT2D(#96,#97);
#99=IFCSINESPIRAL(#98,785398.16339744814,-3535.533905932738,125000.);
```

$$D(0\ m) = (100\ m)^2 \left(\frac{1}{125000\ m} + \left(\frac{-3535.533906\ m}{|-3535.533906\ m|} \right) \left(\frac{1}{-3535.533906\ m} \right)^2 (0\ m) + \frac{1}{785398.1634\ m} \sin\left(2\pi \frac{0\ m}{100\ m} \right) \right) = 0.08\ m$$

$$D'(0\ m) = (100\ m)^2 \left(\frac{-3535.533906}{|-3535.533906|} \left(\frac{1}{-3535.533906} \right)^2 + \frac{2\pi}{(100\ m)(785398.1634\ m)} \cos\left(2\pi \frac{0\ m}{100\ m} \right) \right) = 0.$$

$$\theta_0 = 0$$

$$dx_x = \cos(\theta_0) = 1$$

$$dx_y = \sin(\theta_0) = 0$$

The cross-slope at the start of the segment is

$$\phi_s = \cos^{-1}\left(\frac{D_{sr} - D_{sl}}{D_{rh}}\right) = \cos^{-1}\left(\frac{0.16 - 0.0}{1.5}\right) = 1.463926346$$

The cross slope orientation is

$$dy_z = \cos(\phi_s) = \cos(1.463926346) = 0.106666667$$

$$dz_z = \sin(\phi_s) = \sin(1.463926346) = 0.994294837$$

```
#100=IFCCARTESIANPOINT((0.,0.08,0.));
#101=IFCDIRECTION((0.,0.1066666666666667,0.99429483666678176));
#102=IFCDIRECTION((1.,0.,0.));
#103=IFCAXIS2PLACEMENT3D(#100,#101,#102);
#104=IFCCURVESEGMENT(.CONTSAMEGRADIENTSAMECURVATURE.,#103,IFCLENGTHMEASURE(0.
),IFCLENGTHMEASURE(100.),#99);
```

4.8.3 Compute Point on Curve

Compute the cant placement matrix for a point $\ell = 50\ m$ from the start of the curve segment using the algorithm in Section 4.2.

Step 1 — Form the curve segment placement matrix M_{CSP}

$$\boldsymbol{RefDir}_p = (1,\ 0,\ 0)$$

$$\boldsymbol{Axis}_p = (0,\ 0.10666666666666667,\ 0.9942948366667818)$$

$$\boldsymbol{Y}_p = \frac{\boldsymbol{Axis}_p \times \boldsymbol{RefDir}_p}{\left|\boldsymbol{Axis}_p \times \boldsymbol{RefDir}_p\right|} = (0,\ 0.9942948367,\ -0.1066666667)$$

$$M_{CSP} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0.994294836666782 & 0.106666666666667 & 0.08 \\ 0 & -0.106666666666667 & 0.994294836666782 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Step 2 — Evaluate the parent curve at the trim start — M_{PCS}

$$s_0 = 0,\ D(0) = 0.08\ m,\ D'(0) = 0,\ \theta_s = \tan^{-1}(D'(0)) = 0$$

The cross slope at the trim start is that of the segment's `Placement.Axis`, $\phi_s = \tan^{-1}\left(\frac{Axis_p.dz}{Axis_p.dy}\right) = 1.463926346$.

$$\boldsymbol{RefDir}_s = (\cos\theta_s,\ \sin\theta_s,\ 0) = (1,\ 0,\ 0)$$

$$\boldsymbol{Axis}_s = (0,\ \cos\phi_s,\ \sin\phi_s) = (0,\ 0.1066666667,\ 0.9942948367)$$

$$\boldsymbol{Y}_s = \boldsymbol{Axis}_s \times \boldsymbol{RefDir}_s = (0,\ 0.9942948367,\ -0.1066666667)$$

$$M_{PCS} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0.994294836666782 & 0.106666666666667 & 0.08 \\ 0 & -0.106666666666667 & 0.994294836666782 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

The rotation part of M_{PCS} equals that of M_{CSP} because $\theta_s = 0$ and the parent curve starts with the placement's cross slope.

Step 3 — Evaluate the parent curve at $\ell = 50\ m$ — $M_{PC\ell}$

$$D(50\ m) = 0.04\ m,\ D'(50\ m) = -0.0016,\ \theta_\ell = \tan^{-1}(D'(50\ m)) = -0.001599998635$$

The cross slope at the end of the segment is that of the next segment's `Placement.Axis`, $\phi_e = 1.570796327$, and the parent curve's deviating elevation changes by $\Delta D = D(100\ m) - D(0) = -0.08\ m$ over the segment.

$$\phi(\ell) = 1.463926346 + \left(\frac{1.570796327 - 1.463926346}{-0.08}\right)(0.04 - 0.08) = 1.517361336$$

$$\boldsymbol{RefDir}_\ell = (\cos\theta_\ell,\ \sin\theta_\ell,\ 0) = (0.99999872,\ -0.001599997952,\ 0)$$

$$\boldsymbol{Axis}_\ell = (0,\ \cos\phi(\ell),\ \sin\phi(\ell)) = (0,\ 0.05340956531,\ 0.9985726906)$$

$$\boldsymbol{Y}_\ell = \boldsymbol{Axis}_\ell \times \boldsymbol{RefDir}_\ell = (0.00159771426,\ 0.9985714124,\ -0.05340949695)$$

$$M_{PC\ell} = \begin{bmatrix} 0.999998720002458 & 0.00159771425982398 & 0 & 50 \\ -0.00159999795200393 & 0.998571412389988 & 0.0534095653100558 & 0.04 \\ 0 & -0.0534094969459435 & 0.998572690560578 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Step 4 — Compute the cant placement matrix M_c

Rotation

$$R_c = R_{PC\ell} \cdot R_{PCS}^T \cdot R_{CSP}$$

Translation

$$\boldsymbol{T}_c = \boldsymbol{T}_{PC\ell} - \boldsymbol{T}_{PCS} + \boldsymbol{T}_{CSP}$$

Assemble

$$M_c = \begin{bmatrix} R_c & \boldsymbol{T}_c \\ \boldsymbol{0}^T & 1 \end{bmatrix}$$

$$= \begin{matrix} \\ \\ \\ \end{matrix} \overset{M_c}{\begin{bmatrix} 0.999998720002458 & 0.00159771425982398 & 0 & 50 \\ -0.00159999795200393 & 0.998571412389988 & 0.0534095653100559 & 0.04 \\ 0 & -0.0534094969459435 & 0.998572690560578 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}}$$

4.9 Viennese Bend

A Viennese Bend transition in cant is represented with an `IfcSeventhOrderPolynomialSpiral`.

4.9.1 Parent Curve Parametric Equations

The deviating elevation and its rate of change are given by the following equations.

$$\frac{D(s)}{L^2} = \frac{A_7}{|A_7^9|}s^7 + \frac{1}{A_6^7}s^6 + \frac{A_5}{|A_5^7|}s^5 + \frac{1}{A_4^5}s^4 + \frac{A_3}{|A_3^5|}s^3 + \frac{1}{A_2^3}s^2 + \frac{A_1}{2|A_1^3|}s + \frac{1}{A_0}$$

$$\frac{d}{ds}D(s) = L^2\left(\frac{7A_7}{|A_7^9|}s^6 + \frac{6}{A_6^7}s^5 + \frac{5A_5}{|A_5^7|}s^4 + \frac{4}{A_4^5}s^3 + \frac{3A_3}{|A_3^5|}s^2 + \frac{2}{A_2^3}s + \frac{A_1}{2|A_1^3|}\right)$$

$$D_s = \frac{D_{sl} + D_{sr}}{2},\ D_e = \frac{D_{el} + D_{er}}{2},\ \Delta D = D_e - D_s$$

Constant Term

$$a_0 = D_s, A_0 = \frac{L^2}{|a_0|}\frac{a_0}{|a_0|}$$

Linear Term

$$a_1 = 0, A_1 = \frac{L^{\frac{3}{2}}}{|a_1|^{\frac{1}{2}}}\frac{a_1}{|a_1|}$$

Quadratic Term

$$a_2 = 0, A_2 = \frac{L^{\frac{4}{3}}}{|a_2|^{\frac{1}{3}}}\frac{a_2}{|a_2|}$$

Cubic Term

$$a_3 = 0, A_3 = \frac{L^{\frac{5}{4}}}{|a_3|^{\frac{1}{4}}}\frac{a_3}{|a_3|}$$

Quartic Term

$$a_4 = 35\Delta D, A_4 = \frac{L^{\frac{6}{5}}}{|a_4|^{\frac{1}{5}}} \frac{a_4}{|a_4|}$$

Quintic Term

$$a_5 = -84\Delta D, A_5 = \frac{L^{\frac{7}{6}}}{|a_5|^{\frac{1}{6}}} \frac{a_5}{|a_5|}$$

Sextic Term

$$a_6 = 70\Delta D, A_6 = \frac{L^{\frac{8}{7}}}{|a_6|^{\frac{1}{7}}} \frac{a_6}{|a_6|}$$

Septic Term

$$a_7 = -20\Delta D, A_7 = \frac{L^{\frac{9}{8}}}{|a_7|^{\frac{1}{8}}} \frac{a_7}{|a_7|}$$

4.9.2 Semantic Definition to Geometry Mapping

Consider a Viennese Bend cant transition on a left-hand curve where the curvature decreases — a transition from a tighter arc to an arc of larger radius. The right rail begins at 100 mm of superelevation and decreases to 30 mm over 100 m. The left rail carries no cant throughout.

```
#64=IFCALIGNMENTCANTSEGMENT($,$,0.,100.,0.,0.,0.10,0.03,.VIENNESEBEND.);
```

Compute the parent curve parameters.

$$D_{sl} = 0.0\ m$$

$$D_{el} = 0.0\ m$$

$$D_{sr} = 0.1\ m$$

$$D_{er} = 0.03\ m$$

$$D_s = \frac{0\ m + 0.1\ m}{2} = 0.05\ m, D_e = \frac{0\ m + 0.03\ m}{2} = 0.015\ m,\ \Delta D = 0.015 - 0.05 = -0.035\ m$$

Constant Term

$$a_0 = 0.05\ m, A_0 = \frac{(100\ m)^2}{|0.05\ m|} \frac{0.05\ m}{|0.05\ m|} = 200000\ m$$

Linear Term

$$a_1 = 0, A_1 = 0\ m$$

Quadratic Term

$$a_2 = 0, A_2 = 0\ m$$

Cubic Term

$$a_3 = 0, A_3 = 0\ m$$

Quartic Term

$$a_4 = 35(-0.035\ m) = -1.225\ m, A_4 = \frac{(100\ m)^{\frac{6}{5}}}{|-1.225\ m|^{\frac{1}{5}}}\ \frac{-1.225\ m}{|-1.225\ m|}$$
$$= -241.1974890085123\ m$$

Quintic Term

$$a_5 = -84(-0.035\ m) = 2.94\ m, A_5 = \frac{(100\ m)^{\frac{7}{6}}}{|2.94\ m|^{\frac{1}{6}}}\ \frac{2.94\ m}{|2.94\ m|} = 180.0012184608678\ m$$

Sextic Term

$$a_6 = 70(-0.035\ m) = -2.45\ m, A_6 = \frac{(100\ m)^{\frac{8}{7}}}{|-2.45\ m|^{\frac{1}{7}}}\ \frac{-2.45\ m}{|-2.45\ m|} = -169.87095595653895\ m$$

Septic Term

$$a_7 = -20(-0.035\ m) = 0.7\ m, A_7 = \frac{(100\ m)^{\frac{9}{8}}}{|0.7\ m|^{\frac{1}{8}}}\ \frac{0.7\ m}{|0.7\ m|} = 185.93568367635672\ m$$

The parent curve is

```
#97=IFCDIRECTION((1.,0.));
#98=IFCAXIS2PLACEMENT2D(#96,#97);
#99=IFCSEVENTHORDERPOLYNOMIALSPIRAL(#98,185.93568367635649,-
169.87095595653892,180.00121846086768,-241.19748900851218,$,$,$,200000.);
```

$$D(0\ m) = (100\ m)^2\left(\frac{185.93568367635672\ m}{|(185.93568367635672\ m)^9|}(0\ m)^7\right.$$
$$+\frac{1}{(-169.87095595653895\ m)^7}(0\ m)^6$$
$$+\frac{180.0012184608678\ m}{|(180.0012184608678\ m)^7|}(0\ m)^5+\frac{1}{(-241.1974890085123\ m)^5}(0\ m)^4$$
$$\left.+\frac{1}{200000\ m}\right) = 0.05\ m$$

$$D'(0) = 0,\ \theta_0 = 0$$

$$dx_x = \cos(\theta) = 1$$

$$dx_y = \sin(\theta) = 0$$

The cross-slope at the start of the segment is

$$\phi_s = \cos^{-1}\left(\frac{D_{sr} - D_{sl}}{D_{rh}}\right) = \cos^{-1}\left(\frac{0.1 - 0.0}{1.5}\right) = 1.504080178$$

The cross slope orientation is

$$dy_z = \cos(\phi_s) = 0.066666667$$

$$dz_z = \sin(\phi_s) = 0.997775303$$

```
#100=IFCCARTESIANPOINT((0.,0.05,0.));
#101=IFCDIRECTION((0.,0.06666666666666666,0.99777530313971774));
#102=IFCDIRECTION((1.,0.,0.));
#103=IFCAXIS2PLACEMENT3D(#100,#101,#102);
#104=IFCCURVESEGMENT(.CONTSAMEGRADIENT.,#103,IFCLENGTHMEASURE(0.),IFCLENGTHME
ASURE(100.),#99);
```

4.9.3 Compute Point on Curve

Compute the cant placement matrix for a point $\ell = 50\ m$ from the start of the curve segment using the algorithm in Section 4.2.

Step 1 — Form the curve segment placement matrix M_{CSP}

$$\boldsymbol{RefDir}_p = (1,\ 0,\ 0)$$

$$\boldsymbol{Axis}_p = (0,\ 0.06666666666666667,\ 0.9977753031397177)$$

$$\boldsymbol{Y}_p = \frac{\boldsymbol{Axis}_p \times \boldsymbol{RefDir}_p}{\left|\boldsymbol{Axis}_p \times \boldsymbol{RefDir}_p\right|} = (0,\ 0.9977753031,\ -0.06666666667)$$

$$M_{CSP} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0.997775303139718 & 0.0666666666666667 & 0.05 \\ 0 & -0.0666666666666667 & 0.997775303139718 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Step 2 — Evaluate the parent curve at the trim start — M_{PCS}

$$s_0 = 0,\ D(0) = 0.05\ m,\ D'(0) = 0,\ \theta_s = \tan^{-1}(D'(0)) = 0$$

The cross slope at the trim start is that of the segment's `Placement.Axis`, $\phi_s = \tan^{-1}\left(\frac{Axis_p.dz}{Axis_p.dy}\right) = 1.504080178$.

$$\boldsymbol{RefDir}_s = (\cos\theta_s,\ \sin\theta_s,\ 0) = (1,\ 0,\ 0)$$

$$\boldsymbol{Axis}_s = (0,\ \cos\phi_s,\ \sin\phi_s) = (0,\ 0.06666666667,\ 0.9977753031)$$

$$\boldsymbol{Y}_s = \boldsymbol{Axis}_s \times \boldsymbol{RefDir}_s = (0,\ 0.9977753031,\ -0.06666666667)$$

$$M_{PCS} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0.997775303139718 & 0.0666666666666667 & 0.05 \\ 0 & -0.0666666666666667 & 0.997775303139718 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

The rotation part of M_{PCS} equals that of M_{CSP} because $\theta_s = 0$ and the parent curve starts with the placement's cross slope.

Step 3 — Evaluate the parent curve at $\ell = 50\ m$ — $M_{PC\ell}$

$$D(50\ m) = 0.0325\ m,\ D'(50\ m) = -0.000765625,\ \theta_\ell = \tan^{-1}(D'(50\ m)) = -0.0007656248504$$

The cross slope at the end of the segment is that of the next segment's `Placement.Axis`, $\phi_e = 1.550794993$, and the parent curve's deviating elevation changes by $\Delta D = D(100\ m) - D(0) = -0.035\ m$ over the segment.

$$\phi(\ell) = 1.504080178 + \left(\frac{1.550794993 - 1.504080178}{-0.035}\right)(0.0325 - 0.05) = 1.527437586$$

$$\boldsymbol{RefDir}_\ell = (\cos\theta_\ell,\ \sin\theta_\ell,\ 0) = (0.9999997069,\ -0.0007656247756,\ 0)$$

$$\boldsymbol{Axis}_\ell = (0,\ \cos\phi(\ell),\ \sin\phi(\ell)) = (0,\ 0.04334515667,\ 0.999060157)$$

$$\boldsymbol{Y}_\ell = \boldsymbol{Axis}_\ell \times \boldsymbol{RefDir}_\ell = (0.0007649052086,\ 0.9990598642,\ -0.04334514397)$$

$$M_{PC\ell} = \begin{bmatrix} 0.999999706909309 & 0.000764905208550319 & 0 & 50 \\ -0.000765624775602475 & 0.999059864228941 & 0.0433451566714392 & 0.032499999999999 \\ 0 & -0.0433451439673772 & 0.999060157044173 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Step 4 — Compute the cant placement matrix M_c

Rotation

$$R_c = R_{PC\ell} \cdot R_{PCS}^T \cdot R_{CSP}$$

Translation

$$\boldsymbol{T}_c = \boldsymbol{T}_{PC\ell} - \boldsymbol{T}_{PCS} + \boldsymbol{T}_{CSP}$$

Assemble

$$M_c = \begin{bmatrix} R_c & \boldsymbol{T}_c \\ \boldsymbol{0}^T & 1 \end{bmatrix}$$

$$M_c$$

$$= \begin{bmatrix} 0.999999706909309 & 0.000764905208550319 & 0 & 50 \\ -0.000765624775602475 & 0.999059864228941 & 0.0433451566714392 & 0.032499999999999 \\ 0 & -0.0433451439673772 & 0.999060157044174 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

4.10 Combined 3D

The cant segment result is computed with the 2D vertical and 2D horizontal to complete the evaluation of a point on the alignment. The result is a 3D point and reference frame.

Using the example horizontal Bloss curve from Section 2.8 M_h at $s = 50\ m$ is

$$M_h = \begin{bmatrix} 0.999512 & -0.031245 & 0 & 49.9962109 \\ 0.031245 & 0.999512 & 0 & 0.416638875 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Assume a vertical alignment that is flat. At $s = 50\ m$

$$M_v = \begin{bmatrix} 1 & 0 & 0 & 50 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

From the Bloss cant example in Section 4.6

$$M_c$$

$$= \begin{bmatrix} 0.999999280000778 & -0.00119828636590682 & 0 & 50 \\ 0.00119999913600094 & 0.998571971589017 & 0.0534095653100558 & 0.0400000000000001 \\ 0 & -0.0534095268552103 & 0.998572690560578 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Step 5 — Combine with horizontal and vertical to produce the 3D placement matrix

Construct M_v' as described in Section 3.2: zero the distance along and swap rows and columns 2 and 3. For a flat grade M_v has no rotation, so swapping both its rows and its columns leaves the identity:

$$M_v' = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Construct M_c' by zeroing the distance-along ℓ component in row 1, column 4 and moving the deviating elevation D from row 2 to row 3 in column 4 of M_c:

$$M_c'$$

$$= \begin{bmatrix} 0.999999280000778 & -0.00119828636590682 & 0 & 0 \\ 0.00119999913600094 & 0.998571971589017 & 0.0534095653100558 & 0 \\ 0 & -0.0534095268552103 & 0.998572690560578 & 0.0400000000000001 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Extract position vectors from each modified matrix, M_v', M_c', (setting row 4 to zero), then zero column 4:

$$P_v = M_v' \text{ column 4, row 4 set to 0} = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \end{bmatrix}, \qquad P_c = M_c' \text{ column 4, row 4 set to 0}$$

$$= \begin{bmatrix} 0 \\ 0 \\ 0.0400000000000001 \\ 0 \end{bmatrix}$$

$$M_v'' = M_v' \text{ with column 4 set to } (0,0,0,1)^T, \qquad M_c'' = M_c' \text{ with column 4 set to } (0,0,0,1)^T$$

$$M_v'' = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$M_c'' = \begin{bmatrix} 0.999999280000778 & -0.00119828636590682 & 0 & 0 \\ 0.00119999913600094 & 0.998571971589017 & 0.0534095653100558 & 0 \\ 0 & -0.0534095268552103 & 0.998572690560578 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Multiply the three orientation matrices, then add back both position offsets:

$$M' = M_h \cdot M_v'' \cdot M_c''$$

$$M'$$

$$= \begin{bmatrix} 0.999512 & -0.031245 & 0 & 49.9962109 \\ 0.031245 & 0.999512 & 0 & 0.416638875 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 0.999999280000778 & -0.0011982863 \\ 0.00119999913600094 & 0.998571971 \\ 0 & -0.053409526 \\ 0 & 0 \end{bmatrix}$$

$$M' = \begin{bmatrix} 0.9994738 & -0.03239808 & -0.001668782 & 49.99621 \\ 0.03244439 & 0.9980472 & 0.0533835 & 0.4166389 \\ 0 & -0.05340953 & 0.9985727 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$M_{3Dcant} = \begin{bmatrix} 0.9994738 & -0.03239808 & -0.001668782 & 49.99621 \\ 0.03244439 & 0.9980472 & 0.0533835 & 0.4166389 \\ 0 & -0.05340953 & 0.9985727 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} + \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix} + \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0.04 \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

$$M_{3Dcant} = \begin{bmatrix} 0.9994738 & -0.03239808 & -0.001668782 & 49.99621 \\ 0.03244439 & 0.9980472 & 0.0533835 & 0.4166389 \\ 0 & -0.05340953 & 0.9985727 & 0.04 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

4.11 Deviation from EnrichIfc4x3 Reference Implementation

The cant calculations in this section deviate from the ones performed by the EnrichIfc4x3 reference implementation. The calculations in the reference implementation, published at IFC-Rail-Unit-Test-Reference-Code/EnrichIFC4x3/EnrichIFC4x3/business2geometry at master · bSI-RailwayRoom/IFC-Rail-Unit-Test-Reference-Code (github.com), yield coefficients for sine, cosine, clothoid, and polynomial spirals that are not in units of length. The IFC specification is clear that the coefficients have units of length and are represented by `IfcLengthMeasure`.

The basic form of the coefficient of the ith term is

$$A_i = \frac{L^{\frac{i+2}{i+1}}}{|a_i|^{\frac{1}{(i+1)}}} \frac{a_i}{|a_i|}$$

Performing a dimensional analysis with l representing term with length units, we see that the resulting coefficient A_i has units of length.

$$\frac{l^{\frac{i+2}{i+1}}}{|l|^{\frac{1}{(i+1)}}} \frac{l}{|l|} = \frac{l^{\frac{i+2}{i+1}}}{l^{\frac{1}{i+1}}} = l^{\frac{i+2}{i+1}} l^{\frac{-1}{(i+1)}} = l^{\frac{i+1}{i+1}} = l$$

The error in the EnrichIfc4x3 reference implementation is illustrated by way of an example. Consider the Bloss Curve example GENERATED__CantAlignment_BlossCurve_100.0_1000_300_1_Meter.ifc.

The semantic definition of the cant transition segment is

```
#64=IFCALIGNMENTCANTSEGMENT($,$,0.,100.,0.,0.,0.,0.16,.BLOSSCURVE.);
```

The EnrichIfc4x3 reference implementation maps the semantic definition to the geometric representation as follows:

$$D_s = \frac{0.0 + 0.0}{2} = 0\ m,\ D_e = \frac{0.0 + 0.16}{2} = 0.08\ m,\ \Delta D = 0.08 - 0.0 = 0.08\ m$$

Quadratic Term:

$$a_2 = 3\Delta D = 3(0.08\ m) = 0.24\ m$$

$$A_2 = \frac{(100\ m)}{|0.24\ m|^{\frac{1}{3}}}\left(\frac{0.24\ m}{|0.24\ m|}\right) = 160.917897\ m^{2/3}$$

Cubic Term:

$$a_3 = -2\Delta D = -0.016\ m$$

$$A_3 = \frac{100\ m}{|-0.16\ m|^{\frac{1}{4}}}\frac{-0.16\ m}{|-0.16\ m|} = -158.113883\ m^{3/4}$$

Notice that the polynomial coefficients do not have whole length units (e.g. the units aren't m).

The mapped geometric representation is

```
#127 = IFCTHIRDORDERPOLYNOMIALSPIRAL(#128, -158.113883008419,
160.914897434272, $, $);
```

Recalling from Bloss Curve example above,

Quadratic Term

$$A_2 = \frac{(100m)^{\frac{4}{3}}}{|0.24m|^{\frac{1}{3}}}\left(\frac{0.24m}{|0.24m|}\right) = 746.9007911\ m$$

Cubic Term

$$A_3 = \frac{(100\ m)^{\frac{5}{4}}}{|-0.16\ m|^{\frac{1}{4}}}\left(\frac{-0.16\ m}{|-0.16\ m|}\right) = -500\ m$$

The polynomial coefficients have units of length as required by IFC.

The resulting geometric representation is

```
#127=IFCTHIRDORDERPOLYNOMIALSPIRAL(#98,-500.,746.900791092861,$,$);
```

Table 4.11-1 compares the third order polynomial terms.

Polynomial Term	EnrichIfc4x3	This Guide
Constant	0.0 (unitless)	$0\ m$
Linear	$0\ m^{1/2}$	$0\ m$

Polynomial Term	EnrichIfc4x3	This Guide
Quadratic	$160.91789\ m^{2/3}$	$746.90078\ m$
Cubic	$-158.113883\ m^{3/4}$	$-500\ m$

Table 4.11-1 — Comparison of polynomial coefficients

The EnrichIfc4x3 reference implementation computes the correct value for the deviating elevation. This is because there is a compensating error in the implementation. The deviating elevation is computed as

$$D(s) = L\left(\frac{A_3}{|A_3^5|}s^3 + \frac{1}{A_2^3}s^2\right)$$

Evaluated at $s = 50\ m$

$$D(50\ m) = (100\ m)\left(\frac{-158.113883\ m^{3/4}}{|(-158.113883\ m^{3/4})^5|}(100\ m)^3 + \frac{1}{(160.914897434272\ m^{2/3})^3}(100\ m)^2\right) = (100\ m)(-0.0002 + 0.0006) = 0.04\ m$$

The deviating elevation as computed in this guide is

$$D(s) = L^2\left(\frac{A_3}{|A_3^5|}s^3 + \frac{1}{A_2^3}s^2\right)$$

Notice that the first term is L^2, not L as in the reference implementation.

$$D(50\ m) = (100\ m)^2\left(\frac{-500\ m}{|(-500\ m)^5|}(100\ m)^3 + \frac{1}{(746.90079\ m)^3}(100\ m)^2\right) = (100\ m)^2(-0.000002\ m^{-1} + 0.000006\ m^{-1}) = 0.04\ m$$

Both approaches give the same result, however, a serious problem emerges if the EnrichIfc4x3 semantic to geometry mapping is mixed with the deviating elevation equation from this guide and vice versa. Table 4.11-2 compares the results for all the combinations of the semantic to geometry mapping and deviating elevation computations for the Bloss curve evaluated at $s = 50\ m$. The deviating elevation can be incorrect proportional (or inversely proportional) to the length of the segment (100 or 1/100 in this example).

Mapping	$D(s)$ Equation	$D(50\ m)$
EnrichIfc4x3	EnrichIfc4x3	$0.04\ m$
This Guide	This Guide	$0.04\ m$
EnrichIfc4x3	This Guide	$4.0\ m$
This Guide	EnrichIfc4x3	$0.0004\ m$

Table 4.11-2 — Comparison of deviating elevation results

4.12 Zero-Length Closing Segment and `IfcSegmentedReferenceCurve.EndPoint`

The zero-length closing segment requirement described in Section 2.12 applies equally to `IfcSegmentedReferenceCurve`. The final `IfcAlignmentCantSegment` nested within `IfcAlignmentCant` must have `SegmentLength = 0`, and the corresponding geometric counterpart in `IfcSegmentedReferenceCurve` must be a zero-length `IfcCurveSegment` placed at the endpoint of the cant alignment with its placement matching the end cant state of the preceding segment.

The same guidance regarding `EndPoint` described in §3.8 applies to `IfcSegmentedReferenceCurve.EndPoint`: it is optional, secondary to the zero-length segment, and must be consistent with the zero-length segment placement when present.

4.13 Summary and Implementation Checklist

#	Item	Notes
1	Use `IfcAxis2Placement3D` (not `IfcAxis2Placement2D`) for cant curve segment placements	Cant encodes a cross-slope rotation that a 2D placement cannot represent; the `Axis` vector carries the cross-slope angle ϕ at the segment start
2	The cross-slope angle ϕ is interpolated using deviating elevation as the interpolation parameter, not arc length	The interpolation formula in Section 4.1.3 uses $D(\ell) - D_s$ as the parameter; ϕ does not vary linearly with distance along the segment
3	When forming M_c' for Step 5, zero the distance-along in column 4, row 1 and move the deviating elevation D from row 2 to row 3 in column 4	This maps the deviating elevation onto the Z axis where M_h expects it; failure to do so incorrectly applies the translation offsets to the rotated frame
4	Do not mix the EnrichIfc4x3 semantic-to-geometry mapping with the deviating elevation equations from this guide	EnrichIfc4x3 contains two compensating errors that yield correct results within its own implementation; mixing its coefficient mapping with this guide's deviating elevation equation (or vice versa) breaks the cancellation and produces results off by a factor of L or $1/L$ — see Section 4.11
5	Include a zero-length `IfcCurveSegment` at the end of `IfcSegmentedReferenceCurve`; treat `IfcSegmentedReferenceCurve.EndPoint` as secondary	The zero-length segment is required by the IFC Concept Templates and is the normative endpoint; `EndPoint` is optional — if present, it must agree with the zero-length segment placement
6	Interpolate the cross slope between the	The end cross slope ϕ_e comes from the

#	Item	Notes
	segment's own `Placement.Axis` and the next segment's `Placement.Axis`	next segment's placement (the zero-length closing segment for the last segment), not from the segment's own design parameters (Section 4.1.3)
7	Build the parent-curve frame from $\boldsymbol{RefDir} = (\cos\theta, \sin\theta, 0)$ and $\boldsymbol{Axis} = (0, \cos\phi, \sin\phi)$ without re-orthogonalizing them	Re-orthogonalizing gives `Axis` an x component and changes `RefDir`; results then differ from IfcOpenShell by up to about 10^{-5} (Section 4.2, Steps 2 and 3)

Chapter 5 — Offset Curves

5.0 Introduction

Offset curves are very common in infrastructure geometry. Some examples are:

- Edge of pavement
- Bridge girder centerlines
- Lane striping
- Utility centerline

`IfcOffsetCurveByDistances` defines the geometry of offset curves. It has three attributes:

- `BasisCurve`: The curve from which offsets are measured
- `OffsetValues`: Defines the offsets along the curve
- `Tag` (optional): An identifier for the curve, used to correlate offset curve points with positions in a variable cross-section (see Chapter 10)

A single offset value indicates a constant offset over the entire length of the curve.

If the offsets do not span the full length of the curve, the first and last offset are implicitly continued to the head and tail of the basis curve, respectively.

The `IfcOffsetCurveByDistances.OffsetValues` are of type `IfcPointByDistanceExpression`. The `IfcOffsetCurveByDistances.BasisCurve` and `IfcPointByDistanceExpression.BasisCurve` should logically reference the same curve since the offset value is defining an offset from `IfcOffsetCurveByDistances.BasisCurve`. However, this is not an explicitly stated requirement in the IFC specifications nor is it enforced by the bSI Validation Service [8].

The `IfcOffsetCurveByDistances` geometry is derived by interpolating the `OffsetValues`.

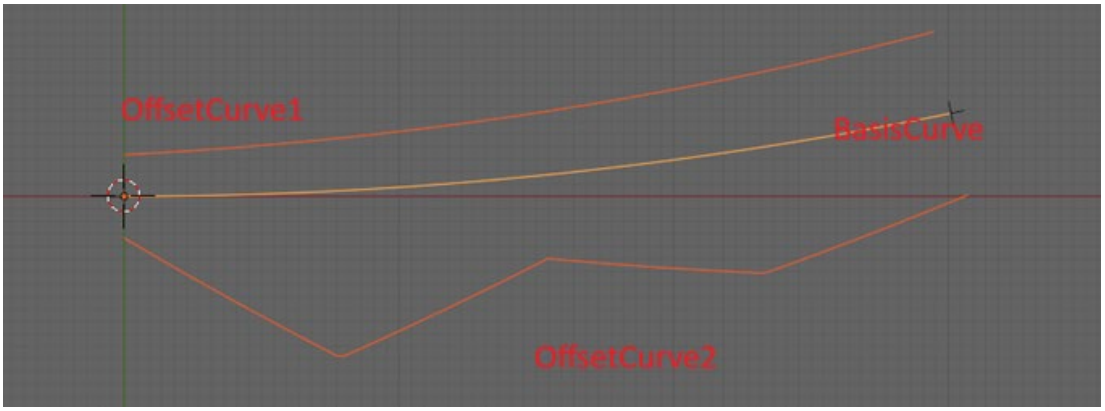


Figure 5.0-1 Offset curves

Figure 5.0-1 shows two offset curves that share a single basis curve. Offset curve 1 has different offset values defined at the start and the end of the basis curve. The start offset is 10 ft and the end offset is 20 ft. The basis curve is 100 ft long. The point on Offset curve 1 that corresponds to the mid-point of the basis curve is determined by evaluating the basis curve at 50 ft, then applying the linearly interpolated offset of 15 ft.

Offset curve 2 has several offsets. Linear interpolation is used to determine the offset from the basis curve to the offset curve between each pair of consecutive offset points.

The following IFC STEP excerpt illustrates how these two offset curves are encoded. The basis curve `#20` is a circular arc.

Offset1 — left side, linearly varying:

```
#88=IFCPOINTBYDISTANCEEXPRESSION(IFCLENGTHMEASURE(0.),10.,$,$,#20);
#89=IFCPOINTBYDISTANCEEXPRESSION(IFCLENGTHMEASURE(200.),20.,$,$,#20);
#94=IFCOFFSETCURVEBYDISTANCES(#20,(#88,#89),$); /* Tag omitted */
```

The offset begins at +10 at distance 0 and increases linearly to +20 at distance 200 along the basis curve. At the midpoint of the basis curve (distance 100) the interpolated offset is +15.

Offset2 — right side, multi-point:

```
#102=IFCPOINTBYDISTANCEEXPRESSION(IFCLENGTHMEASURE(0.),-10.,$,$,#20);
#103=IFCPOINTBYDISTANCEEXPRESSION(IFCLENGTHMEASURE(50.),-40.,$,$,#20);
#104=IFCPOINTBYDISTANCEEXPRESSION(IFCLENGTHMEASURE(100.),-20.,$,$,#20);
#105=IFCPOINTBYDISTANCEEXPRESSION(IFCLENGTHMEASURE(150.),-30.,$,$,#20);
#106=IFCPOINTBYDISTANCEEXPRESSION(IFCLENGTHMEASURE(200.),-20.,$,$,#20);
#111=IFCOFFSETCURVEBYDISTANCES(#20,(#102,#103,#104,#105,#106),$); /* Tag
omitted */
```

The right-side offset widens quickly from −10 to −40 over the first 50 distance units along the basis curve, then narrows to −20 at mid-curve, widens again to −30, and finishes at −20. Positive lateral offsets place the offset curve to the left of the basis curve and negative offsets to the right, measured perpendicular to the curve looking in the direction of increasing distance.

Both offset curves share the same basis curve `#20`. Each `IfcPointByDistanceExpression` also references `#20` directly — the logical choice, since each offset value defines a lateral distance from that curve. The case where these two basis curve references differ is explored in §5.5.3.

The STEP excerpt above is drawn from the example models `IfcOffsetCurveByDistances_2D.ifc` and `IfcOffsetCurveByDistances_3D.ifc`, which are discussed further in §5.5.

5.1 Offset Curves and IfcAlignment

In IFC4x3, `IfcOffsetCurveByDistances` is a resource-layer geometric entity and cannot exist independently in a model. Validation rule IFC105 [5] requires that every resource entity be reachable from a rooted entity. In practice this means the offset curve must be assigned as the shape representation of an `IfcAlignment` (or another product), and that alignment must be aggregated into the project. Without this, the IFC validation service will report an IFC105 violation.

The consequence is that offset curves in IFC4x3 models are represented as alignments. This is natural: an offset alignment has its own object identity, can carry properties, can support linear placement of objects, and participates in the project's spatial breakdown just like the basis alignment.

The example file `IfcOffsetCurveByDistances_2D.ifc` demonstrates this structure. The basis alignment "E-Line" and its two offsets, "Offset1" and "Offset2," are all `IfcAlignment` instances aggregated under the project. Each offset alignment's `Representation` references an `IfcShapeRepresentation` whose item is the corresponding `IfcOffsetCurveByDistances`.

When `IfcOffsetCurveByDistances` is used as a guide curve for `IfcSectionedSurface` or `IfcSectionedSolidHorizontal`, the same requirement applies with an additional complication. The guide curve is geometrically referenced by the surface or solid through the tag mechanism, and the surface or solid is itself a rooted entity — so the chain of references back to a rooted entity exists. The validation service does not trace this path, however, and still flags guide curves that are not directly assigned as a representation of a rooted entity. See §10.5.2 for the practical consequence.

5.2 Accuracy of Offset Curves

`IfcOffsetCurveByDistances` produces an exact geometric parallel only for lines and circular arcs with a constant offset. Even with a constant offset, transition curves have no closed-form geometric parallel — the true parallel of a Clothoid spiral, for example, is not a Clothoid. The resulting geometry is always a piecewise-linear approximation of the true offset, regardless of whether the offset is constant or varying. Accuracy improves by adding more `IfcPointByDistanceExpression` values at shorter intervals, densifying the approximation.

For most infrastructure applications, the linear interpolation error is small relative to construction tolerances. For applications requiring tighter accuracy — such as rail superelevation ramps or precision survey control — the approximation should be evaluated against the required tolerance and the offset point spacing adjusted accordingly.

A further consequence of the approximation is that **distance along** the offset curve is also approximate. When `IfcOffsetCurveByDistances` is used as the `BasisCurve` in

IfcPointByDistanceExpression for IfcLinearPlacement, two independent implementations may compute different arc lengths for the same curve definition.

The arc length of IfcOffsetCurveByDistances at any given location is obtained by numerically integrating the offset curve geometry between the sample points. Because the IFC specification does not prescribe the interpolation method (see §5.3), different software may produce different intermediate curve shapes from identical OffsetValues, and therefore different arc lengths. Even if the interpolation method is agreed upon, the arc length integral has no closed form for a general offset from a spiral, so implementations must use numerical quadrature with finite precision or approximate series expansion solutions.

The consequence is that a DistanceAlong value specified against an IfcOffsetCurveByDistances does not map to a unique, reproducible point across implementations. This is in direct contrast to DistanceAlong measured along an IfcCompositeCurve or IfcGradientCurve, where arc length is determined exactly by the curve equations.

For precise linear placement, the recommended practice (discussed fully in §8.5) is to reference the **parent alignment** as the BasisCurve in IfcPointByDistanceExpression and use OffsetLateral to account for the transverse distance — rather than placing objects directly along the offset curve. This avoids the arc-length approximation entirely and keeps placement reproducible across software implementations.

5.3 Interpolation of Offset Values

The IFC specification does not prescribe how offset values are to be interpolated between IfcPointByDistanceExpression entries. Several methods are possible:

- **Linear interpolation of offset values** — the lateral offset distance is interpolated linearly between consecutive offset points, and the interpolated distance is applied perpendicular to the basis curve at each chainage. This keeps the offset curve anchored to the basis curve geometry.
- **Linear interpolation between 3D points** — the 3D positions of consecutive offset points are computed and connected by straight line segments. This produces a true piecewise-linear curve in space, but one that drifts away from the basis curve between offset points.
- **Spline interpolation** — cubic spline, B-spline, Bézier, or NURBS methods applied to either the offset values or the 3D positions, producing smoother curves but with greater implementation complexity.

Different interpolation methods produce materially different geometry from the same IfcOffsetCurveByDistances data. This is illustrated in Figure 5.3-1 which overlays a spline interpolation onto the linear interpolation from Figure 5.0-1. Without a formal implementation agreement, two compliant applications may render the same model differently.

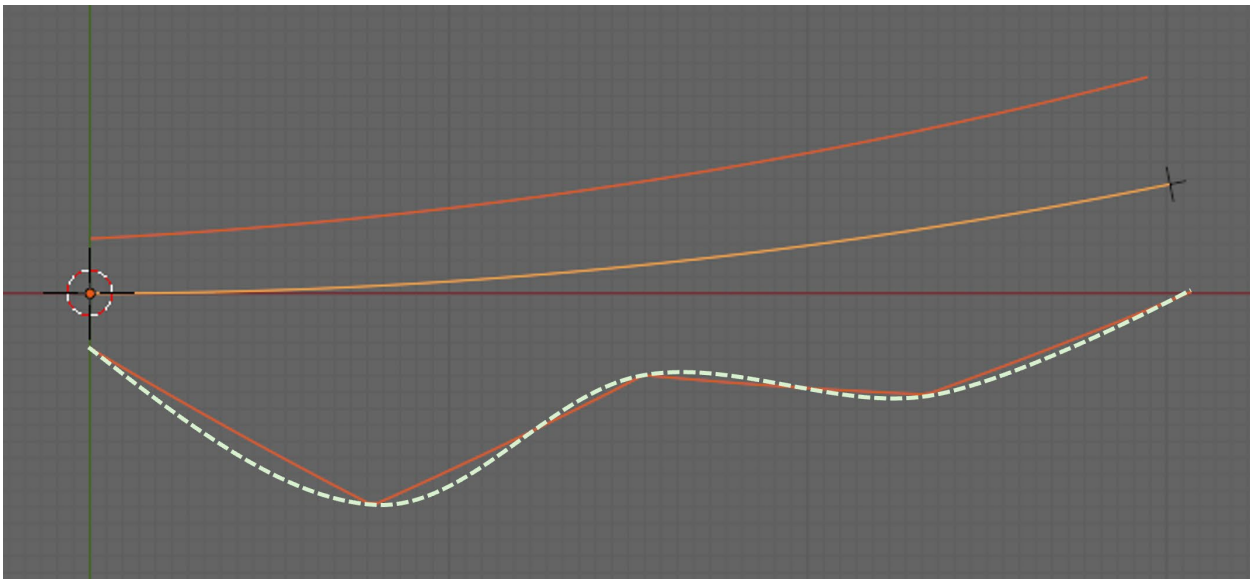


Figure 5.3-1 — Comparison of linear and spline interpolation

The recommended best practice is **linear interpolation of offset values**: interpolate the lateral distance at each chainage position, then apply that distance perpendicular to the basis curve. This method is the most natural given the structure of `IfcOffsetCurveByDistances`, produces predictable geometry, and is the least ambiguous for interchange.

5.4 The Tag Attribute

The `Tag` attribute of `IfcOffsetCurveByDistances` is an optional label that assigns an identifier to the offset curve. Its primary purpose is to correlate an offset curve with a named point in a variable cross-section definition.

`IfcSectionedSurface` and `IfcSectionedSolidHorizontal` define geometry by sweeping a series of cross-sections along a basis curve. Each cross-section can contain named control points — edge-of-pavement, top-of-rail, daylight line, and so on. The `Tag` on an `IfcOffsetCurveByDistances` matches one of those named points, allowing the surface or solid geometry to be driven by the offset curve's lateral positions rather than by fixed cross-section ordinates. This is the mechanism by which variable-width surfaces and solids are defined in IFC4x3.

The full treatment of `IfcSectionedSurface` and `IfcSectionedSolidHorizontal`, including how `Tag` values are resolved against cross-section geometry, is covered in Chapter 10.

5.5 Example Models

Three example models illustrate `IfcOffsetCurveByDistances` in progressively more demanding configurations. The 2D and 3D variants share the same geometry and offset profiles; the difference is whether the basis curve is an `IfcCompositeCurve` (horizontal

only) or an IfcGradientCurve (3D). The split-basis variant tests a configuration the IFC specification does not prohibit but leaves geometrically ambiguous.

5.5.1 Two-Dimensional Basis Curve

The file IfcOffsetCurveByDistances_2D.ifc contains a basis alignment and two offset alignments. The basis curve is an IfcCompositeCurve — a horizontal-only curve with no elevation component.

Validation. Both offset alignments in this model fail the validation rule IfcShapeRepresentation.CorrectItemsForType. The IfcShapeRepresentation for each offset alignment uses RepresentationType = "Curve3D", which requires its items to be 3D curves. The IfcOffsetCurveByDistances entities here are 2D because their BasisCurve is an IfcCompositeCurve — a planar horizontal curve with no elevation component. The IfcAlignment specification permits IfcOffsetCurveByDistances to represent either a 2D or 3D alignment curve, but the validation service's CorrectItemsForType rule does not account for this — it requires Curve3D items to be 3D regardless. This is a gap in the validation service, not a conflict in the specification. It is anticipated that the next version of the buildingSMART Validation Service will not flag this as an error.

5.5.2 Three-Dimensional Basis Curve

The file IfcOffsetCurveByDistances_3D.ifc extends the 2D example by adding a flat vertical alignment to the basis alignment, making its basis curve an IfcGradientCurve. The two offset alignments are constructed to demonstrate how basis curve choice affects dimensionality and validation outcome.

Offset1 references the IfcGradientCurve as the BasisCurve in both the IfcOffsetCurveByDistances and all IfcPointByDistanceExpression offset values. Because IfcGradientCurve is a 3D curve, the resulting offset curve is 3D. Offset1 satisfies IfcShapeRepresentation.CorrectItemsForType.

Offset2 references IfcGradientCurve.BaseCurve — the underlying IfcCompositeCurve — as the BasisCurve for its offset values. IfcCompositeCurve is a 2D horizontal curve, so the resulting offset curve is 2D regardless of whether the parent alignment has a vertical component. Offset2 fails IfcShapeRepresentation.CorrectItemsForType for the same reason as the 2D example. It is anticipated that the next version of the buildingSMART Validation Serivce will not flag this as an error.

The key finding is that the dimensionality of an IfcOffsetCurveByDistances is determined by which curve object is referenced as its BasisCurve — not by the dimensionality of the alignment that owns it. Referencing an IfcGradientCurve produces a 3D offset curve; referencing its constituent IfcCompositeCurve produces a 2D one.

5.5.3 Split Basis Curves

The file `IfcOffsetCurveByDistances_split_basis.ifc` demonstrates the edge case described in §5.0 where `IfcOffsetCurveByDistances.BasisCurve` and `IfcPointByDistanceExpression.BasisCurve` reference different curves. The model contains two basis alignments — a circular arc and a straight line — each with a flat vertical profile and an `IfcGradientCurve` basis curve.

The single offset alignment has its `IfcOffsetCurveByDistances.BasisCurve` set to the circular arc's `IfcGradientCurve`, while its `IfcPointByDistanceExpression` offset values reference the straight line's `IfcGradientCurve`. Because the two curves have different shapes, the parameterization of the offset values is inconsistent with the curve to which they are applied.

The IFC specification does not prohibit this configuration, and the model passes all validation service checks. This is itself a gap: the split-basis configuration is semantically ambiguous and should be flagged as either an error or a best-practices violation. A future validation rule requiring that `IfcOffsetCurveByDistances.BasisCurve` and the `BasisCurve` of all its `OffsetValues` reference the same curve object would close this gap without breaking any legitimate use case.

Chapter 6 — Approximate Alignment Geometry

6.0 Introduction

The parametric `IfcCurveSegment`-based representations described in Sections 2 through 4 provide closed-form equations for position, tangent, and curvature at any arc-length along the alignment. That precision is essential when geometry drives design, construction staking, linear placement, or quantity takeoff.

Not all alignment data arrives in this form. Field surveys of existing infrastructure may produce a series of measured points rather than fitted curve parameters. Early-stage planning alignments may exist only as a sketched centerline digitized from a map. GIS systems commonly store linear features as coordinate strings. IFC accommodates these cases through two representation types: `IfcPolyline` and `IfcIndexedPolyCurve`.

Both are **discrete** rather than parametric: geometry is defined by a sequence of control points with linear interpolation between them. The result is a piecewise-linear approximation that encodes position only.

The IFC specification lists both types as valid `IfcAlignment` geometry for 2D horizontal alignments (planning and mapping contexts) and 3D alignments (survey-derived). See Section 1.5 for the complete list of valid representation types.

6.1 Geometric Forms

Both `IfcPolyline` and `IfcIndexedPolyCurve` provide simplified, approximate representations of alignment geometry. Rather than encoding the mathematical intent of the design — curve type, radius, spiral parameter — they encode only sampled positions. The two types differ in storage layout and in how much approximation they can absorb.

`IfcPolyline` holds geometry as an ordered list of `IfcCartesianPoint` instances — each vertex is a discrete entity in the model. Consecutive vertices are connected by straight line segments, so every curve in the original alignment is approximated by a series of chords. The curve accepts 2D or 3D coordinates, making it suitable for both plan-view sketches and full survey point strings.

`IfcIndexedPolyCurve` stores coordinates compactly in an `IfcCartesianPointList2D` or `IfcCartesianPointList3D` and references individual vertices by integer index. When the optional `Segments` attribute is omitted, the result is functionally identical to `IfcPolyline`. When `Segments` is provided, each entry is either an `IfcLineIndex` (two indices, straight segment) or an `IfcArcIndex` (three indices: start, a through-point on the arc, and end). Arc segments allow circular curves to be represented exactly rather than approximated by chords, reducing the number of vertices needed for a given accuracy. This makes `IfcIndexedPolyCurve` better suited to map-digitized or GIS-derived data where circular arcs are common but transition spirals are absent.

6.2 Alignments Without Semantic Layout

In contrast to Concept Templates 4.1.7.1.1.1 through 4.1.7.1.1.4, the IFC specification does not provide concept templates that show how `IfcPolyline` or `IfcIndexedPolyCurve` provide a geometric representation of an `IfcAlignment`. (The same gap applies to `IfcOffsetCurveByDistances`, covered in Chapter 5.)

Section 1.4 establishes that every `IfcAlignmentHorizontal`, `IfcAlignmentVertical`, and `IfcAlignmentCant` must terminate with a zero-length `IfcAlignmentSegment`. When a geometric representation accompanies the semantic definition, the corresponding last `IfcCurveSegment` must also be zero length.

Neither type is composed of `IfcCurveSegment` entities and therefore neither can satisfy the requirements for the geometric counterpart to a full alignment layout.

While `IfcPolyline` and `IfcIndexedPolyCurve` could each serve as an `IfcCurveSegment.ParentCurve`, they cannot produce the required zero-length closing segment. A zero-length polyline segment requires two coincident vertices; `IfcPolyline` WHERE rule WR1 explicitly prohibits this. `IfcIndexedPolyCurve` has no equivalent schema-level prohibition, but a coincident point pair produces a degenerate segment with an undefined tangent direction. In practice this means an application cannot pair a complete semantic alignment layout with a simple chorded polyline for visualization — the two are mutually exclusive in IFC4x3.

For this reason, `IfcPolyline` and `IfcIndexedPolyCurve` (as well as `IfcOffsetCurveByDistances`) can only represent an `IfcAlignment` without an accompanying semantic layout definition.

Chapter 7 — Alignments Reusing Horizontal Layout

7.0 Introduction

In infrastructure projects it is common for several alignments to share the same horizontal path. Three situations arise frequently:

Design alternatives. A corridor study fixes the horizontal alignment and evaluates multiple vertical profiles. Each profile becomes its own alignment sharing the common horizontal baseline. Comparison software can overlay the profiles without any coordinate transformation.

Existing versus proposed. When an existing road is to be reconstructed, the existing centerline may be surveyed and represented as one alignment; the proposed centerline shares the same horizontal but carries a new vertical profile.

Parallel infrastructure elements. Features that follow the same horizontal path at different elevations — top of sub-grade, finished grade, top of rail — can each be modelled as a separate alignment sharing a common horizontal. This is more precise than using `IfcOffsetCurveByDistances` for vertical offset, because each element carries its own parametric vertical definition rather than a sampled elevation difference.

IFC4x3 accommodates all three cases directly. Because horizontal layout is represented by discrete entities in the model, those entities can be referenced by more than one alignment. The horizontal curve is defined once and shared; each alignment then adds its own vertical and, if applicable, cant definition on top of that shared base.

The shared-horizontal pattern relies on a **parent** `IfcAlignment` that owns the common horizontal layout, and **child** `IfcAlignment` instances — each with its own vertical profile — aggregated under that parent. This parent/child structure is prescribed by Concept Template 4.1.4.4.1.2 *Alignment Layout — Reusing Horizontal Layout*.

This section describes the alignment layout structure for the reusing-horizontal-layout pattern, the geometric representations of the child alignments, and the open implementation questions that remain to be resolved in the specification.

7.1 Alignment Layout and Geometric Representation

7.1.1 Alignment Layout — Parent/Child Structure

The alignment layout uses a three-tier hierarchy:

1. A **parent** `IfcAlignment` nests a single shared `IfcAlignmentHorizontal` via `IfcRelNests`.
2. One or more **child** `IfcAlignment` instances are aggregated under the parent via `IfcRelAggregates`.

3. Each child `IfcAlignment` nests its own `IfcAlignmentVertical` (and, for rail alignments, its own `IfcAlignmentCant`) via `IfcRelNests`.

The shared `IfcAlignmentHorizontal` exists exactly once in the model, owned by the parent via `IfcRelNests`. No child alignment nests an `IfcAlignmentHorizontal` of its own; they reuse the horizontal definition from the parent. The parent itself carries no vertical layout; its sole role is to hold the shared horizontal and aggregate the children. Children that require cant also nest an `IfcAlignmentCant` via `IfcRelNests`. Figure 7.1.1-1 shows the complete parent/child layout structure.

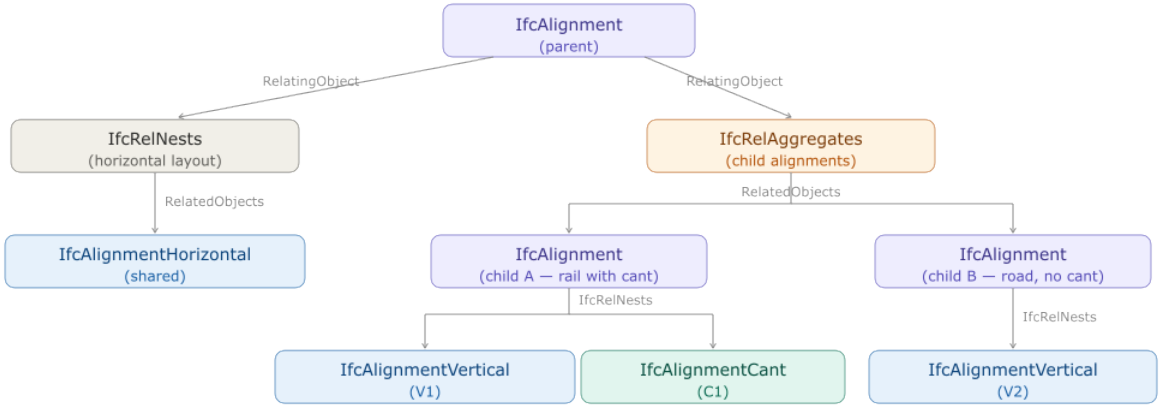


Figure 7.1.1-1 — Alignment layout for the reusing-horizontal-layout pattern.

Only the `IfcAlignmentHorizontal` is shared. Each child alignment owns its `IfcAlignmentVertical` and `IfcAlignmentCant` exclusively — those layout entities are never shared between children.

7.1.2 Geometric Representation — Shared BaseCurve

Each child alignment's 3D curve references the horizontal `IfcCompositeCurve` through its `BaseCurve` attribute. `IfcGradientCurve` carries `BaseCurve: IfcBoundedCurve` as the 2D horizontal curve, and two `IfcGradientCurve` instances can reference the same `BaseCurve` entity. The horizontal geometry — typically an `IfcCompositeCurve` composed of `IfcCurveSegment` instances — exists in the model exactly once and is referenced by both gradient curves. Figure 7.1.2-1 illustrates this shared reference.

At the geometric level, `IfcSegmentedReferenceCurve` adds cant to a gradient curve by taking an `IfcGradientCurve` as its own `BaseCurve`. Each child alignment that needs a cant definition gets its own `IfcSegmentedReferenceCurve` and its own `IfcGradientCurve`; only the `IfcCompositeCurve` at the base of the chain is the shared instance, as shown in Figure 7.1.2-1:

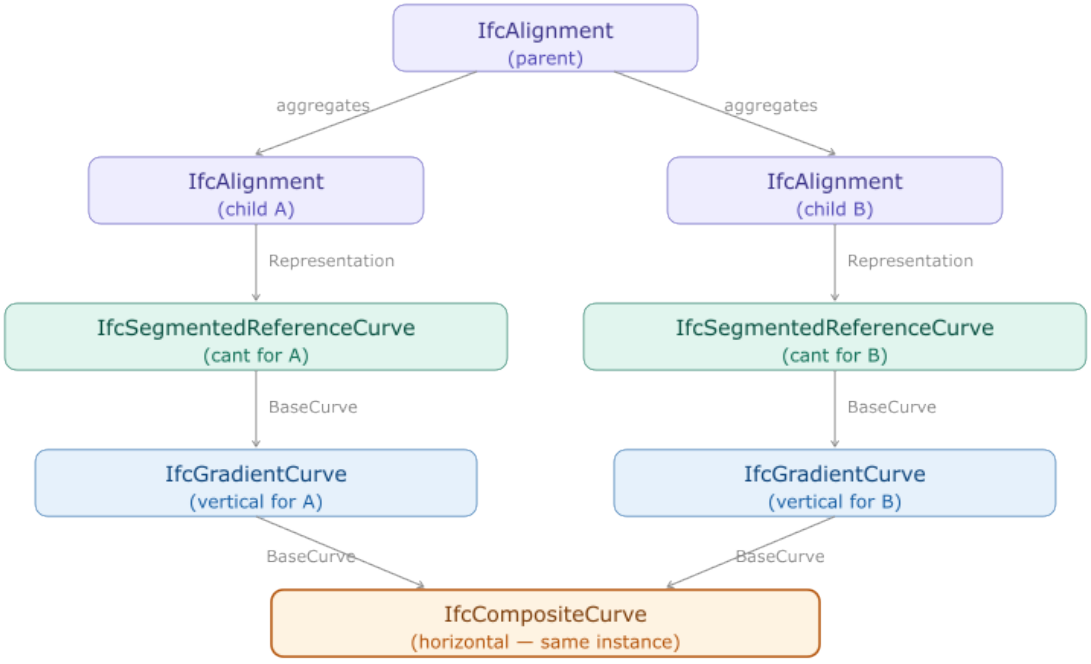


Figure 7.1.2-1 — Geometric chain for child alignments with cant. Each child alignment has its own `IfcSegmentedReferenceCurve` and `IfcGradientCurve`; the `IfcCompositeCurve` is the shared instance.

Shape Representation Placement

Pending resolution of the open questions in §7.2, the following placement convention is recommended based on implementation experience:

- The parent `IfcAlignment` entity carries the `IfcCompositeCurve` as its own `IfcShapeRepresentation` with `RepresentationIdentifier` = `"FootPrint"` and `RepresentationType` = `"Curve2D"`.
- Each child `IfcAlignment` carries its full 3D curve as its own `IfcShapeRepresentation` with `RepresentationIdentifier` = `"Axis"` and `RepresentationType` = `"Curve3D"`.

  This recommendation consistently relates the geometric representation to an instance of `IfcAlignment`.

7.2 Open Implementation Questions

The IFC4x3 specification does not yet provide a concept template for the geometric representation when reusing horizontal layout. The following questions are not resolved and remain active topics for a future normative concept template:

- **Representation ownership.** Should all representations be contained within the parent `IfcAlignment.Representation.Representations`, within each child alignment's `Representation.Representations`, or split between them?
- **Horizontal curve ownership.** Should the `IfcCompositeCurve` representation of the horizontal layout be a representation on the parent `IfcAlignment` or on the `IfcAlignmentHorizontal`?
- **All-or-nothing geometry.** Is the presence of a geometric representation for one alignment a requirement for all alignments in the group to have geometric representations?
- **Mixed representation types.** Are mixed representation types valid within the same group — for example, one child alignment represented as an `IfcGradientCurve` and another as an `IfcOffsetCurveByDistances`?

Until these questions are resolved normatively, implementations should follow the convention described in §7.1.2 and document which representation placement strategy has been adopted.

7.3 Concept Template Transitions in Live Model Management

An application that maintains an IFC model incrementally — adding and removing vertical layouts as the designer works — must handle transitions between two different concept templates:

Vertical count	Required concept template
1	CT 4.1.4.4.1.1 — Alignment Layout – Horizontal, Vertical and Cant
2 or more	CT 4.1.4.4.1.2 — Alignment Layout – Reusing Horizontal Layout

These two templates have structurally different models. CT 4.1.4.4.1.1 nests both `IfcAlignmentHorizontal` and `IfcAlignmentVertical` directly under a single `IfcAlignment` via `IfcRelNests`. CT 4.1.4.4.1.2 uses the parent/child structure described in §7.1.1: the parent nests only the `IfcAlignmentHorizontal`, and each vertical lives in its own child `IfcAlignment` aggregated under the parent. Switching between them is not additive — it requires restructuring the existing model.

7.3.1 Transitioning to Reusing Horizontal Layout

Figure 7.3.1-1 shows the CT 4.1.4.4.1.1 structure used when a single vertical is present:

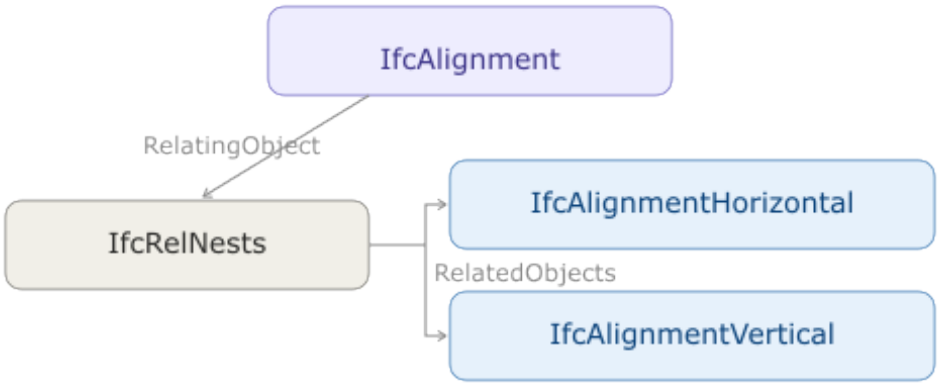


Figure 7.3.1-1 — CT 4.1.4.4.1.1: single `IfcAlignmentVertical` nested directly under `IfcAlignment`.

When a second vertical is added, the model must be restructured to CT 4.1.4.4.1.2. The `IfcAlignmentVertical (V1)` that was directly nested under the original `IfcAlignment` must be moved into a new child alignment. A second child alignment is created for V2, yielding the structure shown in Figure 7.3.1-2:

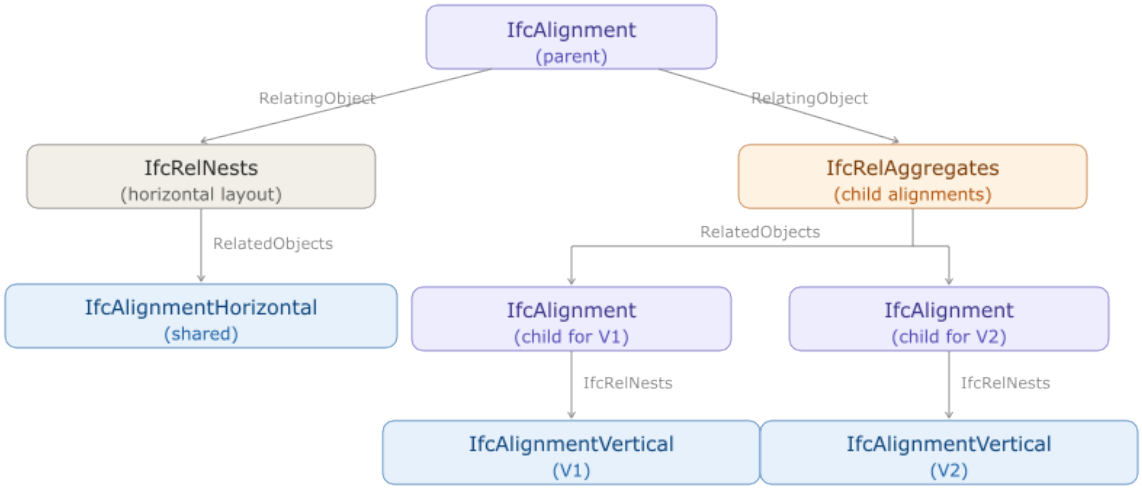


Figure 7.3.1-2 — CT 4.1.4.4.1.2: vertical layouts moved into child alignments aggregated under the parent.

The inverse transition is equally important but easier to overlook. When the designer removes a vertical layout and only one remains, the model must revert back to CT 4.1.4.4.1.1. The sole remaining `IfcAlignmentVertical` is re-nested directly under the alignment, the child `IfcAlignment` instances are dissolved, and the `IfcRelAggregates` relationship is removed — restoring the structure shown in Figure 7.3.1-1.

7.4 Relationship to Offset Curves

Chapter 5 describes `IfcOffsetCurveByDistances`, which defines a new curve laterally displaced from a basis alignment. Offset curves address a different need: they shift the curve horizontally (in plan) rather than reusing the same horizontal path at a different elevation. A left edge-of-shoulder offset and a right edge-of-shoulder offset use `IfcOffsetCurveByDistances` because their horizontal positions differ from the centerline. Features that share the exact same horizontal trace but differ only in elevation use the shared-horizontal-layout pattern described in this section.

The two patterns can coexist. For example, a road model might define: - The centerline as a parent `IfcAlignment` with a full horizontal definition, plus a child carrying the design vertical profile - The left and right edges of pavement as offset curves (lateral displacement, Chapter 5) - The sub-grade profile as another child `IfcAlignment` under the same parent, sharing the centerline's horizontal but carrying an independent vertical profile (this section)

7.5 Use Cases Not Supported by Alignment - Reusing Horizontal Layout Concept Template

The following configurations arise in practice but cannot be fully represented using Concept Template 4.1.4.4.1.2 as currently defined. The IFC specification does not provide other concept templates that support these use cases.

Double-track railway. In this configuration, sometimes called the **7-line geometry** in railway practice, a single central horizontal reference defines both tracks. Each track carries its own vertical profile and cant, while stationing is carried along the common central alignment. The reusing-horizontal-layout template accommodates independent verticals per child alignment, but it provides no mechanism to associate independent horizontal offsets (one track to each side of the reference) with the shared horizontal. The two tracks would require separate horizontal definitions even where they are geometrically parallel, which defeats the purpose of horizontal sharing.

Dual carriageway. A highway with a central reference alignment defining two independent carriageways faces the same problem: each carriageway has a distinct horizontal position offset from the reference, so the shared-horizontal-layout pattern does not apply. Modeling each carriageway as a child of a common parent would require the child alignments to carry their own `IfcAlignmentHorizontal`, which is not permitted under the concept template. IFC4x3 has no explicit concept template for dual-carriageway or multi-track configurations.

Chapter 8 — Linear Placement

8.0 Introduction

Most elements in the built environment for infrastructure works are located **relative to an alignment** rather than by absolute coordinates. A bridge pier, a drainage inlet, a light pole, a traffic sign — all are described in traditional engineering plans by where they fall along a road or railway centerline, how far they sit to the left or right of that centerline, and how high above (or below) a reference elevation they stand. This intuitive "station, offset, elevation" system has been the language of civil engineers for over a century.

`IfcLinearPlacement` is the IFC mechanism that formalizes this concept in accordance with ISO 19148 *Geographic information - Linear referencing*. It places and orients an object relative to a *basis curve* — typically an alignment curve — using an `IfcPointByDistanceExpression` to define the location and `IfcAxis2PlacementLinear` to define the axis directions.

This section covers:

- The concept of linear placement and how it maps from traditional engineering practice into IFC.
- The IFC class hierarchy: `IfcLinearPlacement`, `IfcAxis2PlacementLinear`, and `IfcPointByDistanceExpression`.
- How the local coordinate system is constructed at a placement point.
- Special cases: longitudinal offset for unreachable points, and placement along `IfcOffsetCurveByDistances`.

8.1 Linear Placement in Practice

8.1.1 The Traditional Approach

Before examining IFC, it helps to recall how placement is expressed in conventional civil engineering plans. Two common cases illustrate the idea:

Case 1 — Bridge Pier (2D plan view). A bridge pier is located in plan by its station along the roadway alignment and its lateral offset from the centerline. A plan note might read "Pier 2 CL: Sta. 142+35.75, 12.50 ft Lt." The elevation of the pier top is read separately from a profile view. No absolute coordinates are needed; the alignment provides the reference frame.

Case 2 — Drainage Inlet (3D). A storm drain inlet is placed at Sta. 198+12.42, offset 18.0 ft right of centerline, with its top grate set at elevation 312.75 ft (NAVD88). Here all three spatial dimensions are given: station gives position along the alignment, lateral offset gives the transverse position, and the elevation fixes the vertical position independently of the alignment profile.

Both cases share the same structure: **a distance along a reference curve, plus offsets from it**. `IfcLinearPlacement` captures exactly this structure.

8.1.2 The IFC Object Graph

The IFC classes that implement linear placement form a short chain, shown in Figure 8.1.2-1.

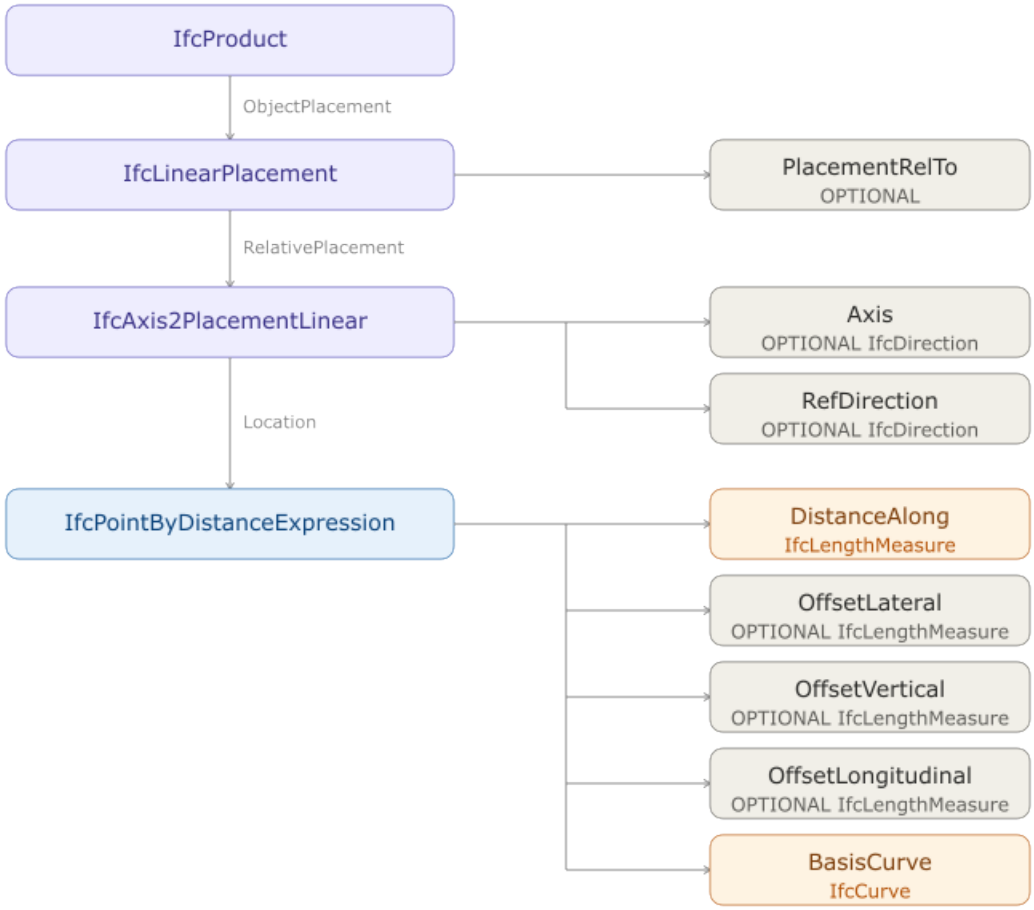


Figure 8.1.2-1 — IFC object graph for linear placement.

The `PlacementRelTo` attribute of `IfcLinearPlacement` establishes the reference context. When it is **omitted**, the placement is measured from the start of the basis curve defined in `IfcPointByDistanceExpression`. This is the standard case when the basis curve is an alignment defined in the project coordinate system.

Figure 8.1.2-2 schematically represents the linear placement of a bridge pier and a drain inlet.

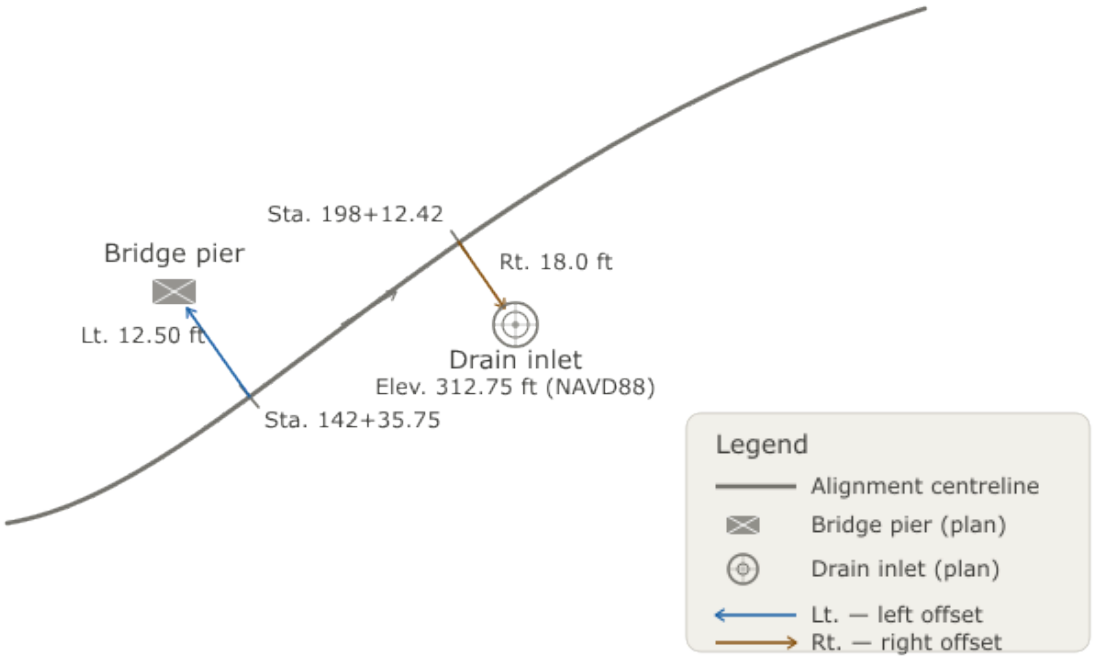


Figure 8.1.2-2 — Conceptual diagram showing a plan view of an alignment with a bridge pier placed at a station/offset and a drain inlet placed at station/offset/elevation.

8.2 IfcLinearPlacement and IfcAxis2PlacementLinear

`IfcAxis2PlacementLinear` performs two independent jobs:

1. **Location** — fixing a point in space, via `Location` (typed `IfcPoint`, but constrained by a WHERE rule to be `IfcPointByDistanceExpression`). The point is found by walking `DistanceAlong` a `BasisCurve` and then applying up to three offsets (§8.2.1).
2. **Orientation** — fixing the direction of the placement's local axes, via the optional `Axis` and `RefDirection` attributes (§8.3).

These two jobs are independent. The offsets that determine `Location` are measured in a frame derived directly from the geometry of `BasisCurve` at `DistanceAlong` — not from whatever `Axis`/`RefDirection` values are chosen for orientation. An implementation evaluates `Location` the same way regardless of how, or whether, `Axis` and `RefDirection` are supplied.

8.2.1 Distance Along the Basis Curve

`IfcPointByDistanceExpression` class has five key attributes:

Attribute	Type	Description
`BasisCurve`	`IfcCurve`	The curve relative to which the point's location is defined.
`DistanceAlong`	`IfcCurveMeasureSelect`	The parametric distance measured along the *basis curve*. Typically a plain `IfcLengthMeasure`.
`OffsetLateral`	`OPTIONAL IfcLengthMeasure`	Signed lateral offset. Positive to the left of the curve's forward tangent; negative to the right (consistent with ISO 19148).
`OffsetVertical`	`OPTIONAL IfcLengthMeasure`	Signed vertical offset. Positive upward relative to `BasisCurve`.
`OffsetLongitudinal`	`OPTIONAL IfcLengthMeasure`	Signed offset in the tangent direction of the `BasisCurve`. See §8.2.1.1 for uses.

For a full 3D alignment (`IfcGradientCurve` or `IfcSegmentedReferenceCurve`), the `DistanceAlong` is measured along the **horizontal projection** of the 3D curve onto the global XY plane — that is, along the underlying `IfcCompositeCurve` that forms the basis of the 2.5D geometry and represents the plan view curve geometry. This is consistent with how stationing is defined in civil engineering: stationing is a horizontal measure.

> The `IfcPointByDistanceExpression.BasisCurve` attribute is poorly named. This curve is the 3D alignment geometry curve composed from the 2D composite curves defining the 2.5D geometry. A better name for this attribute would be Directrix. This would be consistent with `IfcSectionedSolidHorizontal.Directrix`. The basis curve is the curve that forms the basis of the 2.5D geometry. This is the horizontal projection of `IfcPointByDistanceExpression.BasisCurve` onto a horizontal plane. Basis curve and `IfcPointByDistanceExpression.BasisCurve` are similarly named but have very different definitions. For this reason `IfcPointByDistanceExpression.BasisCurve` (or `BasisCurve`) is used when referring to the 3D curve and *basis curve* is used when referring to the 2D projection of the 3D curve onto a horizontal plane.

The three offsets are measured relative to a frame derived from `BasisCurve` itself at `DistanceAlong` — not relative to the `Axis`/`RefDirection` of the enclosing `IfcAxis2PlacementLinear` (§8.3). That curve-derived frame, shown in Figure 8.2.1-1, is defined as follows:

- Let **T** be the tangent to `BasisCurve` at `DistanceAlong`, pointing in the direction of increasing distance.
- **OffsetLateral** is measured perpendicular to **T**, in the horizontal plane (parallel to the global XY plane): positive to the left of **T**, negative to the right (ISO 19148 convention).
- **OffsetVertical** is measured perpendicular to **T**, within the *vertical plane* (the plane perpendicular to the global XY plane) containing **T**.
- **OffsetLongitudinal** is measured along **T**, applied last, from the point already displaced by `OffsetLateral` and `OffsetVertical` (see §8.2.1.1).

This is not the same as elevation. `OffsetVertical` is perpendicular to the tangent **T**, not to the global XY plane. When `BasisCurve` is horizontal at that point, the two happen to coincide, and `OffsetVertical` behaves exactly like a change in elevation. But on a graded alignment the tangent is inclined, so "perpendicular to the tangent" points partly sideways and partly up — `OffsetVertical` is *not* a pure elevation change. §8.3.3 works through a numeric example where confusing "perpendicular to the curve" with "vertical (elevation)" produces different placements.

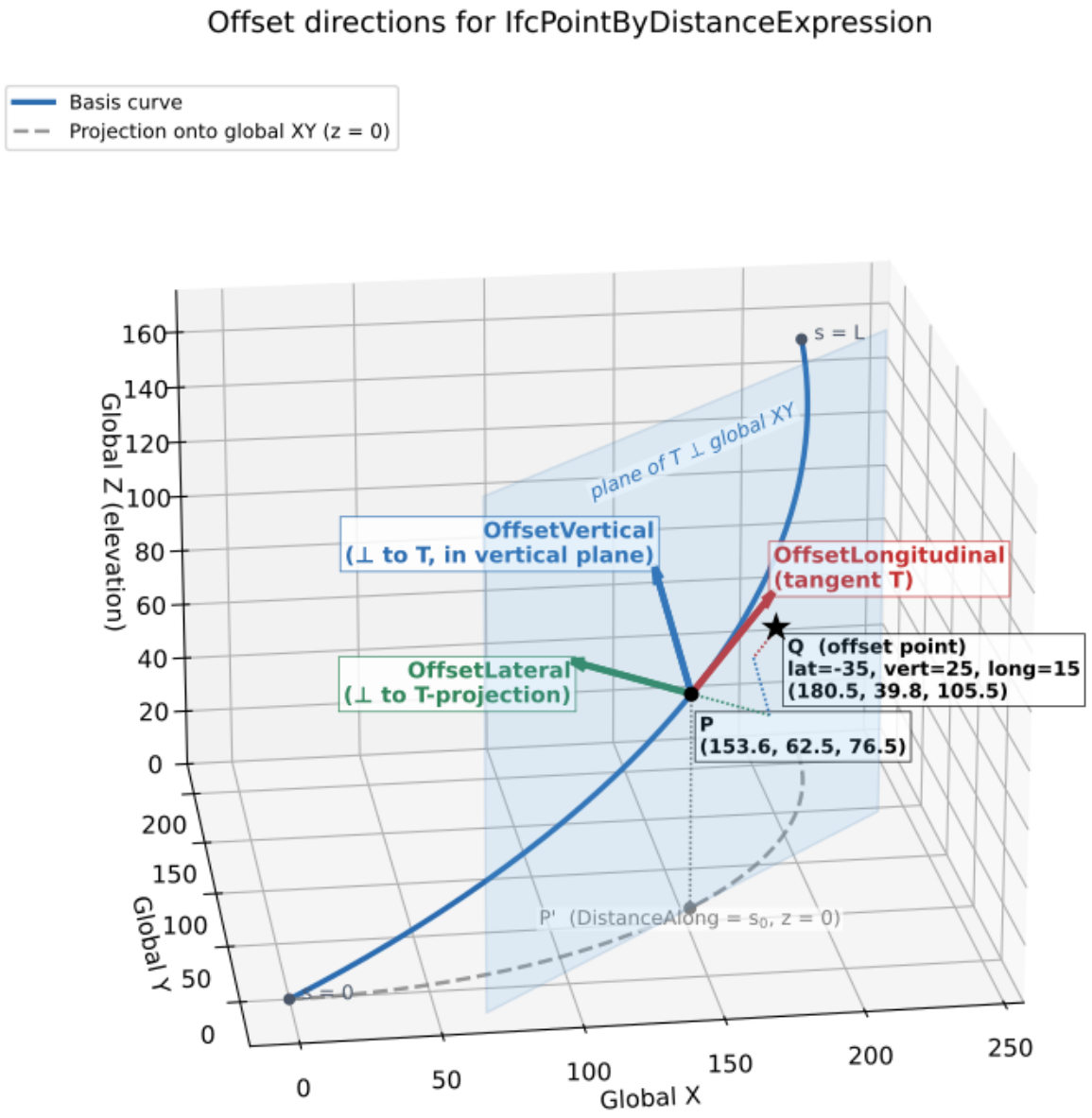


Figure 8.2.1-1 — Offset directions used by `IfcPointByDistanceExpression` to locate a point P on an alignment, where T is the tangent to the alignment at P: `OffsetLateral` measured horizontal and perpendicular to T, `OffsetVertical` measured in the vertical plane containing T, and `OffsetLongitudinal` measured along T. The dashed path shows offset point Q constructed by applying `OffsetLateral`, then `OffsetVertical`, then `OffsetLongitudinal`, in that order.

8.2.1.1 Longitudinal Offset and Unreachable Points

What Is an Unreachable Point?

In plane geometry, every point off a smooth curve can be reached by some combination of distance along the curve plus a perpendicular offset. However, certain geometric configurations in horizontal alignment create locations that **cannot be expressed** as a station plus a purely transverse offset.

The classic example is an **angle point** — the intersection of two tangents in a horizontal alignment where no curve has been inserted. At an angle point, the curve has a sharp corner. A point located “outside” the angle — beyond the apex — lies in a zone where the perpendicular from the curve never reaches. This is depicted in Figure 8.2.1.1-1.

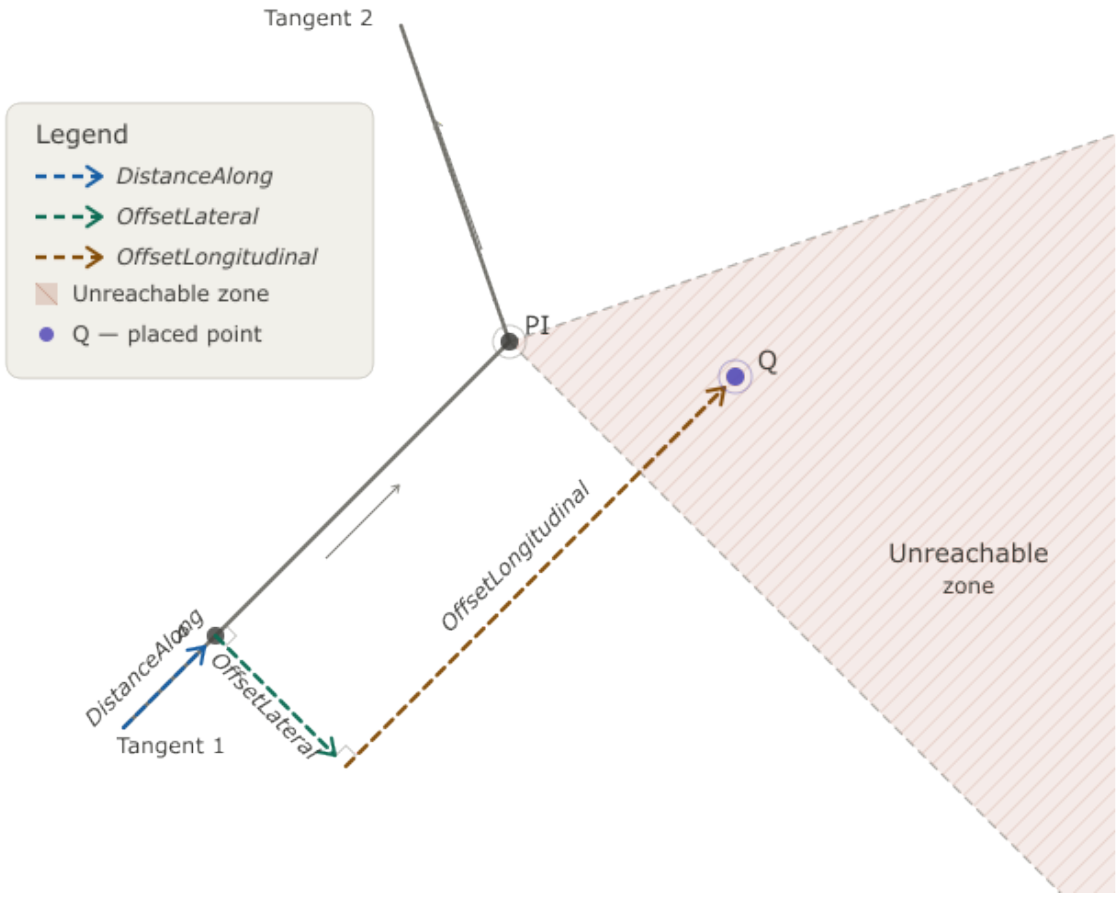


Figure 8.2.1.1-1 — Plan view showing two tangent lines meeting at an angle point (PI). The shaded region outside the angle cannot be reached by a station + lateral offset alone. An object in this region requires a longitudinal offset.

Using OffsetLongitudinal

`IfcPointByDistanceExpression.OffsetLongitudinal` provides the solution. A non-zero `OffsetLongitudinal` moves the placement point along **T** — the same tangent direction

used to define the other offsets in §8.2.1 — after the lateral and vertical offsets are applied. The procedure is:

1. Locate the point on the *basis curve* at `DistanceAlong`, establish the corresponding point on the `BasisCurve`, and determine tangent **T** at that point.
2. Apply `OffsetLateral` perpendicular to **T**, in the horizontal plane.
3. Apply `OffsetVertical` perpendicular to **T**, within the vertical plane containing **T**.
4. Apply `OffsetLongitudinal` along **T**.

The resulting point is no longer "on" a perpendicular to the *basis curve* at `DistanceAlong`, but it is precisely located in 3D space.

> **Practical note:** `OffsetLongitudinal` should be used only when necessary. For all ordinary station-offset placements, it should be omitted.

8.2.2 Stationing and DistanceAlong

Stationing is addressed comprehensively in Chapter 9. For the purposes of linear placement, the key point is that `DistanceAlong` is a **geometric distance from the start of the basis curve**, not a station label. These two quantities are related but not identical:

- **Geometric distance** begins at zero and increases continuously to the total length of the curve.
- **Station value** may begin at an arbitrary value (e.g., 10+00.00 = 1000 ft from some project reference), may include equation gaps or overlaps where stationing is reset, and may use different units (feet vs. meters).

When using `IfcPointByDistanceExpression`, supply the **geometric distance**, not the raw station label. If the alignment has an `IfcReferent` that defines the starting station, the geometric distance equals the station value minus the starting station value (adjusted for any station equations encountered along the way).

8.3 The Placement Coordinate System

§8.2 covered the first job of `IfcAxis2PlacementLinear` — fixing `Location` using offsets measured in a frame derived purely from `BasisCurve`'s own geometry. This section covers the second, independent job: establishing the *orientation* of the placement's local axes via `Axis` and `RefDirection`. Nothing here changes how `Location` was computed in §8.2 — location and orientation are established independently, and an implementation must not let one influence the other.

8.3.1 Role of Axis and RefDirection

The orientation job of `IfcAxis2PlacementLinear` defines a local right-handed coordinate system at the placement point fixed in §8.2. Two optional attributes control that orientation:

Attribute	Role in local CS	Default
`RefDirection`	X-axis (forward direction)	Tangent to the 3D curve at `Location`
`Axis`	Z-axis (up direction)	See §8.3.2

The Y-axis is derived as the cross product of `Axis` × `RefDirection` (after normalization).

When `Axis` and `RefDirection` are both omitted, the implementation must supply defaults. This is the most common scenario and is described below.

8.3.2 Default Axis — An Open Issue

The ambiguity originates in how `IfcAxis2PlacementLinear` describes its optional attributes compared to its 3D counterpart. `IfcAxis2Placement3D` is explicit: its specification states that when `Axis` is omitted it defaults to `(0, 0, 1)` and when `RefDirection` is omitted it defaults to `(1, 0, 0)`. No interpretation is required.

`IfcAxis2PlacementLinear` takes a different approach. Its specification states:

> "Relative placement axes (`Axis` and `RefDirection`) are relative to the curve used for linear referencing provided in `IfcPlacement.Location` (`IfcPointByDistanceExpression.BasisCurve`), maintaining the relationship to the tangent of the curve."

This sentence says the axes are *derived from the curve*, not from global coordinates. The natural reading is that `RefDirection` defaults to the curve tangent — which implementations broadly agree on — and that `Axis` defaults to something perpendicular to that tangent in the general upward direction. On a flat alignment those two readings coincide: the upward perpendicular to a horizontal tangent is `(0, 0, 1)`. On a graded alignment they diverge: "perpendicular to the 3D tangent in the upward direction" tilts with the grade and is no longer `(0, 0, 1)`. The specification never resolves what "upward relative to the curve" concretely means, and that silence is the source of the ambiguity. A known open issue in the buildingSMART community tracks it (see IFC4.x-IF Issue #125 and IFC4.x-development Issue #732); a proposed resolution has been submitted as IFC4.x-development Pull Request #1118 but has not yet been accepted or merged.

Two interpretations exist in practice:

1. **Axis = global Z = (0, 0, 1).** The Z-axis is always vertical regardless of alignment slope. The X-axis follows the horizontal tangent direction even when the curve has a vertical grade. This is natural for plan-oriented placement and matches traditional station-offset-elevation thinking.
2. **Axis = perpendicular to the `IfcPointByDistanceExpression.BasisCurve` tangent, in the vertical plane containing that tangent (the plane containing the tangent that is normal to the global XY plane).** The Z-axis tilts with the grade of the alignment. The local X-axis is truly tangent to the 3D curve. This reading follows directly from the curve-relative language in the specification, and produces the

same orientation that `IfcExtrudedAreaSolid` would use when sweeping along the same path.

The connection between interpretation 2 and the broader IFC linear referencing model is not incidental. When evaluating a point along a 3D curve such as `IfcGradientCurve` or `IfcSegmentedReferenceCurve`, the resulting coordinate frame at that point is derived from the curve itself — the tangent at the evaluation distance determines the local forward direction, and the normal plane at that point governs the transverse and vertical axes. This is the frame that implementations compute when moving along a 3D alignment, as illustrated in the discussions of vertical and cant alignment geometry in Chapters 3 and 4. The perpendicular-to-3D-tangent interpretation of `Axis` is most consistent with that model: both derive orientation from the same 3D curve tangent, so a placement using interpretation 2 produces a frame congruent with what an implementation computes when evaluating the directrix at the same distance.

The practical consequence of this ambiguity is graphically illustrated in §10.5.6 — Figure 10.5.6-2 shows the same `IfcSectionedSolidHorizontal` model with `Axis` omitted rendered by two viewers that adopted opposite defaults, producing visibly different geometry on a 10% grade.

8.3.3 Linear Placement Example

This example evaluates an `IfcLinearPlacement` for an object located along an alignment. The object is to be located at a `DistanceAlong` of 201.0 m and a lateral offset of 2.0 m. The object is to be located 2.5 m above the alignment, measured vertically, and is to be oriented vertically. The desired placement of the object is at `(201.0, 2.0, 52.75)`.

The horizontal alignment is straight towards the East. The vertical alignment is a constant gradient at a slope of 0.25 and starts at elevation 0.0.

The gradient parameters for a slope of 0.25 are $d_x = 0.9701425,\ d_y = 0.2425356.$

One option is to define the object's position relative to the gradient curve using horizontal and vertical offsets. The relevant IFC is:

```
#5208 = IFCGRADIENTCURVE(/*details omitted*/)
#5297 = IFCLINEARPLACEMENT($, #5303, $);
#5303 = IFCAXIS2PLACEMENTLINEAR(#5304, #5305, $);

/* Placement defined relative to gradient curve */
#5304 = IFCPOINTBYDISTANCEEXPRESSION(IFCLENGTHMEASURE(201.), 2., 2.5, $,
#5208);

/* Explicitly defined Axis direction */
#5305 = IFCDIRECTION((0., 0., 1.));
```

The object will be oriented vertically because the `IfcAxis2PlacementLinear.Axis` is explicitly defined as `(0,0,1)`. If `IfcAxis2PlacementLinear.Axis` is omitted, the default orientation of the object may depend on the implementation as discussed in §8.3.2.

The point on the gradient curve is:

$$x = 201.0$$

$$z = (201.0)\frac{(0.2425356)}{(0.9701425)} = 50.25$$

The vertical offset is perpendicular to the vertical gradient. The offset from the gradient curve point is:

$$\Delta x = (2.5)(-0.2425356) = -0.606$$

$$\Delta y = (2.5)(0.9701425) = 2.425$$

The resulting placement is:

$$x + \Delta x = 201.0 - 0.606 = 200.394$$

$$y + \Delta y = 50.25 + 2.425 = 52.675$$

The object's placement is `(200.394, 2.0, 52.675)`. This is not the desired placement. The object is not in the correct location because `OffsetVertical` is measured perpendicular to the curve's tangent (§8.2.1), not as a pure elevation change — on this 0.25 gradient the two are not the same.

The approach to achieve the desired placement is to use an `IfcLinearPlacement.PlacementRelTo` with $z = 2.5$.

```
/* Local placement with z = 2.5. Linear placement is relative to this
coordinate frame. */
#5139 = IFCLOCALPLACEMENT($, #5143);
#5140 = IFCDIRECTION((0., 0., 1.));
#5141 = IFCDIRECTION((1., 0., 0.));
#5142 = IFCCARTESIANPOINT((0., 0., 2.5));
#5143 = IFCAXIS2PLACEMENT3D(#5142, #5140, #5141);

#5208 = IFCGRADIENTCURVE(/*details omitted*/)

/* Linear placement relative to the local placement defined above. */
#5297 = IFCLINEARPLACEMENT(#5139, #5303, $);

#5303 = IFCAXIS2PLACEMENTLINEAR(#5304, #5305, $);

/* Placement defined relative to gradient curve. Note that vertical offset is
omitted */
#5304 = IFCPOINTBYDISTANCEEXPRESSION(IFCLENGTHMEASURE(201.), 2., $, $,
#5208);

/* Explicitly defined Axis direction */
#5305 = IFCDIRECTION((0., 0., 1.));
```

From before, the point on the gradient curve is:

$$x = 201.0, \quad z = 50.25$$

There is no vertical offset relative to the gradient curve. However, this point is relative to a local placement with $z = 2.5$. The elevation of the object is $50.25 + 2.5 = 52.75$.

The object's placement is `(201.0, 2.0, 52.75)`.

In this approach, the location of the object relative to the gradient curve is treated as an incremental offset from the origin of the `IfcLinearPlacement.PlacementRelTo` coordinate system.

8.4 Fallback Cartesian Position

`IfcLinearPlacement.CartesianPosition` is an optional `IfcAxis2Placement3D` attribute that provides a fallback geometry placement for receiving applications that do not support linear placement. An importing application that does not implement linear placement evaluation can read the stored Cartesian frame directly, without evaluating any curve geometry.

The attribute is populated by the exporting application, which evaluates `RelativePlacement` against the alignment geometry and stores the resulting absolute 3D frame in the project coordinate system before writing the file. The buildingSMART Validation Service treats `CartesianPosition` as a best-practice requirement: a file that omits it will be flagged as not conforming to best practices, and a file that includes it must pass consistency checking — the origin and axes of `CartesianPosition` must match the evaluated result of `RelativePlacement` to within model tolerance. A mismatched fallback is worse than no fallback, because it silently places the object at the wrong location for receivers that fall back to the Cartesian frame.

8.5 Linear Placement along IfcOffsetCurveByDistances

`IfcOffsetCurveByDistances` is an interpolated curve defined by a series of offset values measured from a basis curve. The offset values at intermediate positions are linearly interpolated between the defined sample points, forming a piecewise-linear offset profile. This is used, for example, to define a road edge line whose lateral distance from the centerline varies gradually. Offset curves are comprehensively discussed in Chapter 5.

8.5.1 The Approximate Length Problem

Because `IfcOffsetCurveByDistances` is a sampled, interpolated curve rather than an analytically defined curve, its **arc length is only approximate**. The arc length depends on the density of the sample points along the curve: more sample points produce a more accurate length estimate, but the length is never exact for a truly curved basis.

This approximation has a critical consequence for linear placement: when `IfcPointByDistanceExpression.BasisCurve` is an `IfcOffsetCurveByDistances`, the `DistanceAlong` value cannot be mapped to a unique, precisely determined point on the

curve. Two implementations with different sampling densities may compute slightly different positions for the same `DistanceAlong` value.

8.5.2 Recommendations

For applications requiring precise linear placement:

1. **Prefer using the parent alignment** (e.g., the `IfcCompositeCurve` representing the horizontal alignment) as the `BasisCurve`, and use `OffsetLateral` to account for any transverse offset from the centerline. This avoids the approximation problem entirely.
2. **If placement along an offset curve is unavoidable**, document the sampling density of the `IfcOffsetCurveByDistances` so that receivers can evaluate the precision of derived positions.
3. **Do not rely on** `DistanceAlong` values along an `IfcOffsetCurveByDistances` being reproducible across different software implementations.

8.6 Summary and Implementation Checklist

#	Item	Notes
1	Use `IfcLinearPlacement` for all infrastructure elements located relative to an alignment.	Prefer this over `IfcLocalPlacement` with absolute coordinates for alignment-relative objects.
2	Supply `DistanceAlong` as the geometric arc length from the start of `BasisCurve`.	Convert station labels to geometric distances first; use `IfcReferent` to record station label metadata.
3	Use signed `OffsetLateral`: positive = left, negative = right.	Consistent with ISO 19148 convention.
4	Do not assume `OffsetVertical` is a true elevation offset.	`OffsetVertical` is measured perpendicular to the curve's tangent (§8.2.1), which only equals elevation when the alignment is flat at that point. For a true vertical (elevation) offset on a graded alignment, place the object relative to a `PlacementRelTo` frame instead (see the worked example in §8.3.3).
5	Set `Axis = (0,0,1)` explicitly to remove ambiguity.	Do not rely on the default; the default is not unambiguously defined in the IFC schema.
6	Use `OffsetLongitudinal` only for geometrically unreachable points (e.g., outside an angle point).	For all ordinary placements, omit or set to zero.
7	Avoid using `IfcOffsetCurveByDistances`	Use the parent alignment with

#	Item	Notes
	as `BasisCurve` for precise placement.	`OffsetLateral` instead.
8	Provide `CartesianPosition` only when exchanging with applications that lack linear placement support; omit otherwise.	Regenerate it after any alignment geometry change; validate consistency with the buildingSMART Validation Service.

Chapter 9 — Referents and Stationing

9.0 Introduction

Chapter 8 established that `IfcPointByDistanceExpression.DistanceAlong` is a **geometric distance** — a raw arc-length measure from the start of a curve. But civil engineering practice does not communicate location this way. Engineers say "Pier 3 is at Station 14+235.75," not "Pier 3 is 14,235.75 feet from the start of the survey baseline." The station label is a human-readable, project-specific identifier that may begin at an arbitrary value, may use different units than the project, and may not even progress continuously along the alignment.

`IfcReferent` is the IFC mechanism that bridges these two worlds. It is a positioned object that attaches 1D semantic location information — most importantly, stationing — to a specific geometric point on an alignment. It does not change the geometry; it annotates it.

`IfcReferent` plays two distinct roles, and this chapter is organized around them:

- What `IfcReferent` is and the range of uses it serves beyond stationing.
- Stationing as a concept: what a station is, why station equations (gaps and overlaps) exist, how stationing can run *against* the alignment direction (reverse stationing), and how to convert between station labels and `DistanceAlong` in each case.
- The **geometric role** — how a referent is attached to its own alignment: `Pset_Stationing`, `IfcRelNests`, and the nesting/placement edge cases that follow directly from that mechanism (the mixed-nesting problem, `ObjectPlacement` for semantic-only alignments, key-point referents marking a layout's own segment transitions, and — per ISO 19148 — why an `INTERSECTION` referent is owned by one alignment rather than shared between two).
- The **semantic role** — how a referent labels another product's position via `IfcRelPositions`, covering the single-point pattern (Product Relative Positioning) and the start/end-extent pattern (Product Span Positioning).
- How the fact that two alignments physically cross — as opposed to the station label at that location — is recorded independently of any referent, using `IfcRelInterferesElements` between the facilities the alignments lay out.
- ISO 19148 Linear Referencing: the underlying model of Linear Referencing Methods (LRM) that stationing, kilometer-posting, and similar conventions all instantiate, and how it relates to `DistanceAlong`.

9.1 What Is a Referent?

`IfcReferent` is a subtype of `IfcPositioningElement`, which is itself a subtype of `IfcProduct`. Like any product, it has an `ObjectPlacement` (typically `IfcLinearPlacement`)

that locates it geometrically on an alignment, and it can carry property sets that provide additional semantic information at that location.

The single attribute `IfcReferent` adds beyond its supertypes is `PredefinedType` (`IfcReferentTypeEnum`), which guides receivers on how to interpret the referent and which property sets are relevant:

- **Stationing system definition** (`STATION`). Defines the alignment's own stationing system: the starting station or a station equation. A `STATION` referent is nested under `IfcAlignment` via `IfcRelNests` (§9.3.1) — it establishes stationing, it does not label another object.
- **Linear referencing events** (`SUPERELEVATIONEVENT`, `WIDTHEVENT`). Locations where a design parameter changes — superelevation transitions, width changes, and similar cross-section events.
- **Reference markers and mileposts** (`REFERENCEMARKER`, `KILOPOINT`, `MILEPOINT`). Physical objects in the right-of-way that are part of a real-world linear referencing system.
- **Administrative boundaries** (`BOUNDARY`). Locations where a jurisdiction, maintenance zone, or asset ownership boundary crosses the alignment.
- **Intersection locations** (`INTERSECTION`). Points where another road or route crosses the alignment.
- **Position of another object** (`POSITION`). The most common use. Gives the station label of some other product's location — a bridge pier, a sign, a guardrail run — without defining the stationing system itself. A `POSITION` referent is linked to the object it labels via `IfcRelPositions` (§9.4).

`STATION` and `POSITION` are easy to conflate, since both carry `Pset_Stationing` and both resolve to a station label. The distinction is structural, not just semantic: a `STATION` referent is *part of* the alignment's stationing system (`IfcRelNests`), while a `POSITION` referent *consumes* that system to label something else (`IfcRelPositions`) — the two are never the same referent instance.

9.2 Stationing

9.2.1 What Is a Station?

Stationing (also called *chainage* in British practice) is a distance-based coordinate system used to identify locations along a linear route. In North American highway practice, stations are expressed in the form `ccc+dd.dd`, where the number before the `+` is the count of hundreds of feet and the digits after are the remainder. For example, Sta. 142+35.75 means 14,235.75 feet from the alignment start station.

The project datum — the zero point of stationing — is defined by the first `IfcReferent` nested in the alignment (see §9.3.1). Stationing often begins at a round number such as

10+00 or 100+00 rather than 0+00, to avoid negative stations at points that may be surveyed upstream of the formal project start.

Stationing also has a **direction**. Normally station values increase in the direction of increasing `DistanceAlong` — the direction the alignment geometry runs. An alignment may instead be *reverse-stationed*, with station values that decrease as `DistanceAlong` increases; §9.2.7 covers this case and the `Pset_Stationing.HasIncreasingStation` flag that records it.

Stationing is a **display convention**, not a geometric quantity. The geometric position of a point is fully determined by `DistanceAlong` in `IfcPointByDistanceExpression`. The station label is metadata that identifies that point to engineers and field crews in human-readable terms.

9.2.2 Why Station Equations Exist

A station equation (also called a *chainage break* or *stationing equation*) is a discontinuity in the stationing system at which the station value jumps. Equations arise in several common situations:

Realignment. When a road is realigned — a curve is straightened or a new bypass is built — the new alignment geometry is a different length than the old. Rather than re-station the entire project downstream of the change (which would invalidate all existing plan sheets, permits, and as-built records), engineers introduce a station equation at the point where the old and new alignments reconnect. The geometry changes; the stationing on the downstream portion is preserved.

Survey accumulation. Long routes are often surveyed in segments. Small differences in measurement between survey crews produce a gap or overlap at the junction point. Rather than re-survey everything, an equation is introduced.

Existing route adoption. When a new project ties into an existing road with its own stationing, an equation reconciles the two stationing systems at the junction.

9.2.3 Gap Equations

A **gap equation** (also called a *station ahead* or *forward equation*) occurs when stationing jumps forward. The alignment geometry is continuous, but the station number increases by more than the physical length at the equation point.

Example: The alignment arrives at a point with incoming station 142+35.75. Due to a realignment that shortened the route, the outgoing station is 145+00.00. There is a gap of 264.25 ft. A distance of 14,235.75 ft of geometry corresponds to a label of 145+00, not 142+35.75.

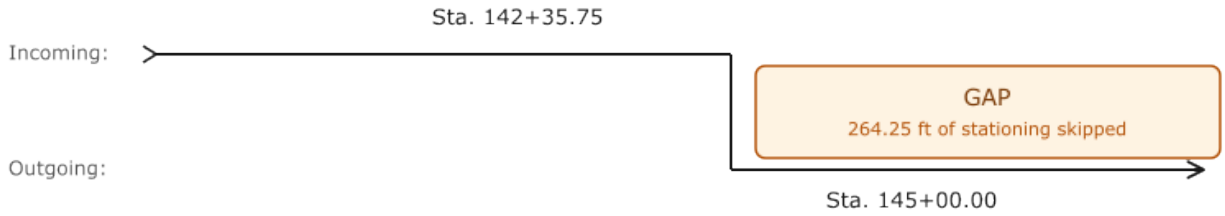


Figure 9.2.3-1 — Station gap equation. Geometry is continuous; station numbers jump forward.

9.2.4 Overlap Equations

An **overlap equation** (also called a *station back* or *backward equation*) occurs when stationing jumps backward. The alignment geometry is continuous, but the station number decreases at the equation point.

Example: Incoming station 78+42.10, outgoing station 76+00.00. There is an overlap of 242.10 ft. Two different geometric points — one upstream and one downstream of the equation — carry station labels between 76+00 and 78+42. Software implementations may consider adding an indicator, such as a suffix, to the station label to indicate which stationing region the station label applies to.

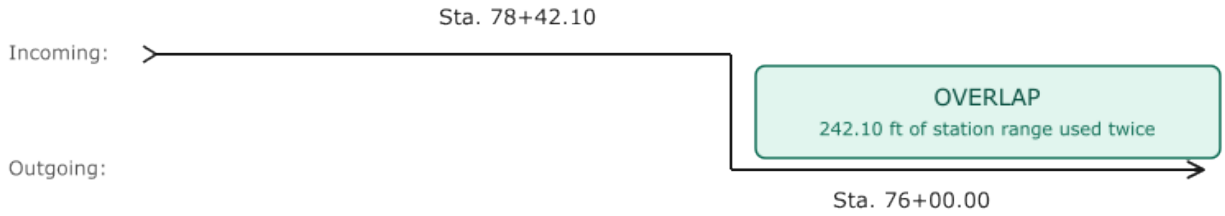


Figure 9.2.4-1 — Station overlap equation. Geometry is continuous; station numbers jump backward.

9.2.5 Effect on `DistanceAlong` Conversion

The presence of station equations means that **you cannot convert a station label to `DistanceAlong` by simple subtraction of the starting station**. You must account for every equation encountered between the alignment start and the point of interest.

The conversion algorithm is:

1. Sort all referents on the alignment by `DistanceAlong`. For each referent record two values:
    - D_i: the `DistanceAlong` read directly from its `IfcLinearPlacement`
    - S_i: the outgoing station — `Pset_Stationing.Station`

2. *(Optional validation)* For any referent that also carries `IncomingStation`, verify that $S_{i-1} + (D_i - D_{i-1}) \approx$ IncomingStation. This confirms that the station count in the preceding segment matches the geometry.
3. To convert a target station S to `DistanceAlong`, find the last referent i such that $S_i \leq S$. Then:

$$\text{DistanceAlong} = D_i + (S - S_i)$$

Before applying the formula, check whether the station overshoots the next referent: if a next referent exists and $S - S_i > D_{i+1} - D_i$, the station falls inside a gap — it was skipped by a forward equation — and `DistanceAlong` is undefined.

The algorithm above assumes station values **increase** with `DistanceAlong`. When a referent carries `Pset_Stationing.HasIncreasingStation = FALSE`, the region it governs runs the opposite way (§9.2.7): the selection test in step 3 becomes *the last referent i, ordered by* `DistanceAlong`*, with* $S_i \geq S$, and the conversion generalizes to

$$\text{DistanceAlong} = D_i + \sigma_i\,(S - S_i), \qquad \sigma_i = \begin{cases} +1 & \text{increasing region} \\ -1 & \text{decreasing region} \end{cases}$$

The step-2 validation, the gap/overshoot check (compare $\sigma_i\,(S - S_i)$ against $D_{i+1} - D_i$), and the sign convention that distinguishes a gap equation from an overlap equation all flip with σ_i. §9.2.7 works through a decreasing-stationing example.

Figure 9.2.5-1 illustrates how the geometric distance axis and the station label axis diverge when equations are present.

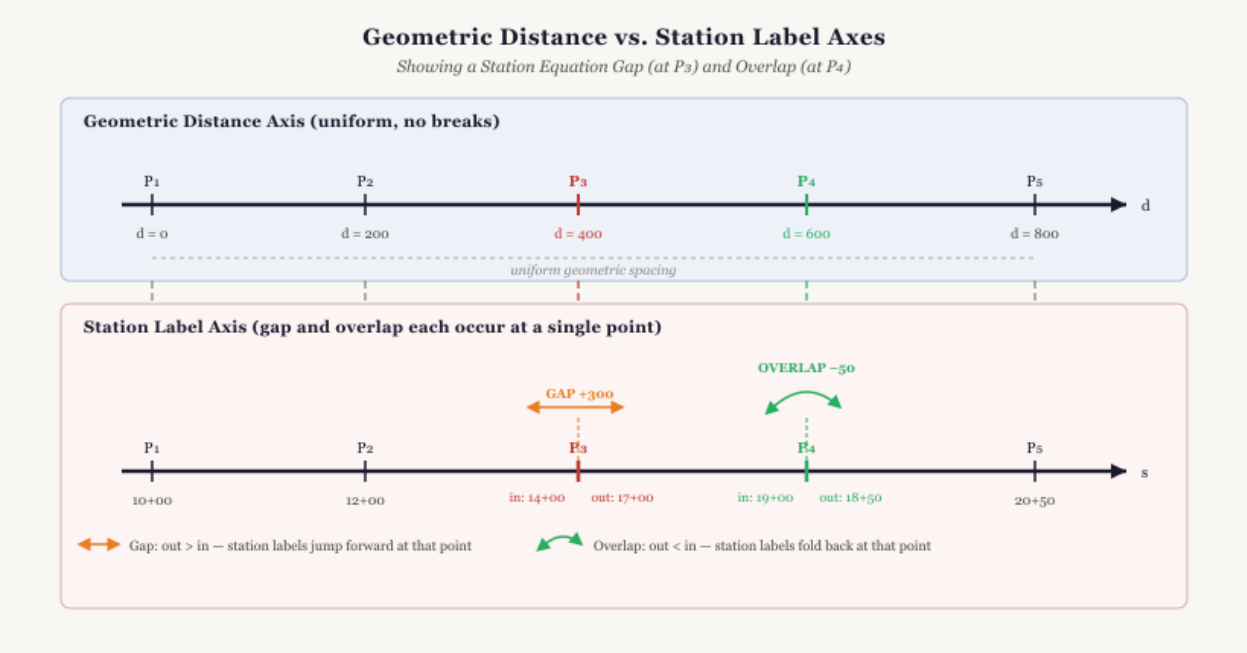


Figure 9.2.5-1 — Two-axis diagram comparing DistanceAlong (geometric distance) and station label for an alignment with one gap equation and one overlap equation.

9.2.6 Example: `DistanceAlong` Conversion with Gap and Overlap Equations

The table below reflects the alignment shown in Figure 9.2.5-1: five points at 200-unit intervals along a generic alignment, a gap equation at P3, and an overlap equation at P4. Only P1, P3, and P4 map to actual `IfcReferent` instances (§9.3.1) — P1 as the starting station, P3 and P4 at the equations. P2 and P5 are ordinary points on the alignment with no referent of their own; they appear here only to anchor the geometric spacing.

Point	D_i	S_i outgoing	`IncomingStation`	Note
P1	0	1000.00 (Sta. 10+00)	—	Starting station
P2	200	1200.00 (Sta. 12+00)	—	
P3	400	1700.00 (Sta. 17+00)	1400.00 (Sta. 14+00)	Gap = 300
P4	600	1850.00 (Sta. 18+50)	1900.00 (Sta. 19+00)	Overlap = 50
P5	800	2050.00 (Sta. 20+50)	—	

Between P2 and P3 — Sta. 13+00 (1300.00):

Last i where $S_i \leq 1300$: P2 ($S_2 = 1200$, $D_2 = 200$).

$$\text{DistanceAlong} = 200 + (1300 - 1200) = 300$$

Between P3 and P4 — Sta. 18+00 (1800.00):

Last i where $S_i \leq 1800$: P3 ($S_3 = 1700$, $D_3 = 400$).

$$\text{DistanceAlong} = 400 + (1800 - 1700) = 500$$

Between P4 and P5 — Sta. 19+25 (1925.00):

Last i where $S_i \leq 1925$: P4 ($S_4 = 1850$, $D_4 = 600$).

$$\text{DistanceAlong} = 600 + (1925 - 1850) = 675$$

Without accounting for the equations, naive subtraction gives $1925 - 1000 = 925$ — an error of 250. The error is the net station inflation introduced by the two equations: the gap at P3 added 300 station units with no corresponding geometry, and the overlap at P4 recovered 50, leaving a net surplus of $300 - 50 = 250$ station units.

Gap check — Sta. 15+00 (1500.00):

- P1: $S_1 = 1000 \leq 1500$ ✓
- P2: $S_2 = 1200 \leq 1500$ ✓

- P3: $S_3 = 1700 > 1500$ ✗

Last is P2 ($D_2 = 200, S_2 = 1200$). Before computing, check whether the station overshoots the next referent: $S - S_2 = 1500 - 1200 = 300$, but the geometric span to P3 is only $D_3 - D_2 = 400 - 200 = 200$. Since $300 > 200$, Sta. 15+00 falls inside the gap — it was skipped by the forward equation at P3 and `DistanceAlong` is undefined. Any station between Sta. 14+00 and Sta. 16+99 produces the same result.

Implementations must handle this case gracefully — returning an error, `null`, or a domain-specific sentinel value rather than silently producing a geometrically meaningless `DistanceAlong`.

Overlap ambiguity — Sta. 18+75 (1875.00):

The overlap zone spans Sta. 18+50 (1850) to Sta. 19+00 (1900) — the range of station values that appear on both the incoming segment (P3→P4) and the outgoing segment (P4→P5). Station 18+75 falls inside this zone and resolves to two distinct geometric points:

As a point in the P3→P4 segment (before the equation): last i where $S_i \leq 1875$ ignoring P4 gives P3 ($S_3 = 1700, D_3 = 400$):

$$\text{DistanceAlong} = 400 + (1875 - 1700) = 575$$

As a point in the P4→P5 segment (after the equation): the algorithm as defined returns P4 ($S_4 = 1850, D_4 = 600$):

$$\text{DistanceAlong} = 600 + (1875 - 1850) = 625$$

Both answers are geometrically valid. The algorithm as defined in §9.2.5 always returns the post-equation result (625 here), silently discarding the pre-equation match. Implementations that need to resolve ambiguous overlap stations must detect the overlap zone — any S where $S_i^{\text{outgoing}} \leq S \leq S_i^{\text{incoming}}$ at an overlap equation referent — and either return both candidate distances or apply project-specific rules to select one.

9.2.7 Decreasing (Reverse) Stationing

Sections 9.2.1–9.2.6 assume station values increase in the direction the alignment geometry runs — the direction of increasing `DistanceAlong`. This is the usual case, but it is not required. An alignment is **reverse-stationed** (or *decreasing-stationed*) when its station labels decrease as `DistanceAlong` increases. It is recorded by setting `Pset_Stationing.HasIncreasingStation = FALSE` on the starting-station referent (§9.3.1).

`DistanceAlong` is unchanged. `DistanceAlong` is always the geometric arc length from the start of the basis curve, always increasing (Chapter 8). Reverse stationing reverses only the *label* axis. For the same reason, the geometric conventions that depend on travel direction — the positive-curvature sign ("curves to the left"), the cross-section local axes of Chapter 10 — follow increasing `DistanceAlong`, not the station-label direction, and are unaffected.

Referent ordering is unchanged. Referents are still nested in `IfcRelNests.RelatedObjects` in order of increasing `DistanceAlong` (§9.3.1) — which, under reverse stationing, means order of *decreasing* `Station`. Read referents by sorting on `DistanceAlong`, never on `Station`.

Converting station to `DistanceAlong`. The algorithm of §9.2.5 generalizes by attaching a direction sign σ_i to each referent's region:

$$\sigma_i = \begin{cases} +1 & \text{referent } i \text{ governs an increasing-station region} \\ -1 & \text{referent } i \text{ governs a decreasing-station region} \end{cases}$$

- **Select the governing referent.** For an increasing region, take the last referent i (by `DistanceAlong`) with $S_i \leq S$. For a decreasing region, take the last referent i (by `DistanceAlong`) with $S_i \geq S$.
- **Convert.** DistanceAlong $= D_i + \sigma_i\,(S - S_i)$.
- **Validate** (optional). At a referent carrying `IncomingStation`, check $S_{i-1} + \sigma_{i-1}\,(D_i - D_{i-1}) \approx$ IncomingStation.
- **Gap / overshoot check.** If a next referent exists and $\sigma_i\,(S - S_i) > D_{i+1} - D_i$, the station was skipped by an equation and `DistanceAlong` is undefined.

The sign convention that distinguishes a gap equation from an overlap equation flips with direction:

Region direction	Gap equation (labels skipped)	Overlap equation (labels repeated)
Increasing	`Station` > `IncomingStation`	`Station` < `IncomingStation`
Decreasing	`Station` < `IncomingStation`	`Station` > `IncomingStation`

9.2.7.1 Worked Example: Reverse Stationing with a Gap Equation

An alignment is stationed against an agency convention whose values increase from the route's north terminus. The alignment geometry was captured running south from that terminus, so `DistanceAlong = 0.0` sits at the *highest*-station end and station labels count down as `DistanceAlong` increases. A past realignment shortened the middle of the route, leaving one gap equation. Three referents are nested under the alignment, in order of increasing `DistanceAlong`:

Point	D_i	S_i outgoing	`IncomingStation`	Note
R1	0	2000.00 (Sta. 20+00)	—	Starting station; `HasIncreasingStation = FALSE`
R2	400	1550.00 (Sta. 15+50)	1600.00 (Sta. 16+00)	Gap = 50
R3	800	1150.00 (Sta. 11+50)	—	End referent

All three regions are decreasing, so $\sigma_i = -1$ throughout.

Consistency of the input. Advancing 400 along the geometry from R1 reaches a label of $2000 - 400 = 1600.00$ — matching R2's `IncomingStation`. At R2 the outgoing `Station` (1550.00) is *less than* the incoming (1600.00); by the table above, that is a **gap** in a decreasing region, skipping the 50 from 1600 down to 1550. Advancing another 400 from R2 reaches $1550 - 400 = 1150.00$ — matching R3's `Station`.

Sta. 18+00 (1800.00). Last referent with $S_i \geq 1800$ is R1 ($S_1 = 2000$, $D_1 = 0$):

$$\text{DistanceAlong} = 0 + (-1)(1800 - 2000) = 200$$

Sta. 13+00 (1300.00). Last referent with $S_i \geq 1300$ is R2 ($S_2 = 1550$, $D_2 = 400$):

$$\text{DistanceAlong} = 400 + (-1)(1300 - 1550) = 650$$

Naive subtraction from the starting station — $2000 - 1300 = 700$ — overstates the distance by 50. The error is exactly the gap: the equation at R2 dropped 50 station units with no corresponding geometry, so every station past R2 sits 50 units lower than its distance from R1 alone would imply.

Gap check — Sta. 15+75 (1575.00). This label lies between R2's outgoing 1550 and incoming 1600.

- R1: $S_1 = 2000 \geq 1575$ ✓
- R2: $S_2 = 1550 \geq 1575$ ✗

The governing referent is R1. Before converting, apply the overshoot check: $\sigma_1 (S - S_1) = (-1)(1575 - 2000) = 425$, but the geometric span to R2 is only $D_2 - D_1 = 400$. Since $425 > 400$, Sta. 15+75 falls in the range the equation at R2 skipped — it has no geometric position, and `DistanceAlong` is undefined. Any station between Sta. 15+50 and Sta. 16+00 gives the same result. As in §9.2.6, implementations must return an error, `null`, or a sentinel rather than a meaningless distance.

Figure 9.2.7-1 shows the two axes for this alignment.

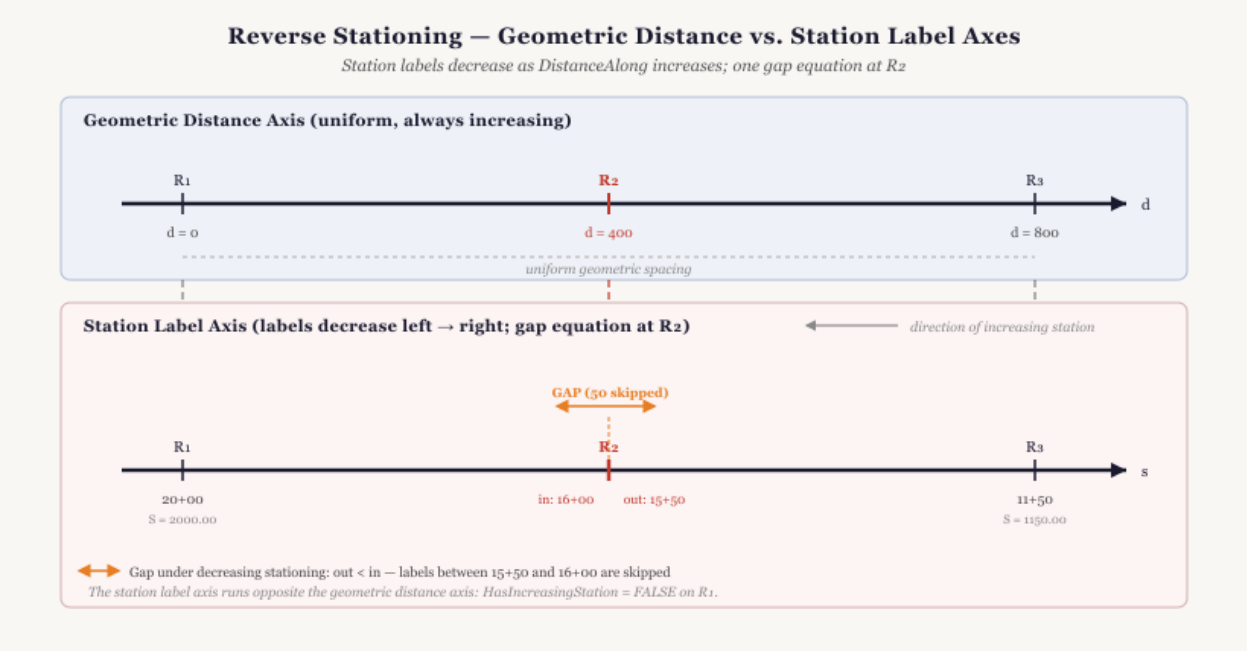


Figure 9.2.7-1 — Reverse stationing: the station label axis decreases in the direction of increasing `DistanceAlong`. A gap equation at R2 skips 50 station units. Contrast with Figure 9.2.5-1, where the label axis increases.

9.3 Attaching Referents to an Alignment

This section covers the **geometric role** of `IfcReferent`: how a referent is nested under, ordered along, and placed on the alignment it belongs to.

9.3.1 Defining Stationing in IFC

With the stationing concept established in §9.2, this section describes how it is encoded: `IfcReferent` carries the station value via `Pset_Stationing`, and referents are ordered along the alignment via `IfcRelNests`.

Stationing metadata is stored in `Pset_Stationing`, attached to an `IfcReferent` via `IfcRelDefinesByProperties`. The property set has three properties:

Property	Type	Description
`Station`	`IfcLengthMeasure`	The station value at this referent's location. For the first referent, this defines the starting station of the alignment.
`IncomingStation`	`IfcLengthMeasure`	Present only at a station equation. The station value on the *incoming* (upstream) side of the break. The `Station` property holds the *outgoing* value immediately after the break.
`HasIncreasingStation`	`IfcBoolean`	Records the direction of stationing for the referents nested *after* this one. If `TRUE` or absent, those referents are

Property	Type	Description
		expected to carry greater `Station` values (increasing stations); if `FALSE`, lower values (decreasing stations). Set `FALSE` on the starting referent for a reverse-stationed alignment — see §9.2.7.

The `Station` value is expressed in the same units as the project length unit (typically meters or feet), **not** in the `ccc+dd.dd` string form. A station of 142+35.75 ft is stored as `14235.75` with feet as the project unit.

`IfcReferent` instances are connected to their parent `IfcAlignment` through `IfcRelNests`. This relationship has two key properties:

1. **`RelatingObject`** — the `IfcAlignment` (the parent).
2. **`RelatedObjects`** — an ordered list of objects nested within. Referents must appear in the list in order of increasing `DistanceAlong`. This ordering is required by the IFC concept template (4.1.4.4.3 Object Nesting) but is not enforced by a schema WHERE rule. Implementations that read referent data should sort by `DistanceAlong` before processing and not assume the list arrives pre-sorted; implementations that write referent data must produce a sorted list.

The first `IfcReferent` in `RelatedObjects` defines the **starting station** of the alignment. Its `Pset_Stationing.Station` value is the station label at `DistanceAlong = 0.0` of the alignment curve. As a best practice, place this referent at `DistanceAlong = 0.0` so that the station origin and the geometric origin of the alignment coincide.

Figure 9.3.1-1 shows this mechanism populated with the starting-station referent (P1) and the two equation referents (P3, P4) from the §9.2.6 worked example.

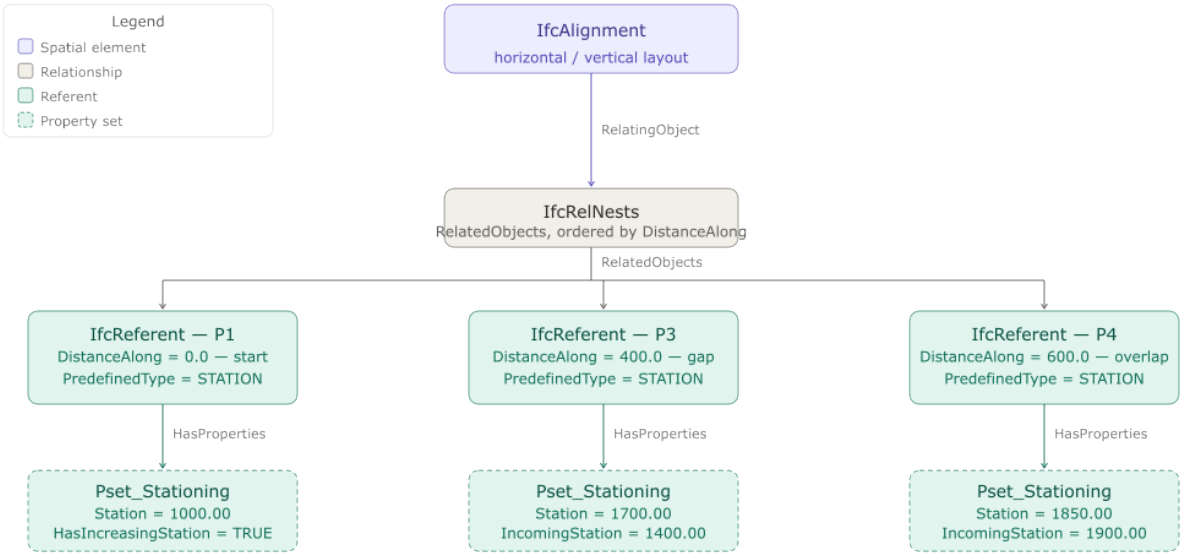


Figure 9.3.1-1 — `IfcRelNests` establishing the stationing order for `IfcReferent` instances P1, P3, and P4, with `Pset_Stationing` values matching the §9.2.6 worked example. P2 and P5 are omitted because they are ordinary points on the alignment, not referents — of the five points in the §9.2.6 example, only P1, P3, and P4 have a corresponding `IfcReferent`.

Reverse-stationed alignments. To station an alignment so that station values *decrease* in the direction of increasing `DistanceAlong` (§9.2.7), set `Pset_Stationing.HasIncreasingStation = FALSE` on the starting-station referent — the first entry in `RelatedObjects`. The nesting order does not change: referents are still listed by increasing `DistanceAlong`, which for a reverse-stationed alignment means by *decreasing* `Station`. Figure 9.3.1-2 shows the mechanism for the reverse-stationed alignment of §9.2.7.1, whose starting station is Sta. 20+00 and whose one equation is a gap.

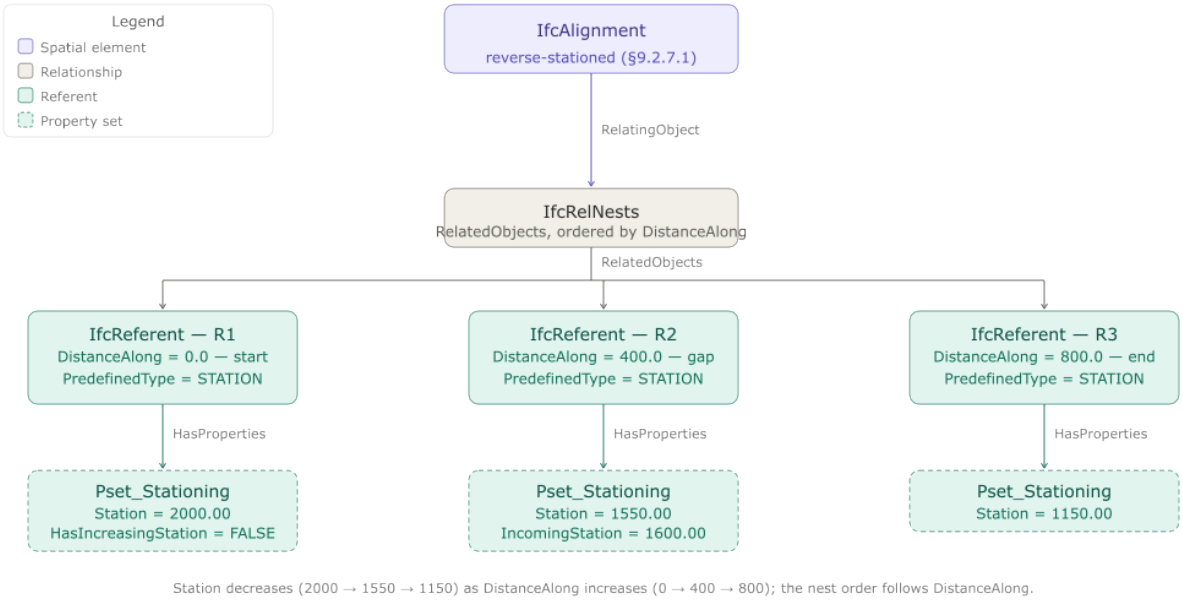


Figure 9.3.1-2 — `IfcRelNests` for the reverse-stationed alignment of §9.2.7.1. `HasIncreasingStation = FALSE` on the starting referent R1 makes the subsequently nested referents decrease in `Station` even though they increase in `DistanceAlong`.

9.3.2 The Mixed-Nesting Problem

`IfcRelNests` is also used to attach alignment layout components (`IfcAlignmentHorizontal`, `IfcAlignmentVertical`, `IfcAlignmentCant`) to `IfcAlignment`. Both layout sub-objects and referents decompose the same parent through the same relationship type. This creates ambiguity: should layout sub-objects and referents share a single `IfcRelNests` instance, or should they use separate ones?

The IFC specification does not resolve this clearly. Two patterns are seen in practice:

Pattern A — Shared nest. A single `IfcRelNests` instance contains both layout objects and referents interleaved or grouped. This works but can make parsing complex, because consumers must distinguish layout objects from referents by type.

Pattern A also has an ordering problem. `RelatedObjects` is an *ordered* list, and referents have a natural order — each sits at a single `DistanceAlong`. Layout sub-objects do not: `IfcAlignmentHorizontal`, `IfcAlignmentVertical`, and `IfcAlignmentCant` each span the whole alignment. In a shared nest there is no schema rule, and no settled convention, for where a layout sub-object falls relative to a referent that lies partway along it — a `SUPERELEVATIONEVENT` at a cant transition, a `POSITION` referent at the low point of a vertical curve, an `INTERSECTION` referent inside a segment. A reader cannot recover the intended order without a convention that IFC does not supply, and one would have to be standardized for Pattern A to be interoperable.

Pattern B — Separate nests (recommended). Two distinct `IfcRelNests` instances are used: one whose `RelatedObjects` contains only the alignment layout sub-objects, and one whose `RelatedObjects` contains only `IfcReferent` instances. The ordering problem disappears: each list holds one kind of object with an unambiguous order — layout sub-objects by discipline, referents by `DistanceAlong`.

Recommendation: Use Pattern B — separate nests. This is the pattern illustrated in the existing Figure 9.3.2-1 and is more robust for software implementations. The layout nest and the referent nest are independent, and each can be processed without concern for the content of the other.

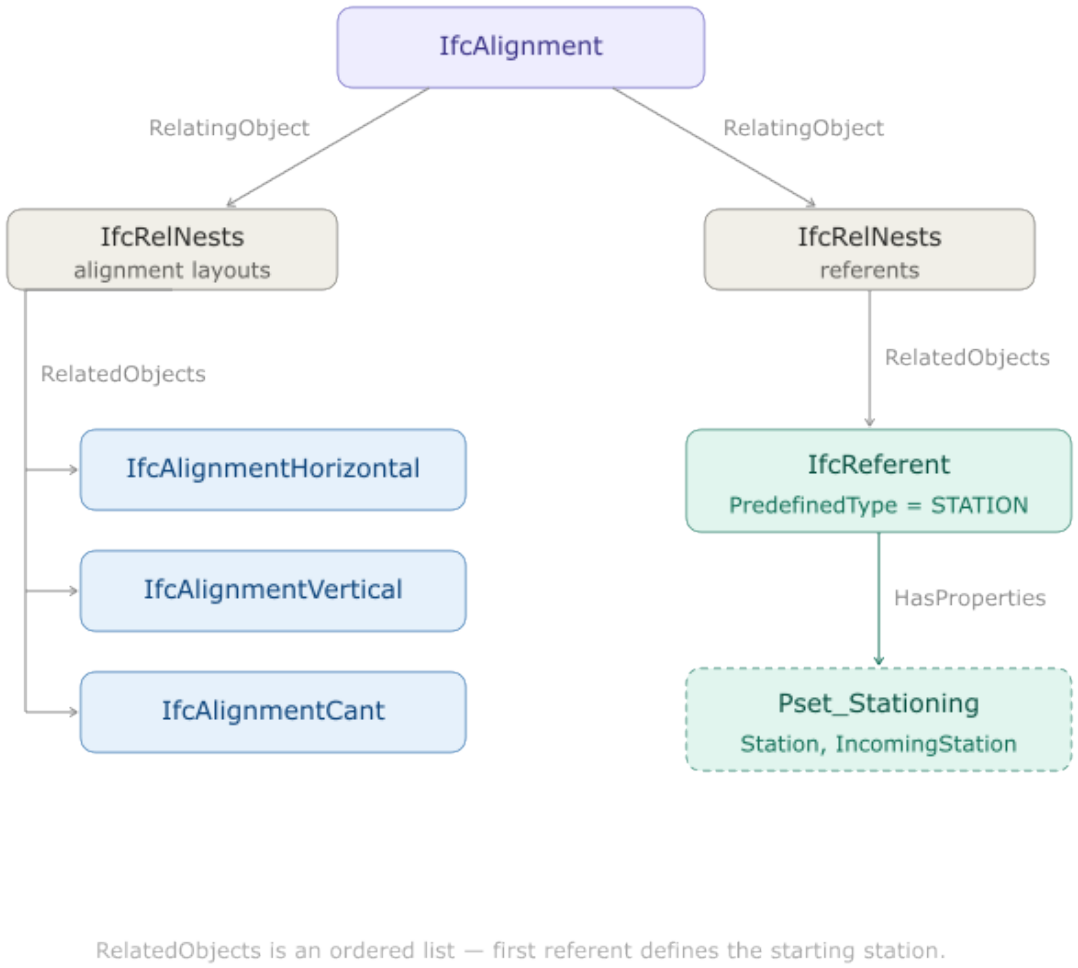


Figure 9.3.2-1 — Two `IfcRelNests` relationships: one for alignment layout sub-objects (horizontal, vertical, cant) and one for `IfcReferent` instances.

9.3.3 `ObjectPlacement` for `IfcReferent`

`IfcReferent` is a subtype of `IfcPositioningElement`, which requires an `ObjectPlacement` (omitting `ObjectPlacement` results in a bSI Validation Service error). Although the schema permits any subtype of `IfcObjectPlacement`, only `IfcLinearPlacement` is practical for referents on an alignment with geometry. `IfcLocalPlacement` and `IfcGridPlacement` carry no `DistanceAlong` value, so a consumer reading a model that uses these placement types has no reliable basis for ordering referents or performing station-to-distance conversions.

Two cases arise in practice:

Alignment with geometry. Use `IfcLinearPlacement` with `IfcPointByDistanceExpression` referencing the alignment's horizontal geometric curve. The `DistanceAlong` value establishes the referent's geometric position and is the basis for ordering.

Semantic-only alignment. When the alignment carries no geometric representation, `IfcLinearPlacement` cannot be evaluated — there is no basis curve from which to measure distance. Set `IfcReferent.ObjectPlacement` to the same placement instance as `IfcAlignment.ObjectPlacement`. Implementations that later add a geometric representation must update all referent placements accordingly; retaining the copied alignment placement after geometry is added will cause the referent's resolved position to diverge from its geometric intent.

9.3.4 Key-Point Referents

Beyond stationing (§9.3.1) and labeling another product's position (§9.4), a third, entirely discretionary use of `IfcReferent` marks the geometric transition points within the alignment's own layout: where a line meets a curve, where one curve type gives way to another, where the layout begins and ends. On a horizontal layout these are the familiar Point of Beginning, Point of Curvature, Point of Tangency, Point of Intersection, and similar transition points; on a vertical layout, the analogous Point of Vertical Curvature, Point of Vertical Tangency, and Point of Vertical Intersection. Modeling these points as referents is a choice, not a requirement — a consumer can always re-derive a transition's station by walking `IfcAlignmentSegment`/`IfcCurveSegment` lengths from the start of the layout. Key-point referents exist purely so that a downstream consumer — a viewer, a plan-production tool, a reviewer opening the model directly — can read these locations and their station labels without performing that walk itself.

Predefined type and relationship. A key-point referent identifies the position of a point on the alignment's own geometry — it does not define the alignment's stationing system, so `STATION` (§9.3.1) does not apply. It uses the same `PredefinedType = POSITION` value already introduced in §9.1 for labeling another product's location (elaborated in §9.4.1), but the relationship pattern is different: a key-point referent is related to `IfcAlignment` by `IfcRelNests` (the geometric role, exactly like the `STATION` referent's relationship to `IfcAlignment` in §9.3.1), not by `IfcRelPositions` (the semantic role used in §9.4.1 to label another product). Both flavors of `POSITION` referent carry `Pset_Stationing`; only the relationship distinguishes "this referent marks a point on the alignment's own geometry" from "this referent labels some other product's position."

Nesting. Concept Template 4.1.4.4.3 Object Nesting lists `IfcAlignment` as `RelatingObject` for `IfcReferent` `RelatedObjects` — the same pairing already used for the `STATION` referent in §9.3.1.

- **One nest per discipline.** Each applicable discipline — horizontal, vertical, cant — gets its own `IfcRelNests`, `RelatingObject` = `IfcAlignment`, `RelatedObjects` = only that discipline's key-point referents. This is independent of, but structurally identical to, the stationing nest of §9.3.1: several `IfcRelNests` instances can share the same `RelatingObject` (`IfcAlignment`), distinguished only by which referents each one's `RelatedObjects` holds.

- **Possible but discouraged: a single combined nest for all disciplines.** Every horizontal, vertical, and cant key-point referent in one `IfcRelNests.RelatedObjects` list is schema-legal and still `RelatingObject = IfcAlignment`, but it reproduces the same problem Pattern A in §9.3.2 already warned against: a consumer must inspect each referent's placement or label just to determine which discipline it belongs to, rather than being able to read that from the nest itself.

Whichever form is chosen, the key-point nest (or nests) is independent of both the horizontal/vertical/cant layout nest (§9.3.2) and the stationing nest (§9.3.1) — a third category that extends, rather than replaces, the mixed-nesting guidance of §9.3.2: layout sub-objects, key-point referents, and stationing referents all coexist under `IfcAlignment.IsNestedBy`, each in its own `IfcRelNests`.

A note on Object Nesting and the Alignment-based View MVD. Concept Template 4.1.4.4.3 is not itself tagged as part of the Alignment-based View (AbV), even though it is the only schema mechanism that establishes alignment stationing (§9.3.1) or key-point referents at all. Apply it regardless — omitting it would leave no conforming way to model stationing or key-point referents on an AbV alignment. (§9.5.2 notes a similar AbV-tagging gap for `IfcConnectionGeometry`.)

Naming. `Name` is free-form text, not a schema-constrained enumeration — nothing in IFC fixes the abbreviation vocabulary. The worked example below (§9.3.4.1) uses `"{alignment name} {station string} ({abbreviation})"`, e.g. `"E-Line 119+56.79 (P.C.)"`. Leading with the alignment name makes every referent nested under an alignment identifiable by its name alone, without inspecting its placement or `Pset_Stationing`. The abbreviations are typical of North American highway practice: `P.O.B.`/`P.O.E.` for the layout's start/end, `P.C.`/`P.T.` at line-to-arc and arc-to-line transitions on the horizontal layout, and `V.P.O.B.`/`P.V.C.`/`P.V.T.`/`V.P.O.E.` for the corresponding vertical-layout points. Other jurisdictions and agencies use different abbreviations, additional key-point types (e.g. `P.I.` at a line-to-line deflection, `P.C.C.` at an arc-to-arc transition), or different station-string formats — the guide shows one typical, commonly-seen convention, not a vocabulary implementations must reproduce.

Placement and properties. `ObjectPlacement` follows the same two cases already described in §9.3.3: `IfcLinearPlacement`/`IfcPointByDistanceExpression` against the layout's curve when geometry exists, or a placement copied from the alignment itself for a semantic-only layout. Each referent carries `Pset_Stationing` (§9.3.1), giving it a station value in addition to its geometric position. As with other referents in this chapter, no `Representation` is created — placement and property data are sufficient.

Figure 9.3.4-1 shows the nesting pattern.

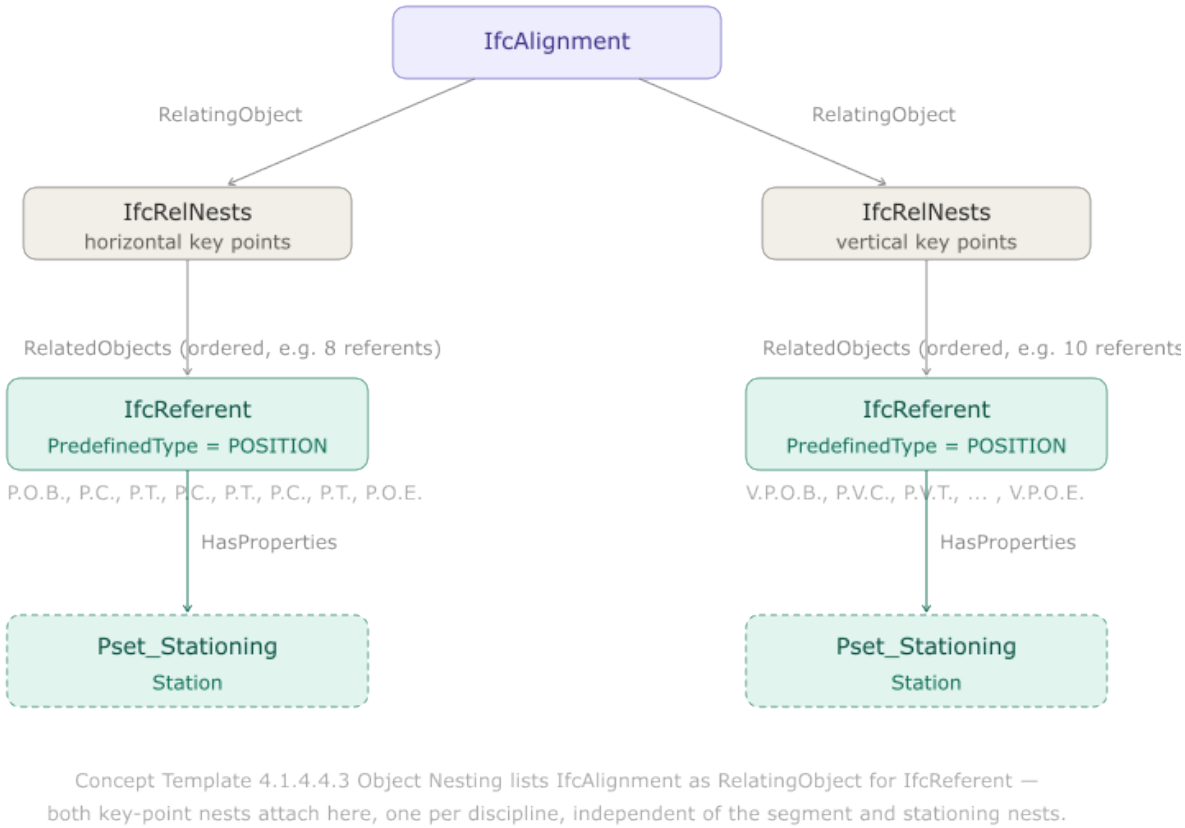


Figure 9.3.4-1 — Key-point-referent nesting: one `IfcRelNests` per discipline, `RelatingObject` = `IfcAlignment`, `RelatedObjects` = that discipline's `POSITION` referents.

9.3.4.1 Worked Example: the FHWA E-Line

The FHWA Bridge Geometry Manual E-Line alignment already used as the real-world test case in Chapter 12 demonstrates this pattern: `testset/real-world-alignments/FHWA_Alignment.ifc` carries one `IfcRelNests` nested to `IfcAlignment` with eight `POSITION` referents (`E-Line 100+00.00 (P.O.B.)` through `E-Line 223+37.07 (P.O.E.)`, with `P.C.`/`P.T.` pairs at each of the three horizontal curves), and a second `IfcRelNests`, also nested to `IfcAlignment`, with ten `POSITION` referents (`E-Line 100+00.00 (V.P.O.B.)` through `E-Line 228+00.00 (V.P.O.E.)`, with `P.V.C.`/`P.V.T.` pairs at each of the four vertical curves) — independent of, and in addition to, the single starting-station `STATION` referent nested to `IfcAlignment` per §9.3.1. All three nests share `RelatingObject = IfcAlignment`, distinguished only by which referents each one's `RelatedObjects` holds.

9.4 Product Positioning

This section covers the **semantic role** of `IfcReferent`: how it semantically labels the position of a product. IFC formalizes two patterns for this under Object Connectivity:

Product Relative Positioning (Concept Template 4.1.5.8), which uses a single `IfcReferent` to label one point, and **Product Span Positioning** (Concept Template 4.1.5.9).

It is important to understand that `IfcReferent` serves two distinct roles that must not be conflated:

Geometric role. When an `IfcReferent` carries an `IfcLinearPlacement`, it has a precise geometric location on the alignment.

Semantic role. An `IfcReferent` can *annotate* the position of another object — not by moving it, but by providing a human-readable label (typically a station) that names its location. This is purely informational metadata.

The geometric location of an infrastructure element (a bridge pier, a sign, a drain inlet) is defined entirely by its own `IfcObjectPlacement`. An associated referent gives that location a name. Removing or changing the referent does not move the object.

9.4.1 Product Relative Positioning

The relationship between an `IfcReferent` and the object whose position it annotates is `IfcRelPositions`:

- `RelatingPositioningElement` — the `IfcPositioningElement` providing the semantic position.
- `RelatedProducts` — the `IfcProduct` instances positioned by the `RelatingPositioningElement`.

This is exactly the `IfcRelPositions` pattern defined by Concept Template 4.1.5.8, Product Relative Positioning: one positioning element, one or more related products.

Suppose the alignment's starting station is Sta. 100+00 — that is, the first referent nested under the alignment (§9.3.1) carries `Pset_Stationing.Station = 10000.0` at `DistanceAlong = 0.0`. A bridge pier (`IfcBridgePart.PIER`) is geometrically placed at `DistanceAlong = 14235.75`. Its station label is *not* 14235.75 — it is the starting station plus the distance along: 10000.0 + 14235.75 = 24235.75, i.e., Sta. 242+35.75. An `IfcReferent` with `PredefinedType = POSITION` and `Pset_Stationing.Station = 24235.75` is connected to the pier via `IfcRelPositions`. The pier's station label is now Sta. 242+35.75. The pier's geometry — `DistanceAlong = 14235.75` on its own `IfcLinearPlacement` — is entirely unaffected by the starting station value; only the label changes. However, the semantic position of the pier is incomplete. We know it is at Sta. 242+35.75 but the `IfcReferent` does not inform which alignment that station refers to, nor where that alignment's stationing begins. Two further relationships resolve this. First, the `IfcReferent` that was the `RelatingPositioningElement` for the pier is now itself `RelatedProducts` in a second `IfcRelPositions`, where `IfcAlignment` is the `RelatingPositioningElement`. This ties the pier's referent to the specific alignment it stations. Second, and independently, the alignment's starting-station referent

(`Pset_Stationing.Station = 10000.0`, `DistanceAlong = 0.0`) is nested under `IfcAlignment` by `IfcRelNests`, per §9.3.1 — this is what fixes the 10000.0 offset that turns `DistanceAlong = 14235.75` into Sta. 242+35.75. The positioning relationships are shown in Figure 9.4.1-1.

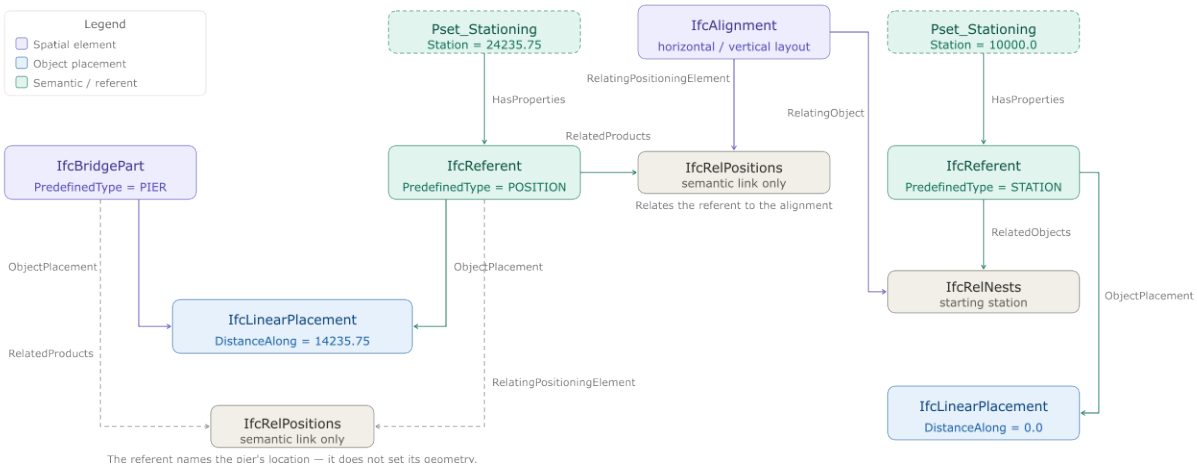


Figure 9.4.1-1 — Object graph showing an `IfcBridgePart.PIER` with its own `IfcLinearPlacement`, an `IfcReferent` (POSITION type) with `Pset_Stationing`, and the `IfcRelPositions` link between them. To the right, `IfcAlignment` is the `RelatingPositioningElement` for a second `IfcRelPositions` that has the pier's referent as `RelatedProducts`, and a starting-station `IfcReferent` (`Station = 10000.0`) is nested under `IfcAlignment` by `IfcRelNests`, supplying the offset used to derive Sta. 242+35.75 from `DistanceAlong = 14235.75`. Below `IfcRelNests`, that starting-station referent has its own `IfcLinearPlacement` at `DistanceAlong = 0.0`.

Note the two different relationships an `IfcReferent` can have with the alignment, depending on its role: an `IfcReferent` that *defines* stationing — such as the starting-station referent or a station equation — is related to the `IfcAlignment` with `IfcRelNests`. An `IfcReferent` that *provides the stationing value of another object* — such as the pier's referent — is related to the `IfcAlignment` with `IfcRelPositions`. This is a subtle but important distinction.

9.4.2 Product Span Positioning

Some infrastructure elements are not defined by a single point but by a linear extent — a guardrail run, a noise barrier, a pavement marking, a work-zone limit. Concept Template 4.1.5.9, Product Span Positioning, covers this case: an `IfcProduct` is positioned relative to *two* `IfcReferent` instances, marking the boundaries of the span, via **two** separate `IfcRelPositions` relationships — one per referent, each with the referent as `RelatingPositioningElement` and the product as `RelatedProducts`.

Example: Continuing the alignment from §9.4.1 (starting station Sta. 100+00), a guardrail run (`IfcRailing`) begins at Sta. 240+00 and ends at Sta. 250+00. Two `IfcReferent` instances, each with `PredefinedType = POSITION`, mark these locations, each carrying its

own `Pset_Stationing` (`Station = 24000.00` and `Station = 25000.00` respectively) and its own `IfcLinearPlacement`. Toward the `IfcRailing`, Product Span Positioning applies; toward the `IfcAlignment`, Product Relative Positioning applies (§9.4.1). As with the single-point pattern, `Pset_Stationing` values on the span referents are independent of the guardrail's own geometry — moving the guardrail's `IfcLinearPlacement` does not move the referents, and vice versa.

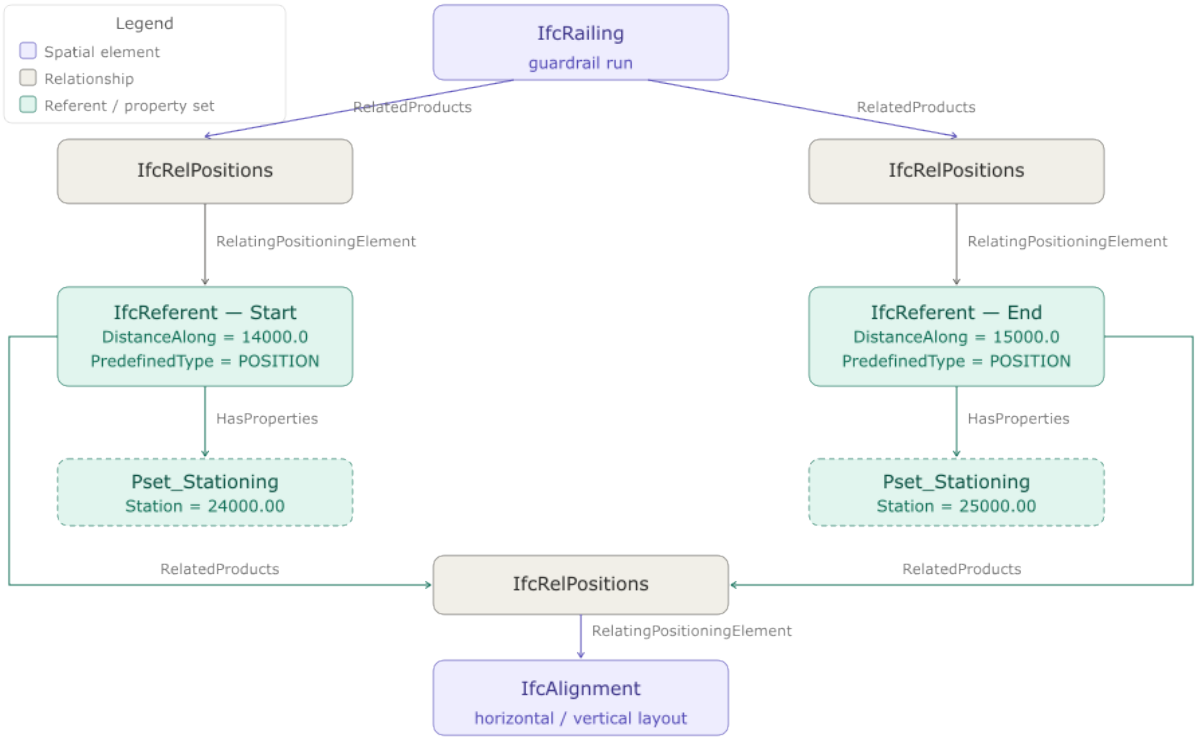


Figure 9.4.2-1 — Two concept templates combined: Product Span Positioning (two `IfcRelPositions` relationships linking the `IfcRailing` to its start and end `IfcReferent` instances) and Product Relative Positioning (a single `IfcRelPositions` linking both referents to the `IfcAlignment` they are stationed against).

9.5 The `INTERSECTION` Referent

An `IfcReferent` with `PredefinedType = INTERSECTION` marks the point where two or more routes cross. A single crossing uses **two** `IfcReferent` instances of type `INTERSECTION`, one per intersecting alignment, each related to its own single owning alignment by `IfcRelPositions`. That referent pattern (§9.5.1) only ever names a station on one alignment — it is not, by itself, a record that the two alignments cross. This section works through two related but distinct questions: how each `INTERSECTION` referent is owned (§9.5.1), and how the crossing itself — a fact about the two facilities, established independently of any referent — is recorded with `IfcRelInterferesElements` (§9.5.2).

9.5.1 Referent Ownership: One Referent Per Alignment

In each case, the `IfcRelPositions` has that `IfcAlignment` as `RelatingPositioningElement` and the referent as `RelatedProducts` — the same alignment-link pattern used for the `POSITION` referent's second `IfcRelPositions` in §9.4.1 — and each referent carries its own `Pset_Stationing.Station` value in its owning alignment's stationing. This interpretation follows directly from ISO 19148 §6.2.12.1, which IFC's linear referencing model is built on (§9.6):

> Referents are owned by a single feature. For example, the reference markers along Interstate 95 are owned by the feature that represents Interstate 95. The referent representing the intersection with First Avenue along Washington Street is owned by Washington Street if it is used to specify relative linearly referenced locations along Washington Street. The location of this referent is most likely specified with a position expression along Washington Street. A different referent can represent the intersection with Washington Street along First Avenue. This referent is owned by First Avenue and is used to specify relative linearly referenced locations along First Avenue.

Applied to that example: Washington Street's `INTERSECTION` referent is placed via `IfcLinearPlacement`/`DistanceAlong` on Washington Street's curve, carries `Pset_Stationing.Station = 1000.00` (Sta. 10+00) in Washington Street's stationing, and is related to `IfcAlignment` — Washington Street by its own `IfcRelPositions`. First Avenue's `INTERSECTION` referent is an entirely separate instance: placed on First Avenue's own curve, carrying `Pset_Stationing.Station = 1200.00` (Sta. 12+00) in First Avenue's stationing, and related to `IfcAlignment` — First Avenue by a second, independent `IfcRelPositions`. Figure 9.5-1 shows the two independent object graphs.

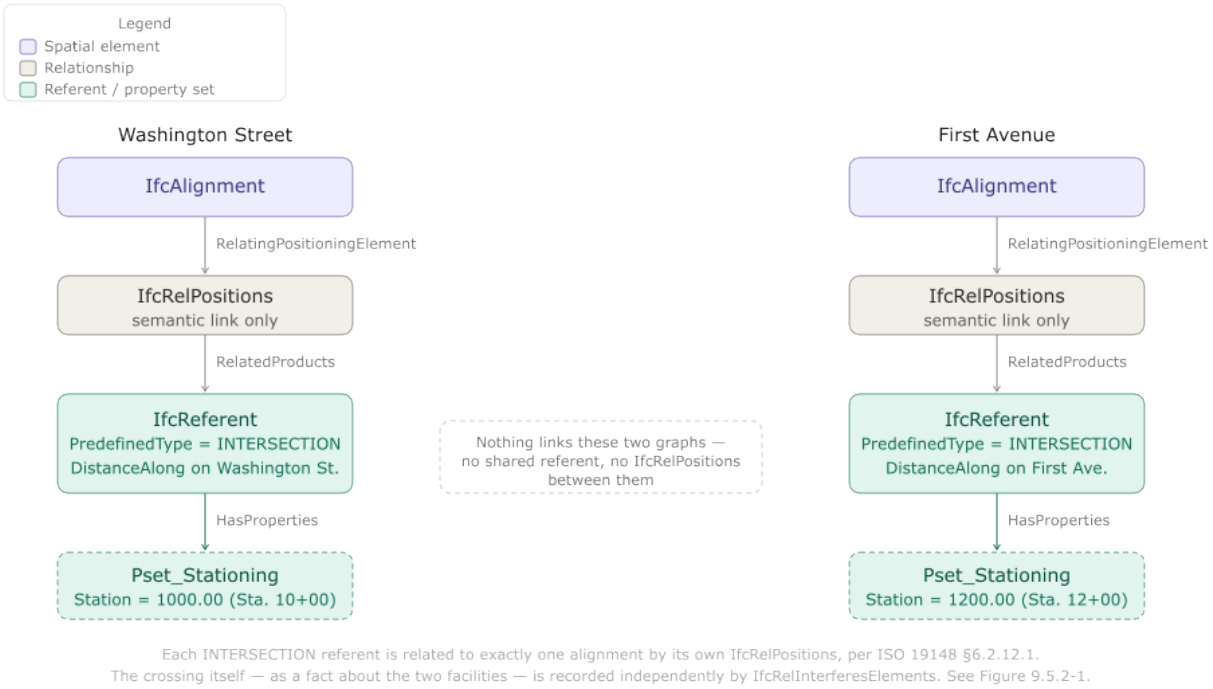


Figure 9.5-1 — Two independent `INTERSECTION` referents, each related to its own alignment by its own `IfcRelPositions`, following the same alignment-link pattern as the `POSITION` referent in Figure 9.4.1-1 — applied once per alignment rather than shared between two. Nothing in this graph connects the two referents or the two alignments; the fact that they cross is established separately, in §9.5.2.

This pattern scales to a multi-leg intersection without modification — one `INTERSECTION` referent per crossing alignment.

9.5.2 `IfcRelInterferesElements`: Recording That Two Facilities Cross

The referent and the `IfcRelPositions` instance that ties it to its owning alignment (§9.5.1) do exactly one thing: they give a *name* — a station — to a location on one alignment. Neither asserts anything about the alignment on the other side of the crossing. It is easy to misread two `INTERSECTION` referents at coincident locations as if that, by itself, recorded "these two roads cross," but nothing in §9.5.1 relates Washington Street's referent to First Avenue's, or either referent to the other alignment. Whether the crossing needs to be recorded as queryable schema data — as distinct from a fact two engineers can see on a plan sheet — is answered by a relationship that has nothing to do with referents or stationing: `IfcRelInterferesElements`.

`IfcRelInterferesElements` (a subtype of `IfcRelConnects`) is an objectified relationship stating that two elements interfere, physically or logically, independent of either element's own definition. Its attributes:

- `RelatingElement`, `RelatedElement` — typed `IfcInterferenceSelect`, meaning each must be a subtype of `IfcElement` or of `IfcSpatialElement`.
- `InterferenceGeometry` — optional `IfcConnectionGeometry`, the shape of the interference itself, expressed in `RelatingElement`'s object coordinate system.
- `InterferenceType` — optional `IfcIdentifier`. This is free text, **not a restricted enumeration** — the schema documentation offers values like `Crosses`, `PassesOver`, `PassesUnder`, `PassesThrough`, `Along`, and `Clash` only as examples a project might adopt as a convention, not as a fixed value list IFC enforces.
- `ImpliedOrder` — `IfcLogical`, indicating whether `RelatingElement` should be read as a source and `RelatedElement` as a target for the chosen `InterferenceType` (meaningful for a directional type like `PassesOver`; not meaningful for an undirected `Clash`).
- `InterferenceSpace` — optional `IfcSpatialZone`, the space in which the interference occurs.

`IfcAlignment` cannot itself be a party to this relationship. `IfcAlignment` is a subtype of `IfcPositioningElement` (via `IfcLinearPositioningElement`), not of `IfcElement` or `IfcSpatialElement` — the only two branches `IfcInterferenceSelect` accepts. That follows from what an alignment *is* in this schema: a positioning/reference curve, not a physical or spatial occurrence. What can carry the relationship is whatever `IfcElement` or `IfcSpatialElement` the alignment describes the layout of — typically the `IfcRoad` or `IfcRailway` facility the alignment lays out, or, where the crossing needs to be tied to a specific occurrence rather than the facility as a whole, a physical element such as a pavement course or a bridge structure.

The facility does not *aggregate* the alignment, either. `IfcAlignment` is conventionally aggregated directly under `IfcProject` (per the concept template in IFC 4.1.4.1.1, Alignment Aggregation To Project), and the bSI Validation Service's SPS002 rule (Correct spatial breakdown) restricts what `IfcRelAggregates` may nest under an `IfcRoad` to other facility/space-type entities — `IfcAlignment` is not among them, so aggregating an alignment under a road fails validation. Instead, the road references its alignment with `IfcRelReferencedInSpatialStructure` — the same non-hierarchical relationship used elsewhere for an element that logically belongs to more than one spatial structure (§9.3, e.g. a multi-storey column) — with the `IfcRoad` as `RelatingStructure` and the `IfcAlignment` as one of its `RelatedElements`.

Example, continuing §9.5.1: Washington Street and First Avenue are each modeled as an `IfcRoad`, each referencing its respective `IfcAlignment` (which remains aggregated under `IfcProject`). The at-grade crossing is recorded independently of either `INTERSECTION` referent:

- `RelatingElement` = Washington Street (`IfcRoad`)
- `RelatedElement` = First Avenue (`IfcRoad`)
- `InterferenceType` = `"Crosses"`

- `ImpliedOrder` = `FALSE` — an at-grade crossing has no source/target direction

Figure 9.5.2-1 shows this alongside the `IfcRelReferencedInSpatialStructure` that ties each `IfcRoad` to its `IfcAlignment`.

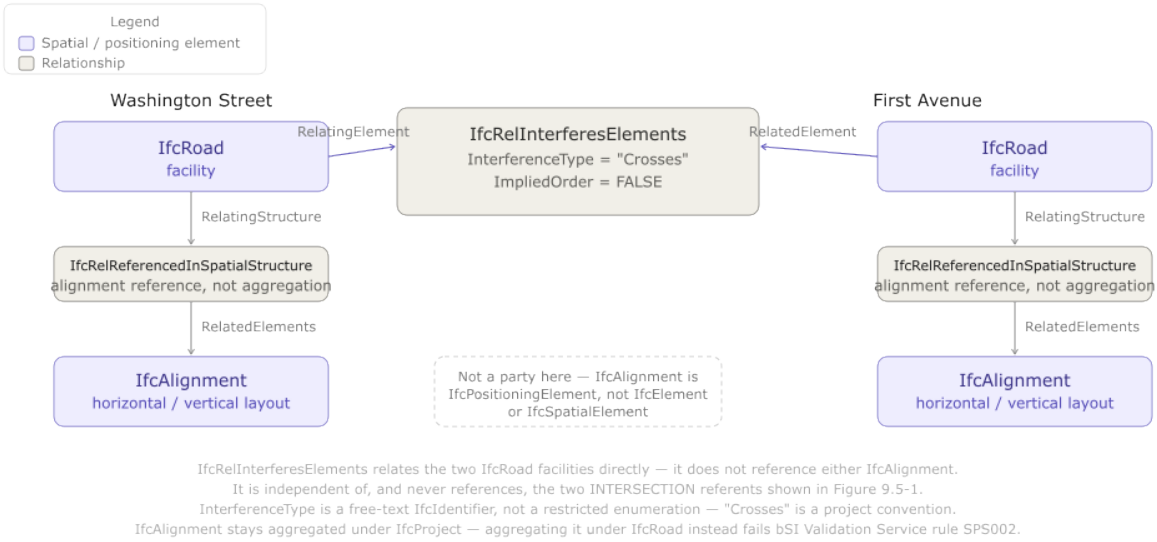


Figure 9.5.2-1 — A single `IfcRelInterferesElements` relating the two `IfcRoad` facilities. Each facility separately references the `IfcAlignment` that lays it out via `IfcRelReferencedInSpatialStructure`, not `IfcRelAggregates` — the alignments themselves are outside the interference relationship, since `IfcInterferenceSelect` does not accept `IfcPositioningElement`.

This `IfcRelInterferesElements` instance is what lets software ask “does Washington Street cross another facility, and which one?” without parsing referents or comparing coordinates. It complements, but does not replace, the two `INTERSECTION` referents from §9.5.1: the referents answer “at what station,” which `IfcRelInterferesElements` does not carry, while `IfcRelInterferesElements` answers “is this a real, queryable crossing,” which the referents do not assert. The two relationships never reference each other. A model can legitimately have `INTERSECTION` referents with no corresponding `IfcRelInterferesElements` (the crossing is only implied by proximity — common in low-detail concept models), or an `IfcRelInterferesElements` instance with no referents yet (the crossing is flagged before it has been given a station label).

A grade-separated (multi-level) crossing — a road passing over another road, or over a railway — is recorded the same way, but with a directional `InterferenceType` and `ImpliedOrder = TRUE`: `InterferenceType = "PassesOver"`, `RelatingElement` = the facility physically above, `RelatedElement` = the facility below. Nothing about the two `INTERSECTION` referents changes between the at-grade and grade-separated cases — both still use the identical `PredefinedType = INTERSECTION` and carry no elevation information

of their own — so IfcRelInterferesElements is also the only place in the model that distinguishes an at-grade crossing from a grade-separated one.

Two example files demonstrate this pattern:

- IntersectionReferent_AtGradeCrossing.ifc — two alignments crossing at a true 3D intersection, each referenced by its own IfcRoad; a single IfcRelInterferesElements between the two roads carries InterferenceType = "Crosses", ImpliedOrder = FALSE, and an IfcConnectionPointGeometry InterferenceGeometry built from the same two IfcPointByDistanceExpression instances already used by the INTERSECTION referents' own placements, rather than duplicating the location as new IfcCartesianPoints.
- IntersectionReferent_GradeSeparatedCrossing.ifc — the same plan-view crossing, but with one alignment's grade reversed so the two roads no longer meet in 3D; the IfcRelInterferesElements between them carries InterferenceType = "PassesOver", ImpliedOrder = TRUE, and an IfcConnectionPointEccentricity InterferenceGeometry — the same two reused points, plus EccentricityInZ recording the 30 ft vertical separation between them.

A note on `InterferenceGeometry` and the Alignment-based View MVD. IfcConnectionGeometry is tagged as part of the Alignment-based View (AbV), but it is abstract, and none of its five concrete descendants — including IfcConnectionPointGeometry and IfcConnectionPointEccentricity, used above — carry that tag themselves, the same kind of AbV-tagging gap noted for Object Nesting in §9.3.4. The two example files were checked against the bSI Validation Service and were not flagged as non-conforming to AbV.

9.6 ISO 19148 Linear Referencing

9.6.1 Background

ISO 19148 *Geographic information — Linear referencing* is the international standard that formalizes the concept of locating features along a linear element. IFC4x3's infrastructure extensions draw on ISO 19148 concepts, and Pset_LinearReferencingMethod (applicable to IfcAlignment and IfcReferent) is defined in terms of ISO 19148.

Understanding the ISO 19148 model helps implementers correctly interpret DistanceAlong values, especially when data is exchanged between systems that use different linear referencing conventions.

9.6.2 Key ISO 19148 Concepts

Linear Referencing Method (LRM). An LRM defines the rules for measuring distance along a linear element. The most common types are:

LRM Type	Description	Highway Example

LRM Type	Description	Highway Example
Absolute	Distance measured continuously from a fixed start point.	Route mileage from a state border.
Relative	Distance measured from a nominated referent (not the start).	“0.4 km past interchange 12.”
Interpolated Position	Location expressed as a fraction between two known referents.	Between mileposts 43 and 44.

`Pset_LinearReferencingMethod` records the LRM type (`LRMType`), its name (`LRMName`), and the units of measure (`LRMUnit`) for an alignment or referent element.

Referents and Milestones vs. Stationing. In European road practice, distance along a route is often expressed using *kilometer posts* (KP) or *reference posts* — physical markers at known locations. A KP system is a Relative or Reference Post LRM: distance is measured to the nearest upstream post, plus an offset. In North American highway practice, *stationing* is an Absolute LRM: every point on the alignment is assigned a cumulative distance from the project start, expressed as `ccc+dd.dd` (hundreds of feet) or in meters.

The key difference:

- **Stationing (Absolute LRM):** `DistanceAlong` in IFC closely matches the station value (after accounting for any starting station offset defined by an `IfcReferent`, per §9.3.1, and any station equations, per §9.2.5). A reverse-stationed alignment (§9.2.7) is still an Absolute LRM — `DistanceAlong` still runs forward from the curve start; only the sign relating it to the station value changes.
- **KP / Reference Post (Relative LRM):** `DistanceAlong` in IFC is always the absolute geometric distance from the curve start. A KP value must be converted to a geometric distance before use in `IfcPointByDistanceExpression`.

9.6.3 LRM Name Examples from ISO 19148 Annex C

ISO 19148 Annex C lists recognized LRM name aliases. Common examples include:

Name	Common Alias	Typical Region
milepoint	milepost, MP	USA, Canada
kilometer point	KP, PK	Europe, Latin America
chainage	ch	UK, Australia, India
reference post	RP	Rail, some roads

`Pset_LinearReferencingMethod.LRMName` should use one of these recognized names where possible for maximum interoperability.

9.6.4 Impact on DistanceAlong

Regardless of the LRM in use for labelling purposes, `IfcPointByDistanceExpression.DistanceAlong` is always the **geometric distance from the start of `BasisCurve`**. LRM labels (station values, KP values, etc.) are a display convention managed through `IfcReferent` and `Pset_LinearReferencingMethod`, not through `DistanceAlong` directly.

See §9.2 (Stationing) and §9.3 (Attaching Referents to an Alignment) above for a detailed treatment of how station labels are stored and how to convert between station labels and geometric distances.

9.7 Summary and Implementation Checklist

#	Item	Notes
1	Use `IfcReferent` to attach station labels and other semantic location information to alignment positions.	Referents are informational overlays on geometry, not geometry themselves.
2	Store the station value in `Pset_Stationing.Station` as a plain `IfcLengthMeasure` in project length units.	Do not store the `ccc+dd.dd` string — compute it from the numeric value when displaying.
3	Set `Pset_Stationing.HasIncreasingStation = FALSE` on the starting referent when station values decrease in the direction of increasing `DistanceAlong` (reverse stationing). It governs the referents nested after it.	When absent or `TRUE`, stationing increases with `DistanceAlong` — the common case. `DistanceAlong` itself never reverses; only the sign relating it to `Station` changes. See §9.2.7.
4	At a station equation, provide both `IncomingStation` and `Station` on the same referent.	`IncomingStation` = upstream label; `Station` = downstream label.
5	Do not convert station labels to `DistanceAlong` by simple subtraction. For each referent record D_i and outgoing S_i; find the last i where $S_i \leq S$; then `DistanceAlong` $= D_i + (S - S_i)$.	See §9.2.5 for the full algorithm. For a reverse-stationed region, select the last i where $S_i \geq S$ and use `DistanceAlong` $= D_i - (S - S_i)$. See §9.2.7.
6	Before applying the conversion formula, check for a gap: if $\sigma_i\ (S - S_i) > D_{i+1} - D_i$ (with $\sigma_i = +1$ increasing, -1 decreasing), the station was skipped by an equation and `DistanceAlong` is undefined.	Return an error or sentinel value — do not silently produce a meaningless distance. See §9.2.6, §9.2.7.
7	Stations in an overlap zone map to two geometric points. Detect the zone (the	Which of `Station`/`IncomingStation` is the greater — i.e. what counts as an overlap

#	Item	Notes
	`Station`–`IncomingStation` interval at an overlap referent) and either return both candidates or apply project rules to select one.	versus a gap — flips with stationing direction. See §9.2.6, §9.2.7.
8	Use separate `IfcRelNests` instances for alignment layout sub-objects and referents.	Mixing them in a single nest is allowed but makes parsing harder, and leaves the list order between a whole-alignment layout sub-object and a referent partway along it undefined. See §9.3.2.
9	Order referents in `IfcRelNests.RelatedObjects` by increasing `DistanceAlong` — which, for a reverse-stationed alignment, means by *decreasing* `Station`.	Required by the nesting concept template; sort defensively on `DistanceAlong` when reading, never on `Station`. See §9.2.7.
10	Place the first referent at `DistanceAlong = 0.0` and set `Pset_Stationing.Station` to the starting station value.	Commonly a round number like 1000.00 (Sta. 10+00). For semantic-only alignments, `ObjectPlacement` = alignment's placement; for geometric alignments, use `IfcLinearPlacement` at distance 0. Update when adding geometry.
11	Use `IfcRelPositions` to link a referent to the object(s) whose station it names.	This is semantic annotation only; it does not affect geometry.
12	An `IfcReferent` that provides stationing should itself be linked to its `IfcAlignment` via `IfcRelPositions`, with `IfcAlignment` as `RelatingPositioningElement` and the referent as `RelatedProducts`.	Without this link, a station label is ambiguous — nothing states which alignment's stationing it belongs to. See §9.4.1, §9.4.2.
13	For linear-extent elements (guardrails, barriers, work-zone limits) spanning between two referents, link the product to each referent with its own `IfcRelPositions` (Product Span Positioning, Concept Template 4.1.5.9).	Toward the alignment, several referents share a single `IfcRelPositions` under Product Relative Positioning (§9.4.1). See §9.4.2.
14	Model an intersection as **one `INTERSECTION` referent per crossing alignment** (two for a simple crossing, more for a multi-leg intersection), each with its own `IfcRelPositions` to its own alignment and its own	Do not link one shared referent to multiple alignments — `Pset_Stationing` holds only one `Station` value. Per ISO 19148 §6.2.12.1. See §9.5.1.

#	Item	Notes
	Pset_Stationing.Station.	
15	Record the crossing itself — as a fact about the two facilities, independent of either INTERSECTION referent — with a single IfcRelInterferesElements relating the two IfcRoad/IfcRailway (or other IfcElement/IfcSpatialElement) instances the alignments lay out.	IfcAlignment cannot be RelatingElement/RelatedElement — IfcInterferenceSelect accepts only IfcElement or IfcSpatialElement, and IfcAlignment is IfcPositioningElement. InterferenceType is a free-text IfcIdentifier, not an enumeration. Use ImpliedOrder = TRUE with a directional type (e.g. PassesOver) for grade separations; FALSE for an undirected at-grade Crosses. See §9.5.2.
16	Keep IfcAlignment aggregated under IfcProject; link a facility (IfcRoad/IfcRailway) to the alignment it lays out with IfcRelReferencedInSpatialStructure (facility as RelatingStructure, alignment as one of RelatedElements), not IfcRelAggregates.	The bSI Validation Service's SPS002 rule (Correct spatial breakdown) restricts what IfcRelAggregates may nest under an IfcRoad/IfcRailway to other facility/space-type entities; IfcAlignment is not among them, and aggregating it under a facility fails validation. See §9.5.2.
17	Model InterferenceGeometry with a concrete IfcConnectionGeometry subtype — IfcConnectionPointGeometry for a coincident point, IfcConnectionPointEccentricity when the two points differ (e.g. the vertical offset at a grade separation) — reusing the alignments' existing IfcPointByDistanceExpression points rather than duplicating coordinates as new IfcCartesianPoints.	PointOnRelatingElement/PointOnRelatedElement accept any IfcPoint, including an already-built IfcPointByDistanceExpression. See §9.5.2.
18	Don't be alarmed that none of IfcConnectionGeometry's concrete subtypes are individually tagged Alignment-based View (AbV) — only the abstract supertype is.	A documentation ambiguity in the MVD, not a modeling error; confirmed against the bSI Validation Service on this guide's example files. See §9.5.2.
19	Key-point referents (P.C., P.T., and similar segment-transition markers) are optional. When used, give them PredefinedType = POSITION and relate them with IfcRelNests, one nest per	POSITION here does not mean the same relationship as the product-labeling POSITION referent of §9.4.1 — same enum value, different relationship. See §9.3.4.

#	Item	Notes
	applicable discipline, `RelatingObject = IfcAlignment` — not `IfcRelPositions`, and not `STATION`.	
20	Don't be alarmed that Concept Template 4.1.4.4.3 (Object Nesting) isn't itself tagged Alignment-based View (AbV) — apply it anyway for stationing and key-point referents.	A gap in the MVD's tagging, not a reason to avoid the concept template; the same kind of gap noted for `IfcConnectionGeometry` in item 18. See §9.3.4.
21	Treat the abbreviation text used in a key-point referent's `Name` as a display convention, not a schema enumeration.	The guide's worked example uses a typical North American convention; other jurisdictions/agencies may use different abbreviations or additional key-point types. See §9.3.4.
22	Record the LRM type and units in `Pset_LinearReferencingMethod` on the `IfcAlignment`.	Required for correct interpretation of `IfcReferent` stationing labels; `DistanceAlong` itself is always a geometric distance regardless of the LRM in use. See §9.6.
23	For a reverse-stationed region, apply the direction-aware conversion: select the last referent (by `DistanceAlong`) with $S_i \geq S$, then `DistanceAlong` $= D_i + \sigma_i\,(S - S_i)$ with $\sigma_i = -1$. Reverse stationing is a whole-region property; an overlap equation is a single-point discontinuity — do not conflate them.	`HasIncreasingStation = FALSE` records the direction. Reverse stationing does not change referent nesting order, `DistanceAlong` direction, or any geometry convention (curvature sign, cross-section axes). See §9.2.7.

Chapter 10 — Sectioned Surfaces and Solids

10.0 Introduction

A common approach in roadway design software is template-based modeling: define a cross-section template, apply it at regular intervals along the alignment, and interpolate the geometry between positions. For simple, uniform roadways — constant width, constant cross-slope — this works well. But most roadways are not uniform. They taper in and out; include medians that appear and disappear; require superelevation transitions; and feature traffic islands, widenings, and complex intersection geometry. For these cases, interpolating between evenly-spaced template stamps produces piecewise-linear geometry that can only approximate the intended design.

Road design software has long addressed this limitation through the concept of *strings* — continuous three-dimensional threads tracing significant features along the route: the edge of pavement, the shoulder break, the top of curb, the back of curb. Rather than deriving edges from template interpolation, strings define the edges explicitly. The cross-sections through the design are understood as sections through the string model, not the primary definition.

IFC4x3 introduces two geometry entities specifically for infrastructure: `IfcSectionedSurface` and `IfcSectionedSolidHorizontal`. Both define geometry by sweeping cross-sections along a `Directrix` — typically an alignment curve. `IfcSectionedSurface` uses open cross-sections to produce a surface; `IfcSectionedSolidHorizontal` uses closed cross-sections to produce a volumetric solid. Both support two complementary approaches to defining the geometry: template-based (cross-sections at defined distances along the directrix, linearly interpolated between) and string-line-based (guide curves that control where tagged cross-section points travel between sections). The IFC specification leaves several aspects of these entities underspecified — particularly around guide curve behavior, tag scoping, and the interaction between authored sections and guide curves. These gaps are significant enough that consistent implementations cannot be guaranteed without additional guidance; they are identified and discussed in §10.5.

10.1 Template-Based Approach

In the template-based approach, cross-sections are defined at discrete distances along the `Directrix`. The geometry engine interpolates linearly between consecutive cross-section positions to generate the surface or solid. The cross-sections need not be identical — width, slope, and shape can all vary from one distance to the next — but the transition between any two consecutive sections is linear.

For simple geometry this is entirely adequate. A road with a constant typical section can be fully described by two cross-sections with linear interpolation between them. A superelevation transition with a uniform rotation rate can be captured with sections at

each end of the transition. The template approach becomes strained when the geometry between defined sections is not linear — a curved edge taper through a widening, for example — requiring progressively denser section spacing to reduce the approximation error.

Inevitably, all roadways encounter other roadways, requiring complex intersection designs that must move smoothly through 3D space. This is where template-based design becomes difficult.

10.2 String Lines

A string line is a continuous three-dimensional curve tracing the path of a specific feature along the full extent of the design, such as edge of pavement, shoulder break, or top of curb. Where the template approach interpolates linearly between section endpoints, a string line defines the exact trajectory of a tagged cross-section point, independent of section density.

In IFC, string lines are represented as `IfcOffsetCurveByDistances` instances. The `Tag` attribute on `IfcOffsetCurveByDistances` connects a guide curve to the cross-section points that carry the matching tag value. A tagged point in a cross-section profile is constrained to follow the guide curve carrying the same tag.

> **Note:** The IFC specification's terminology is inconsistent. `IfcSectionedSolidHorizontal` uses "string line" and "guide curve," while `IfcSectionedSurface` uses "transitions curves" [sic]. In the context of this chapter, the terms *string line* and *guide curve* are used; however, all three terms mean the same thing.
>
> **Note:** `IfcElement.Tag` is an entirely different attribute and is not used by the string line mechanism described in this chapter. `IfcElement.Tag` is an optional identifier string for a physical element instance (e.g. a manufacturer's tag or asset number) and has no relationship to the `Tag` attributes discussed here, which appear on `IfcOffsetCurveByDistances`, `IfcOpenCrossProfileDef`, and `IfcCartesianPointList2D`. The name is coincidental.

The tag mechanism differs between the two entities:

- For `IfcSectionedSurface`, tags are carried by `IfcOpenCrossProfileDef.Tags` — a list of $N + 1$ labels, one per vertex of the open profile. Each vertex can be independently associated with a guide curve.
- For `IfcSectionedSolidHorizontal`, tags are carried by `IfcCartesianPointList2D.TagList` — one label per coordinate in the point list. Closed profiles are typically `IfcArbitraryClosedProfileDef` whose `OuterCurve` is an `IfcIndexedPolyCurve` referencing an `IfcCartesianPointList2D`; the `TagList` on that point list provides a label for each profile vertex.

In both cases, the intended behavior is that the geometry at a tagged cross-section point follows the corresponding guide curve — allowing, for example, a widening with a curved edge taper to be defined by sections only at the start and end of the transition, with the guide curve carrying the taper geometry between them.

The string line approach is not an alternative to the template approach but an augmentation of it. Cross-sections are always required; guide curves control interpolation between them.

10.3 IfcSectionedSurface

`IfcSectionedSurface` defines a surface by sweeping open cross-sections along a directrix curve.

Attribute	Type	Description
`Directrix`	`IfcCurve`	The curve along which sections are swept
`CrossSectionPositions`	`LIST [2:?] OF IfcAxis2PlacementLinear`	Positions along the directrix at which sections are placed
`CrossSections`	`LIST [2:?] OF IfcProfileDef`	Open cross-section profiles, one per position

Table 10.3-1 — IfcSectionedSurface attributes

The surface is generated by sweeping the cross-sections between the defined `CrossSectionPositions`. It does not extend to the head or tail of the `Directrix` — the surface is bounded by its first and last cross-section positions. The `CrossSectionPositions` and `CrossSections` lists must be equal in length. Figure 10.3-1 illustrates a sectioned surface through a superelevation transition, showing how the open cross-section profile varies along the directrix.

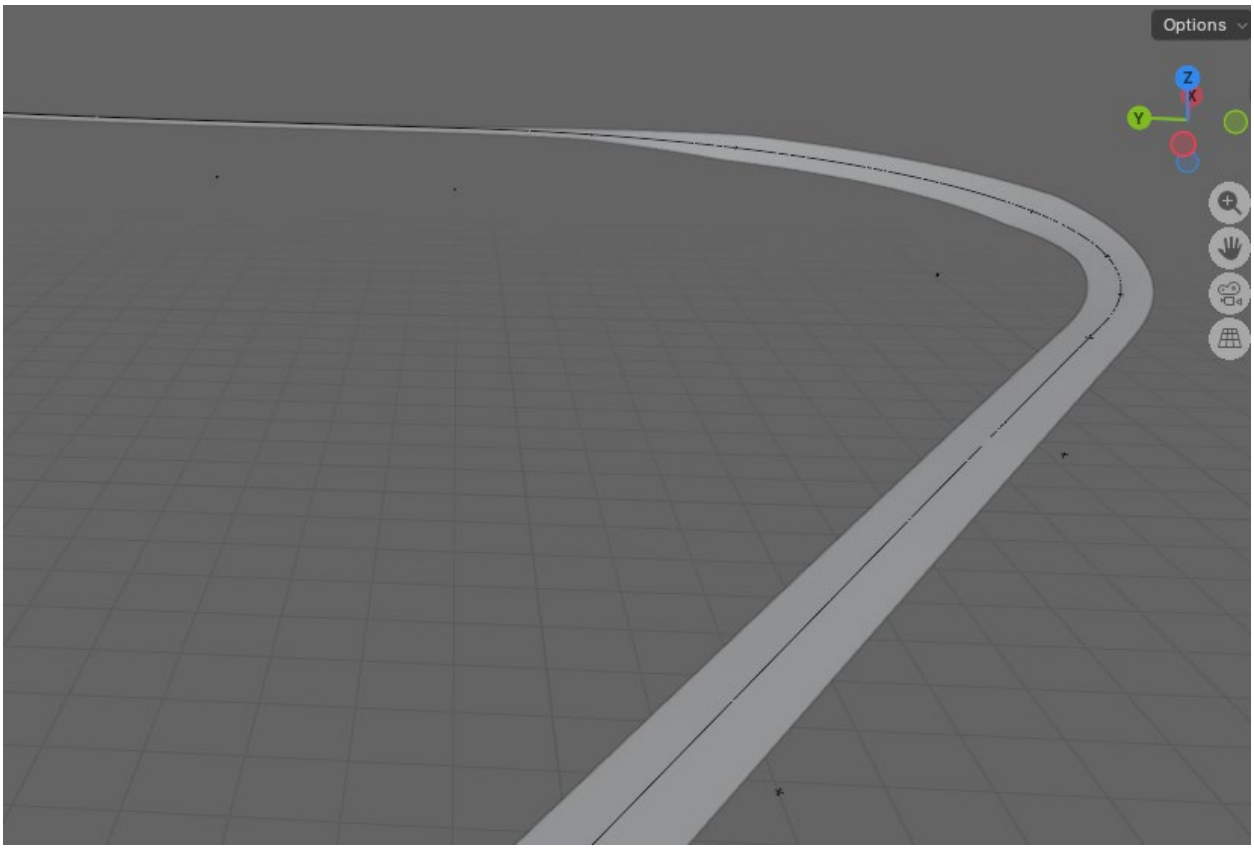


Figure 10.3-1 — Sectioned surface illustrating a superelevation transition

10.3.1 Breaklines

In the context of `IfcSectionedSurface`, a breakline is a line along the surface where the cross-slope changes abruptly — an edge of pavement, a shoulder break, the face of a curb. This usage is distinct from DTM breaklines, which force TIN triangulation to honor a feature edge. Here the term describes a topological feature of the swept surface itself: a ridge or valley line where adjacent surface facets meet at a non-tangent angle.

Breaklines arise from the tag mechanism when consecutive cross-sections have different numbers of segments. A widening lane introduces a new vertex at the distance along the directrix where the widening begins. Tags identify which vertices in one section correspond to vertices in the next, even when the two sections have different segment counts. A vertex present in one section but absent in the adjacent section causes the surface to initiate or terminate at that longitudinal position, which is the geometric definition of a breakline in this context. `IfcOpenCrossProfileDef` is the only profile that supports breaklines. Figure 10.3.1-1 illustrates a sectioned surface with breaklines resulting from cross-section topology changes along the route. `IfcSectionedSolidHorizontal` does not support the concept of breaklines.

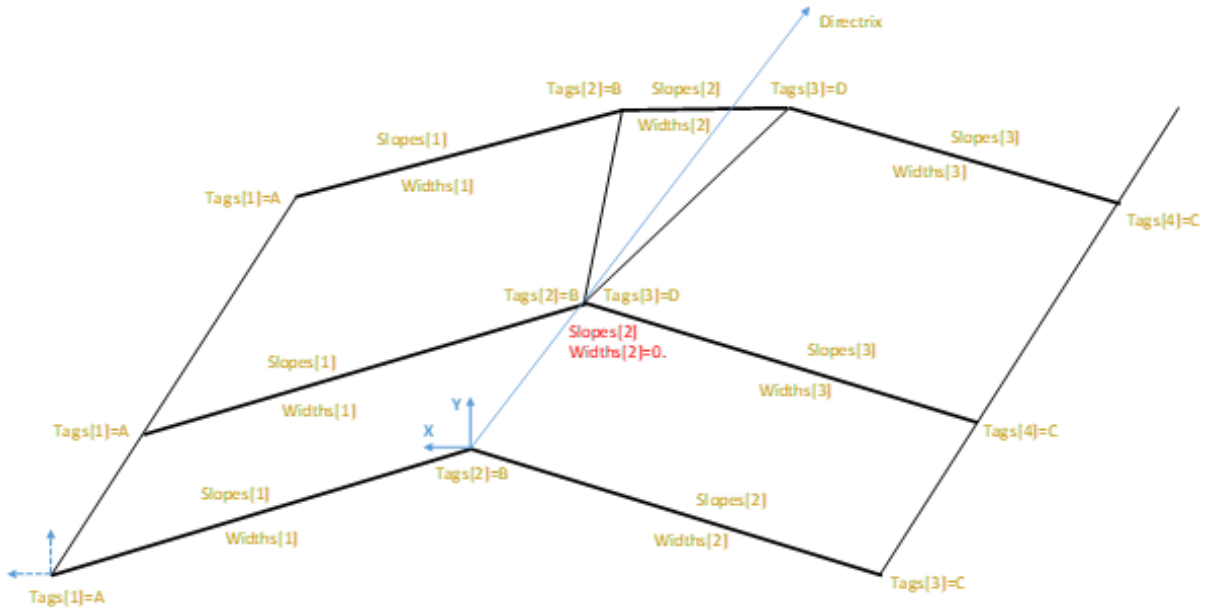


Figure 10.3.1-1 — Sectioned surface with breaklines at cross-section topology changes. (source bSI)

Figure 10.3.1-2 shows a sectioned surface inspired by Figure 10.3.1-1 with both a widening and narrowing of the surface.

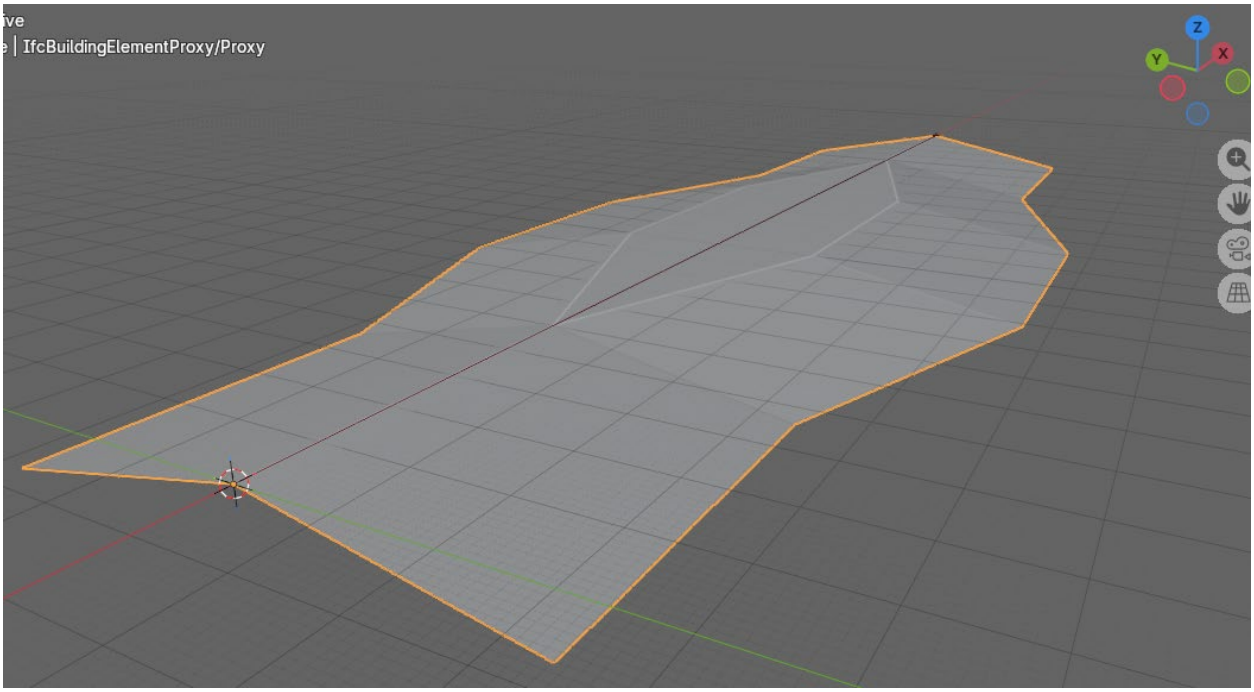


Figure 10.3.1-2 — Sectioned surface with breaklines at cross-section topology changes. This figure shows the surface widening and returning to its original width.

Although the IFC specification does not state this explicitly, the `Slope` and `Width` of an `IfcOpenCrossProfileDef` segment must be zero at the distance along the directrix where a breakline initiates or terminates. When a new feature vertex first appears — the start of a widening lane, for example — the cross-section at that distance must include the new tagged vertex with a zero-width segment. In Figure 10.3.1-1, this is why points B and D coincide in the second cross-section: the zero-width segment collapses those tagged vertices to the same position, establishing the breakline origin without introducing lateral extent. Without a zero-width segment at the breakpoint position, the surface has no defined starting position for the new feature and the geometry is ambiguous. The branching example in §10.6.1.3 illustrates this pattern for a surface where breaklines branch from the main cross-section.

The tag mechanism serves both breaklines and string lines simultaneously. A tag on a profile vertex identifies that vertex's correspondence across sections with different topology and, when a matching `IfcOffsetCurveByDistances` guide curve exists, controls the vertex's trajectory between sections. The two functions — topological correspondence and geometric guidance — share the same tag infrastructure.

10.3.2 String Lines

Within `IfcSectionedSurface`, string lines are implemented through the `Tags` attribute of `IfcOpenCrossProfileDef`. This attribute holds a list of $N + 1$ labels — one per vertex of the open profile, where N is the number of segments. An `IfcOffsetCurveByDistances` guide curve whose `Tag` matches one of these vertex labels takes control of that vertex's trajectory along the surface.

The `OffsetPoint` attribute of `IfcOpenCrossProfileDef` positions the first tagged vertex in the directrix's local coordinate system. Subsequent vertices are derived from there by accumulating the segment `Widths` and `Slopes`. It is unclear whether all tagged vertices participate in the matching process independently, allowing a surface to have some vertices guided by offset curves and others governed by linear interpolation between authored sections (see §10.5.2).

10.3.3 Cross-Section Coordinate System

The profiles of `IfcSectionedSurface` and `IfcSectionedSolidHorizontal` are defined in a local coordinate system established at each cross-section position by the corresponding `IfcAxis2PlacementLinear`. The axes of this local frame are:

- **X** — the lateral axis, pointing to the **left** of a traveler moving in the direction of increasing chainage.
- **Y** — the vertical axis, aligned with the `Axis` direction of `IfcAxis2PlacementLinear` (typically global vertical, $(0,0,1)$).
- **Z** — the extrusion direction, tangent to the directrix in the direction of increasing chainage.

The left-pointing X direction follows from the right-hand rule. Denoting the unit tangent as $\hat{t}$ and the `Axis` direction as $\hat{u}$:

$$\hat{x} = \hat{u} \times \hat{t}$$

With $\hat{u}$ pointing up and $\hat{t}$ in the direction of travel, $\hat{x}$ points to the left, and the frame satisfies $\hat{x} \times \hat{y} = \hat{z}$. Assigning X to the right would reverse this cross product, producing a left-handed system — incompatible with the extrusion direction occupying the +Z axis.

Contrast with drafting convention. When engineers draw a cross-section on paper, the conventional choice is X to the right and Y upward. The IFC cross-section coordinate system is the mirror image of this: X points to the left and Y remains upward. The visual appearance of the cross-section drawing is unchanged — features left of the alignment still appear on the left side of the drawing — but the X axis arrow points in the opposite direction. The practical consequence is that **positive X profile coordinates are to the left of the alignment centerline, and negative X coordinates are to the right**. Engineers accustomed to computing profile widths with positive values to the right must reverse the sign when authoring IFC profile geometry. The `OffsetPoint` of `IfcOpenCrossProfileDef` and the vertex coordinates in `IfcCartesianPointList2D` (for `IfcSectionedSolidHorizontal`) are both expressed in this left-X frame.

10.4 IfcSectionedSolidHorizontal

`IfcSectionedSolidHorizontal` defines a solid by sweeping closed cross-sections along a directrix curve. It follows the same fundamental structure as `IfcSectionedSurface` but produces a volumetric solid bounded by the swept faces and the cross-section planes at each end.

Attribute	Type	Description
`Directrix`	`IfcCurve`	The curve along which sections are swept
`CrossSectionPositions`	`LIST [2:?] OF IfcAxis2PlacementLinear`	Positions along the directrix at which sections are placed
`CrossSections`	`LIST [2:?] OF IfcProfileDef`	Closed cross-section profiles, one per position

Table 10.4-1 — IfcSectionedSolidHorizontal attributes

As with `IfcSectionedSurface`, the solid is generated by sweeping only between the defined `CrossSectionPositions`. It does not extend to the head or tail of the `Directrix` (see §10.5.5). The `CrossSectionPositions` and `CrossSections` lists must be equal in length, and the position expressions must not use longitudinal offsets.

Cross-sections are typically `IfcArbitraryClosedProfileDef` profiles. The profile outline is defined as an `IfcIndexedPolyCurve` referencing an `IfcCartesianPointList2D`. Profile points are listed in counter-clockwise order when viewed from the direction the profile

normal points. The `IfcCartesianPointList2D.TagList` attribute assigns a label to each point in the coordinate list, enabling the string line mechanism of §10.2. Guide curve hosting and tag scoping follow the same sibling-representation construction described for `IfcSectionedSurface` in §10.5.2, demonstrated through `IfcSectionedSolidHorizontal` examples in §10.6.2.5 and §10.6.2.6.

For a note on a documentation error in the profile orientation specification for this entity, see §10.5.6.

10.4.1 Rotations

Cross-section rotation accommodates superelevation — the banking of a road or rail cross-section along a curve. `IfcSectionedSolidHorizontal` supports two approaches.

Single superelevation applies a uniform rotation to the entire cross-section at each defined position along the directrix. `IfcDerivedProfileDef` expresses this rotation, with its `ParentProfile` referencing the base (unrotated) profile. Each entry in `CrossSections` can reference its own `IfcDerivedProfileDef` instance with a different rotation angle, allowing the rotation to vary along the directrix while the underlying profile shape remains constant.

Multiple independent superelevations apply when different parts of the cross-section rotate independently — for example, a divided highway where left and right carriageways have different cross-slopes, or a cross-section with a varying median shape. In this case each `CrossSection` is a distinct profile instance. The tagged points in `IfcCartesianPointList2D.TagList` and their corresponding `IfcOffsetCurveByDistances` guide curves control where each point travels independently along the route. The guide curves effectively encode the superelevation variation for each tagged feature point without requiring explicit rotation parameters.

10.5 Specification Gaps and Implementation Notes

10.5.1 Design History: The Guided Sweep Proposal

This section documents the origin of the tag-based guide curve mechanism used throughout this chapter. §10.5.2 and §10.5.3 rely on it as evidence of the specification writers' intended construction — not as historical background to a still-open question, but as the basis for stating that construction directly.

The tag-based guide curve mechanism described in this chapter originates from buildingSMART issue bSI-InfraRoom/IFC-Specification #118, "[IFC Tunnel] Guided sweep representation" [9], opened to address a requirement for sectioned sweeps with varying profiles whose paths between profiles could be described in a non-linear manner. Two competing EXPRESS formalizations were drafted and posted to the issue (guided_sweep_1.txt, guided_sweep_2.txt), alongside a discussion document (2021-08-11_GuidedSweeps.pdf). An intermediate proposal described guiding curves independently as `IfcSegmentedReferenceCurve` instances positioned using `IfcPointByDistanceExpression`; the mechanism that ultimately shipped in the schema

instead uses `IfcOffsetCurveByDistances`, though both approaches share the same underlying `IfcPointByDistanceExpression`-based referencing.

One considered formalization, `IfcLinearlyReferencedProfileDef`, was opposed during review specifically because it conflicted with the assumption, used throughout the schema, that `IfcProfileDef` is defined in a local x-y plane suitable for arbitrary extrusions and lofts — a reviewer objected that it was unclear how an `IfcExtrudedAreaSolid` would even be constructed from such a profile, and that the proposal "messes with too many well established concepts."

The mechanism that was adopted instead — tag matching between profile vertices and an independently-defined guide curve — was accepted with its principal weakness stated explicitly by its own proposer:

> The downside, however, is that the `IfcCartesianPoint` of the profile and the corresponding `IfcPointByDistanceExpression` of the GuidingCurve must be linked explicitly and might have slightly diverging locations. Consistency must be ensured/checked by the implementors, potentially by defining an upper threshold for the acceptable distance between them.

This confirms that the redundancy discussed in §10.5.3 is a known and accepted trade-off, not an oversight: the specification writers deliberately left consistency between a tagged vertex's authored coordinate and its guide curve's path as an implementer responsibility, with no schema-level enforcement proposed or adopted.

The resolution that ultimately closed the issue favored a simpler approach over a dedicated new profile-definition entity: a review comment pointed to the existing pattern of carrying an `Axis` (2D reference geometry) and a `Body` (swept solid geometry) as separate representations on the same product — as already used on `IfcWall` — as precedent for describing guiding curves and swept geometry as siblings rather than inventing new geometry types to carry them.

10.5.2 Guide Curve Hosting and Tag Resolution

Guide curves belong as `Items` in a second `IfcShapeRepresentation` — `RepresentationIdentifier = "Axis"`, `RepresentationType = "GeometricCurveSet"` — that is a sibling entry, alongside the representation holding the surface or solid, within the same `IfcProductDefinitionShape.Representations` list. This is the `Axis`/`Body` pattern already used elsewhere in IFC (for example, on `IfcWall`) and is the construction described in §10.5.1. Guide curves hosted this way are scoped to the `IfcProductDefinitionShape` they belong to, so tags need only be unique within that scope.

This does not close every case, however: if a model author places more than one `IfcSectionedSurface` or `IfcSectionedSolidHorizontal` across the representations of a single `IfcProductDefinitionShape` — schema-legal — representation scoping alone cannot determine which guide curves belong to which surface. Implementations should avoid this configuration.

Even under this construction, no IFC attribute, inverse relationship, or STEP entity identifier points from a tagged vertex to the guide curve that controls it — the only link is the shared string value of `Tag`. Because every guide curve for a surface or solid is collected into the same `IfcProductDefinitionShape`, resolving a guided vertex only requires comparing tags among that representation's `Items`, not searching the entire model. However, a model-wide search may be required (depending on the implementation) to find all the `IfcRepresentation`s that contain the surface or solid as an `IfcRepresentationItem`. From there the owning `IfcProductDefinitionShape` (either directly or by traversing `IfcRepresentationMap` -> `IfcMappedItem` -> `IfcShapeRepresentation`) can be interrogated for the sibling `"Axis"/"GeometricCurveSet"` representation.

10.5.3 Consistency Between Authored Coordinates and Guide Curves

A guided vertex's position is defined twice, through two independent mechanisms: once by the profile's authored coordinate, and once by the string line's own geometry. When a cross-section vertex is authored at a specific position and a guide curve with a matching tag passes through a different position for the same cross-section, the two descriptions disagree. The mechanism's own proposer (§10.5.1) stated plainly that the two "must be linked explicitly and might have slightly diverging locations," with consistency to be "ensured/checked by the implementors, potentially by defining an upper threshold for the acceptable distance between them." The intent was never that these two descriptions could legitimately diverge and be arbitrated by some precedence rule; it was that they are supposed to coincide, and keeping them so is an authoring responsibility, not a question a consuming application resolves case by case.

What remains a genuine, unresolved limitation is that this consistency has no schema-level enforcement: the specification provides no mechanism, tolerance value, or validation rule for detecting when an authored coordinate and its guide curve have drifted apart. A model can therefore still carry a defect — a divergence beyond whatever tolerance the authoring tool intended — that no consuming application can distinguish from a small, accepted numerical difference. Interoperability issues that follow from such a defect (visualization, surface area and volume estimates, clash detection) are a data-quality problem in the authored file, not a case of the specification failing to state which value is authoritative.

10.5.4 Partial Guide Curve Coverage

The IFC specification does not define what happens when a tagged cross-section vertex has no corresponding guide curve. The ambiguity is meaningful: if a tagged vertex with no matching guide curve falls back to linear interpolation between authored sections, then guide curves and linear interpolation can be freely mixed within the same surface or solid — some vertices precisely guided, others interpolated. If instead every tagged vertex is required to have a matching guide curve, models must supply redundant zero-offset guide curves for vertices whose trajectory is already linear (as in the centerline vertex of §10.6.1.4 and §10.6.2.5). The practical use case for mixing is clear: exterior edge vertices follow

string line guide curves through a curved widening while interior breakline vertices use linear interpolation — combining the geometric precision of string lines with the simplicity of template-based sections. An implementation agreement is needed to confirm that a tagged vertex with no matching guide curve is treated as unguided and linearly interpolated, rather than as a modeling error.

10.5.5 Extent Along the Directrix

`IfcSectionedSurface` and `IfcSectionedSolidHorizontal` are bounded by their `CrossSectionPositions` — geometry is generated by sweeping only between the first and last defined position and does not extend to the head or tail of the `Directrix`. Figure 8.8.3.35.C in the IFC specification incorrectly depicts the solid extending to the head and tail of the `Directrix` when `CrossSectionPositions` are at interior distances, contradicting the specification text. The practical consequence of such extension would be severe: multiple `IfcSectionedSolidHorizontal` instances modeling sequential prisms along the same alignment — pavement layers, cut sections, fill sections — would each extend to the full alignment endpoints and overlap one another. Implementers should follow the specification text, not the figure.

Guide curve extent follows the opposite rule. The `IfcOffsetCurveByDistances` specification states that if the defined offset values do not span the full extent of the basis curve, the offsets implicitly continue with the same value toward the head and tail. For guide curves this means a constant offset string line — such as the edge of a uniform-width shoulder — can be expressed with a single `IfcPointByDistanceExpression` without requiring offset values at every cross-section position. The two behaviors are independent: guide curves extend implicitly because they control interpolation *within* the surface or solid's defined span, not because they extend that span.

10.5.6 Cross-Section Orientation: Default Axis Direction

Each `IfcAxis2PlacementLinear` entry in `CrossSectionPositions` defines the local coordinate system at which its corresponding cross-section is placed. The `Axis` attribute specifies the local "up" direction — the Z-axis of that coordinate system — which determines how the cross-section is oriented in 3D space. As discussed in §8.3.2, the default value of `Axis` when the attribute is omitted is not unambiguously defined in the IFC schema.

Historical context. During early development of `IfcSectionedSolidHorizontal` under IFC4x3, the `Axis` direction at each cross-section position was envisioned as always vertical — `(0, 0, 1)` in global coordinates. As the standard evolved through RC1–RC4 and ADD1, this assumption was replaced with an author-controlled `IfcAxis2PlacementLinear` allowing `Axis` to be specified explicitly per cross-section position. The resolved specification permits both interpretations, but the default when `Axis` is omitted remains undefined (see §8.3.2).

Geometric consequence. On a flat (zero-grade) alignment the two interpretations produce identical results. On a graded alignment they diverge:

- **Axis = (0, 0, 1).** The cross-section "up" direction is always global vertical. Cross-section faces are plumb — their planes are perpendicular to the horizontal projection of the directrix, not to the 3D curve tangent. For infrastructure design this is the natural interpretation: end views and cross-sections are in a vertical plane, heights are measured vertically, widths are measured horizontally.
- **Axis perpendicular to the 3D tangent.** The cross-section "up" direction tilts with the grade, remaining in the vertical plane containing the 3D tangent. Cross-section faces are truly perpendicular to the 3D curve. On a graded alignment the faces lean forward or backward relative to the direction of travel, producing a different solid with different volume and cross-sectional area at any given distance. This interpretation yields the same solid that `IfcExtrudedAreaSolid` would produce for the same profile swept along the same path.

On alignments with cant, the Axis = (0,0,1) interpretation would exclude the cross-section rotation from the sweep, requiring it to be explicitly constructed into the `IfcSectionedSurface` profile or the `IfcSectionedSolidHorizontal` section rotation mechanism. The other interpretation would allow for cant-defined rotation to occur by simply using the default axis direction, as discussed in §10.6.1.1 and §10.6.2.1.

Figures 10.5.6-1 and 10.5.6-2 show the retaining wall example of §10.6.2.4 swept along a 10% graded directrix. In Figure 10.5.6-1, `Axis` is supplied explicitly as `(0, 0, 1)` at each cross-section position and the end face is plumb. Figure 10.5.6-2 shows the same scenario with `Axis` omitted, rendered by two different IFC viewers that have adopted opposite defaults: the upper half uses the perpendicular-to-3D-tangent interpretation; the lower half uses `(0, 0, 1)`. The divergence is most apparent at the start and end of the wall, where the two viewers produce noticeably different face orientations from an identical input file.

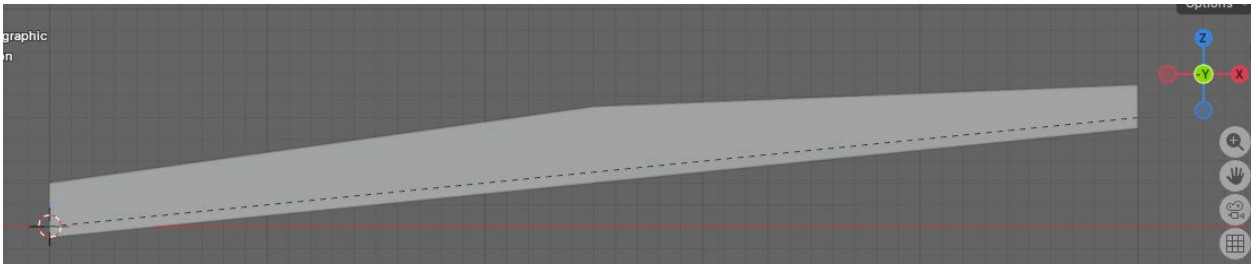


Figure 10.5.6-1 — Elevation view of `IfcSectionedSolidHorizontal` on a 10% graded alignment with `Axis = (0, 0, 1)`: end face is vertical (plumb).

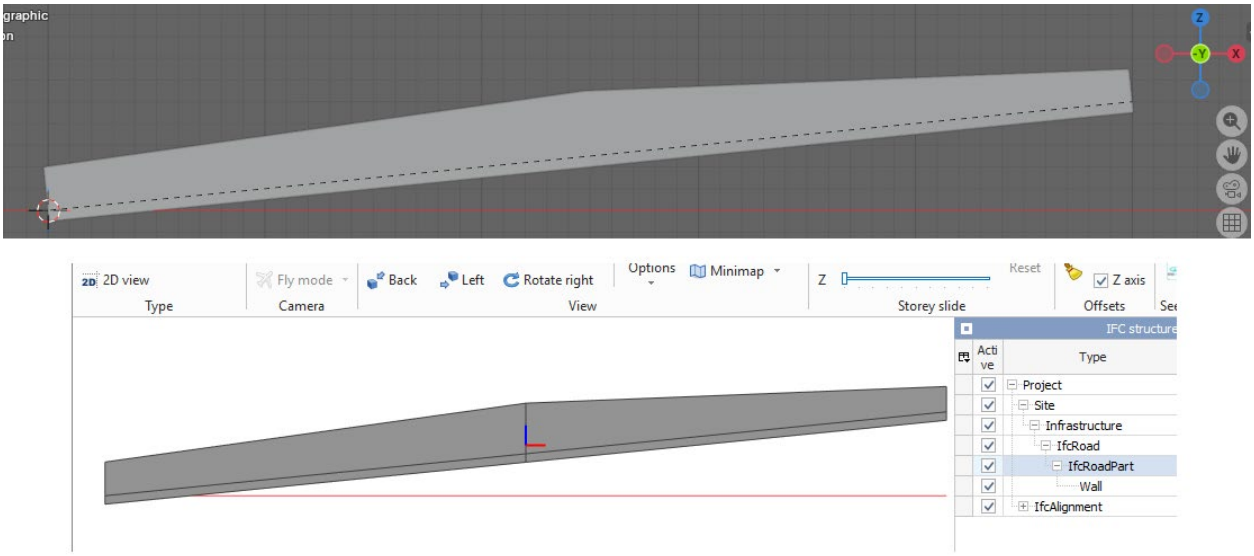


Figure 10.5.6-2 — The same `IfcSectionedSolidHorizontal` model (retaining wall, 10% grade, `Axis` omitted) rendered by two IFC viewers: the upper half assumes `Axis` perpendicular to the 3D tangent (inclined end face); the lower half assumes `Axis = (0, 0, 1)` (plumb end face).

Practical consequences. The discrepancy extends beyond geometry. Quantity takeoffs derived from these two solids yield different volumes — the differing end-face angles shift the amount of material at each terminus, and the effect grows with grade steepness. A bill-of-materials or payment verification workflow can produce measurably different results from the same model file depending solely on which viewer performed the calculation.

Clash detection is similarly affected. The two solids occupy different regions of 3D space: an element near a wall endpoint may show as clear in one viewer and conflicting in another. A clash report's findings become tool-dependent, with nothing in the model file to signal the ambiguity.

In construction, the disagreement extends to physical work. Formwork geometry, excavation limits, and bearing surface angles at the wall ends all differ between interpretations. A contractor working from one viewer's output and an inspector referencing another's may reach opposite conclusions about what was designed — each correctly reading what their own tool shows.

Recommendation. Until such time as this discrepancy can be resolved, authors should always supply `Axis` explicitly in every `IfcAxis2PlacementLinear` entry within `CrossSectionPositions`. While this is onerous and redundant for cant alignments, it will accurately convey the modeling intent.

10.5.7 Profile Orientation Documentation Error

The specification for `IfcSectionedSolidHorizontal` states:

> The profile X axis is the direction of `RefDirection` from `IfcAxis2PlacementLinear`, and the profile Y axis is the direction of `Axis`.

This is incorrect. The profile is defined in a 2D XY plane and has a normal — the axis perpendicular to that plane — which determines how the profile is oriented in 3D space when swept along the directrix. It is that normal, not the profile X axis, that is the direction of `RefDirection`. When `RefDirection` is omitted from `IfcAxis2PlacementLinear` it defaults to the curve tangent direction. If the profile X axis were to align with the tangent, the profile plane would lie parallel to the sweep direction rather than perpendicular to it, producing degenerate geometry. The correct behavior is that the profile normal aligns with `RefDirection` (or the curve tangent when `RefDirection` is absent), placing the profile face perpendicular to the sweep path.

The specification for `IfcSectionedSurface` states this correctly: “the profile normal is derived from the associated `IfcAxis2PlacementLinear`.” The `IfcSectionedSolidHorizontal` documentation should be read consistently with the `IfcSectionedSurface` text. This error is documented in buildingSMART issues IFC4.x-IF #147 and IFC4.x-development #1010 [6].

10.6 Example Models

The following example models illustrate the concepts of this chapter in progressively more complex configurations, organized by entity type.

10.6.1 IfcSectionedSurface

10.6.1.1 Minimal Definition

The file `IfcSectionedSurface.ifc` demonstrates `IfcSectionedSurface` with an `IfcSegmentedReferenceCurve` as the `Directrix`. The alignment is a 50 m Bloss transition with a matching cant transition from zero to 500 mm right-rail elevation — exaggerated to make the banking clearly visible. The cross-section is a single flat segment spanning the 1.5 m track gauge placed at both ends of the alignment; the `IfcSegmentedReferenceCurve` rotates the profile frame continuously with cant, so the banking arises entirely from the directrix rather than the profile geometry.

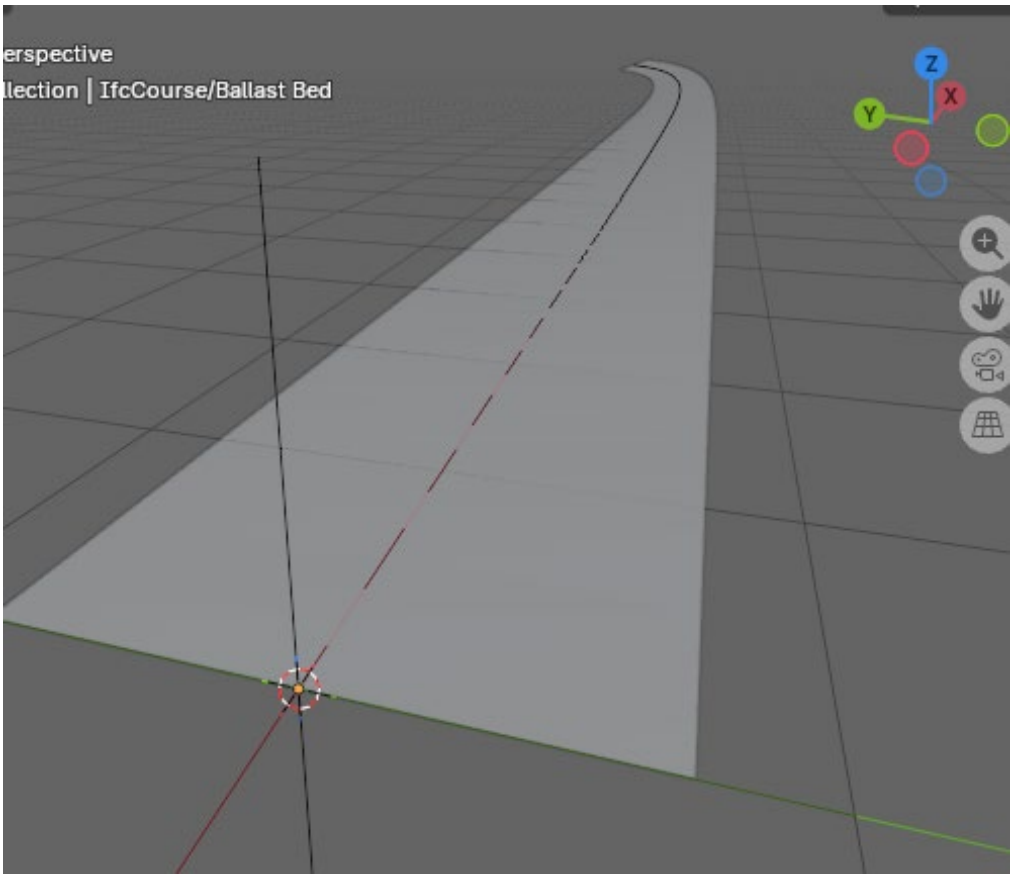


Figure 10.6.1.1-1 — `IfcSectionedSurface` (ballast bed top surface, 1.5 m track gauge) swept along an `IfcSegmentedReferenceCurve` directrix. The 50 m Bloss cant transition banks the surface from level at the start to 500 mm right-rail elevation at the end.

10.6.1.2 Superelevation

The file `IfcSectionedSurface_superelevation.ifc` is a template-based `IfcSectionedSurface` illustrating a superelevation transition, shown in Figure 10.3-1. The surface is defined by cross-sections at discrete positions along a `Directrix`, with linear interpolation between them. No guide curves are used. This is the simplest configuration and produces adequate geometry when the variation between sections is itself linear — which is the case for uniform superelevation rotation. The model also includes `IfcReferent` instances marking the superelevation events along the alignment.

10.6.1.3 Sectioned Surface with Branching

The file `IfcSectionedSurface_with_branching.ifc` is shown in Figure 10.3.1-2. It is based on the breakline example given in the `IfcSectionedSurface` documentation, enhanced to show both a widening and a subsequent narrowing of the section — a median or auxiliary lane that emerges from the main roadway and then returns.

10.6.1.4 String Lines — Guide Curves Sharing the Directrix

The file `IfcSectionedSurface_with_stringlines_shared_basiscurve.ifc` demonstrates guide curves offset directly from the surface's own `Directrix`: each guide curve's `BasisCurve` is `Main-Line`, the same curve the surface sweeps along. This contrasts with §10.6.1.5, where the guide curves instead follow independent curve geometry unrelated to the `Directrix`. Guide curves here are hosted using the sibling-representation construction §10.5.1 and §10.5.2 identify as the intended one.

The directrix is a 200 m straight alignment (`Main-Line`) with a flat vertical profile. The symmetric open profile has two segments and three tagged vertices: `A` (left edge), `B` (centerline), and `C` (right edge), each with a corresponding `IfcOffsetCurveByDistances` guide curve sharing `Main-Line` as its `BasisCurve`. Only two cross-sections are authored — one at each end of the surface. The guide curves carry all intermediate geometry: guide curves `A` and `C` define a variable-width surface, both edges beginning at ±30 m, widening to ±45 m at the 100 m midpoint, and returning to ±30 m at 200 m; guide curve `B` holds the centerline at a constant zero offset throughout. Without the guide curves, linear interpolation between the two end sections would produce a uniform 30 m width along the full 200 m — the guide curves are what create the widening.

> **Note:** `Main-Line` cannot itself serve as the guide curve for the centerline vertex: `IfcAlignment` does not have a `Tag` attribute. Guide curve `B` is therefore an `IfcOffsetCurveByDistances` with a zero lateral offset from the alignment, existing solely to carry the tag `"B"` — though this may not be required if IFC allows a mix of guide curves and linear interpolation to define a sectioned surface or solid. This ambiguity is discussed in §10.5.4.

The three tagged `IfcOffsetCurveByDistances` instances are collected into a second `IfcShapeRepresentation` — `RepresentationIdentifier = "Axis"`, `RepresentationType = "GeometricCurveSet"` — that is a sibling entry, alongside the `"Body"`/`SectionedSurface` representation holding the `IfcSectionedSurface`, in the same `IfcPavement`'s `IfcProductDefinitionShape.Representations` list. Because that `IfcProductDefinitionShape` is only ever reachable through the rooted `IfcPavement`, the guide curves inherit a traceable path to a rooted entity — see §10.5.2 for the general technique. Because all three guide curves belong to this single `IfcProductDefinitionShape`, tag matching is unambiguously scoped: tags `"A"`, `"B"`, and `"C"` need only be unique within this representation, not globally across the model — whether or not a guide curve's `BasisCurve` happens to match the `Directrix`, as it does for all three here.

Figure 10.6.1.4-1 shows the expected geometry of this example.

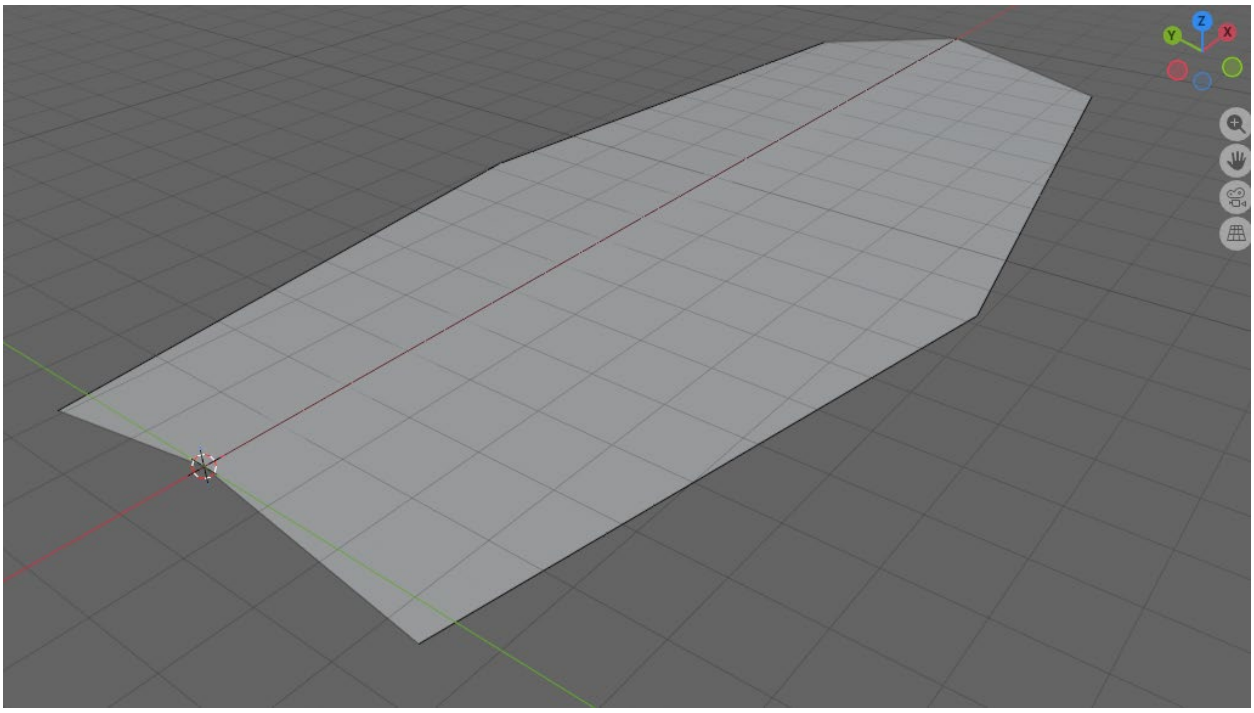


Figure 10.6.1.4-1 — Expected geometry of `IfcSectionedSurface` with string line guide curves. This rendering has been synthesized to illustrate the correct geometry; no viewer capable of resolving string line guide curves has been found at the time of writing.

Figure 10.6.1.4-2 shows the resulting entity structure: two sibling shape representations under one `IfcProductDefinitionShape`.

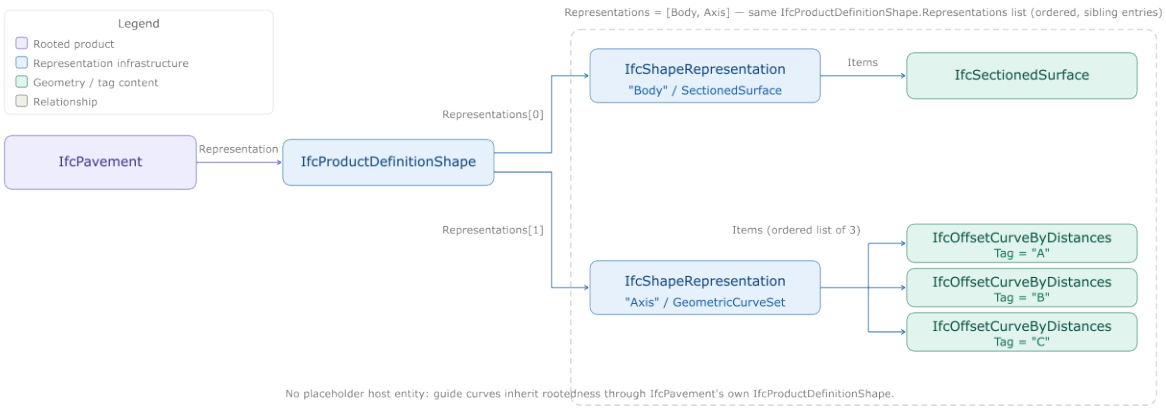


Figure 10.6.1.4-2 — Guide curve rootedness via sibling representation: the surface and its three guide curves hang off the same `IfcProductDefinitionShape`.

§10.6.2.5 demonstrates the same technique applied to the `IfcSectionedSolidHorizontal` counterpart of this example; see Figure 10.6.2.5-1 for comparison.

10.6.1.5 String Lines — Independent Guide Curves

The file `IfcSectionedSurface_with_stringlines_independent_guide_curves.ifc` takes a different approach: the edge string lines follow independent curve geometry — circular arcs — rather than parametric offsets from the centerline. `Main-Line` is a 200 m straight alignment. The left and right edge curves start tangent-parallel to `Main-Line`, offset ±30 m from the centerline, and curve away from it at a 1010 m radius — the radius that puts each arc exactly ±50 m from centerline, the width authored on the cross-section there, at the point 200 m forward, matching `Main-Line`'s own endpoint. Their arc length is 205 m, a bit longer than the ~201.3 m strictly needed to reach that point: a guide curve may run past what's required (§10.5.5); only falling short is a defect. Unlike `Main-Line`, the edge curves are not `IfcAlignment` products at all — each is a bare `IfcGradientCurve`, built directly from `IfcCurveSegment`/`IfcCircle` entities with no semantic alignment layer (`IfcAlignmentHorizontal`/`Vertical`, segments, or nesting) above it, referenced only as the `BasisCurve` of an `IfcOffsetCurveByDistances`. The guide curves for tags `"A"` and `"C"` are zero-offset curves from these bare edge curves; the guide curve for tag `"B"` is a zero-offset from `Main-Line`. All three are hosted in a sibling `"Axis"` `IfcShapeRepresentation` alongside the surface's `"Body"` representation, as in §10.6.1.4.

Although the guide curves for tags `"A"` and `"C"` have a different `BasisCurve` than the surface `Directrix`, this has no effect on tag scoping here: all three guide curves are hosted in the same sibling `"Axis"` representation as the surface, so scoping is governed by that shared `IfcProductDefinitionShape` (§10.5.2) regardless of whether a guide curve's `BasisCurve` matches the `Directrix`.

This configuration is the most conceptually faithful to the idea of string lines — each edge path is an independent geometric entity. The edge arcs' radius is fixed entirely by the requirement that each pass through the correct point at `Main-Line`'s endpoint. Getting that arithmetic right is the model author's responsibility: nothing in the specification checks that an independently-parameterized guide curve reaches the correct offset at the correct distance, and a model that gets it wrong carries a defect of the same kind discussed in §10.5.3.

Figure 10.6.1.5-1 shows the expected geometry of this example.

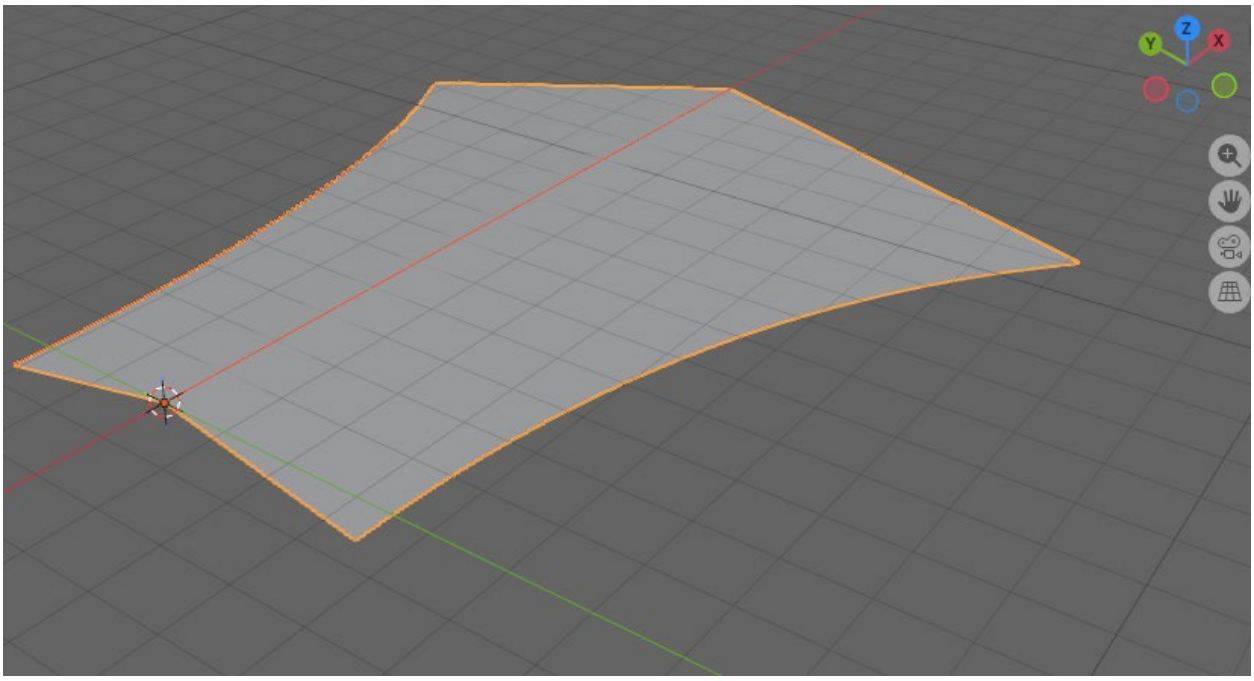


Figure 10.6.1.5-1 — Expected geometry of `IfcSectionedSurface_with_stringlines_independent_guide_curves.ifc`. This rendering has been synthesized to illustrate the correct geometry; no viewer capable of resolving string line guide curves has been found at the time of writing.

10.6.1.6 String Lines — Dense Section Approximation

The file `IfcSectionedSurface_with_stringlines_dense_section_approximation.ifc` reproduces the same edge geometry as §10.6.1.5 — a straight centerline with curved edges defined by 1010 m radius arcs — but using the template approach rather than guide curves. Cross-section widths at four evenly-spaced stations are computed analytically from the arc geometry, and the resulting `IfcOpenCrossProfileDef` instances embed the exact width at each station — deliberately few, so the piecewise-linear approximation error remains visible rather than vanishing as more sections are added. This example retains the true edge geometry — `Left_Edge` and `Right_Edge`, ordinary `IfcAlignment` instances with real circular-arc geometry — not as functioning guide curves (this is a pure template-approach model; nothing here references them by `Tag`), but purely so Figure 10.6.1.6-2 has an exact curve to compare the template-approximated surface against. Figure 10.6.1.6-1 shows the resulting surface, widening along its length according to the four analytically computed cross-sections.

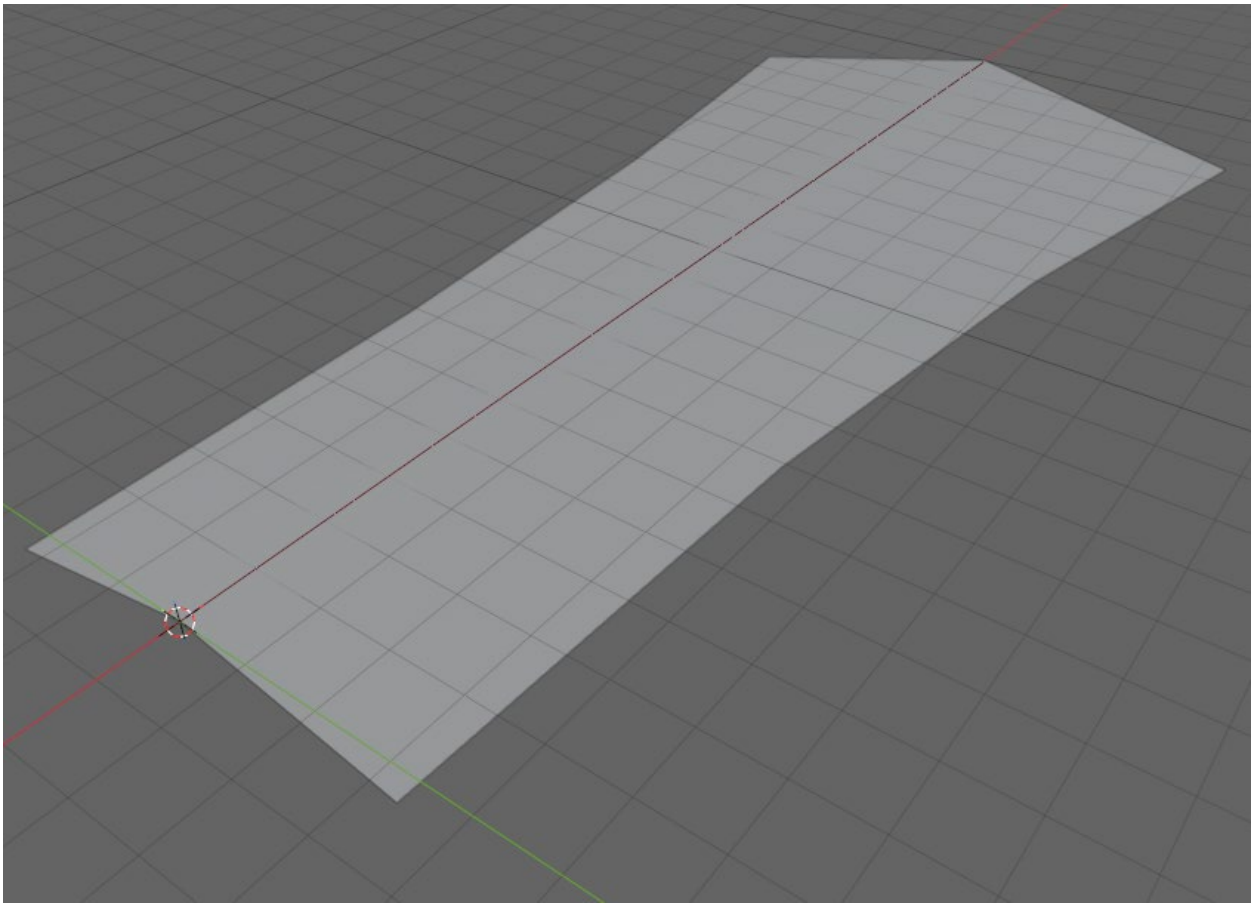


Figure 10.6.1.6-1 — Sectioned surface with four cross-sections approximating curved edge geometry; the edge paths between them are piecewise-linear, not the true arc.

Figure 10.6.1.6-2 zooms in on the far-right corner of the surface, where the gap between the exact circular arc guide curve (shown in orange) and the piecewise-linear surface edge is most visible. With only four sections, the approximation error is perceptible at the scale of the model.

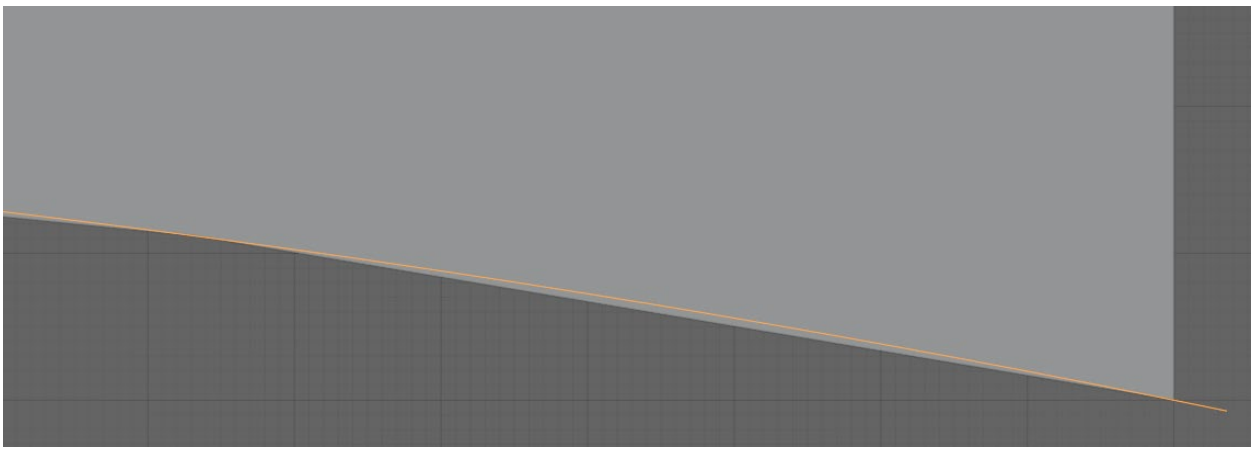


Figure 10.6.1.6-2 — Zoomed view of the far-right corner showing the gap between the piecewise-linear surface edge and the exact circular arc guide curve (orange). The error is largest at the midpoint between cross-sections.

This example illustrates the trade-off between the two approaches. The template approach with dense sections can approximate curved edge geometry to any desired tolerance, but requires explicit computation of profile widths at each station and produces a model with many cross-section instances. The string line approach of §10.6.1.5 requires only two sections and lets the guide curves carry the geometry exactly — but only if the model author gets the underlying arc geometry exactly right (§10.5.3); the template approach here sidesteps that precision requirement by sampling the known curve directly, at the cost of only approximating it.

10.6.2 IfcSectionedSolidHorizontal

10.6.2.1 Minimal Definition

The file `IfcSectionedSolidHorizontal.ifc` is the solid counterpart to §10.6.1.1, using the same `IfcSegmentedReferenceCurve` directrix (50 m Bloss transition with exaggerated cant) and the same two-section minimal structure. The open surface profile is replaced by a closed trapezoidal ballast bed cross-section — 1.8 m wide at the base, 1.5 m at the top, and 0.25 m deep — with no profile rotation, superelevation, or guide curves.

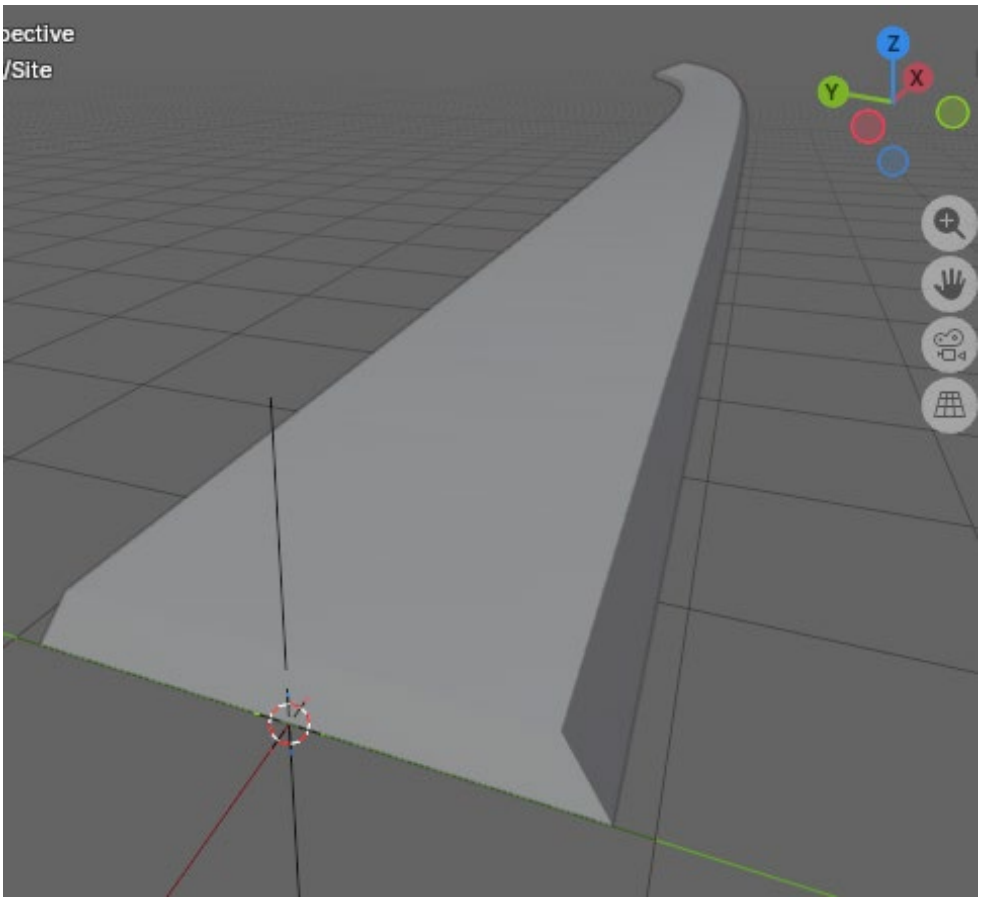


Figure 10.6.2.1-1 — `IfcSectionedSolidHorizontal` (trapezoidal ballast bed, 1.5 m top width) swept along an `IfcSegmentedReferenceCurve` directrix. The 50 m Bloss cant transition banks the solid from level at the start to 500 mm right-rail elevation at the end.

10.6.2.2 Single Superelevation

The file `IfcSectionedSolidHorizontal_single_superelevation.ifc` demonstrates the `IfcDerivedProfileDef` superelevation approach described in §10.4.1. The cross-slope transitions from flat (0%) at station 0 m to a 10% superelevation at station 50 m, with the geometry engine interpolating linearly between the two defined positions.

The alignment is a 50 m straight line due East at zero grade with no cant. The directrix is therefore an `IfcGradientCurve` — no `IfcSegmentedReferenceCurve` is needed because the superelevation is encoded entirely in the profile rotation rather than in the alignment cant.

The base cross-section is a 60 m wide trapezoid — vertices `(-30, 0)`, `(30, 0)`, `(29.5, 0.5)`, `(-29.5, 0.5)`, encoded as an `IfcArbitraryClosedProfileDef` — exaggerated in width relative to §10.6.2.1's rail ballast bed to make the rotation clearly visible. A single instance of this base profile is shared by both cross-section positions; each position wraps it in its own `IfcDerivedProfileDef` whose `IfcCartesianTransformationOperator2D` carries the rotation.

The rotation angle is $\theta = \tan^{-1}(e)$, where e is the cross-slope (rise/run): $\theta = 0$ at station 0 m and $\theta \approx 5.71^\circ$ at station 50 m. The geometry engine interpolates the rotation linearly between the two positions.

10.6.2.3 Crown Cross-Section with Superelevation

The file `IfcSectionedSolidHorizontal_superelevation.ifc` is the solid counterpart to `IfcSectionedSurface_superelevation.ifc`. It demonstrates `IfcSectionedSolidHorizontal` with a distinct `IfcArbitraryClosedProfileDef` at each cross-section position, each directly encoding the cross-slope geometry for that station. The alignment structure, station pattern, and superelevation progression match the surface example; the difference is that the cross-section here is a closed solid profile rather than an open surface profile.

The alignment is a tangent–arc–tangent sequence: a 600 m leading straight, a 300 m radius circular arc of 585 m arc length, and a 600 m trailing straight, at a constant 0% grade starting at elevation 30 m. The directrix is an `IfcGradientCurve`. Start station is 3000 m.

Each cross-section is a V-shaped pavement slab 60 m wide and 0.5 m thick. The crown point is at the profile origin $(0,0)$ — the top centerline. From the crown, the road surface slopes to each shoulder at a rate defined by the superelevation state at that station. The soffit is 0.5 m vertically below each top-surface point. There are no beveled edges.

Six `IfcArbitraryClosedProfileDef` instances — one per position — directly encode the cross-slope at each station, mirroring the six `IfcOpenCrossProfileDef` instances in the

surface counterpart. Without guide curves, vertex trajectories between stations are linearly interpolated; explicit guide curve control is illustrated in §10.6.2.5 and §10.6.2.6.

10.6.2.4 Retaining Wall with Variable Section

Two files demonstrate `IfcSectionedSolidHorizontal` with a cross-section whose shape transforms along the alignment:

- `IfcSectionedSolidHorizontal_retaining_wall_explicit_axis.ifc` — `Axis = (0, 0, 1)` supplied explicitly at each `CrossSectionPosition`; cross-section faces are plumb (Figure 10.5.6-1).
- `IfcSectionedSolidHorizontal_retaining_wall_implicit_axis.ifc` — `Axis` omitted; the resulting orientation is implementation-defined (Figure 10.5.6-2).

The alignment is a 50 m straight grade at 10% due East. Three cross-sections are defined — at 0 m, 25 m, and 50 m — and the geometry engine interpolates linearly between them. The cross-section is a retaining wall with footing; the stem height varies from 2.0 m at the start, to 3.0 m at the midpoint, and back to 1.5 m at the end. The footing width also varies: 2.0 m at the start and end, widening to 2.5 m at the midpoint, while the toe and wall thickness remain constant throughout. See §10.5.6 for a discussion of the `Axis` direction and its geometric consequences.

10.6.2.5 String Lines — Guide Curves Sharing the Directrix

The file `IfcSectionedSolidHorizontal_with_stringlines_shared_basiscurve.ifc` demonstrates guide curves controlling tagged vertices of an `IfcArbitraryClosedProfileDef`, replacing the `IfcOpenCrossProfileDef.Tags` mechanism of the surface with `IfcCartesianPointList2D.TagList`. It is the solid counterpart to `IfcSectionedSurface_with_stringlines_shared_basiscurve.ifc` (§10.6.1.4), using the same sibling-representation technique — the construction §10.5.1 and §10.5.2 identify as the intended one — rather than a contrasting alternative.

The directrix is the same 200 m straight alignment (`Main-Line`) at flat grade. Two cross-section positions are authored — one at each end. Six guide curves — bare `IfcOffsetCurveByDistances` instances tagged `"A_top"`, `"B_top"`, `"C_top"`, `"A_bot"`, `"B_bot"`, `"C_bot"` — share `Main-Line` as their `BasisCurve`, the same curve as the solid `Directrix`. Because all six belong to this single `IfcProductDefinitionShape`, tag matching is unambiguously scoped (see §10.5.2) regardless of that shared `BasisCurve`. The top guide curve offsets are unchanged from §10.6.1.4's `A`/`B`/`C`: `A_top` and `C_top` widen from ±30 m to ±45 m at the 100 m midpoint and return; `B_top` holds the crown at zero offset throughout. The soffit guide curves (`A_bot`, `B_bot`, `C_bot`) follow the same lateral path as their top counterparts, offset a further 1 m downward to hold the slab thickness constant.

The six tagged `IfcOffsetCurveByDistances` instances are collected into a second `IfcShapeRepresentation` — `RepresentationIdentifier = "Axis"`, `RepresentationType = "GeometricCurveSet"` — that is a sibling entry, alongside the `"Body"`/`"SolidModel"` representation holding the `IfcSectionedSolidHorizontal`, in the same `IfcPavement`'s

`IfcProductDefinitionShape.Representations` list. Because that `IfcProductDefinitionShape` is only ever reachable through the rooted `IfcPavement`, the guide curves inherit a traceable path to a rooted entity — see §10.5.2 for the general technique, and for the gap that arises if multiple sectioned entities were to share one `IfcProductDefinitionShape`. Figure 10.6.2.5-1 illustrates the resulting entity structure; compare it with Figure 10.6.1.4-2, which shows the same sibling-representation structure applied to the surface counterpart.

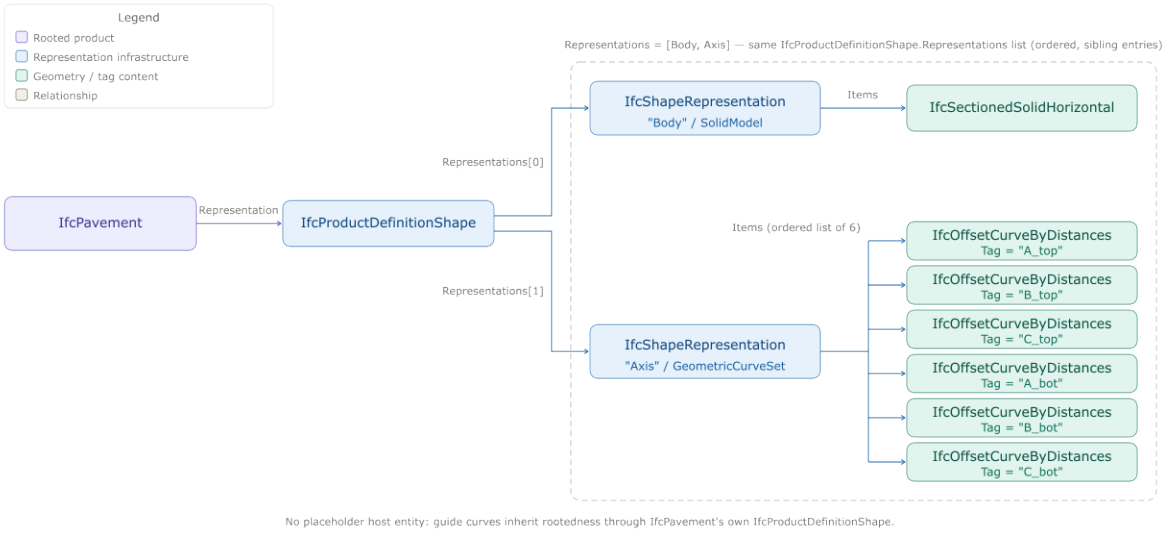


Figure 10.6.2.5-1 — Guide curve rootedness via sibling representation: the solid and its six guide curves hang off the same `IfcProductDefinitionShape`.

The closed cross-section is a V-shaped pavement slab 60 m wide (nominal) and 1 m thick, with a 20% crown slope. Tags are assigned through `IfcCartesianPointList2D.TagList`: all six vertices carry matching guide curve tags — `"A_top"`, `"B_top"`, `"C_top"` on the top surface and `"A_bot"`, `"B_bot"`, `"C_bot"` on the soffit — so every vertex, top and bottom, follows its own guide curve.

Key difference from the surface counterpart. In `IfcOpenCrossProfileDef`, the `Tags` attribute is a list of vertex labels on the open profile. In `IfcArbitraryClosedProfileDef`, there is no `Tags` attribute; vertex labels are carried by `IfcCartesianPointList2D.TagList`. Both attributes achieve the same match — an offset curve's `Tag` is compared against the vertex label, and matching vertices are guided. A vertex with a tag that matches no guide curve is treated as unguided — its position is linearly interpolated between its authored positions at the `CrossSectionPositions` — though the specification does not explicitly confirm this behavior (see §10.5.2).

10.6.2.6 String Lines — Independent Guide Curves

The file `IfcSectionedSolidHorizontal_with_stringlines_independent_guide_curves.ifc` is

the solid counterpart to `IfcSectionedSurface_with_stringlines_independent_guide_curves.ifc` (§10.6.1.5), using the identical edge-curve construction. As in §10.6.2.5, guide curves are hosted in a sibling `"Axis"` `IfcShapeRepresentation` (§10.5.2); the difference here is that the `"A_top"`/`"C_top"` guide curves have a `BasisCurve` independent of the solid `Directrix`, rather than sharing it.

The directrix is the same 200 m straight `Main-Line` alignment at flat grade — a real `IfcAlignment` product, unlike the edge geometry described next. Two bare edge curves define the edge paths, built the same way as §10.6.1.5's — an `IfcGradientCurve` composed directly from `IfcCurveSegment`/`IfcCircle` entities, with no `IfcAlignment` product above it:

- Left edge curve — a 205 m circular arc with radius 1010 m, curving left, starting at $(0,\ +30)$ m from the origin.
- Right edge curve — a 205 m circular arc with radius 1010 m (curving right; the direction is encoded by the sign of the segment's arc length, not the radius), starting at $(0,\ -30)$ m from the origin.

Both edge curves carry a constant-height vertical profile at $-\frac{1}{2} \times 30 \times 0.2 = -3$ m, placing each shoulder 3 m below the crown at the start. The tag-carrying guide curves are bare, zero-offset `IfcOffsetCurveByDistances` entities: `"A_top"` references the left edge curve as its `BasisCurve`, `"B_top"` references `Main-Line`, and `"C_top"` references the right edge curve — all three placed in this file's own sibling `"Axis"` representation. Only the top vertices are guided; the `"A_bot"`, `"B_bot"`, and `"C_bot"` tags on the soffit match no guide curve and fall back to linear interpolation.

As in §10.6.1.5, an independent `BasisCurve` has no effect on tag scoping here — scoping is governed by shared representation membership (§10.5.2), not by whether `BasisCurve` matches `Directrix`.

Two profiles are authored: `cs1` at distance 0 m (half-width 30 m) and `cs2` at distance 200 m (half-width 50 m) — the width the edge curves are sized to reach exactly at that distance (§10.6.1.5).

Chapter 11 — Precision and Tolerance

11.0 Introduction

Alignment geometry is rarely mathematically perfect. Design parameters are rounded to convenient values, segment coordinates are computed independently by different tools, and transition curve ordinates are approximated by numerical integration — all of which introduce small discrepancies at segment joints. Alignments derived from historical sources carry additional uncertainty from the original survey or drafting. IFC provides mechanisms for both declaring the intended geometric continuity at segment joints and communicating the tolerance within which that continuity should be evaluated.

The primary declaration mechanism is `IfcCurveSegment.Transition`, which specifies the intended relationship between the end of one segment and the start of the next. Table 11.0-1 lists the available transition codes.

Transition Value	Description
`CONTINUOUS`	The segments join but no condition on their tangents is implied.
`CONTSAMEGRADIENT`	The segments join and their tangent vectors or tangent planes are parallel and have the same direction at the joint; equality of derivative is not required.
`CONTSAMEGRADIENTSAMECURVATURE`	For a curve, the segments join, their tangent vectors are parallel and in the same direction and their curvatures are equal at the joint: equality of derivatives is not required. For a surface this implies that the principle curvatures are the same and the principle directions are coincident along the common boundary.
`DISCONTINUOUS`	The segments do not join. This is permitted only at the boundary of the curve or surface to indicate that it is not closed.

Table 11.0-1 — IfcCurveSegment transition codes

Figure 11.0-1 illustrates the various transition types.

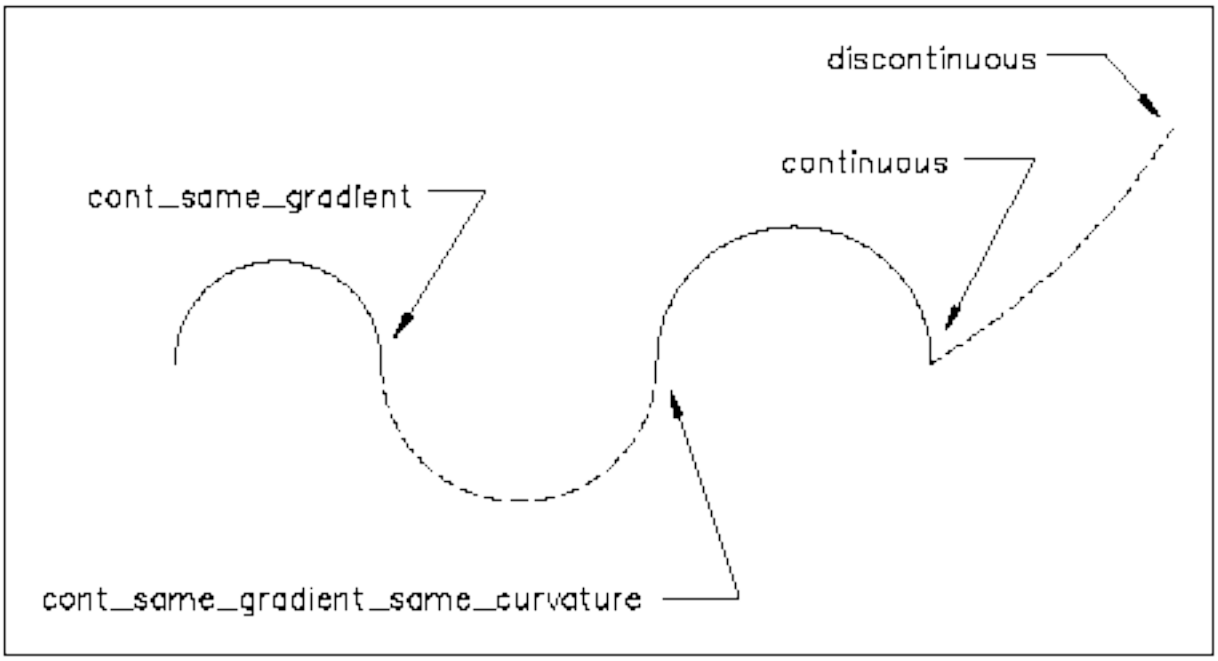


Figure 11.0-1 — Illustration of segment transition types

These transition codes declare intent, not guarantee. Adjacent segment end and start points may not exactly line up in position, gradient, or curvature for several reasons:

- **Rounded design parameters.** Designers routinely round curve parameters — radius, arc length, tangent bearing — to convenient values. Geometry computed from those rounded values does not land exactly on the next segment's stored start point.
- **Independent segment computation.** Each segment's coordinates are often computed independently from its own parameters rather than chained forward from the previous segment's computed end. Floating-point rounding differences between two independent computations of the same nominal point produce small gaps or overlaps at joints.
- **Numerical integration of transition curves.** The x/y ordinates at the end of a spiral are transcendental integrals — Fresnel integrals for Clothoids, analogous forms for Bloss, Cosine, and the other spiral types — that different software approximates with different precision. The same nominal parameters can yield slightly different endpoint coordinates depending on the tool that produced them.
- **Unit conversion.** Converting between feet and meters, or between degrees/minutes/seconds and decimal degrees, introduces rounding that propagates to all downstream coordinates.
- **Legacy source fidelity.** Alignments derived from historical plan sheets, right-of-way plats, or field surveys carry the measurement uncertainty inherent in the original source material. Many times, the accuracy is "good enough" for its purpose.

`Pset_Tolerance` indicates the acceptable tolerance under which the calculated end point of a preceding segment would be considered geometrically continuous with the provided start point of the following segment. No implementation agreement exists for how its properties should be used; the `OverallTolerance` property is recommended. Similarly, `Pset_Uncertainty` indicates the uncertainty inherent in historical information gleaned from plan sets and other historic sources, and carries no implementation agreement; the `LinearUncertainty` property is recommended. As a fallback if `Pset_Tolerance` is not provided, tolerance can be taken from `IfcGeometricRepresentationContext.Precision`.

11.1 Handling Geometry Mismatches at Segment Joints

The IFC specification is silent on how an implementation should respond when the computed end point of one segment does not exactly match the stored start point of the next, even when a `CONTSAMEGRADIENT` or `CONTSAMEGRADIENTSAMECURVATURE` transition is declared.

Mismatches are inevitable in practice. They are especially common in alignments derived from historical sources, where design parameters were rounded for readability on paper plan sheets. Tangent bearings are a typical case: a bearing recorded in degrees-minutes-seconds with the seconds term truncated to a whole number introduces a small angular error whose positional effect accumulates over the length of the tangent run. Segment lengths and curve delta angles are similarly simplified — rounded to the nearest foot, or to a convenient angle. Each segment is computed independently from its own rounded parameters, and the positional differences at joints can reach non-trivial magnitudes. The BPaimio–Kupittaa railway alignment in Chapter 12 is a real-world example: its `IfcGeometricRepresentationContext.Precision` is set to 0.1 m, reflecting exactly this kind of plan-rounding heritage. Critically, the gaps exist only in the data — the physical railway has no abrupt offsets or kinks at segment boundaries. The infrastructure is continuous; the apparent discontinuity is an artifact of parameter rounding in the original record or the inherent limitations of the surveying techniques used to capture it.

Several reconciliation strategies are possible; the right choice depends on the application's purpose:

- **Accept and evaluate as-is.** Each segment is evaluated from its own stored start point. Gaps and overlaps at joints are ignored. The declared transition code is taken as a statement of design intent rather than a geometric guarantee. This is the most faithful reading of the file and is appropriate for applications whose primary task is geometry display or quantity take-off where sub-meter positional accuracy is sufficient.

- **Chain forward (snap).** The computed end point of each segment is carried forward as the start point of the next, overriding the stored value. This produces a geometrically continuous chain but subtly distorts the individual segment parameters. It is appropriate when downstream processing requires strict continuity — for example, when constructing a composite curve for clash detection

or BIM integration — provided the caller understands the parameters have been adjusted.

- **Interpolate across the gap.** A blending zone is constructed at each joint to smooth over the mismatch. This is geometrically the most forgiving approach but adds complexity, introduces geometry not present in the original design, and is difficult to implement consistently. It is rarely the right choice for engineering applications.
- **Reject or warn.** Files in which a declared continuous transition is violated beyond a threshold are flagged. This is appropriate for authoring and validation tools whose role is to enforce data quality. The warning should report the mismatch magnitude so the sender can investigate its source.

No single strategy is universally correct. A staking or track geometry application demands accuracy that the accept-as-is strategy cannot provide; a visualisation tool has no need to chain-forward. Implementations should document which strategy they apply and, where possible, expose it as a configurable option.

11.2 IfcGeometricRepresentationContext.Precision

`IfcGeometricRepresentationContext.Precision` declares the intended coordinate precision of the model — the smallest meaningful distance in the model's unit system. It is the appropriate fallback when `Pset_Tolerance` is not attached to the alignment.

The precision value varies widely depending on the source and purpose of the model. A freshly computed design model may carry a precision of 10^{-5} m or smaller, consistent with double-precision floating-point arithmetic. A model converted from historical plan data — such as the BPaimio–Kupittaa example — may carry 0.1 m, reflecting the resolution of the original source.

An implementation reading an alignment should retrieve this value and use it to contextualise segment joint mismatches: a gap smaller than the declared precision is likely a rounding artefact and should be treated accordingly; a gap that substantially exceeds it warrants investigation.

Chapter 12 — Alignment Geometry Testset

12.1 Background

The test cases in this chapter are inspired by the alignment test sets developed by Andreas Pinzenöhler for the buildingSMART Germany chapter's BIM Fit Check competition [7], and by the EnrichIFC4x3 reference implementation developed by Peter Bonsma and published in the IFC-Rail-Unit-Test-Reference-Code repository. Both bodies of work established the principle of pairing semantic IFC files with independently computed reference coordinates as a practical validation strategy.

The goal of this testset is to give implementors a concrete, mechanically verifiable check for their alignment geometry code. Each test case is deliberately simple: a single 100 m segment of one curve type, with known input parameters and published reference coordinates sampled at 1 m intervals. Each test case is provided in three unit variants — SI meters, US survey feet, and international feet — making the set a direct check for unit-conversion correctness as well as geometric accuracy.

12.2 Testset Structure

The testset provides each test case in three complementary forms, together with plots for visual inspection, all located under the `testset/` folder in this repository.

```
testset/
├── Alignment-semantic-testset/    # Semantic IFC files (no geometry)
├── Alignment-geometry-testset/    # IFC files with geometric representations
├── Alignment-reference-testset/   # Reference coordinate tables
└── Alignment-plots/               # SVG plots for visual inspection
```

All four folders share the same internal hierarchy: alignment type then curve type subfolder.

Table 12.2-1 — Curve type subfolders by alignment type

Alignment type	Curve type subfolders
HorizontalAlignment	Line, CircularArc, Clothoid, BlossCurve, CosineCurve, HelmertCurve, SineCurve, VienneseBend
VerticalAlignment	ConstantGradient, ParabolicArc, CircularArc
CantAlignment	ConstantCant, LinearTransition, BlossCurve, CosineCurve, HelmertCurve, SineCurve, VienneseBend

12.2.1 Unit Systems

Every test case is provided in three unit variants, identified by the `{Unit}` token at the end of each filename:

Token	Unit	Conversion to meters

Token	Unit	Conversion to meters
`Meter`	SI meter	1.0 (exact)
`SurveyFoot`	US survey foot	1200 / 3937 ≈ 0.304 800 609 … m
`IntlFoot`	International foot	0.3048 m (exact)

All dimensional quantities in each IFC file — segment lengths, curve radii, elevations, cant values, `RailHeadDistance`, and spiral coefficients — are expressed in the declared project unit. Gradients are dimensionless percentage values and are identical across all three variants. The reference coordinates are likewise in project units, so a `SurveyFoot` file contains distances and positions in survey feet.

12.2.2 Semantic IFC Files

The semantic IFC files express the design intent of each alignment through `IfcAlignmentSegment` subtypes (`IfcAlignmentHorizontalSegment`, `IfcAlignmentVerticalSegment`, `IfcAlignmentCantSegment`). No geometric representation (`IfcCurveSegment`) is attached. These files are useful for testing the semantic layer of an implementation: can the software read the alignment parameters, classify the segment type, and display or export the design intent without requiring geometric evaluation? They also serve as inputs for testing geometric representation generation — an implementation can construct its own geometry IFC files from the semantic definitions and benchmark the result against the reference geometry files in `Alignment-geometry-testset/`.

File naming. Each filename is prefixed with the alignment type — `Horizontal`, `Vertical`, or `Cant` — so the alignment type is unambiguous without inspecting the file or its folder path. All numeric tokens in the filename (length, radii) use meter-based values regardless of unit system, so the same filename prefix identifies the same physical alignment across all three unit variants.

Horizontal and cant:

```
Horizontal_{CurveType}_{Length}_{StartRadius}_{EndRadius}_{Unit}.ifc
Cant_{CurveType}_{Length}_{StartRadius}_{EndRadius}_{Unit}.ifc
```

The radius tokens use `inf` and `-inf` for infinite radius (straight tangent). Positive radii denote left curves; negative radii denote right curves.

Examples: - `Horizontal_Clothoid_100.0_inf_300_Meter.ifc` — Clothoid entry spiral, R = ∞ → +300 m, SI units - `Horizontal_Clothoid_100.0_inf_300_SurveyFoot.ifc` — same alignment, US survey feet - `Horizontal_HelmertCurve_100.0_-300_-1000_Meter.ifc` — Helmert arc-to-arc, R = −300 → −1000 m - `Cant_BlossCurve_100.0_-inf_-300_Meter.ifc` — Bloss cant entry spiral, right curve

Vertical:

```
Vertical_{CurveType}_{Length}_{StartHeight}_{StartGrade}_{EndGrade}_{Unit}.ifc
```

Examples: - `Vertical_ParabolicArc_100.0_10.0_0.5_-1.0_Meter.ifc` — parabolic crest, +0.5 % to −1.0 %, starting at elevation 10.0 m - `Vertical_ConstantGradient_100.0_10.0_-0.5_-0.5_Meter.ifc` — constant descent at −0.5 %

12.2.3 Geometry IFC Files

The geometry IFC files extend the semantic files with `IfcCurveSegment`-based geometric representations attached to each layout curve (`IfcCompositeCurve`, `IfcGradientCurve`, `IfcSegmentedReferenceCurve`). Each geometric curve also carries a zero-length terminator segment that marks the end of the parametric domain.

The primary purpose of these files is to validate an implementation's mapping from semantic to geometric alignment definitions — specifically, whether the `IfcAlignmentSegment` attributes are correctly translated into the corresponding `IfcCurveSegment` representations. They also provide everything needed to evaluate the alignment curve at any point along its length, making them the input for coordinate comparison against the reference values in `Alignment-reference-testset/`.

File naming. Geometry files carry the same name as their semantic counterpart with `Generated_` prepended, distinguishing machine-generated files from the hand-authored semantic definitions:

```
Generated_Horizontal_{CurveType}_{Length}_{StartRadius}_{EndRadius}_{Unit}.ifc
Generated_Vertical_{CurveType}_{Length}_{StartHeight}_{StartGrade}_{EndGrade}_{Unit}.ifc
Generated_Cant_{CurveType}_{Length}_{StartRadius}_{EndRadius}_{Unit}.ifc
```

12.2.4 Reference Coordinates

The reference coordinate files are CSV tables of coordinates sampled at every 1 m of horizontal distance along each alignment, from 0.0 m to 100.0 m inclusive (101 rows per file). Their purpose is to provide concrete values that any implementation can compare its output against, independent of the IFC files. The file format and column definitions are described in §12.4.

Vertical and cant alignment folders each contain an additional `3D/` subfolder with a second set of reference files in the same 13-column format used for real-world alignments (§12.4.2). These 3D files are produced by evaluating the combined 3D curve (`IfcGradientCurve` for vertical, `IfcSegmentedReferenceCurve` for cant) at the same 101 sample points. They let an implementor validate the complete evaluation pipeline — not just the component-plane output — against controlled, single-component synthetic geometry.

The following excerpt shows the first five rows of `HorizontalAlignment/BlossCurve/Horizontal_BlossCurve_100.0_300_1000_Meter.csv` — a Bloss arc-to-arc transition from R = +300 m to R = +1000 m:

```
dist_along,X,Y,RefDir_dx,RefDir_dy
0.000000,0.000000,0.000000,1.000000,0.000000
1.000000,0.999998,0.001667,0.999994,0.003333
2.000000,1.999985,0.006666,0.999978,0.006665
3.000000,2.999950,0.014995,0.999950,0.009994
4.000000,3.999882,0.026652,0.999911,0.013318
```

At the start of the segment the position is at the origin, the tangent points due East (`RefDir_dx` = 1, `RefDir_dy` = 0), and the curve is already bending left — the Y coordinate and `RefDir_dy` both increase as the alignment curves away from the initial straight.

File naming. Reference CSV files use the same name as the corresponding semantic IFC file with the `.ifc` extension replaced by `.csv`:

```
Horizontal_{CurveType}_{Length}_{StartRadius}_{EndRadius}_{Unit}.csv
Vertical_{CurveType}_{Length}_{StartHeight}_{StartGrade}_{EndGrade}_{Unit}.cs
v
Cant_{CurveType}_{Length}_{StartRadius}_{EndRadius}_{Unit}.csv
```

12.2.5 Plots

The `Alignment-plots/` folder contains plots for visual inspection, organized into three subfolders that mirror the alignment type structure:

- **`HorizontalAlignment/{type}/`** — one plan-view plot per horizontal file.
- **`VerticalAlignment/{type}/`** — one elevation profile plot per vertical file.
- **`CantAlignment/{type}/`** — one two-panel plot per cant file, pairing the cant profile with its matching horizontal plan view.

Figure 12.2.5-1 shows a representative example: a Helmert arc-to-arc transition from R = +300 m to R = +1000 m. The upper panel shows the horizontal plan view; the middle panel shows the corresponding cant profile; the lower panel shows the cross-slope direction.

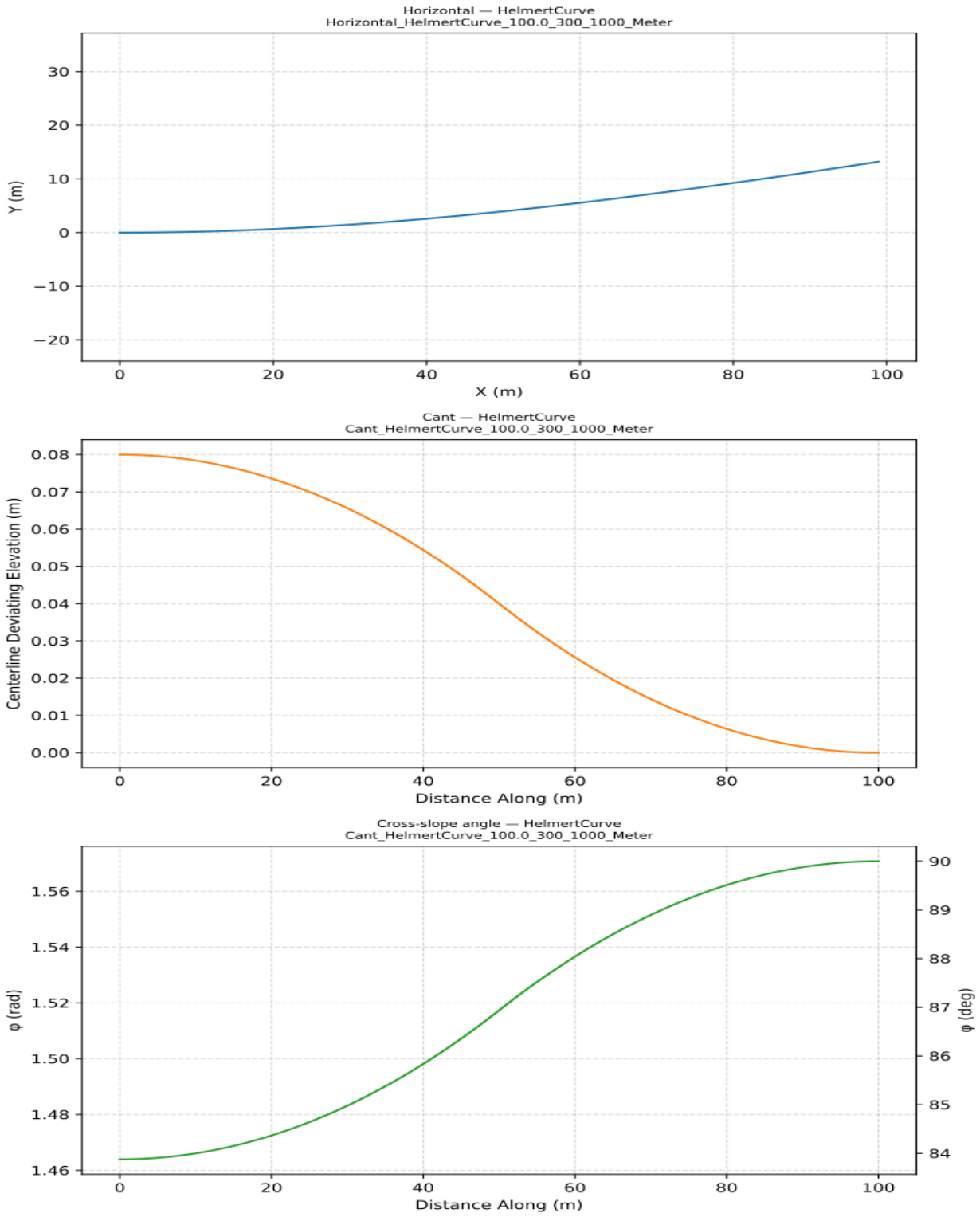


Figure 12.2.5-1 — Combined horizontal and cant plot for `Cant_HelmertCurve_100.0_300_1000_Meter`.

12.3 Test Cases

The test cases are organized by alignment component — horizontal, vertical, and cant — and each component is tested in isolation. Horizontal cases contain no vertical or cant layouts. Vertical cases use a straight horizontal to avoid introducing horizontal curvature. Cant cases use a flat zero-grade vertical for the same reason. This isolation strategy means that discrepancies in the reference coordinates can be attributed unambiguously to a single geometric component. Real-world alignments combining all three components are covered in Section 12.3.4.

12.3.1 Horizontal Alignment

All horizontal segments are 100 m long. These cases contain only a horizontal layout — no vertical or cant layouts are present — so any coordinate discrepancy is attributable entirely to horizontal curve evaluation. Table 12.3.1-1 summarises the curve types and case counts. All six transition types share the same eight geometry cases shown in Table 12.3.1-2.

Table 12.3.1-1 — Horizontal curve types

Curve type	IFC predefined type	Cases
Line	`LINE`	1
Circular arc	`CIRCULARARC`	4
Clothoid	`CLOTHOID`	8
Bloss curve	`BLOSSCURVE`	8
Cosine curve	`COSINECURVE`	8
Helmert curve	`HELMERTCURVE`	8
Sine curve	`SINECURVE`	8
Viennese bend	`VIENNESEBEND`	8

Table 12.3.1-2 — Horizontal transition cases (applied to all six transition types)

Start radius	End radius	Description
∞	+300 m	Entry spiral — straight to tight left arc
+300 m	∞	Exit spiral — tight left arc to straight
∞	−300 m	Entry spiral — straight to tight right arc
−300 m	∞	Exit spiral — tight right arc to straight
+300 m	+1000 m	Arc-to-arc — tighter to looser (left)
+1000 m	+300 m	Arc-to-arc — looser to tighter (left)
−300 m	−1000 m	Arc-to-arc — tighter to looser (right)
−1000 m	−300 m	Arc-to-arc — looser to tighter (right)

Viennese bend note: The Viennese bend (`VIENNESEBEND`) is unique among horizontal transition types: its geometric representation requires cant parameters to be present. The VienneseBend files in `HorizontalAlignment/VienneseBend/` therefore include all three alignment layouts (horizontal, vertical, and cant), with the same parameter values as the corresponding `CantAlignment/VienneseBend/` files. When extracting horizontal coordinates, only the horizontal layout is evaluated; the vertical and cant layouts are present solely to satisfy the geometry generation requirement.

12.3.2 Vertical Alignment

All vertical segments are 100 m horizontal length, starting at an elevation of 10.0 m. Gradients are expressed in percent (e.g. `0.5` = 0.5 %).

Each vertical test file includes a horizontal layout consisting of a single straight `LINE` segment of the same length. A straight horizontal alignment satisfies the IFC schema constraint that a vertical layout must have an associated horizontal, while isolating the vertical geometry so results can be verified independently.

Table 12.3.2-1 — Vertical curve types

Curve type	IFC predefined type	Cases
Constant gradient	`CONSTANTGRADIENT`	5
Parabolic arc	`PARABOLICARC`	12
Circular arc	`CIRCULARARC`	12

Note: Clothoid vertical curves are not included. See Chapter 3 for discussion.

The five constant gradient cases cover 0.0 %, ±0.5 %, and ±1.0 % grade. The twelve vertical transition cases cover the full range of sag and crest combinations across flat, ascending, and descending grade pairs.

12.3.3 Cant Alignment

All cant segments are 100 m horizontal length. The `RailHeadDistance` is 1.5 m for all cases.

Each cant file includes a matching horizontal layout of the corresponding curve type and a flat vertical layout at 0.0 % grade and elevation 0.0 m. The flat vertical isolates the cant geometry from vertical curvature, so reference coordinate errors reflect only the cant evaluation. Table 12.3.3-1 lists the cant curve types, their paired horizontal types, and case counts.

Table 12.3.3-1 — Cant curve types

Cant type	IFC predefined type	Paired horizontal type	Cases	Cant range

Cant type	IFC predefined type	Paired horizontal type	Cases	Cant range
Constant cant	CONSTANTCANT	CIRCULARARC	4	0.16 m constant
Linear transition	LINEARTRANSITION	CLOTHOID	8	0.0 ↔ 0.16 m
Bloss curve	BLOSSCURVE	BLOSSCURVE	8	0.0 ↔ 0.16 m
Cosine curve	COSINECURVE	COSINECURVE	8	0.0 ↔ 0.16 m
Helmert curve	HELMERTCURVE	HELMERTCURVE	8	0.0 ↔ 0.16 m
Sine curve	SINECURVE	SINECURVE	8	0.0 ↔ 0.16 m
Viennese bend	VIENNESEBEND	VIENNESEBEND	8	0.03 ↔ 0.10 m

12.3.4 Real-World Alignments

Four real-world alignments are provided as 3D test cases under `testset/real-world-alignments/`. Unlike the single-component cases in Sections 12.3.1–12.3.3, these exercise the complete evaluation pipeline — from combined horizontal, vertical, and (where present) cant layouts — and produce full 3D placement matrices at each sample point.

FHWA Bridge Geometry Manual

The first case is an alignment from Appendix B of the US Federal Highway Administration *Bridge Geometry Manual* (see Chapter 1). It combines horizontal and vertical layouts only — no cant is present — and exercises `IfcGradientCurve` evaluation. The alignment is approximately 12,337 ft (3,760 m) long and is expressed in international feet.

IFC file	`testset/real-world-alignments/FHWA_Alignment.ifc`
Reference coordinates	`testset/real-world-alignments/FHWA_Alignment.csv`
Alignment name	Unnamed Alignment
Project unit	International foot
3D curve type	`IfcGradientCurve`

FHWA Bridge Geometry Manual — Linear Placement Variant

A second variant of the FHWA alignment is provided to support validation of `IfcLinearPlacement` evaluation. It carries the same horizontal and vertical geometry as the base case and adds 124 `IfcReferent` objects, one at every 100 ft station from 100+00 to 223+00, each with a populated `CartesianPosition` fallback. Each `IfcPointByDistanceExpression.BasisCurve` is the alignment's `IfcGradientCurve`, not the 2D horizontal composite curve, so `DistanceAlong` is measured along the gradient curve's horizontal projection while the evaluated position carries the true 3D (graded) elevation. Linear placement is discussed in Chapter 8.

IFC file	`testset/real-world-alignments/FHWA_Alignment_with_Linear_Placement.ifc`

Reference coordinates	`testset/real-world-alignments/FHWA_Alignment_with_Linear_Placement.csv`
Alignment name	E-Line
Project unit	International foot
3D curve type	`IfcGradientCurve`

The reference CSV uses the same 13-column format as the other real-world alignment files (Section 12.4.2), but contains 124 rows rather than 101 — one row per `IfcLinearPlacement`, ordered by station distance rather than sampled at uniform intervals. An implementation validates its linear placement support by resolving each `IfcLinearPlacement`'s full placement (not merely evaluating the bare gradient curve at its `DistanceAlong`) and comparing the result against the matching row in the CSV.

FHWA Bridge Geometry Manual — Vertical Offset / PlacementRelTo Variant

A third variant of the FHWA alignment tests two mechanisms for shifting a linear placement vertically, contrasted in Chapter 8 section 8.3.3: `IfcPointByDistanceExpression.OffsetVertical` (measured perpendicular to the curve's tangent, per section 8.2.1) versus `IfcLinearPlacement.PlacementRelTo` (a true elevation shift). At each of the same 124 100 ft stations as the Linear Placement Variant, four `IfcReferent` objects are provided:

- `OffsetVertical = +2.5` / `-2.5` (project units), `PlacementRelTo` omitted
- `PlacementRelTo` referencing a shared `IfcLocalPlacement` at z = +2.5 / −2.5, with `OffsetVertical` omitted

On a graded segment the two mechanisms diverge: `OffsetVertical` perturbs X and Y slightly (it is perpendicular to an inclined tangent, not to the global XY plane) and lands at an elevation that only approximates the intended ±2.5 ft shift, while `PlacementRelTo` leaves X and Y unchanged and lands at exactly the intended elevation.

`CartesianPosition` is populated on all four variants by resolving each placement's full placement chain — including `PlacementRelTo` — so every `CartesianPosition` matches its corresponding row in the reference CSV. (This file was originally built while an implementation's fallback logic evaluated only the immediate `RelativePlacement` and ignored `PlacementRelTo`; it continues to serve as a regression check for that behavior.)

IFC file	`testset/real-world-alignments/FHWA_Alignment_LinearPlacement_VerticalOffset.ifc`
Reference coordinates	`testset/real-world-alignments/FHWA_Alignment_LinearPlacement_VerticalOffset.csv`
Alignment name	E-Line
Project unit	International foot
3D curve type	`IfcGradientCurve`

Unlike the Linear Placement Variant, this file's reference CSV is generated by fully resolving each `IfcLinearPlacement` — including `OffsetVertical` and, for the `PlacementRelTo` variants, the nested local placement — rather than by evaluating the bare 3D curve at `DistanceAlong` alone. This is necessary because the two variants being contrasted only differ in attributes that a bare curve evaluation would ignore. The CSV uses a long format instead of the 13-column format used elsewhere in this section: one row per `IfcReferent` (four rows per station — `DistAlong, Variant, X, Y, Z, RefDir_dx/dy/dz, Y_dx/dy/dz, Axis_dx/dy/dz`), identified by a `Variant` column (`OffsetVertical+2.5`, `OffsetVertical-2.5`, `PlacementRelTo+2.5`, `PlacementRelTo-2.5`).

BPaimio–Kupittaa

The second case is alignment `001` from the BPaimio–Kupittaa railway line in Finland. The IFC file was provided by the Finnish Transport Infrastructure Agency (Väylävirasto) and distributed as part of the buildingSMART Germany BIM Fit Check competition. It is a full 3D railway alignment including horizontal, vertical, and cant layouts, exercising the complete `IfcSegmentedReferenceCurve` evaluation pipeline. The alignment is approximately 25.6 km long and is expressed in SI meters with ETRS-TM35FIN coordinates.

IFC file	`testset/real-world-alignments/BPaimio-Kupittaa_GK23_N2000_2020.ifc`
Reference coordinates	`testset/real-world-alignments/BPaimio-Kupittaa_GK23_N2000_2020.csv`
Alignment name	001
Project unit	Meter (ETRS-TM35FIN)
3D curve type	`IfcSegmentedReferenceCurve`

The horizontal layout contains ten Helmert transition curves (`HELMERTCURVE`). In the source data, these segments do not end exactly where the next segment starts. A Helmert curve evaluated from its design parameters, as defined in Section 2.7, ends 0.5–9.6 cm from the following segment's `StartPoint` and 2.5–6.3 × 10^{-4} rad from its `StartDirection`. These gaps are a property of the design data, not of the evaluation, so the reference coordinates jump at those locations. The curve segments that follow each Helmert curve are placed at their own `StartPoint`.

Exit Turnout — Huni Valley Station

The third case is the mainline alignment of an exit turnout at Huni Valley Station on a Swiss railway line. The IFC file was created using IfcOpenShell from LandXML source data. It is a full 3D railway alignment including horizontal, vertical, and cant layouts, exercising the complete `IfcSegmentedReferenceCurve` evaluation pipeline. The alignment runs from station 84+521.000 to 88+524.656 (approximately 4003.656 m) and is expressed in SI meters with LV95 coordinates (Swiss national coordinate system). The horizontal layout consists of clothoid transitions and circular arcs; the vertical layout is a single flat constant-gradient segment; and the cant layout uses linear transitions and constant-cant segments with a rail head distance of 1.0 m.

IFC file	`testset/real-world-alignments/Exit_Turnout_Huni_Valley_Station.ifc`
Reference coordinates	`testset/real-world-alignments/Exit_Turnout_Huni_Valley_Station.csv`
Alignment name	Centerline
Project unit	Meter (LV95)
3D curve type	`IfcSegmentedReferenceCurve`

Reference coordinate format

All three CSV files are sampled at every whole metre of distance along the 3D curve, starting at 0, plus a final row at the end of the curve when it does not fall on a whole metre. The sampling interval is 1 m (SI) regardless of the project unit, so in `FHWA_Alignment.csv` (international feet) `DistAlong` advances by 3.280840 ft per row. All positions are in project units; direction components are dimensionless. The column definitions are given in Section 12.4.2.

12.4 Reference Coordinates

12.4.1 File Format

Each reference file contains 101 rows plus a header row. The rows are sampled at 1 m intervals of horizontal distance (0.0 m to 100.0 m), so `dist_along` values in `SurveyFoot` and `IntlFoot` files are not round numbers. All length columns (`dist_along`, `X`, `Y`) are in the project unit declared by the corresponding IFC file. Direction columns (`RefDir_dx`, `RefDir_dy`, `Axis_dy`, `Axis_dz`) are dimensionless and identical across all three unit variants. The columns differ by alignment type.

Horizontal alignment — `HorizontalAlignment/**/*.csv`

Column	Description
`dist_along`	Distance along the alignment from the segment start (project units)
`X`	Easting in the alignment plane (project units)
`Y`	Northing in the alignment plane (project units)
`RefDir_dx`	X component of the forward tangent unit vector (`RefDirection`)
`RefDir_dy`	Y component of the forward tangent unit vector (`RefDirection`)

The segment starts at the origin (0, 0) with a forward tangent pointing due East (+X direction). `RefDir_dx` and `RefDir_dy` together form a unit vector; the tangent angle $\theta = \tan^{-1}(dy/dx)$.

Vertical alignment — `VerticalAlignment/**/*.csv`

Column	Description
`dist_along`	Horizontal distance from the segment start (project units)

Column	Description
`Y`	Elevation (project units)
`RefDir_dx`	X component of the tangent direction ($\cos\theta$)
`RefDir_dy`	Y component of the tangent direction ($\sin\theta$, proportional to grade)

The parameter passed to the evaluator is horizontal distance, not arc length, even though `SegmentLength` in the IFC file stores the arc length of the curve. This is consistent with the IFC specification for `IfcAlignmentVerticalSegment.HorizontalLength`.

Cant alignment — `CantAlignment/**/*.csv`

Column	Description
`dist_along`	Distance along the horizontal alignment (project units)
`Y`	Track centerline deviating elevation due to cant (project units)
`RefDir_dx`	X component of the forward tangent (`RefDirection`)
`RefDir_dy`	Y component of the forward tangent (`RefDirection`)
`Axis_dy`	Y component of the banked “up” axis vector
`Axis_dz`	Z component of the banked “up” axis vector

The banked axis (`Axis_dy`, `Axis_dz`) is the cross-track “up” direction after applying the cant rotation about the forward tangent. For a planar track the X component of the axis is always 0.0 and `Axis_dz` approaches 1.0 when cant is zero.

12.4.2 3D Alignment Reference Coordinates

Most of the 3D CSV files under `testset/real-world-alignments/` are produced by evaluating the combined 3D curve (`IfcGradientCurve` or `IfcSegmentedReferenceCurve`) directly at each sample point. Each evaluation returns a 4 × 4 placement matrix; the CSV extracts all three orientation vectors and the position.

`FHWA_Alignment_with_Linear_Placement.csv` and `FHWA_Alignment_LinearPlacement_VerticalOffset.csv` (§12.3.4) instead resolve each `IfcLinearPlacement`’s full placement chain rather than sampling the bare 3D curve, so that the reference values actually exercise `IfcLinearPlacement` evaluation. `FHWA_Alignment_with_Linear_Placement.csv` uses the same 13-column format as the curve-sampled files, just ordered by station rather than sampled at uniform intervals. `FHWA_Alignment_LinearPlacement_VerticalOffset.csv` uses a 14-column layout with an added `Variant` column, since it contrasts four placement variants per station. Its other columns match the table below.

Column	Description
`DistAlong`	Distance along the 3D curve from the start (project units)
`X`	X coordinate of the point (project units)

Column	Description
`Y`	Y coordinate of the point (project units)
`Z`	Z (vertical) coordinate of the point (project units)
`RefDir_dx`	X component of the forward tangent unit vector
`RefDir_dy`	Y component of the forward tangent unit vector
`RefDir_dz`	Z component of the forward tangent unit vector
`Y_dx`	X component of the cross-track unit vector
`Y_dy`	Y component of the cross-track unit vector
`Y_dz`	Z component of the cross-track unit vector
`Axis_dx`	X component of the “up” axis unit vector
`Axis_dy`	Y component of the “up” axis unit vector
`Axis_dz`	Z component of the “up” axis unit vector

The three direction vectors (`RefDir`, `Y`, `Axis`) form an orthonormal right-handed frame at each point. For alignments without cant, `Axis` is perpendicular to the horizontal plane and `Y_dz` is zero. For railway alignments with cant, `Axis` is banked about the forward tangent and both `Y_dz` and `Axis_dy` become non-zero.

12.4.3 3D Synthetic Testset Coordinates

The `3D/` subfolders under `Alignment-reference-testset/VerticalAlignment/` and `…/CantAlignment/` contain combined 3D reference coordinates for each synthetic test case. Each file uses the same 13-column header as §12.4.2 and is sampled at the same 101 points (0–100 m at 1 m intervals in SI metres, reported in project units).

File naming follows the same convention as the 2D files, with the `.ifc` extension replaced by `.csv`:

```
VerticalAlignment/{CurveType}/3D/Vertical_{CurveType}_{Length}_{StartHeight}_
{StartGrade}_{EndGrade}_{Unit}.csv
CantAlignment/{CurveType}/3D/Cant_{CurveType}_{Length}_{StartRadius}_{EndRadi
us}_{Unit}.csv
```

Curve evaluated. For vertical test cases (which include a straight horizontal layout), the evaluated curve is `IfcGradientCurve`. For cant test cases (which include a matching horizontal and a flat zero-grade vertical), the evaluated curve is `IfcSegmentedReferenceCurve`.

Interpreting the output. Because vertical test cases use a straight horizontal alignment, the X and Y position columns are collinear with distance along, and curvature appears only in the Z (elevation) column and the `RefDir_dz` component. Because cant test cases add a flat zero-elevation vertical to the same straight horizontal, Z is always 0.0 at position, but the `Axis` and `Y` vectors rotate about the forward tangent to reflect the applied cant. This isolation makes the 3D synthetic files a controlled integration check: a position error in X or

Y points to a horizontal evaluation bug; an error in Z or `RefDir_dz` points to a vertical bug; an error in `Axis_dy`/`Axis_dz` (with correct position) points to a cant rotation bug.

12.5 Validation Guidance

12.5.1 What to Check

The three forms of the testset support three distinct validation activities:

Semantic round-trip. Read a semantic IFC file and verify that the `IfcAlignmentSegment` attributes (`StartRadiusOfCurvature`, `EndRadiusOfCurvature`, `SegmentLength`, `PredefinedType`, etc.) are correctly parsed and preserved. This test does not require any geometric evaluation.

Geometric representation. Verify that each `IfcAlignmentSegment` maps to the correct `IfcCurveSegment` parent curve type, placement, and zero-length terminator. This applies both when constructing a geometry IFC file from a semantic file and when reading a reference geometry file — in either case no numerical evaluation is required.

Geometric accuracy. Evaluate a geometry IFC alignment at 1 m intervals and compare the resulting coordinates against the reference values. This is the primary numerical test. For vertical and cant cases, compare both the component-plane 2D output (§12.4.1) and the full 3D placement matrix output (§12.4.3) to isolate pipeline bugs from component evaluation bugs.

12.5.2 Comparison Approach

Load a geometry IFC file, evaluate the alignment curve at $d = 0.0, 1.0, 2.0, \ldots, 100.0$ meters, and compare each computed value against the corresponding row in the reference file.

The most useful comparisons are:

- **Position** (`X`, `Y` for horizontal; `Y` for vertical and cant) — the absolute coordinate values.
- **Tangent direction** (`RefDir_dx`, `RefDir_dy`) — the forward tangent components.
- **Banked axis** (`Axis_dy`, `Axis_dz`, cant only) — the cross-track “up” direction. Errors here indicate a problem in the cant rotation logic.

12.5.3 Cross-Unit Validation

Because the three unit variants encode the same physical alignment, they provide an independent check for unit-handling correctness that does not require any external ground truth. The procedure is:

1. Evaluate the `Meter`, `SurveyFoot`, and `IntlFoot` variants of the same test case.
2. Convert all `SurveyFoot` coordinates to meters by multiplying by 1200/3937.
3. Convert all `IntlFoot` coordinates to meters by multiplying by 0.3048.

4. Compare the three meter-equivalent result sets row by row.

If the implementation handles units correctly, the differences should be below 10^{-6} m at every sample point — consistent with double-precision floating-point rounding in the unit conversion arithmetic. Differences larger than 10^{-3} m, or differences that grow systematically with distance along the alignment, indicate a unit-handling defect.

This cross-unit check is most effective at isolating unit bugs from algorithmic bugs, because a geometric error will appear in all three variants while a unit error will appear only in the scaled variants.

Chapter 13 — References

References

[1] buildingSMART International, *IFC4x3 ADD2: Industry Foundation Classes, Release IFC4X3_ADD2*, buildingSMART International, 2023. [Online]. Available: https://standards.buildingsmart.org/IFC/RELEASE/IFC4_3/. Also published as ISO 16739-1:2024, International Organization for Standardization, Geneva, 2024.

[2] J. Corven, T. Eberhardt, B. Watson, B. Chavel, J. Mash, D. Paterson, and T. Anthony, *Bridge Geometry Manual*, Report No. FHWA-HIF-22-034, U.S. Federal Highway Administration, U.S. Department of Transportation, Washington, DC, 2022. [Online]. Available: https://www.fhwa.dot.gov/bridge/pubs/hif22034.pdf

[3] bSI RailwayRoom, *IFC-Rail-Unit-Test-Reference-Code: Source Code and Synthetic Cases for IFC 4.3 Alignment Geometry Testing and Validation*, GitHub. [Online]. Available: https://github.com/bSI-RailwayRoom/IFC-Rail-Unit-Test-Reference-Code. [Accessed: May 2026].

[4] buildingSMART International, *IFC4.x-IF: IFC Implementers Forum*, GitHub. [Online]. Available: https://github.com/buildingSMART/IFC4.x-IF. [Accessed: May 2026].

[5] buildingSMART International, *ifc-gherkin-rules: Normative Rules for IFC Validation*, GitHub. [Online]. Available: https://github.com/buildingSMART/ifc-gherkin-rules. [Accessed: May 2026].

[6] buildingSMART International, *IFC4.x-development: Repository to Collect Updates to the IFC 4.3 Specification*, GitHub. [Online]. Available: https://github.com/buildingSMART/IFC4.x-development. [Accessed: May 2026].

[7] Š. Jaud, A.-K. Birkenbeul, J. Braunes, R. Brice, C. Clemen, D. Hintz, C. Kautter, E. Kocher, C. Leverenz, A. Pinzenöhler, S. Rabe, R. Raacke, A. Schütz, R. Steyer, and B. Wehrle, *BIM Fit Check — IFC Georeferencing & Alignment Exchanges*, buildingSMART Deutschland, 2026. [Online]. Available: https://www.buildingsmart.de/buildingsmart/aktuelles/erkenntnisse-zu-ifc-und-bcf-aus-bim-fit-check-runden

[8] buildingSMART International, *Validation Service*, buildingSMART International. [Online]. Available: https://www.buildingsmart.org/users/services/validation-service/. [Accessed: May 2026].

[9] bSI InfraRoom, *IFC-Specification: Infrastructure Room Repository for IFC Specification Development*, GitHub. [Online]. Available: https://github.com/bSI-InfraRoom/IFC-Specification. [Accessed: July 2026].

Further Reading

[10] F. Dillen, “The Classification of Hypersurfaces of a Euclidean Space with Parallel Higher Order Fundamental Form,” *Mathematische Zeitschrift*, vol. 203, pp. 635–643, Springer-Verlag, 1990, doi: 10.1007/BF02570761.

[11] European Committee for Standardization, *BS EN 13803:2017: Railway Applications — Track — Track Alignment Design Parameters — Track Gauges 1 435 mm and Wider*, BSI Standards Publication, Brussels, 2017.

[12] M. O. Schmidt and K. W. Wong, *Fundamentals of Surveying*, 3rd ed., PWS Engineering, Boston, MA, 1985, ISBN 0-534-04161-2.

[13] C. H. Waldman, “The Polynomial Spiral and Beyond,” *National Curve Bank*, Deposit #125, 2013. [Online]. Available: https://nationalcurvebank.org/deposits/waldman4.html

[14] buildingSMART International, *IFC Road Phase 2 — Candidate Standard*, buildingSMART International, May 2020. [Online]. Available: https://www.buildingsmart.org/wp-content/uploads/2020/06/IR-CS-WP2-UML_Model_Report_Part-5_.pdf

[15] buildingSMART International, *IFC Road Phase 2 — Conceptual Model*, buildingSMART International. [Online]. Available: https://app.box.com/s/tsmjlt88w6ye0apbqlg5xkucu7hr6kky. [Accessed: June 2026].

[16] buildingSMART International, *IFC Road Phase 2 — Conceptual Model: Annex I — Example Instance Diagrams*, buildingSMART International. [Online]. Available: https://app.box.com/s/d3b6ua3ug047gglpp44sj7t3d84vlphl. [Accessed: June 2026].

[17] buildingSMART International, *IFC Road Phase 2 — Conceptual Model: Annex II — Reading Guide*, buildingSMART International. [Online]. Available: https://app.box.com/s/e1fy291xyv1ux5tzdyuadkup958x0bf6. [Accessed: June 2026].

[18] buildingSMART International, *IFC Road Phase 2 — Prototypical Implementation*, buildingSMART International. [Online]. Available: https://app.box.com/s/d7123nwhe1xr7vpwk1qucj22a3u8zjxu. [Accessed: June 2026].

Appendix A — LandXML to IFC Conversion

A.0 Introduction

LandXML 1.x is the dominant legacy format for civil alignment data; IFC 4x3 is the modern target. The goal of this chapter is to guide implementers through the decisions and pitfalls that arise during conversion, drawn from practical experience implementing LX2IFC, the author's open-source LandXML to IFC conversion program. LX2IFC focuses on alignment conversion and is otherwise incomplete as a general-purpose LandXML to IFC conversion utility; it remains a work in progress.

A.1 Schema Conceptual Comparison

The following table provides a high-level mapping of the two data models.

LandXML	IFC 4x3
`<Alignments>` / `<Alignment>`	`IfcAlignment` nested under `IfcProject` via `IfcRelAggregates`
`<CoordGeom>`	`IfcAlignmentHorizontal` nested via `IfcRelNests`
`<Profile>` / `<ProfAlign>`	`IfcAlignmentVertical` nested via `IfcRelNests`
`<Cant>`	`IfcAlignmentCant` nested via `IfcRelNests`
`<StaEquation>`	`IfcReferent` with stationing attributes and properties
`<Units>`	`IfcUnitAssignment` in `IfcProject`

Table A.1-1 — LandXML to IFC schema conceptual comparison

The key conceptual difference between the two schemas is that LandXML defines geometry through PI and PVI points with derived curve parameters, whereas IFC defines geometry as explicit typed segments each carrying a start condition, length, and curve parameters.

A.2 Units

LandXML carries explicit unit declarations (`<Metric>`, `<Imperial>`) for linear and angular measure. IFC requires all geometry in SI base units (meters, radians) or with an explicit `IfcConversionBasedUnit`.

Read `<Units>` before processing any geometry — all coordinate values depend on it.

Linear units. Convert foot and US Survey foot to meters as needed. Note that the US Survey foot and the international foot are not the same:

$$1 \text{ US Survey foot} = \frac{1200}{3937} \text{ m} \approx 0.3048006 \text{ m}$$

$$1 \text{ international foot} = 0.3048 \text{ m (exact)}$$

Using the wrong conversion factor introduces a systematic error that compounds over long alignments.

Angular units. LandXML directions can be expressed in decimal degrees, grads, or radians. IFC angular units are declared in `IfcUnitAssignment` — the SI base unit is radians, but `IfcConversionBasedUnit` can declare degrees, grads, or any other angular unit. When constructing an IFC model from LandXML, the converter controls what angular unit the output model declares; declaring radians (the default) avoids an extra conversion step and is the most common choice.

Bearing convention. LandXML directions are azimuths measured clockwise from North. IFC plane angles are measured counter-clockwise from the positive X-axis (East). The conversion is

$$\theta_{IFC} = \frac{\pi}{2} - \theta_{bearing}$$

where $\theta_{bearing}$ is in radians.

A.3 Horizontal Alignment

A.3.1 LandXML Horizontal Element Types

The LandXML `<CoordGeom>` element contains a sequence of `<Line>`, `<Curve>`, and `<Spiral>` child elements. Each maps to an `IfcAlignmentSegment` nested under `IfcAlignmentHorizontal`.

A.3.2 Direction from Geometry

LandXML lines and spirals often omit explicit direction attributes (`Dir`, `DirStart`) and rely on the geometry — start/end or start/PI points — to imply direction. `IfcAlignmentHorizontalSegment.StartDirection` is required. When the direction attribute is absent, compute it from the available geometry:

$$\theta = \text{atan2}(\Delta y, \Delta x)$$

then apply the bearing-to-IFC plane angle conversion from Section A.2.

A.3.3 Circular Curve (`<Curve>` → `CIRCULARARC`)

`<Curve>` provides center, start, end, and radius. IFC requires the start point, the start direction (the tangent direction, not the radial direction), the signed radius, and the arc length.

Compute the tangent direction from the radial line by rotating 90°:

$$\theta_{tangent} = \text{atan2}(y_{start} - y_{center},\ x_{start} - x_{center}) + \text{sign} \cdot \frac{\pi}{2}$$

where sign $= +1$ for counter-clockwise (`rot="ccw"`) and sign $= -1$ for clockwise (`rot="cw"`). The signed radius passed to `IfcAlignmentHorizontalSegment` follows the same sign convention.

A.3.4 Spiral Type Mapping

The following table maps LandXML `spiType` values to `IfcAlignmentHorizontalSegmentTypeEnum`.

LandXML `spiType`	IFC Type	Notes
`clothoid`	`CLOTHOID`	Direct mapping
`bloss`	`BLOSSCURVE`	Direct mapping
`biquadratic`	`HELMERTCURVE`	Direct mapping
`biquadraticParabola`	`HELMERTCURVE`	Equivalent to biquadratic
`cubic`	`CUBIC`	Direct mapping
`cubicParabola`	`CUBIC`	Equivalent to cubic
`cosine`	`COSINECURVE`	Direct mapping
`sinusoid`	`SINECURVE`	Direct mapping
`sineHalfWave`	`COSINECURVE`	Approximation — loss of fidelity, see note below
`japaneseCubic`	`VIENNESEBEND`	Direct mapping
`revBiquadratic`	—	IFC mapping unknown, log warning and skip
`revBloss`	—	IFC mapping unknown, log warning and skip
`revCosine`	—	IFC mapping unknown, log warning and skip
`revSinusoid`	—	IFC mapping unknown, log warning and skip
`radioid`	—	IFC mapping unknown, log warning and skip
`weinerBogen`	—	IFC mapping unknown, log warning and skip

Table A.3.4-1 — LandXML spiType to IFC type mapping

> **Note:** The LandXML specification describes the sine half-wavelength spiral as an approximation of a cosine spiral. Mapping it to `COSINECURVE` is therefore reasonable but not exact. Implementations should log a warning when this substitution is made.

A.3.5 Radius Sign Convention

LandXML uses `rot="cw"` / `rot="ccw"` to indicate curve direction. IFC encodes direction in the sign of `StartRadiusOfCurvature` and `EndRadiusOfCurvature`: a positive value indicates a left turn (counter-clockwise) and a negative value indicates a right turn (clockwise).

$$\text{sign} = \begin{cases} -1 & \text{if } rot = \text{cw} \\ +1 & \text{if } rot = \text{ccw} \end{cases}$$

Apply this sign to both `StartRadiusOfCurvature` and `EndRadiusOfCurvature`.

A.3.6 Infinite Radius

LandXML represents an infinite start or end radius by omitting the attribute or by supplying the value `INF`. IFC uses `0.0` in `IfcAlignmentHorizontalSegment.StartRadius` or `EndRadiusOfCurvature` to indicate an infinite radius. Substitute `0.0` whenever the LandXML value is absent or is `INF`.

A.4 Vertical Alignment

A.4.1 LandXML Vertical Data Model

LandXML represents vertical geometry as a sequence of PVI-based elements under `<ProfAlign>`: `<PVI>`, `<ParaCurve>`, `<UnsymParaCurve>`, and `<CircCurve>`. This is fundamentally different from IFC's segment-based model, which requires an explicit start station, start elevation, start grade, end grade, and length for each segment.

A.4.2 PVI-Based to Segment-Based Conversion

The core translation challenge is that LandXML locates vertical curves by their PVI station and curve length, while IFC needs each segment's start station explicitly. The conversion procedure is:

1. Compute the entry grade from the PVI station and the previous PVI or curve endpoint.
2. The start station of a symmetric parabolic curve is PVI station $- L/2$.
3. The start elevation is computed by projecting back from the PVI along the entry grade: $e_{start} = e_{PVI} - g_1 \cdot L/2$.
4. Gradient segments fill any gap between the end of one curve and the start of the next.

A.4.3 Symmetric Parabolic Arc (`<ParaCurve>` → `PARABOLICARC`)

This is the standard vertical curve case. Given PVI station s_{PVI}, PVI elevation e_{PVI}, curve length L, entry grade g_1, and exit grade g_2:

$$s_{start} = s_{PVI} - \frac{L}{2}$$

$$e_{start} = e_{PVI} - g_1 \frac{L}{2}$$

Map to `IfcAlignmentVerticalSegment` with `PredefinedType = PARABOLICARC`, `StartDistAlong = s_{start} - s_{alignment\_start}`, `StartHeight = e_{start}`, `StartGradient = g_1`, `EndGradient = g_2`, `HorizontalLength = L`.

A.4.4 Asymmetric Parabolic Arc (<UnsymParaCurve> → Two PARABOLICARC Segments)

IFC has no asymmetric vertical curve type. The <UnsymParaCurve> must be split into two abutting PARABOLICARC segments. Given entry length L_{in}, exit length L_{out}, entry grade g_1, and exit grade g_2:

Compute the vertical offset from the composite curve to the PVI:

$$h = \frac{L_{in} \cdot L_{out} \cdot (g_2 - g_1)}{2(L_{in} + L_{out})}$$

The transition point between the two sub-curves occurs at the PVI station with elevation:

$$e_{transition} = e_{PVI} + h$$

The intermediate grade at the transition point is:

$$g_{mid} = g_1 + \frac{2h}{L_{in}}$$

Create two PARABOLICARC segments: the first from the curve start to the PVI station with grades g_1 and g_{mid}, and the second from the PVI station to the curve end with grades g_{mid} and g_2.

A.4.5 Circular Vertical Curve (<CircCurve> → CIRCULARARC)

LandXML provides the radius and arc length. IFC IfcAlignmentVerticalSegment for CIRCULARARC accepts the radius directly in the RadiusOfCurvature attribute. Compute the start station and elevation the same way as for a symmetric parabolic arc, using $L/2$ as the half-length. Note that this is an approximation — the true tangent length of a circular vertical curve is $T = R\tan(\Delta/2)$ where $\Delta = L/R$, which differs from $L/2$ except for small deflection angles.

A.4.6 Station to Distance Along

LandXML vertical profile elements are positioned by station. IfcAlignmentVerticalSegment.StartDistAlong is distance along the horizontal alignment measured from the alignment start — a continuous value with no breaks. These are not the same thing whenever station equations are present.

In the simplest case — no station equations — the conversion is:

$$\text{StartDistAlong} = s_{LandXML} - s_{alignment_start}$$

where $s_{alignment_start}$ is read from <Alignment staStart="...">.

When station equations are present, the station numbering is discontinuous: a gap or overlap at each equation point means a given station value does not map to a unique

physical distance without knowing which zone it falls in. The conversion must walk the list of <StaEquation> elements in order, accumulating the physical distance up to each equation point, then apply the appropriate offset for the zone containing the target station. The staInternal attribute on <StaEquation> gives the continuous internal distance and can serve as a cross-check.

A.5 Cant

A.5.1 LandXML Cant Model

LandXML <Cant> uses <CantStation> points and <CantSegment> elements with left and right cant values at the start and end of each segment, along with a curve type designation.

A.5.2 Mapping to IfcAlignmentCant

Each <CantSegment> maps to an IfcAlignmentCantSegment with StartCantLeft, StartCantRight, EndCantLeft, EndCantRight, and PredefinedType. The cant segment type mapping follows the same spiral type table as horizontal alignment (Section A.3.4).

A.5.3 Rail Head Distance

IfcAlignmentCant.RailHeadDistance is the distance between rail heads (track gauge). In LandXML, this value is the gauge attribute on the <Cant> element:

```
<Cant name="001" gauge="1.524" rotationPoint="insideRail">
```

Map gauge directly to IfcAlignmentCant.RailHeadDistance. The value is in the model's declared length units.

A.5.4 Rotation Point Mapping

The rotationPoint attribute on <Cant> indicates which point on the cross-section serves as the pivot for superelevation rotation. LandXML defines three values: insideRail, centerline, and outsideRail. The curvature attribute on each <CantStation> indicates whether the curve is clockwise (cw, right turn) or counter-clockwise (ccw, left turn), which determines which physical rail is on the inside or outside.

IfcAlignmentCantSegment has no explicit rotation point attribute. Instead, rotation point is encoded implicitly through the combination of StartCantLeft, StartCantRight, EndCantLeft, and EndCantRight values. A zero cant value on one side indicates the pivot is on that side; equal and opposite values indicate centerline rotation.

Table A.5.4-1 gives the IFC cant values in terms of the LandXML appliedCant magnitude c for each combination of curvature and rotationPoint:

curvature	rotationPoint	CantLeft	CantRight
ccw (left turn, right = outside)	insideRail	0	$+c$
ccw (left turn, right = outside)	centerline	$-c/2$	$+c/2$

curvature	rotationPoint	CantLeft	CantRight
ccw (left turn, right = outside)	outsideRail	$-c$	0
cw (right turn, left = outside)	insideRail	$+c$	0
cw (right turn, left = outside)	centerline	$+c/2$	$-c/2$
cw (right turn, left = outside)	outsideRail	0	$-c$

Table A.5.4-1 — LandXML curvature and rotationPoint to IFC CantLeft/CantRight mapping

Apply the same logic to both the start and end values of each segment. The `rotationPoint` is constant for the entire `<Cant>` element; `curvature` and `appliedCant` are read from each `<CantStation>` and applied to the corresponding segment start and end positions.

A.6 Stationing and Station Equations

A.6.1 Start Station

The LandXML `<Alignment staStart="...">` attribute gives the station at the beginning of the alignment. Map this to an `IfcReferent` with `PredefinedType = STATION`, with the station value stored in `Pset_Stationing.Station`. See Chapter 9 for the full `IfcReferent` and `Pset_Stationing` structure.

A.6.2 Station Equations (`<StaEquation>`)

LandXML `<StaEquation>` records station breaks where the back station and ahead station differ. Each equation maps to an additional `IfcReferent`. The relevant attributes are:

LandXML Attribute	Meaning
staBack	Station value before the equation (incoming)
staAhead	Station value after the equation (outgoing)
staInternal	Continuous internal station (no breaks)

Table A.6.2-1 — LandXML station equation attributes

Map each `<StaEquation>` to an `IfcReferent` as follows:

- `staBack` → `Pset_Stationing.IncomingStation` (the station value on the incoming side of the break)
- `staAhead` → `Pset_Stationing.Station` (the station value on the outgoing side)
- `staInternal` → the `DistanceAlong` value in `IfcPointByDistanceExpression` for the referent's `IfcLinearPlacement`

`staInternal` is the continuous internal distance along the alignment with no station breaks. It is therefore the correct value to use as `DistanceAlong` — it maps directly to arc-length along the horizontal curve without any equation adjustment. If `staInternal` is absent, compute `DistanceAlong` by walking the equation list as described in Section A.4.6.

The resulting `IfcReferent` instances, including the starting station referent from A.6.1, are nested under the `IfcAlignment` via a dedicated `IfcRelNests` relationship in order of increasing `DistanceAlong`. See Chapter 9 for the complete `IfcReferent` structure, the nesting requirements, and the algorithm for converting station labels to `DistanceAlong` when equations are present.

Index

A

accuracy — 5.2, 11.0
arc length — 2.2, 3.2, 4.2, 5.5
arc length, offset curve — 5.2
angle point — 8.2.1.1
asymmetric parabolic arc — A.4.4
authoring versus consuming — 10.5.3
Axis (coordinate vector) — 4.1.2, 8.3.1

B

banking — *see* cant
Bloss transition curve
  horizontal — 2.8, 2.8.1–2.8.3
  cant — 4.6, 4.6.1–4.6.3
  testset — 12.3.1, 12.3.3
breaklines — 10.3.1

C

cant alignment — 1.3, 4.0–4.12
cant coordinate system — 4.1.2
cant transition — 4.1.1, 4.1.3
`CartesianPosition` — 8.4
chainage — *see* `DistanceAlong`
circular arc
  horizontal — 2.4, 2.4.1–2.4.3
  vertical — 3.4, 3.4.1–3.4.3
  LandXML — A.3.3, A.4.5
Clothoid (Euler spiral)
  horizontal — 2.5, 2.5.1–2.5.3
  vertical — 3.5
  testset — 12.3.1, 12.3.3
closing segment — *see* zero-length segment
combined 3D alignment — 3.7, 4.10
concept template — 7.1, 7.3
constant cant — 4.3, 4.3.1–4.3.3
constant gradient — 3.3, 3.3.1–3.3.3
coordinate system
  cant — 4.1.2
  placement — 8.3

Cosine transition curve
horizontal — 2.9, 2.9.1–2.9.3
cant — 4.7, 4.7.1–4.7.3
testset — 12.3.1, 12.3.3
cross-section — 10.0–10.5
cross-track direction — 4.1.2
cubic transition curve — 2.6, 2.6.1–2.6.3
curvature — 2.2, Notation
curve segment evaluation algorithm
horizontal — 2.2
vertical — 3.2
cant — 4.2

D

default Axis direction — 8.3.2, 10.5.6
deviating elevation — 4.0, 4.1.2, Notation
directrix — 10.3, 10.4, 10.5
`DistanceAlong` — 8.2.1, 8.2.2, 9.2.5, 9.2.7
`DistanceAlong`**, station conversion** — 8.2.2, 9.2.5, 9.2.6, 9.2.7, A.4.6

E

edge of pavement — 5.0, 10.2
EnrichIfc4x3 — 4.11
Euler spiral — *see* Clothoid
example models
offset curves — 5.5
linear placement — 8.3.3
sectioned surfaces — 10.6.1
sectioned solids — 10.6.2

F

fallback Cartesian position — 8.4
Fresnel integrals — 2.5.1

G

gap equation — 9.2.3
gap, station undefined — 9.2.5, 9.2.6, 9.2.7
geometric representation — 1.5, 2.1
grade — 3.3, Notation
grade-separated crossing — 9.5.2
guide curve — 10.2, 10.3, 10.5

H

`HasIncreasingStation` — 9.2.5, 9.2.7, 9.3.1
Helmert transition curve
horizontal — 2.7, 2.7.1–2.7.3
cant — 4.5, 4.5.1–4.5.3
testset — 12.3.1, 12.3.3
horizontal alignment — 1.1, 2.0–2.13

I

IFC105 validation rule — 5.1, 10.5.2
`IfcAlignment` — 1.0, 1.4, 1.5, 5.2, 7.1
`IfcAlignmentCant` — 1.3, 4.0
`IfcAlignmentHorizontal` — 1.1, 2.0, 2.1
`IfcAlignmentSegment` — 1.4, 1.5.2
`IfcAlignmentVertical` — 1.2, 3.0, 3.1
`IfcAxis2PlacementLinear` — 8.2, 8.3
`IfcCircle`
horizontal — 2.4, 2.4.1
vertical — 3.4, 3.4.1
`IfcCompositeCurve` — 1.5, 1.5.1, 2.1
`IfcConnectionPointEccentricity` — 9.5.2
`IfcConnectionPointGeometry` — 9.5.2
`IfcCurveSegment` — 1.5, 1.5.2
`IfcGradientCurve` — 1.5.1, 3.7, 8.3.3
`IfcIndexedPolyCurve` — 6.1
`IfcInterferenceSelect` — 9.5.2
`IfcLine`
horizontal — 2.3, 2.3.1
vertical — 3.3, 3.3.1
`IfcLinearPlacement` — 8.2, 8.5
`IfcOffsetCurveByDistances` — 5.0, 5.1, 5.2, 5.3, 5.4, 5.5, 8.5
`IfcPointByDistanceExpression` — 5.0, 8.2
`IfcPolyline` — 6.1
`IfcPolynomialCurve` — 2.6, 2.7, 2.8, 2.9, 2.10, 2.11
`IfcRailway` — 9.5.2
`IfcReferent` — 9.1, 9.3, 9.3.4, 9.4, 9.5, 9.6
`IfcRelInterferesElements` — 9.5.2
`IfcRelNests` — 9.3.1, 9.3.2, 9.3.4
`IfcRelPositions` — 9.4.1, 9.4.2, 9.5
`IfcRelReferencedInSpatialStructure` — 9.5.2
`IfcRoad` — 9.5.2
`IfcSectionedSolidHorizontal` — 10.4, 10.4.1, 10.6.2
`IfcSectionedSurface` — 10.3, 10.3.1, 10.3.2, 10.6.1

`IfcSegmentedReferenceCurve` — 1.5.1, 4.0, 4.2
implementation checklist
horizontal — 2.13
vertical — 3.9
cant — 4.13
linear placement — 8.6
referents and stationing — 9.7
implementation questions — 7.2
`ImpliedOrder` — 9.5.2
infinite radius — A.3.6
`InterferenceType` — 9.5.2
`INTERSECTION` **referent** — 9.5, 9.5.1
interpolation, offset values — 5.3
ISO 19148 — 9.5.1, 9.6

K

key-point referent — 9.3.4
horizontal (P.O.B., P.C., P.T., P.O.E.) — 9.3.4.1
vertical (V.P.O.B., P.V.C., P.V.T., V.P.O.E.) — 9.3.4.1

L

LandXML — Appendix A
horizontal mapping — A.3
vertical mapping — A.4
cant mapping — A.5
stationing — A.6
lane striping — 5.0
linear placement — 8.0–8.6
linear referencing — 9.6
linear transition (cant) — 4.4, 4.4.1–4.4.3
live model management — 7.3

M

matrix composition
horizontal — 2.2
vertical — 3.2
cant — 4.2
mixed-nesting problem — 9.3.2

N

notation — Notation chapter

O

offset curves — 5.0–5.5
offset sign convention — 5.1
`OffsetLateral` — 5.2, 8.2.1, 8.5.2
`OffsetLongitudinal` — 8.2.1.1
`OffsetVertical` — 8.2.1
overlap equation — 9.2.4
overlap, station ambiguity — 9.2.6, 9.2.7

P

parabolic arc — 3.6, 3.6.1–3.6.3
  LandXML — A.4.3, A.4.4
parent curve — 1.5.2, 2.1
parent/child structure — 7.1.1
`PlacementRelTo` — 8.1.2, 8.3.3
precision — 11.0
Product Relative Positioning — 9.4.1
Product Span Positioning — 9.4.2
profile (cross-section) — 10.1
profile orientation — 10.5.7
`Pset_LinearReferencingMethod` — 9.6.2, 9.6.3, 9.6.4
`Pset_Stationing` — 9.2.7, 9.3.1, 9.3.4
PVI (point of vertical intersection) — 9.3.4, A.4.2

R

radius sign convention — 2.1, A.3.5
real-world alignments (testset) — 9.3.4.1, 12.3.4
reference coordinates — 12.2.4, 12.4
referent ordering — 9.2.7, 9.3.1
referent placement (`ObjectPlacement`) — 9.3.3
representation scoping — 10.5.2
retaining wall — 10.6.2.4
reverse stationing — *see* stationing, decreasing / reverse
RefDirection — 4.1.2, 8.3.1
rotation point — A.5.4

S

sectioned surfaces and solids — 10.0–10.6
specification gaps — 10.5
segment mapping — 1.6, 2.3.2, 2.4.2, 2.5.2, 2.6.2, 2.7.2, 2.8.2, 2.9.2, 2.10.2, 2.11.2
semantic definition — 1.4, 2.1

semantic vs. geometric — 1.4, 1.5, 9.4
sibling representation — 10.5.1, 10.5.2
sign convention
  curvature — 2.1
  offset — 5.0
  radius (LandXML) — A.3.5
Sine transition curve
  horizontal — 2.10, 2.10.1–2.10.3
  cant — 4.8, 4.8.1–4.8.3
  testset — 12.3.1, 12.3.3
spiral — *see* transition curves
start station — 9.2.1, A.6.1
station equations — 9.2.2–9.2.6, A.6.2
stationing — 9.2, 9.3.1, A.6
  decreasing / reverse (`HasIncreasingStation`) — 9.2.7, 9.3.1, 9.6.2
stringlines — 10.2, 10.3.2, 10.6.1.4–10.6.1.6, 10.6.2.5, 10.6.2.6
superelevation — *see* cant

T

Tag attribute — 5.4, 10.3
tangent direction — 4.1.2
testset — 12.1–12.5
  file naming — 12.2
  horizontal test cases — 12.3.1
  vertical test cases — 12.3.2
  cant test cases — 12.3.3
  real-world cases — 12.3.4
  validation guidance — 12.5
tolerance — 11.0
transition curves — 2.5–2.11
  see also individual curve types
two-level placement — 2.7.2

U

units of measure — 1.7, 12.2.1, A.2
unreachable points — 8.2.1.1

V

validation — 5.1, 12.5
vertical alignment — 1.2, 3.0–3.8
Viennese Bend (VienneseBend)
  horizontal — 2.11, 2.11.1–2.11.3

cant — 4.9, 4.9.1–4.9.3
testset — 12.3.1, 12.3.3

W

workflow, alignment reuse — 7.3, 7.3.1

Z

zero-length segment — 2.12, 2.12.1–2.12.2